

CAPÍTULO

4

SISTEMAS DE ARQUIVOS

Todas as aplicações de computadores precisam armazenar e recuperar informações. Enquanto um processo está sendo executado, ele pode armazenar uma quantidade limitada de informações dentro do seu próprio espaço de endereçamento. No entanto, a capacidade de armazenamento está restrita ao tamanho do espaço do endereçamento virtual. Para algumas aplicações esse tamanho é adequado, mas, para outras, como reservas de passagens aéreas, bancos ou sistemas corporativos, ele é pequeno demais.

Um segundo problema em manter informações dentro do espaço de endereçamento de um processo é que, quando o processo é concluído, as informações são perdidas. Para muitas aplicações (por exemplo, bancos de dados), as informações precisam ser retidas por semanas, meses, ou mesmo para sempre. Perdê-las quando o processo que as está utilizando é concluído é algo inaceitável. Além disso, elas não devem desaparecer quando uma falha no computador mata um processo.

Um terceiro problema é que frequentemente é necessário que múltiplos processos acessem (partes de) uma informação ao mesmo tempo. Se temos um diretório telefônico on-line armazenado dentro do espaço de um único processo, apenas aquele processo pode acessá-lo. A maneira para solucionar esse problema é tornar a informação em si independente de qualquer processo.

Assim, temos três requisitos essenciais para o armazenamento de informações por um longo prazo:

1. Deve ser possível armazenar uma quantidade muito grande de informações.

2. As informações devem sobreviver ao término do processo que as está utilizando.
3. Múltiplos processos têm de ser capazes de acessá-las ao mesmo tempo.

Discos magnéticos foram usados por anos para esse armazenamento de longo prazo. Em anos recentes, unidades de estado sólido tornaram-se cada vez mais populares, à medida que elas não têm partes móveis que possam quebrar. Elas também oferecem um rápido acesso aleatório. Fitas e discos óticos também foram amplamente usados, mas são dispositivos com um desempenho muito pior e costumam ser usados como backups. Estudaremos mais sobre discos no Capítulo 5, mas por ora, basta pensar em um disco como uma sequência linear de blocos de tamanho fixo e que dão suporte a duas operações:

1. Leia o bloco k .
2. Escreva no bloco k .

Na realidade, existem mais operações, mas com essas duas, em princípio, você pode solucionar o problema do armazenamento de longo prazo.

No entanto, essas são operações muito inconvenientes, mais ainda em sistemas grandes usados por muitas aplicações e possivelmente múltiplos usuários (por exemplo, em um servidor). Apenas algumas das questões que rapidamente surgem são:

1. Como você encontra informações?
2. Como impedir que um usuário leia os dados de outro?
3. Como saber quais blocos estão livres?

e há muitas mais.

Da mesma maneira que vimos como o sistema operacional abstraía o conceito do processador para criar a abstração de um processo e como ele abstraía o conceito da memória física para oferecer aos processos espaços de endereçamento (virtuais), podemos solucionar esse problema com uma nova abstração: o arquivo. Juntas, as abstrações de processos (e threads), espaços de endereçamento e arquivos são os conceitos mais importantes relacionados com os sistemas operacionais. Se você realmente compreender esses três conceitos do início ao fim, estará bem encaminhado para se tornar um especialista em sistemas operacionais.

Arquivos são unidades lógicas de informação criadas por processos. Um disco normalmente conterá milhares ou mesmo milhões deles, cada um independente dos outros. Na realidade, se pensar em cada arquivo como uma espécie de espaço de endereçamento, você não estará muito longe da verdade, exceto que eles são usados para modelar o disco em vez de modelar a RAM.

Processos podem ler arquivos existentes e criar novos se necessário. Informações armazenadas em arquivos devem ser **persistentes**, isto é, não devem ser afetadas pela criação e término de um processo. Um arquivo deve desaparecer apenas quando o seu proprietário o remove explicitamente. Embora as operações para leitura e escrita de arquivos sejam as mais comuns, existem muitas outras, algumas das quais examinaremos a seguir.

Arquivos são gerenciados pelo sistema operacional. Como são estruturados, nomeados, acessados, usados, protegidos, implementados e gerenciados são tópicos importantes no projeto de um sistema operacional. Como um todo, aquela parte do sistema operacional lidando com arquivos é conhecida como **sistema de arquivos** e é o assunto deste capítulo.

Do ponto de vista do usuário, o aspecto mais importante de um sistema de arquivos é como ele aparece, em outras palavras, o que constitui um arquivo, como os arquivos são nomeados e protegidos, quais operações são permitidas e assim por diante. Os detalhes sobre se listas encadeadas ou mapas de bits são usados para o armazenamento disponível e quantos setores existem em um bloco de disco lógico não lhes interessam, embora sejam de grande importância para os projetistas do sistema de arquivos. Por essa razão, estruturamos o capítulo como várias seções. As duas primeiras dizem respeito à interface do usuário para os arquivos e para os diretórios, respectivamente. Então segue uma discussão detalhada de como o sistema de arquivos é implementado e gerenciado. Por fim, damos alguns exemplos de sistemas de arquivos reais.

4.1 Arquivos

Nas páginas a seguir examinaremos os arquivos do ponto de vista do usuário, isto é, como eles são usados e quais propriedades têm.

4.1.1 Nomeação de arquivos

Um arquivo é um mecanismo de abstração. Ele fornece uma maneira para armazenar informações sobre o disco e lê-las depois. Isso deve ser feito de tal modo que isole o usuário dos detalhes de como e onde as informações estão armazenadas, e como os discos realmente funcionam.

É provável que a característica mais importante de qualquer mecanismo de abstração seja a maneira como os objetos que estão sendo gerenciados são nomeados; portanto, começaremos nosso exame dos sistemas de arquivos com o assunto da nomeação de arquivos. Quando um processo cria um arquivo, ele lhe dá um nome. Quando o processo é concluído, o arquivo continua a existir e pode ser acessado por outros processos usando o seu nome.

As regras exatas para a nomeação de arquivos variam de certa maneira de sistema para sistema, mas todos os sistemas operacionais atuais permitem cadeias de uma a oito letras como nomes de arquivos legais. Desse modo, *andrea*, *bruce* e *cathy* são nomes de arquivos possíveis. Não raro, dígitos e caracteres especiais também são permitidos, assim nomes como *2*, *urgente!* e *Fig.2-14* são muitas vezes válidos também. Muitos sistemas de arquivos aceitam nomes com até 255 caracteres.

Alguns sistemas de arquivos distinguem entre letras maiúsculas e minúsculas, enquanto outros, não. O UNIX pertence à primeira categoria; o velho MS-DOS cai na segunda. (Como nota, embora antigo, o MS-DOS ainda é amplamente usado em sistemas embarcados, portanto ele não é obsoleto de maneira alguma.) Assim, um sistema UNIX pode ter todos os arquivos a seguir como três arquivos distintos: *maria*, *Maria* e *MARIA*. No MS-DOS, todos esses nomes referem-se ao mesmo arquivo.

Talvez seja um bom momento para fazer um comentário aqui sobre os sistemas operacionais. O Windows 95 e o Windows 98 usavam o mesmo sistema de arquivos do MS-DOS, chamado **FAT-16**, e portanto herdaram muitas de suas propriedades, como a maneira de se formarem os nomes dos arquivos. O Windows 98 introduziu algumas extensões ao FAT-16, levando ao **FAT-32**, mas esses dois são bastante parecidos. Além disso, o Windows NT, Windows 2000, Windows XP, Windows

Vista, Windows 7 e Windows 8 ainda dão suporte a ambos os sistemas de arquivos FAT, que estão realmente obsoletos agora. No entanto, esses sistemas operacionais novos também têm um sistema de arquivos nativo muito mais avançado (**NTFS — native file system**) que tem propriedades diferentes (como nomes de arquivos em Unicode). Na realidade, há um segundo sistema de arquivos para o Windows 8, conhecido como **ReFS** (ou **Resilient File System — sistema de arquivos resiliente**), mas ele é voltado para a versão de servidor do Windows 8. Neste capítulo, quando nos referimos ao MS-DOS ou sistemas de arquivos FAT, estaremos falando do FAT-16 e FAT-32 como usados no Windows, a não ser que especificado de outra forma. Discutiremos o sistema de arquivos FAT mais tarde neste capítulo e NTFS no Capítulo 12, onde examinaremos o Windows 8 com detalhes. Incidentalmente, existe também um novo sistema de arquivos semelhante ao FAT, conhecido como sistema de arquivos **exFAT**, uma extensão da Microsoft para o FAT-32 que é otimizado para flash drives e sistemas de arquivos grandes. ExFAT é o único sistema de arquivos moderno da Microsoft que o OS X pode ler e escrever.

Muitos sistemas operacionais aceitam nomes de arquivos de duas partes, com as partes separadas por um ponto, como em *prog.c*. A parte que vem em seguida ao ponto é chamada de **extensão do arquivo** e costuma indicar algo seu a respeito. No MS-DOS,

por exemplo, nomes de arquivos têm de 1 a 8 caracteres, mais uma extensão opcional de 1 a 3 caracteres. No UNIX, o tamanho da extensão, se houver, cabe ao usuário decidir, e um arquivo pode ter até duas ou mais extensões, como em *homepage.html.zip*, onde *.html* indica uma página da web em HTML e *.zip* indica que o arquivo (*homepage.html*) foi compactado usando o programa *zip*. Algumas das extensões de arquivos mais comuns e seus significados são mostradas na Figura 4.1.

Em alguns sistemas (por exemplo, todas as variações do UNIX), as extensões de arquivos são apenas convenções e não são impostas pelo sistema operacional. Um arquivo chamado *file.txt* pode ser algum tipo de arquivo de texto, mas aquele nome tem a função mais de lembrar o proprietário do que transmitir qualquer informação real para o computador. Por outro lado, um compilador C pode realmente insistir em que os arquivos que ele tem de compilar terminem em *.c*, e se isso não acontecer, pode recusar-se a compilá-los. O sistema operacional, no entanto, não se importa.

Convenções como essa são especialmente úteis quando o mesmo programa pode lidar com vários tipos diferentes de arquivos. O compilador C, por exemplo, pode receber uma lista de vários arquivos a serem compilados e ligados, alguns deles arquivos C e outros arquivos de linguagem de montagem. A extensão então se torna essencial para o compilador dizer quais são os

FIGURA 4.1 Algumas extensões comuns de arquivos.

Extensão	Significado
.bak	Cópia de segurança
.c	Código-fonte de programa em C
.gif	Imagem no formato Graphical Interchange Format
.hlp	Arquivo de ajuda
.html	Documento em HTML
.jpg	Imagem codificada segundo padrões JPEG
.mp3	Música codificada no formato MPEG (camada 3)
.mpg	Filme codificado no padrão MPEG
.o	Arquivo objeto (gerado por compilador, ainda não ligado)
.pdf	Arquivo no formato PDF (Portable Document File)
.ps	Arquivo PostScript
.tex	Entrada para o programa de formatação TEX
.txt	Arquivo de texto
.zip	Arquivo compactado

arquivos C, os arquivos de linguagem de montagem e outros arquivos.

Em contrapartida, o Windows é consciente das extensões e designa significados a elas. Usuários (ou processos) podem registrar extensões com o sistema operacional e especificar para cada uma qual é seu “proprietário”. Quando um usuário clica duas vezes sobre o nome de um arquivo, o programa designado para essa extensão de arquivo é lançado com o arquivo como parâmetro. Por exemplo, clicar duas vezes sobre *file.docx* inicializará o Microsoft Word, tendo *file.docx* como seu arquivo inicial para edição.

4.1.2 Estrutura de arquivos

Arquivos podem ser estruturados de várias maneiras. Três possibilidades comuns estão descritas na Figura 4.2. O arquivo na Figura 4.2(a) é uma sequência desestruturada de bytes. Na realidade, o sistema operacional não sabe ou não se importa sobre o que há no arquivo. Tudo o que ele vê são bytes. Qualquer significado deve ser imposto por programas em nível de usuário. Tanto UNIX quanto Windows usam essa abordagem.

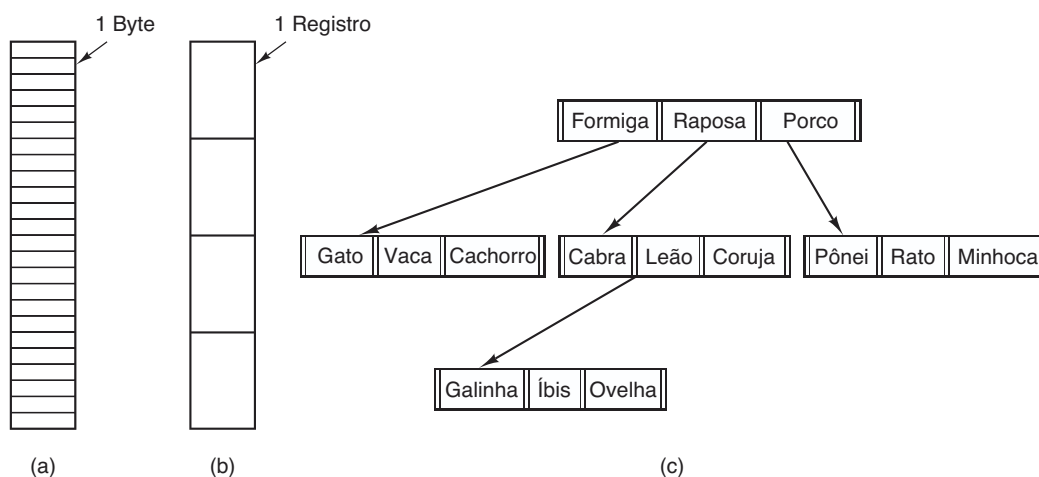
Ter o sistema operacional tratando arquivos como nada mais que sequências de bytes oferece a máxima flexibilidade. Programas de usuários podem colocar qualquer coisa que eles quiserem em seus arquivos e nomeá-los do jeito que acharem conveniente. O sistema operacional não ajuda, mas também não interfere. Para usuários que querem realizar coisas incomuns, o segundo ponto pode ser muito importante. Todas as versões do UNIX (incluindo Linux e OS X) e o Windows usam esse modelo de arquivos.

O primeiro passo na estruturação está ilustrado na Figura 4.2(b). Nesse modelo, um arquivo é uma sequência de registros de tamanho fixo, cada um com alguma estrutura interna. O fundamental para que um arquivo seja uma sequência de registros é a ideia de que a operação de leitura retorna um registro e a operação de escrita sobrepõe ou anexa um registro. Como nota histórica, décadas atrás, quando o cartão de 80 colunas perfurado era o astro, muitos sistemas operacionais de computadores de grande porte baseavam seus sistemas de arquivos em arquivos consistindo em registros de 80 caracteres, na realidade, imagens de cartões. Esses sistemas também aceitavam arquivos com registros de 132 caracteres, destinados às impressoras de linha (que naquela época eram grandes impressoras de corrente com 132 colunas). Os programas liam a entrada em unidades de 80 caracteres e a escreviam em unidades de 132 caracteres, embora os últimos 52 pudessem ser espaços, é claro. Nenhum sistema de propósito geral atual usa mais esse modelo como seu sistema primário de arquivos, mas na época dos cartões perfurados de 80 colunas e impressoras de 132 caracteres por linha era um modelo comum em computadores de grande porte.

O terceiro tipo de estrutura de arquivo é mostrado na Figura 4.2(c). Nessa organização, um arquivo consiste em uma árvore de registros, não necessariamente todos do mesmo tamanho, cada um contendo um campo **chave** em uma posição fixa no registro. A árvore é ordenada no campo chave, a fim de permitir uma busca rápida por uma chave específica.

A operação básica aqui não é obter o “próximo” registro, embora isso também seja possível, mas aquele com a chave específica. Para o arquivo zoológico da Figura 4.2(c), você poderia pedir ao sistema para obter

FIGURA 4.2 Três tipos de arquivos. (a) Sequência de bytes. (b) Sequência de registros. (c) Árvore.



o registro cuja chave fosse *pônei*, por exemplo, sem se preocupar com sua posição exata no arquivo. Além disso, novos registros podem ser adicionados, com o sistema operacional, e não o usuário, decidindo onde colocá-los. Esse tipo de arquivo é claramente bastante diferente das seqüências de bytes desestruturadas usadas no UNIX e Windows, e é usado em alguns computadores de grande porte para o processamento de dados comerciais.

4.1.3 Tipos de arquivos

Muitos sistemas operacionais aceitam vários tipos de arquivos. O UNIX (novamente, incluindo OS X) e o Windows, por exemplo, apresentam arquivos regulares e diretórios. O UNIX também tem arquivos especiais de caracteres e blocos. **Arquivos regulares** são aqueles que contêm informações do usuário. Todos os arquivos da Figura 4.2 são arquivos regulares. **Diretórios** são arquivos do sistema para manter a estrutura do sistema de arquivos. Estudaremos diretórios a seguir. **Arquivos especiais de caracteres** são relacionados com entrada/saída e usados para modelar dispositivos de E/S seriais como terminais, impressoras e redes. **Arquivos especiais de blocos** são usados para modelar discos. Neste capítulo, estaremos interessados fundamentalmente em arquivos regulares.

Arquivos regulares geralmente são arquivos ASCII ou arquivos binários. Arquivos ASCII consistem de linhas de texto. Em alguns sistemas, cada linha termina com um caractere de retorno de carro (carriage return). Em outros, o caractere de próxima linha (line feed) é usado. Alguns sistemas (por exemplo, Windows) usam ambos. As linhas não precisam ser todas do mesmo tamanho.

A grande vantagem dos arquivos ASCII é que eles podem ser exibidos e impressos como são e editados com qualquer editor de texto. Além disso, se grandes números de programas usam arquivos ASCII para entrada e saída, é fácil conectar a saída de um programa com a entrada de outro, como em pipelines do interpretador de comandos (*shell*). (O uso de pipelines entre processos não é nem um pouco mais fácil, mas a interpretação da informação certamente torna-se mais fácil se uma convenção padrão, como a ASCII, for usada para expressá-la.)

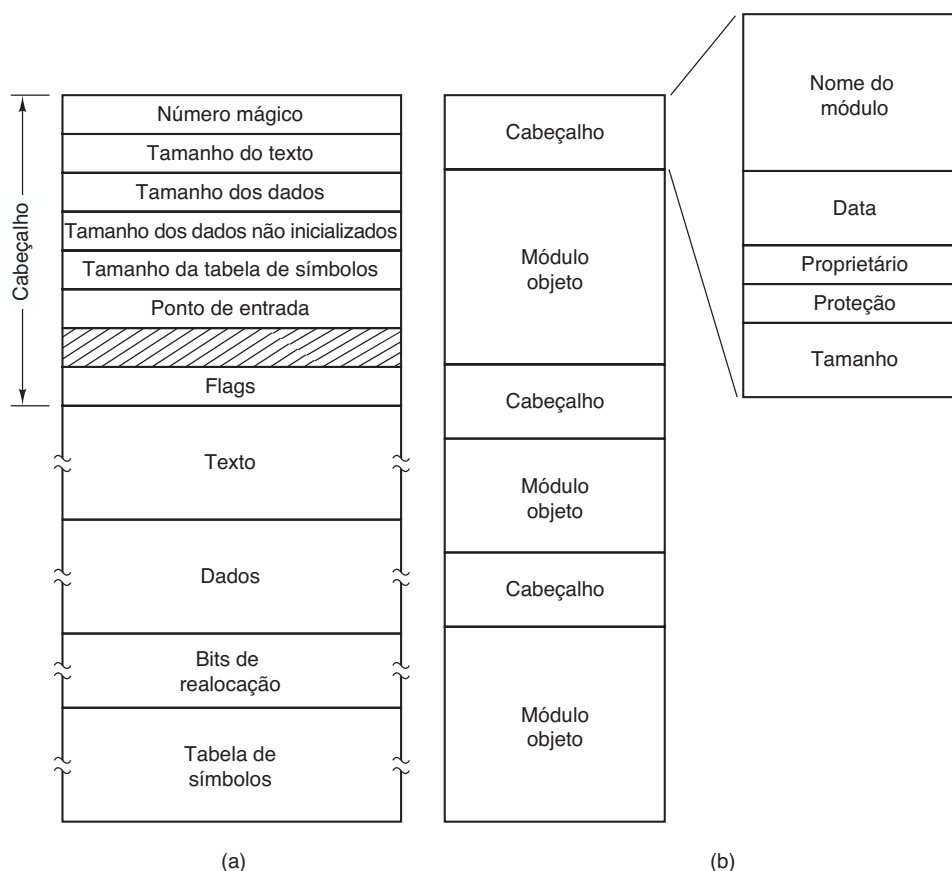
Outros arquivos são binários, o que apenas significa que eles não são arquivos ASCII. Listá-los em uma impressora resultaria em algo completamente incompreensível. Em geral, eles têm alguma estrutura interna conhecida pelos programas que os usam.

Por exemplo, na Figura 4.3(a) vemos um arquivo binário executável simples tirado de uma versão inicial do UNIX. Embora tecnicamente o arquivo seja apenas uma seqüência de bytes, o sistema operacional o executará somente se ele tiver o formato apropriado. Ele tem cinco seções: cabeçalho, texto, dados, bits de realocação e tabela de símbolos. O cabeçalho começa com o chamado **número mágico**, identificando o arquivo como executável (para evitar a execução acidental de um arquivo que não esteja em seu formato). Então vêm os tamanhos das várias partes do arquivo, o endereço no qual a execução começa e alguns bits de sinalização. Após o cabeçalho, estão o texto e os dados do próprio programa, que são carregados para a memória e realocados usando os bits de realocação. A tabela de símbolos é usada para correção de erros.

Nosso segundo exemplo de um arquivo binário é um repositório (archive), também do UNIX. Ele consiste em uma série de rotinas de biblioteca (módulos) compiladas, mas não ligadas. Cada uma é prefaciada por um cabeçalho dizendo seu nome, data de criação, proprietário, código de proteção e tamanho. Da mesma forma que o arquivo executável, os cabeçalhos de módulos estão cheios de números binários. Copiá-los para a impressora produziria puro lixo.

Todo sistema operacional deve reconhecer pelo menos um tipo de arquivo: o seu próprio arquivo executável; alguns reconhecem mais. O velho sistema TOPS-20 (para o DECSysystem 20) chegou ao ponto de examinar data e horário de criação de qualquer arquivo a ser executado. Então ele localizava o arquivo-fonte e via se a fonte havia sido modificada desde a criação do binário. Em caso positivo, ele automaticamente recompilava a fonte. Em termos de UNIX, o programa *make* havia sido embutido no shell. As extensões de arquivos eram obrigatórias, então ele poderia dizer qual programa binário era derivado de qual fonte.

Ter arquivos fortemente tipificados como esse causa problemas sempre que o usuário fizer algo que os projetistas do sistema não esperavam. Considere, como um exemplo, um sistema no qual os arquivos de saída do programa têm a extensão *.dat* (arquivos de dados). Se um usuário escrever um formatador de programa que lê um arquivo *.c* (programa C), o transformar (por exemplo, convertendo-o em um layout padrão de indentação), e então escrever o arquivo transformado como um arquivo de saída, ele será do tipo *.dat*. Se o usuário tentar oferecer isso ao compilador C para compilá-lo, o sistema se recusará porque ele tem a extensão errada. Tentativas de copiar *file.dat* para *file.c* serão rejeitadas

FIGURA 4.3 (a) Um arquivo executável. (b) Um repositório (archive).

pelo sistema como inválidas (a fim de proteger o usuário contra erros).

Embora esse tipo de “facilidade para o usuário” possa ajudar os novatos, é um estorvo para os usuários experientes, pois eles têm de devotar um esforço considerável para driblar a ideia do sistema operacional do que seja razoável ou não.

4.1.4 Acesso aos arquivos

Os primeiros sistemas operacionais forneciam apenas um tipo de acesso aos arquivos: **acesso sequencial**. Nesses sistemas, um processo podia ler todos os bytes ou registros em um arquivo em ordem, começando do princípio, mas não podia pular nenhum ou lê-los fora de ordem. No entanto, arquivos sequenciais podiam ser trazidos de volta para o ponto de partida, então eles podiam ser lidos tantas vezes quanto necessário. Arquivos sequenciais eram convenientes quando o meio de armazenamento era uma fita magnética, em vez de um disco.

Quando os discos passaram a ser usados para armazenar arquivos, tornou-se possível ler os bytes ou registros de um arquivo fora de ordem, ou acessar os

registros pela chave em vez de pela posição. Arquivos ou registros que podem ser lidos em qualquer ordem são chamados de **arquivos de acesso aleatório**. Eles são necessários para muitas aplicações.

Arquivos de acesso aleatório são essenciais para muitas aplicações, por exemplo, sistemas de bancos de dados. Se um cliente de uma companhia aérea liga e quer reservar um assento em um determinado voo, o programa de reservas deve ser capaz de acessar o registro para aquele voo sem ter de ler primeiro os registros para milhares de outros voos.

Dois métodos podem ser usados para especificar onde começar a leitura. No primeiro, cada operação **read** fornece a posição no arquivo onde começar a leitura. No segundo, uma operação simples, **seek**, é fornecida para estabelecer a posição atual. Após um **seek**, o arquivo pode ser lido sequencialmente da posição agora atual. O segundo método é usado no UNIX e no Windows.

4.1.5 Atributos de arquivos

Todo arquivo possui um nome e sua data. Além disso, todos os sistemas operacionais associam outras

informações com cada arquivo, por exemplo, a data e o horário em que foi modificado pela última vez, assim como o tamanho do arquivo. Chamaremos esses itens extras de **atributos** do arquivo. Algumas pessoas os chamam de **metadados**. A lista de atributos varia bastante de um sistema para outro. A tabela da Figura 4.4 mostra algumas das possibilidades, mas existem outras. Nenhum sistema existente tem todos esses atributos, mas cada um está presente em algum sistema.

Os primeiros quatro atributos concernem à proteção do arquivo e dizem quem pode acessá-lo e quem não pode. Todos os tipos de esquemas são possíveis, alguns dos quais estudaremos mais tarde. Em alguns sistemas o usuário deve apresentar uma senha para acessar um arquivo, caso em que a senha deve ser um dos atributos.

As sinalizações (flags) são bits ou campos curtos que controlam ou habilitam alguma propriedade específica. Arquivos ocultos, por exemplo, não aparecem nas listagens de todos os arquivos. A sinalização de arquivamento é um bit que controla se foi feito um backup do

arquivo recentemente. O programa de backup remove esse bit e o sistema operacional o recoloca sempre que um arquivo for modificado. Dessa maneira, o programa consegue dizer quais arquivos precisam de backup. A sinalização temporária permite que um arquivo seja marcado para ser deletado automaticamente quando o processo que o criou for concluído.

O tamanho do registro, posição da chave e tamanho dos campos-chave estão presentes apenas em arquivos cujos registros podem ser lidos usando uma chave. Eles proporcionam a informação necessária para encontrar as chaves.

Os vários registros de tempo controlam quando o arquivo foi criado, acessado e modificado pela última vez, os quais são úteis para uma série de finalidades. Por exemplo, um arquivo-fonte que foi modificado após a criação do arquivo-objeto correspondente precisa ser recompilado. Esses campos fornecem as informações necessárias.

O tamanho atual nos informa o tamanho que o arquivo tem no momento. Alguns sistemas operacionais

FIGURA 4.4 Alguns possíveis atributos de arquivos.

Atributo	Significado
Proteção	Quem tem acesso ao arquivo e de que modo
Senha	Necessidade de senha para acesso ao arquivo
Criador	ID do criador do arquivo
Proprietário	Proprietário atual
Flag de somente leitura	0 para leitura/escrita; 1 para somente leitura
Flag de oculto	0 para normal; 1 para não exibir o arquivo
Flag de sistema	0 para arquivos normais; 1 para arquivos de sistema
Flag de arquivamento	0 para arquivos com backup; 1 para arquivos sem backup
Flag de ASCII/binário	0 para arquivos ASCII; 1 para arquivos binários
Flag de acesso aleatório	0 para acesso somente sequencial; 1 para acesso aleatório
Flag de temporário	0 para normal; 1 para apagar o arquivo ao sair do processo
Flag de travamento	0 para destravados; diferente de 0 para travados
Tamanho do registro	Número de bytes em um registro
Posição da chave	Posição da chave em cada registro
Tamanho da chave	Número de bytes na chave
Momento de criação	Data e hora de criação do arquivo
Momento do último acesso	Data e hora do último acesso do arquivo
Momento da última alteração	Data e hora da última modificação do arquivo
Tamanho atual	Número de bytes no arquivo
Tamanho máximo	Número máximo de bytes no arquivo

de antigos computadores de grande porte exigiam que o tamanho máximo fosse especificado quando o arquivo fosse criado, a fim de deixar que o sistema operacional reservasse a quantidade máxima de memória antecipadamente. Sistemas operacionais de computadores pessoais e de estações de trabalho são inteligentes o suficiente para não precisarem desse atributo.

4.1.6 Operações com arquivos

Arquivos existem para armazenar informações e permitir que elas sejam recuperadas depois. Sistemas diferentes proporcionam operações diferentes para permitir armazenamento e recuperação. A seguir uma discussão das chamadas de sistema mais comuns relativas a arquivos.

1. **Create.** O arquivo é criado sem dados. A finalidade dessa chamada é anunciar que o arquivo está vindo e estabelecer alguns dos atributos.
2. **Delete.** Quando o arquivo não é mais necessário, ele tem de ser removido para liberar espaço para o disco. Há sempre uma chamada de sistema para essa finalidade.
3. **Open.** Antes de usar um arquivo, um processo precisa abri-lo. A finalidade da chamada `open` é permitir que o sistema busque os atributos e lista de endereços do disco para a memória principal a fim de tornar mais rápido o acesso em chamadas posteriores.
4. **Close.** Quando todos os acessos são concluídos, os atributos e endereços de disco não são mais necessários, então o arquivo deve ser fechado para liberar espaço da tabela interna. Muitos sistemas encorajam isso impondo um número máximo de arquivos abertos em processos. Um disco é escrito em blocos, e o fechamento de um arquivo força a escrita do último bloco dele, mesmo que não esteja inteiramente cheio ainda.
5. **Read.** Dados são lidos do arquivo. Em geral, os bytes vêm da posição atual. Quem fez a chamada deve especificar a quantidade de dados necessária e também fornecer um buffer para colocá-los.
6. **Write.** Dados são escritos para o arquivo de novo, normalmente na posição atual. Se a posição atual for o final do arquivo, seu tamanho aumentará. Se estiver no meio do arquivo, os dados existentes serão sobrescritos e perdidos para sempre.

7. **Append.** Essa chamada é uma forma restrita de `write`. Ela pode acrescentar dados somente para o final do arquivo. Sistemas que fornecem um conjunto mínimo de chamadas do sistema raramente têm `append`, mas muitos sistemas fornecem múltiplas maneiras de fazer a mesma coisa, e esses às vezes têm `append`.
8. **Seek.** Para arquivos de acesso aleatório, é necessário um método para especificar de onde tirar os dados. Uma abordagem comum é uma chamada de sistema, `seek`, que reposiciona o ponteiro de arquivo para um local específico dele. Após essa chamada ter sido completa, os dados podem ser lidos da, ou escritos para, aquela posição.
9. **Get attributes.** Processos muitas vezes precisam ler atributos de arquivos para realizar seu trabalho. Por exemplo, o programa `make` da UNIX costuma ser usado para gerenciar projetos de desenvolvimento de software consistindo de muitos arquivos-fonte. Quando `make` é chamado, ele examina os momentos de alteração de todos os arquivos-fonte e objetos e organiza o número mínimo de compilações necessárias para atualizar tudo. Para realizar o trabalho, o `make` deve examinar os atributos, a saber, os momentos de alteração.
10. **Set attributes.** Alguns dos atributos podem ser alterados pelo usuário e modificados após o arquivo ter sido criado. Essa chamada de sistema torna isso possível. A informação sobre o modo de proteção é um exemplo óbvio. A maioria das sinalizações também cai nessa categoria.
11. **Rename.** Acontece com frequência de um usuário precisar mudar o nome de um arquivo. Essa chamada de sistema torna isso possível. Ela nem sempre é estritamente necessária, porque o arquivo em geral pode ser copiado para um outro com um nome novo, e o arquivo antigo é então deletado.

4.1.7 Exemplo de um programa usando chamadas de sistema para arquivos

Nesta seção examinaremos um programa UNIX simples que copia um arquivo do seu arquivo-fonte para um de destino. Ele está listado na Figura 4.5. O programa tem uma funcionalidade mínima e um mecanismo para reportar erros ainda pior, mas proporciona uma ideia razoável de como algumas das chamadas de sistema relacionadas a arquivos funcionam.

FIGURA 4.5 Um programa simples para copiar um arquivo.

```

/* Programa que copia arquivos. Verificacao e relato de erros e minimo.*/

#include <sys/types.h> /* inclui os arquivos de cabecalho necessarios */
#include <fcntl.h>
#include <stdlib.h>
#include <unistd.h>

int main(int argc, char *argv[]); /* prototipo ANS */

#define BUF_SIZE 4096 /* usa um tamanho de buffer de 4096 bytes */
#define OUTPUT_MODE 0700 /* bits de protecao para o arquivo de saida */

int main(int argc, char *argv[])
{
    int in_fd, out_fd, rd_count, wt_count;
    char buffer[BUF_SIZE];

    if (argc != 3) exit(1); /* erro de sintaxe se argc nao for 3 */

    /* Abre o arquivo de entrada e cria o arquivo de saida */
    in_fd = open(argv[1], O_RDONLY); /* abre o arquivo de origem */
    if (in_fd < 0) exit(2); /* se nao puder ser aberto, saia */
    out_fd = creat(argv[2], OUTPUT_MODE); /* cria o arquivo de destino */
    if (out_fd < 0) exit(3); /* se nao puder ser criado, saia */

    /* Laco de copia */
    while (TRUE) {
        rd_count = read(in_fd, buffer, BUF_SIZE); /* le um bloco de dados */
        if (rd_count <= 0) break /* se fim de arquivo ou erro, sai do laco */
        wt_count = write(out_fd, buffer, rd_count); /* escreve dados */
        if (wt_count <= 0) exit(4); /* wt_count <= 0 e um erro */
    }

    /* Fecha os arquivos */
    close(in_fd);
    close(out_fd);
    if (rd_count == 0) /* nenhum erro na ultima leitura */
        exit(0);
    else
        exit(5); /* erro na ultima leitura */
}

```

O programa, *copyfile*, pode ser chamado, por exemplo, pela linha de comando

```
copyfile abc xyz
```

para copiar o arquivo *abc* para *xyz*. Se *xyz* já existir, ele será sobrescrito. De outra maneira, ele será criado. O programa precisa ser chamado com exatamente dois argumentos, ambos nomes legais de arquivos. O primeiro é o fonte; o segundo é o arquivo de saída.

Os quatro comandos *#include* próximos do início do programa fazem que um grande número de definições e protótipos de funções sejam incluídos no programa. Essas inclusões são necessárias para deixá-lo em conformidade com os padrões internacionais relevantes,

mas não nos ocuparemos mais com elas. A linha seguinte é um protótipo da função para *main*, algo exigido pelo ANSI C, mas também não relevante para nossas finalidades.

O primeiro comando *#define* é uma definição macro, que estabelece a sequência de caracteres *BUF_SIZE* como uma macro que se expande no número 4096. O programa lerá e escreverá em pedaços de 4096 bytes. É considerada uma boa prática de programação dar nomes a constantes como essa e usá-los em vez das constantes. Não apenas essa convenção torna os programas mais fáceis de ler, mas também de manter. O segundo comando *#define* determina quem pode acessar o arquivo de saída.

O programa principal é chamado *main* e tem dois argumentos: *argc* e *argv*. Estes são oferecidos pelo sistema operacional quando o programa é chamado. O primeiro diz quantas sequências estão presentes na linha de comando que invocou o programa, incluindo o nome dele. Deveriam ser 3. O segundo é um arranjo de ponteiros para os argumentos. Na chamada de exemplo dada, os elementos desse arranjo conteriam ponteiros para os seguintes valores:

```
argv[0] = "copyfile"
```

```
argv[1] = "abc"
```

```
argv[2] = "xyz"
```

É por meio desse arranjo que o programa acessa os seus argumentos.

Cinco variáveis são declaradas. As duas primeiras, *in_fd* e *out_fd*, conterão os **descritores de arquivos**, valores inteiros pequenos retornados quando um arquivo é aberto. As outras duas, *rd_count* e *wt_count*, são as contagens de bytes retornadas pelas chamadas de sistema *read* e *write*, respectivamente. A última, *buffer*, é um buffer usado para conter os dados lidos e fornecer os dados para serem escritos.

O primeiro comando real confere *argc* para ver se ele é 3. Se não for, o programa terminará com um código de estado 1. Qualquer código de estado diferente de 0 significa que ocorreu um erro. O código de estado é o único meio de reportar erros presente nesse programa. Uma versão comercial normalmente imprimiria mensagens de erros também.

Então tentamos abrir o arquivo-fonte e criar o arquivo-destino. Se o arquivo-fonte for aberto de maneira bem-sucedida, o sistema designa um pequeno inteiro para *in_fd*, a fim de identificá-lo. Chamadas subsequentes devem incluir esse inteiro de maneira que o sistema saiba qual arquivo ele quer. Similarmente, se o destino for criado de maneira bem-sucedida, *out_fd* recebe um valor para identificá-lo. O segundo argumento para *creat* estabelece o modo de proteção. Se a abertura ou a criação falhar, o descritor do arquivo correspondente será definido como -1, e o programa terminará com um código de erro.

Agora entra em cena o laço da cópia. Esse laço começa tentando ler 4 KB de dados para o *buffer*. Ele faz isso chamando a rotina de biblioteca *read*, que na realidade invoca a chamada de sistema *read*. O primeiro parâmetro identifica o arquivo, o segundo dá o buffer e o terceiro diz quantos bytes devem ser lidos. O valor designado para *rd_count* dá o número de bytes que foram realmente lidos. Em geral, esse valor será de 4096, exceto se menos bytes estiverem restando no arquivo. Quando o final do

arquivo tiver sido alcançado, ele será 0. Se o *rd_count* chegar a 0 ou um valor negativo, a cópia não poderá continuar, de maneira que o comando *break* é executado para terminar o laço (de outra maneira interminável).

A chamada *write* descarrega o buffer para o arquivo de destino. O primeiro parâmetro identifica o arquivo, o segundo dá o buffer e o terceiro diz quantos bytes escrever, análogo a *read*. Observe que a contagem de bytes é o número de bytes realmente lidos, não *BUF_SIZE*. Esse ponto é importante porque o último *read* não retornará 4096 a não ser que o arquivo coincidentemente seja um múltiplo de 4 KB.

Quando o arquivo inteiro tiver sido processado, a primeira chamada além do fim do arquivo retornará a 0 para *rd_count*, que a fará deixar o laço. Nesse ponto, os dois arquivos estão próximos e o programa sai com um estado indicando uma conclusão normal.

Embora chamadas do sistema Windows sejam diferentes daquelas do UNIX, a estrutura geral de um programa ativado pela linha de comando no Windows para copiar um arquivo é moderadamente similar àquele da Figura 4.5. Examinaremos as chamadas do Windows 8 no Capítulo 11.

4.2 Diretórios

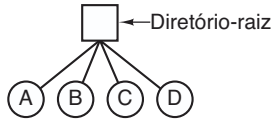
Para controlar os arquivos, sistemas de arquivos normalmente têm **diretórios** ou **pastas**, que são em si arquivos. Nesta seção discutiremos diretórios, sua organização, suas propriedades e as operações que podem ser realizadas por eles.

4.2.1 Sistemas de diretório em nível único

A forma mais simples de um sistema de diretório é ter um diretório contendo todos os arquivos. Às vezes ele é chamado de **diretório-raiz**, mas como ele é o único, o nome não importa muito. Nos primeiros computadores pessoais, esse sistema era comum, em parte porque havia apenas um usuário. Curiosamente, o primeiro supercomputador do mundo, o CDC 6600, também tinha apenas um único diretório para todos os arquivos, embora fosse usado por muitos usuários ao mesmo tempo. Essa decisão foi tomada sem dúvida para manter simples o design do software.

Um exemplo de um sistema com um diretório é dado na Figura 4.6. Aqui o diretório contém quatro arquivos. As vantagens desse esquema são a sua simplicidade e a capacidade de localizar arquivos rapidamente — há apenas um lugar para se procurar, afinal. Às vezes ele ainda

FIGURA 4.6 Um sistema de diretório em nível único contendo quatro arquivos.



é usado em dispositivos embarcados simples como câmeras digitais e alguns players portáteis de música.

4.2.2 Sistemas de diretórios hierárquicos

O nível único é adequado para aplicações dedicadas muito simples (e chegou a ser usado nos primeiros computadores pessoais), mas para os usuários modernos com milhares de arquivos seria impossível encontrar qualquer coisa se todos os arquivos estivessem em um único diretório.

Em consequência, é necessária uma maneira para agrupar arquivos relacionados em um mesmo local. Um professor, por exemplo, pode ter uma coleção de arquivos que juntos formam um livro que ele está escrevendo, uma segunda coleção contendo programas apresentados por estudantes para outro curso, um terceiro grupo contendo o código de um sistema de escrita de compiladores avançado que ele está desenvolvendo, um quarto grupo contendo propostas de doações, assim como outros arquivos para correio eletrônico, minutas de reuniões, estudos que ele está escrevendo, jogos e assim por diante.

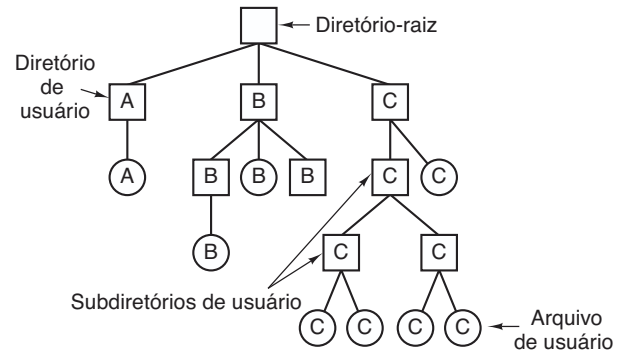
Faz-se necessária uma hierarquia (isto é, uma árvore de diretórios). Com essa abordagem, o usuário pode ter tantos diretórios quantos forem necessários para agrupar seus arquivos de maneira natural. Além disso, se múltiplos usuários compartilham um servidor de arquivos comum, como é o caso em muitas redes de empresas, cada usuário pode ter um diretório-raiz privado para sua própria hierarquia. Essa abordagem é mostrada na Figura 4.7. Aqui, cada diretório A, B e C contido no diretório-raiz pertence a um usuário diferente, e dois deles criaram subdiretórios para projetos nos quais estão trabalhando.

A capacidade dos usuários de criarem um número arbitrário de subdiretórios proporciona uma ferramenta de estruturação poderosa para eles organizarem o seu trabalho. Por essa razão, quase todos os sistemas de arquivos modernos são organizados dessa maneira.

4.2.3 Nomes de caminhos

Quando o sistema de arquivos é organizado com uma árvore de diretórios, alguma maneira é

FIGURA 4.7 Um sistema hierárquico de diretórios.



necessária para especificar os nomes dos arquivos. Dois métodos diferentes são os mais usados. No primeiro, cada arquivo recebe um **nome de caminho absoluto** consistindo no caminho do diretório-raiz para o arquivo. Como exemplo, o caminho `/usr/ast/caixapostal` significa que o diretório-raiz contém um subdiretório `usr`, que por sua vez contém um subdiretório `ast`, que contém o arquivo `caixapostal`. Nomes de caminhos absolutos sempre começam no diretório-raiz e são únicos. No UNIX, os componentes do caminho são separados por `/`. No Windows o separador é `\`. No MULTICS era `>`. Desse modo, o mesmo nome de caminho seria escrito como a seguir nesses três sistemas:

Windows	<code>\usr\ast\caixapostal</code>
UNIX	<code>/usr/ast/caixapostal</code>
MULTICS	<code>>usr>ast>caixapostal</code>

Não importa qual caractere é usado, se o primeiro caractere do nome do caminho for o separador, então o caminho será absoluto.

O outro tipo é o **nome de caminho relativo**. Esse é usado em conjunção com o conceito do **diretório de trabalho** (também chamado de **diretório atual**). Um usuário pode designar um diretório como o de trabalho atual, caso em que todos os nomes de caminho não começando no diretório-raiz são presumidos como relativos ao diretório de trabalho. Por exemplo, se o diretório de trabalho atual é `/usr/ast`, então o arquivo cujo caminho absoluto é `/usr/ast/caixapostal` pode ser referenciado somente como `caixa postal`. Em outras palavras, o comando UNIX

```
cp /usr/ast/caixapostal /usr/ast/caixapostal.bak
```

e o comando

```
cp caixapostal caixapostal.bak
```


O segundo argumento (ponto) refere-se ao diretório atual. Quando o comando *cp* recebe um nome de diretório (incluindo ponto) como seu último argumento, ele copia todos os arquivos para aquele diretório. É claro, uma maneira mais natural de realizar a cópia seria usar o nome de caminho absoluto completo do arquivo-fonte:

```
cp /usr/lib/dictionary .
```

Aqui o uso do ponto poupa o usuário do desperdício de tempo de digitar *dictionary* uma segunda vez. Mesmo assim, digitar

```
cp /usr/lib/dictionary dictionary
```

também funciona bem, assim como

```
cp /usr/lib/dictionary /usr/ast/dictionary
```

Todos esses comandos realizam exatamente a mesma coisa.

4.2.4 Operações com diretórios

As chamadas de sistema que podem gerenciar diretórios exibem mais variação de sistema para sistema do que as chamadas para gerenciar arquivos. Para dar uma impressão do que elas são e como funcionam, daremos uma amostra (tirada do UNIX).

1. **Create.** Um diretório é criado. Ele está vazio exceto por ponto e pontoponto, que são colocados ali automaticamente pelo sistema (ou em alguns poucos casos, pelo programa *mkdir*).
2. **Delete.** Um diretório é removido. Apenas um diretório vazio pode ser removido. Um diretório contendo apenas ponto e pontoponto é considerado vazio à medida que eles não podem ser removidos.
3. **Opendir.** Diretórios podem ser lidos. Por exemplo, para listar todos os arquivos em um diretório, um programa de listagem abre o diretório para ler os nomes de todos os arquivos que ele contém. Antes que um diretório possa ser lido, ele deve ser aberto, de maneira análoga a abrir e ler um arquivo.
4. **Closedir.** Quando um diretório tiver sido lido, ele será fechado para liberar espaço de tabela interno.
5. **Readdir.** Essa chamada retorna a próxima entrada em um diretório aberto. Antes, era possível ler diretórios usando a chamada de sistema *read* usual, mas essa abordagem tem a desvantagem de forçar o programador a saber e lidar com a estrutura interna de diretórios. Por outro lado, *readdir* sempre retorna uma entrada em um formato padrão,

não importa qual das estruturas de diretório possíveis está sendo usada.

6. **Rename.** Em muitos aspectos, diretórios são como arquivos e podem ser renomeados da mesma maneira que eles.
7. **Link.** A ligação (*linking*) é uma técnica que permite que um arquivo apareça em mais de um diretório. Essa chamada de sistema especifica um arquivo existente e um nome de caminho, e cria uma ligação do arquivo existente para o nome especificado pelo caminho. Dessa maneira, o mesmo arquivo pode aparecer em múltiplos diretórios. Uma ligação desse tipo, que incrementa o contador no i-node do arquivo (para monitorar o número de entradas de diretório contendo o arquivo), às vezes é chamada de **ligação estrita** (*hard link*).
8. **Unlink.** Uma entrada de diretório é removida. Se o arquivo sendo removido estiver presente somente em um diretório (o caso normal), ele é removido do sistema de arquivos. Se ele estiver presente em múltiplos diretórios, apenas o nome do caminho especificado é removido. Os outros continuam. Em UNIX, a chamada de sistema para remover arquivos (discutida anteriormente) é, na realidade, *unlink*.

A lista anterior mostra as chamadas mais importantes, mas há algumas outras também, por exemplo, para gerenciar a informação de proteção associada com um diretório.

Uma variação da ideia da ligação de arquivos é a **ligação simbólica**. Em vez de ter dois nomes apontando para a mesma estrutura de dados interna representando um arquivo, um nome pode ser criado que aponte para um arquivo minúsculo que nomeia outro arquivo. Quando o primeiro é usado — aberto, por exemplo — o sistema de arquivos segue o caminho e encontra o nome no fim. Então ele começa todo o processo de localização usando o novo nome. Ligações simbólicas têm a vantagem de conseguirem atravessar as fronteiras de discos e mesmo nomear arquivos em computadores remotos. No entanto, sua implementação é de certa maneira menos eficiente do que as ligações estritas.

4.3 Implementação do sistema de arquivos

Agora chegou o momento de passar da visão do usuário do sistema de arquivos para a do implementador. Usuários estão preocupados em como os arquivos são

nomeados, quais operações são permitidas neles, como é a árvore de diretórios e questões de interface similares. Implementadores estão interessados em como os arquivos e os diretórios estão armazenados, como o espaço de disco é gerenciado e como fazer tudo funcionar de maneira eficiente e confiável. Nas seções a seguir examinaremos uma série dessas áreas para ver quais são as questões e compromissos envolvidos.

4.3.1 Esquema do sistema de arquivos

Sistemas de arquivos são armazenados em discos. A maioria dos discos pode ser dividida em uma ou mais partições, com sistemas de arquivos independentes em cada partição. O Setor 0 do disco é chamado de **MBR (Master Boot Record)** — registro mestre de inicialização) e é usado para inicializar o computador. O fim do MBR contém a tabela de partição. Ela dá os endereços de início e fim de cada partição. Uma das partições da tabela é marcada como ativa. Quando o computador é inicializado, a BIOS lê e executa o MBR. A primeira coisa que o programa MBR faz é localizar a partição ativa, ler seu primeiro bloco, que é chamado de **bloco de inicialização**, e executá-lo. O programa no bloco de inicialização carrega o sistema operacional contido naquela partição. Por uniformidade, cada partição começa com um bloco de inicialização, mesmo que ela não contenha um sistema operacional que possa ser inicializado. Além disso, a partição poderá conter um no futuro.

Fora iniciar com um bloco de inicialização, o esquema de uma partição de disco varia bastante entre sistemas de arquivos. Muitas vezes o sistema de arquivos vai conter alguns dos itens mostrados na Figura 4.9. O primeiro é o **superbloco**. Ele contém todos os parâmetros-chave a respeito do sistema de arquivos e é lido para a memória quando o computador é inicializado ou o sistema de arquivos é tocado pela primeira vez. Informações típicas no superbloco incluem um número

mágico para identificar o tipo de sistema de arquivos, seu número de blocos e outras informações administrativas fundamentais.

Em seguida podem vir informações a respeito de blocos disponíveis no sistema de arquivos, na forma de um mapa de bits ou de uma lista de ponteiros, por exemplo. Isso pode ser seguido pelos i-nodes, um arranjo de estruturas de dados, um por arquivo, dizendo tudo sobre ele. Depois pode vir o diretório-raiz, que contém o topo da árvore do sistema de arquivos. Por fim, o restante do disco contém todos os outros diretórios e arquivos.

4.3.2 Implementando arquivos

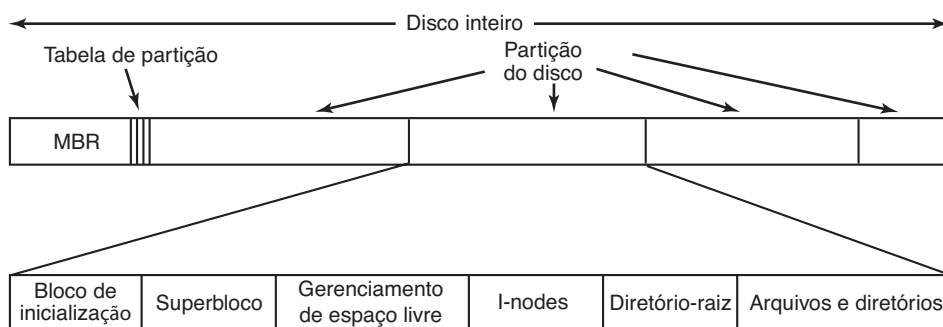
É provável que a questão mais importante na implementação do armazenamento de arquivos seja controlar quais blocos de disco vão com quais arquivos. Vários métodos são usados em diferentes sistemas operacionais. Nesta seção, examinaremos alguns deles.

Alocação contígua

O esquema de alocação mais simples é armazenar cada arquivo como uma execução contígua de blocos de disco. Assim, em um disco com blocos de 1 KB, um arquivo de 50 KB seria alocado em 50 blocos consecutivos. Com blocos de 2 KB, ele seria alocado em 25 blocos consecutivos.

Vemos um exemplo de alocação em armazenamento contíguo na Figura 4.10(a). Aqui os primeiros 40 blocos de disco são mostrados, começando com o bloco 0 à esquerda. De início, o disco estava vazio. Então um arquivo *A*, de quatro blocos de comprimento, foi escrito a partir do início (bloco 0). Após isso, um arquivo de seis blocos, *B*, foi escrito começando logo depois do fim do arquivo *A*.

FIGURA 4.9 Um esquema possível para um sistema de arquivos.



Observe que cada arquivo começa no início de um bloco novo; portanto, se o arquivo *A* realmente ocupar $3\frac{1}{2}$ blocos, algum espaço será desperdiçado ao fim de cada último bloco. Na figura, um total de sete arquivos é mostrado, cada um começando no bloco seguinte ao final do anterior. O sombreadimento é usado apenas para tornar mais fácil a distinção entre os blocos. Não tem significado real em termos de armazenamento.

A alocação de espaço de disco contíguo tem duas vantagens significativas. Primeiro, ela é simples de implementar porque basta se lembrar de dois números para monitorar onde estão os blocos de um arquivo: o endereço em disco do primeiro bloco e o número de blocos no arquivo. Dado o número do primeiro bloco, o número de qualquer outro bloco pode ser encontrado mediante uma simples adição.

Segundo, o desempenho da leitura é excelente, pois o arquivo inteiro pode ser lido do disco em uma única operação. Apenas uma busca é necessária (para o primeiro bloco). Depois, não são mais necessárias buscas ou atrasos rotacionais, então os dados são lidos com a capacidade total do disco. Portanto, a alocação contígua é simples de implementar e tem um alto desempenho.

Infelizmente, a alocação contígua tem um ponto fraco importante: com o tempo, o disco torna-se fragmentado. Para ver como isso acontece, examine a Figura 4.10(b). Aqui dois arquivos, *D* e *F*, foram removidos. Quando um arquivo é removido, seus blocos são naturalmente liberados, deixando uma lacuna de blocos livres no disco. O disco não é compactado imediatamente para eliminá-la, já que isso envolveria copiar todos os blocos seguindo essa lacuna, potencialmente milhões de blocos, o que levaria horas ou mesmo dias

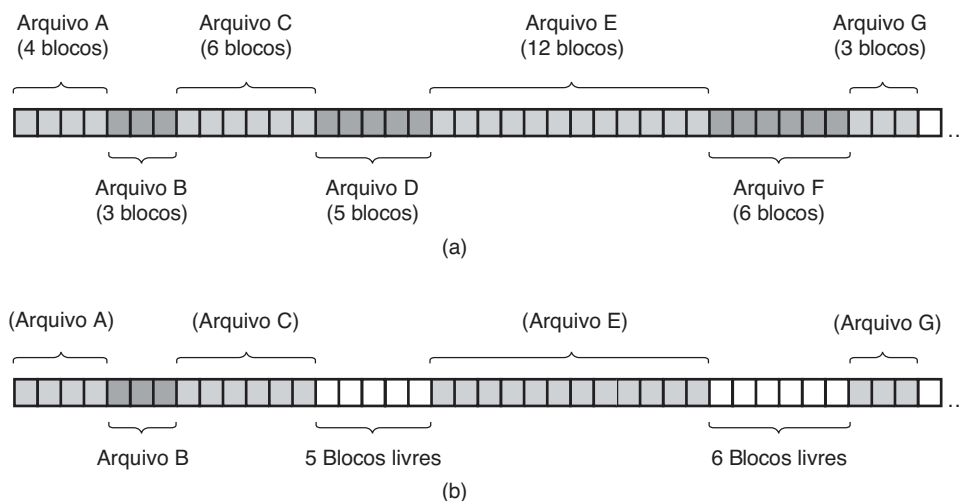
em discos grandes. Como resultado, em última análise o disco consiste em arquivos e lacunas, como ilustrado na figura.

De início, essa fragmentação não é problema, já que cada novo arquivo pode ser escrito ao final do disco, seguindo o anterior. No entanto, finalmente o disco estará cheio e será necessário compactá-lo, o que tem custo proibitivo, ou reutilizar os espaços livres nas lacunas. Reutilizar o espaço exige manter uma lista de lacunas, o que é possível. No entanto, quando um arquivo novo vai ser criado, é necessário saber o seu tamanho final a fim de escolher uma lacuna do tamanho correto para alocá-lo.

Imagine as consequências de um projeto desses. O usuário inicializa um processador de texto a fim de criar um documento. A primeira coisa que o programa pergunta é quantos bytes o documento final terá. A pergunta deve ser respondida ou o programa não continuará. Se o número em última análise provar-se pequeno demais, o programa precisará ser terminado prematuramente, pois a lacuna do disco estará cheia e não haverá lugar para colocar o resto do arquivo. Se o usuário tentar evitar esse problema dando um número irrealisticamente grande como o tamanho final, digamos, 1 GB, o editor talvez não consiga encontrar uma lacuna tão grande e anunciará que o arquivo não pode ser criado. É claro, o usuário estaria livre para inicializar o programa novamente e dizer 500 MB dessa vez, e assim por diante até que uma lacuna adequada fosse localizada. Ainda assim, é pouco provável que esse esquema deixe os usuários felizes.

No entanto, há uma situação na qual a alocação contígua é possível e, na realidade, ainda usada: em CD-ROMs. Aqui todos os tamanhos de arquivos são

FIGURA 4.10 (a) Alocação contígua de espaço de disco para sete arquivos. (b) O estado do disco após os arquivos *D* e *F* terem sido removidos.



conhecidos antecipadamente e jamais mudarão durante o uso subsequente do sistema de arquivos do CD-ROM.

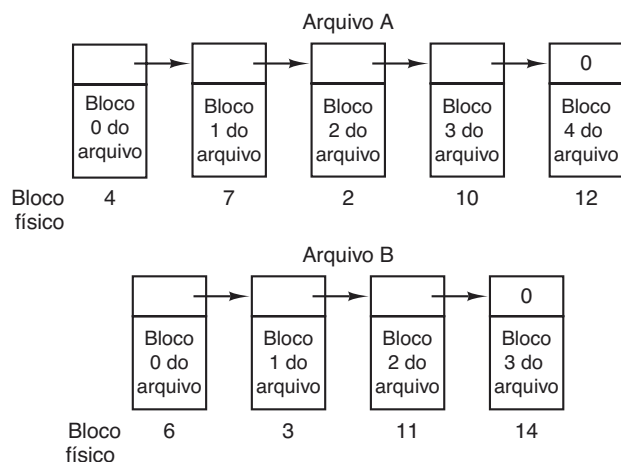
A situação com DVDs é um pouco mais complicada. Em princípio, um filme de 90 minutos poderia ser codificado como um único arquivo de comprimento de cerca de 4,5 GB, mas o sistema de arquivos utilizado, **UDF (Universal Disk Format** — formato universal de disco), usa um número de 30 bits para representar o tamanho do arquivo, o que limita os arquivos a 1 GB. Em consequência, filmes em DVD são em geral armazenados contiguamente como três ou quatro arquivos de 1 GB. Esses pedaços físicos do único arquivo lógico (o filme) são chamados de **extensões**.

Como mencionamos no Capítulo 1, a história muitas vezes se repete na ciência de computadores à medida que surgem novas gerações de tecnologia. A alocação contígua na realidade foi usada nos sistemas de arquivos de discos magnéticos anos atrás pela simplicidade e alto desempenho (a facilidade de uso para o usuário não contava muito à época). Então a ideia foi abandonada por causa do incômodo de ter de especificar o tamanho final do arquivo no momento de sua criação. Mas com o advento dos CD-ROMs, DVDs, Blu-rays e outras mídias óticas para escrita única, subitamente arquivos contíguos eram uma boa ideia de novo. Desse modo, é importante estudar sistemas e ideias antigas que eram conceitualmente limpas e simples, pois elas podem ser aplicáveis a sistemas futuros de maneiras surpreendentes.

Alocação por lista encadeada

O segundo método para armazenar arquivos é manter cada um como uma lista encadeada de blocos de disco, como mostrado na Figura 4.11. A primeira palavra

FIGURA 4.11 Armazenando um arquivo como uma lista encadeada de blocos de disco.



de cada bloco é usada como um ponteiro para a próxima. O resto do bloco é reservado para dados.

Diferentemente da alocação contígua, todos os blocos do disco podem ser usados nesse método. Nenhum espaço é perdido para a fragmentação de disco (exceto para a fragmentação interna no último bloco). Também, para a entrada de diretório é suficiente armazenar meramente o endereço em disco do primeiro bloco. O resto pode ser encontrado a partir daí.

Por outro lado, embora a leitura de um arquivo sequencialmente seja algo direto, o acesso aleatório é de extrema lentidão. Para chegar ao bloco n , o sistema operacional precisa começar do início e ler os blocos $n - 1$ antes dele, um de cada vez. É claro que realizar tantas leituras será algo dolorosamente lento.

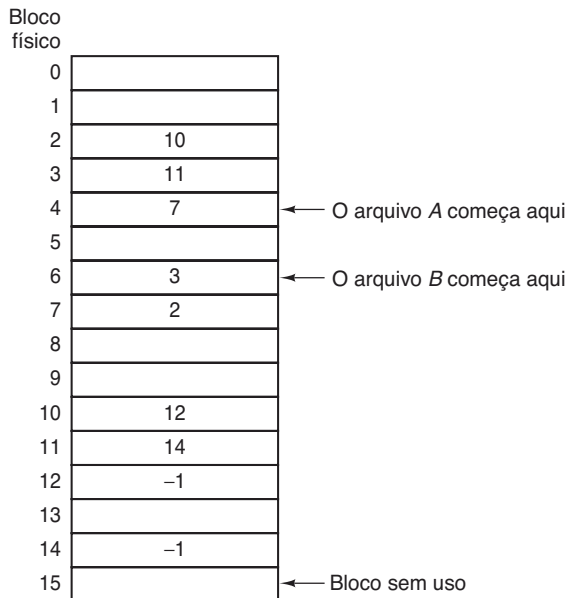
Também, a quantidade de dados que um bloco pode armazenar não é mais uma potência de dois, pois os ponteiros ocupam alguns bytes do bloco. Embora não seja fatal, ter um tamanho peculiar é menos eficiente, pois muitos programas leem e escrevem em blocos cujo tamanho é uma potência de dois. Com os primeiros bytes de cada bloco ocupados por um ponteiro para o próximo bloco, a leitura de todo o bloco exige que se adquira e concatene a informação de dois blocos de disco, o que gera uma sobrecarga extra por causa da cópia.

Alocação por lista encadeada usando uma tabela na memória

Ambas as desvantagens da alocação por lista encadeada podem ser eliminadas colocando-se as palavras do ponteiro de cada bloco de disco em uma tabela na memória. A Figura 4.12 mostra como são as tabelas para o exemplo da Figura 4.11. Em ambas, temos dois arquivos. O arquivo *A* usa os blocos de disco 4, 7, 2, 10 e 12, nessa ordem, e o arquivo *B* usa os blocos de disco 6, 3, 11 e 14, nessa ordem. Usando a tabela da Figura 4.12, podemos começar com o bloco 4 e seguir a cadeia até o fim. O mesmo pode ser feito começando com o bloco 6. Ambos os encadeamentos são concluídos com uma marca especial (por exemplo, -1) que corresponde a um número de bloco inválido. Essa tabela na memória principal é chamada de **FAT (File Allocation Table** — tabela de alocação de arquivos).

Usando essa organização, o bloco inteiro fica disponível para dados. Além disso, o acesso aleatório é muito mais fácil. Embora ainda seja necessário seguir o encadeamento para encontrar um determinado deslocamento dentro do arquivo, o encadeamento está inteiramente na memória, portanto ele pode ser seguido sem fazer quaisquer referências ao disco. Da mesma maneira que no

FIGURA 4.12 Alocação por lista encadeada usando uma tabela de alocação de arquivos na memória principal.



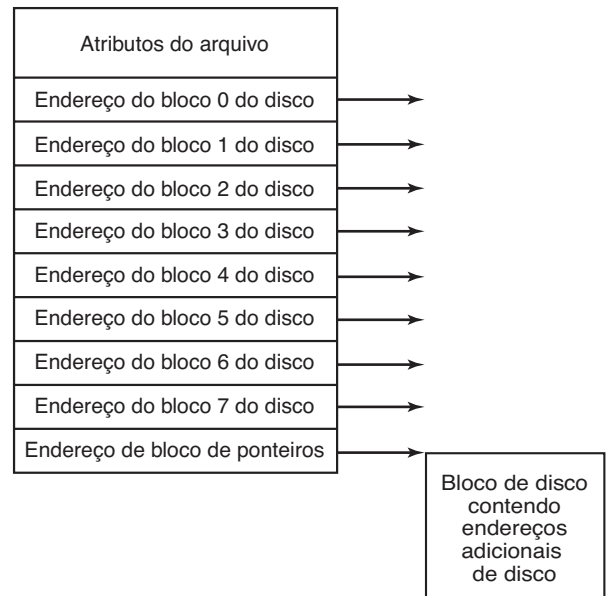
método anterior, é suficiente para a entrada de diretório manter um único inteiro (o número do bloco inicial) e ainda assim ser capaz de localizar todos os blocos, não importa o tamanho do arquivo.

A principal desvantagem desse método é que a tabela inteira precisa estar na memória o todo o tempo para fazê-la funcionar. Com um disco de 1 TB e um tamanho de bloco de 1 KB, a tabela precisa de 1 bilhão de entradas, uma para cada um dos 1 bilhão de blocos de disco. Cada entrada precisa ter no mínimo 3 bytes. Para aumentar a velocidade de consulta, elas deveriam ter 4 bytes. Desse modo, a tabela ocupará 3 GB ou 2,4 GB da memória principal o tempo inteiro, dependendo de o sistema estar otimizado para espaço ou tempo. Não é algo muito prático. Claro, a ideia da FAT não se adapta bem para discos grandes. Era o sistema de arquivos MS-DOS original e ainda é aceito completamente por todas as versões do Windows.

I-nodes

Nosso último método para monitorar quais blocos pertencem a quais arquivos é associar cada arquivo a uma estrutura de dados chamada de **i-node** (**index-node** — nó-índice), que lista os atributos e os endereços de disco dos blocos do disco. Um exemplo simples é descrito na Figura 4.13. Dado o i-node, é então possível encontrar todos os blocos do arquivo. A grande vantagem desse esquema sobre os arquivos encadeados usando

FIGURA 4.13 Um exemplo de i-node.



uma tabela na memória é que o i-node precisa estar na memória apenas quando o arquivo correspondente estiver aberto. Se cada i-node ocupa n bytes e um máximo de k arquivos puderem estar abertos simultaneamente, a memória total ocupada pelo arranjo contendo os i-nodes para os arquivos abertos é de apenas kn bytes. Apenas essa quantidade de espaço precisa ser reservada antecipadamente.

Esse arranjo é em geral muito menor do que o espaço ocupado pela tabela de arquivos descrita na seção anterior. A razão é simples. A tabela para conter a lista encadeada de todos os blocos de disco é proporcional em tamanho ao disco em si. Se o disco tem n blocos, a tabela precisa de n entradas. À medida que os discos ficam maiores, essa tabela cresce linearmente com eles. Por outro lado, o esquema i-node exige um conjunto na memória cujo tamanho seja proporcional ao número máximo de arquivos que podem ser abertos ao mesmo tempo. Não importa que o disco tenha 100 GB, 1.000 GB ou 10.000 GB.

Um problema com i-nodes é que se cada um tem espaço para um número fixo de endereços de disco, o que acontece quando um arquivo cresce além de seu limite? Uma solução é reservar o último endereço de disco não para um bloco de dados, mas, em vez disso, para o endereço de um bloco contendo mais endereços de blocos de disco, como mostrado na Figura 4.13. Mais avançado ainda seria ter dois ou mais desses blocos contendo endereços de disco ou até blocos de disco apontando para outros blocos cheios de endereços. Voltaremos aos i-nodes quando estudarmos o UNIX no Capítulo 10.

De modo similar, o sistema de arquivos NTFS do Windows usa uma ideia semelhante, apenas com i-nodes maiores, que também podem conter arquivos pequenos.

4.3.3 Implementando diretórios

Antes que um arquivo possa ser lido, ele precisa ser aberto. Quando um arquivo é aberto, o sistema operacional usa o nome do caminho fornecido pelo usuário para localizar a entrada de diretório no disco. A entrada de diretório fornece a informação necessária para encontrar os blocos de disco. Dependendo do sistema, essa informação pode ser o endereço de disco do arquivo inteiro (com alocação contígua), o número do primeiro bloco (para ambos os esquemas de listas encadeadas), ou o número do i-node. Em todos os casos, a principal função do sistema de diretórios é mapear o nome do arquivo em ASCII na informação necessária para localizar os dados.

Uma questão relacionada de perto refere-se a onde os atributos devem ser armazenados. Todo sistema de arquivos mantém vários atributos do arquivo, como o proprietário de cada um e seu momento de criação, e eles devem ser armazenados em algum lugar. Uma possibilidade óbvia é fazê-lo diretamente na entrada do diretório. Alguns sistemas fazem precisamente isso. Essa opção é mostrada na Figura 4.14(a). Nesse design simples, um diretório consiste em uma lista de entradas de tamanho fixo, um por arquivo, contendo um nome de arquivo (de tamanho fixo), uma estrutura dos atributos do arquivo e um ou mais endereços de disco (até algum máximo) dizendo onde estão os blocos de disco.

Para sistemas que usam i-nodes, outra possibilidade para armazenar os atributos é nos próprios i-nodes, em vez de nas entradas do diretório. Nesse caso, a entrada do diretório pode ser mais curta: apenas um nome de arquivo e um número de i-node. Essa abordagem está

ilustrada na Figura 4.14(b). Como veremos mais tarde, esse método tem algumas vantagens sobre colocá-los na entrada do diretório.

Até o momento presumimos que os arquivos têm nomes curtos de tamanho fixo. No MS-DOS, os arquivos têm um nome base de 1-8 caracteres e uma extensão opcional de 1-3 caracteres. Na Versão 7 do UNIX, os nomes dos arquivos tinham 1-14 caracteres, incluindo quaisquer extensões. No entanto, quase todos os sistemas operacionais modernos aceitam nomes de arquivos maiores e de tamanho variável. Como eles podem ser implementados?

A abordagem mais simples é estabelecer um limite para o tamanho do nome dos arquivos e então usar um dos designs da Figura 4.14 com 255 caracteres reservados para cada nome de arquivo. Essa abordagem é simples, mas desperdiça muito espaço de diretório, já que poucos arquivos têm nomes tão longos. Por razões de eficiência, uma estrutura diferente é desejável.

Uma alternativa é abrir mão da ideia de que todas as entradas de diretório sejam do mesmo tamanho. Com esse método, cada entrada de diretório contém uma porção fixa, começando com o tamanho da entrada e, então, seguido por dados com um formato fixo, normalmente incluindo o proprietário, momento de criação, informações de proteção e outros atributos. Esse cabeçalho de comprimento fixo é seguido pelo nome do arquivo real, não importa seu tamanho, como mostrado na Figura 4.15(a) em um formato em que o byte mais significativo aparece primeiro (*big-endian*) — SPARC, por exemplo. Nesse exemplo, temos três arquivos, *project-budget*, *personnel* e *foo*. Cada nome de arquivo é concluído com um caractere especial (em geral 0), que é representado na figura por um quadrado com um “X” dentro. Para permitir que cada entrada de diretório comece junto ao limite de uma palavra, cada nome de arquivo é preenchido de modo a completar um número inteiro de palavras, indicado pelas caixas sombreadas na figura.

FIGURA 4.14 (a) Um diretório simples contendo entradas de tamanho fixo com endereços de disco e atributos na entrada do diretório. (b) Um diretório no qual cada entrada refere-se a apenas um i-node.

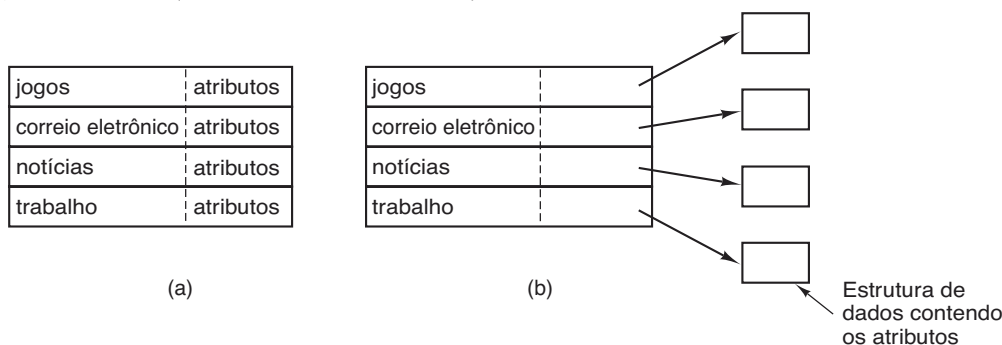
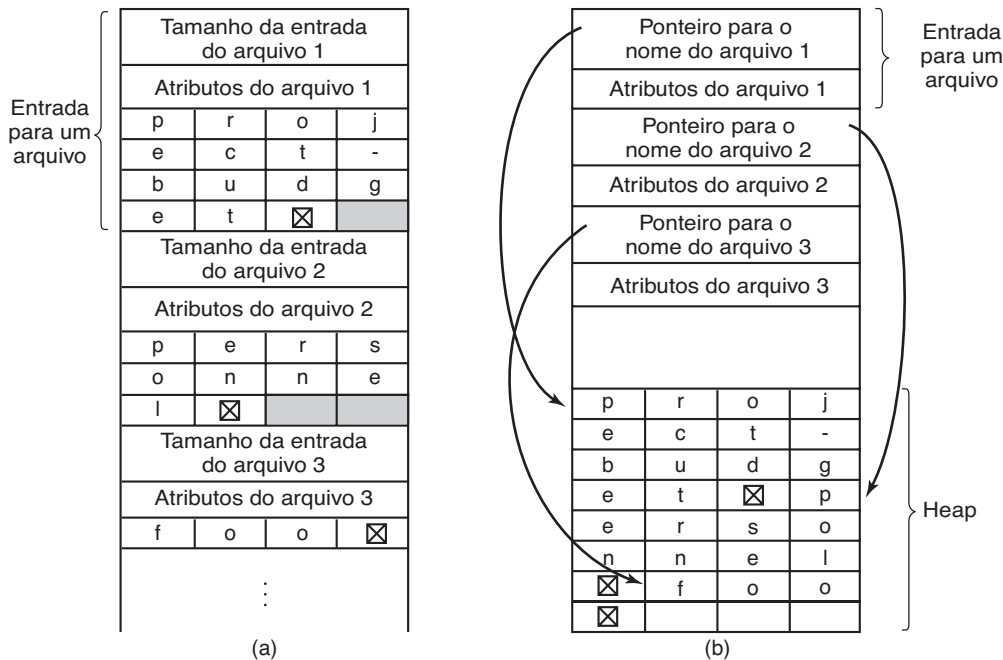


FIGURA 4.15 Duas maneiras de gerenciar nomes de arquivos longos em um diretório. (a) Sequencialmente. (b) No heap.

Uma desvantagem desse método é que, quando um arquivo é removido, uma lacuna de tamanho variável é introduzida no diretório e o próximo arquivo a entrar poderá não caber nela. Esse problema é na essência o mesmo que vimos com arquivos de disco contíguos, apenas agora é possível compactar o diretório, pois ele está inteiramente na memória. Outro problema é que uma única entrada de diretório pode se estender por múltiplas páginas, de maneira que uma falta de página pode ocorrer durante a leitura de um nome de arquivo.

Outra maneira de lidar com nomes de tamanhos variáveis é tornar fixos os tamanhos das próprias entradas de diretório e manter os nomes dos arquivos em um heap (monte) no fim de cada diretório, como mostrado na Figura 4.15(b). Esse método tem a vantagem de que, quando uma entrada for removida, o arquivo seguinte inserido sempre caberá ali. É claro, o heap deve ser gerenciado e faltas de páginas ainda podem ocorrer enquanto processando nomes de arquivos. Um ganho menor aqui é que não há mais nenhuma necessidade real para que os nomes dos arquivos comecem junto aos limites das palavras, de maneira que não é mais necessário completar os nomes dos arquivos com caracteres na Figura 4.15(b) como eles são na Figura 4.15(a).

Em todos os projetos apresentados até o momento, os diretórios são pesquisados linearmente do início ao fim quando o nome de um arquivo precisa ser procurado.

Para diretórios extremamente longos, a busca linear pode ser lenta. Uma maneira de acelerar a busca é usar uma tabela de espalhamento em cada diretório. Defina o tamanho da tabela n . Ao entrar com um nome de arquivo, o nome é mapeado em um valor entre 0 e $n - 1$, por exemplo, dividindo-o por n e tomando-se o resto. Alternativamente, as palavras compreendendo o nome do arquivo podem ser somadas e essa quantidade dividida por n , ou algo similar.

De qualquer maneira, a entrada da tabela correspondendo ao código de espalhamento é verificada. Entradas de arquivo seguem a tabela de espalhamento. Se aquela vaga já estiver em uso, uma lista encadeada é construída, inicializada naquela entrada da tabela e unindo todas as entradas com o mesmo valor de espalhamento.

A procura por um arquivo segue o mesmo procedimento. O nome do arquivo é submetido a uma função de espalhamento para selecionar uma entrada da tabela de espalhamento. Todas as entradas da lista encadeada inicializada naquela vaga são verificadas para ver se o nome do arquivo está presente. Se o nome não estiver na lista, o arquivo não está presente no diretório.

Usar uma tabela de espalhamento tem a vantagem de uma busca muito mais rápida, mas a desvantagem de uma administração mais complexa. Ela é uma alternativa realmente séria apenas em sistemas em que é esperado que os diretórios contenham de modo rotineiro centenas ou milhares de arquivos.

Se *C* subsequentemente tentar remover o arquivo, o sistema se vê diante de um dilema. Se remover o arquivo e limpar o i-node, *B* terá uma entrada de diretório apontando para um i-node inválido. Se o i-node for transferido mais tarde para outro arquivo, a ligação de *B* apontará para o arquivo errado. O sistema pode avaliar, a partir do contador no i-node, que o arquivo ainda está em uso, mas não há uma maneira fácil de encontrar todas as entradas de diretório para o arquivo a fim de removê-las. Ponteiros para os diretórios não podem ser armazenados no i-node, pois pode haver um número ilimitado de diretórios.

A única coisa a fazer é remover a entrada de diretório de *C*, mas deixar o i-node intacto, com o contador em 1, como mostrado na Figura 4.17(c). Agora temos uma situação na qual *B* é o único usuário com uma entrada de diretório para um arquivo cujo proprietário é *C*. Se o sistema fizer contabilidade ou tiver cotas, *C* continuará pagando a conta pelo arquivo até que *B* decida removê-lo. Se *B* o fizer, nesse momento o contador vai para 0 e o arquivo é removido.

Com ligações simbólicas esse problema não surge, pois somente o verdadeiro proprietário tem um ponteiro para o i-node. Os usuários que têm ligações para o arquivo possuem apenas nomes de caminhos, não ponteiros de i-node. Quando o *proprietário* remove o arquivo, ele é destruído. Tentativas subsequentes de usar o arquivo por uma ligação simbólica fracassarão quando o sistema for incapaz de localizá-lo. Remover uma ligação simbólica não afeta o arquivo de maneira alguma.

O problema com ligações simbólicas é a sobrecarga extra necessária. O arquivo contendo o caminho deve ser lido, então ele deve ser analisado e seguido, componente a componente, até que o i-node seja alcançado. Toda essa atividade pode exigir um número considerável de acessos adicionais ao disco. Além disso, um i-node extra é necessário para cada ligação simbólica, assim como um bloco de disco extra para armazenar o caminho, embora se o nome do caminho for curto, o sistema poderá armazená-lo no próprio i-node, como um tipo de otimização. Ligações simbólicas têm a vantagem de poderem ser usadas para ligar os arquivos em máquinas em qualquer parte no mundo, simplesmente fornecendo o endereço de rede da máquina onde o arquivo reside, além de seu caminho naquela máquina.

Há também outro problema introduzido pelas ligações, simbólicas ou não. Quando as ligações são permitidas, os arquivos podem ter dois ou mais caminhos. Programas que inicializam em um determinado diretório e encontram todos os arquivos naquele diretório e

seus subdiretórios, localizarão um arquivo ligado múltiplas vezes. Por exemplo, um programa que salva todos os arquivos de um diretório e seus subdiretórios em uma fita poderá fazer múltiplas cópias de um arquivo ligado. Além disso, se a fita for lida então em outra máquina, a não ser que o programa que salva para a fita seja inteligente, o arquivo ligado será copiado duas vezes para o disco, em vez de ser ligado.

4.3.5 Sistemas de arquivos estruturados em diário (log)

Mudanças na tecnologia estão pressionando os sistemas de arquivos atuais. Em particular, CPUs estão ficando mais rápidas, discos tornam-se muito maiores e baratos (mas não muito mais rápidos), e as memórias crescem exponencialmente em tamanho. O único parâmetro que não está se desenvolvendo de maneira tão acelerada é o tempo de busca dos discos (exceto para discos em estado sólido, que não têm tempo de busca).

A combinação desses fatores significa que um gargalo de desempenho está surgindo em muitos sistemas de arquivos. Pesquisas realizadas em Berkeley tentaram minimizar esse problema projetando um tipo completamente novo de sistema de arquivos, o **LFS (Log-structured File System)** — sistema de arquivos estruturado em diário). Nesta seção, descreveremos brevemente como o LFS funciona. Para uma abordagem mais completa, ver o estudo original sobre LFS (ROSENBLUM e OUSTERHOUT, 1991).

A ideia que impeliu o design do LFS é de que à medida que as CPUs ficam mais rápidas e as memórias RAM maiores, caches em disco também estão aumentando rapidamente. Em consequência, agora é possível satisfazer uma fração muito substancial de todas as solicitações de leitura diretamente da cache do sistema de arquivos, sem a necessidade de um acesso de disco. Segue dessa observação que, no futuro, a maior parte dos acessos ao disco será para escrita, então o mecanismo de leitura antecipada usado em alguns sistemas de arquivos para buscar blocos antes que eles sejam necessários não proporciona mais um desempenho significativo.

Para piorar as coisas, na maioria dos sistemas de arquivos, as operações de escrita são feitas em pedaços muito pequenos. Escritas pequenas são altamente ineficientes, dado que uma escrita em disco de 50 μ s muitas vezes é precedida por uma busca de 10 ms e um atraso rotacional de 4 ms. Com esses parâmetros, a eficiência dos discos cai para uma fração de 1%.

A fim de entender de onde vêm todas essas pequenas operações de escrita, considere criar um arquivo

novo em um sistema UNIX. Para escrever esse arquivo, o i-node para o diretório, o bloco do diretório, o i-node para o arquivo e o próprio arquivo devem ser todos escritos. Embora essas operações possam ser postergadas, fazê-lo expõe o sistema de arquivos a sérios problemas de consistência se uma queda no sistema ocorrer antes que as escritas tenham sido concluídas. Por essa razão, as escritas de i-node são, em geral, feitas imediatamente.

A partir desse raciocínio, os projetistas do LFS decidiram reimplementar o sistema de arquivos UNIX de maneira que fosse possível utilizar a largura total da banda do disco, mesmo diante de uma carga de trabalho consistindo em grande parte de pequenas operações de escrita aleatórias. A ideia básica é estruturar o disco inteiro como um grande diário (log).

De modo periódico, e quando há uma necessidade especial para isso, todas as operações de escrita pendentes armazenadas na memória são agrupadas em um único segmento e escritas para o disco como um único segmento contíguo ao fim do diário. Desse modo, um único segmento pode conter i-nodes, blocos de diretório e blocos de dados, todos misturados. No começo de cada segmento há um resumo do segmento, dizendo o que pode ser encontrado nele. Se o segmento médio puder ser feito com o tamanho de cerca de 1 MB, quase toda a largura de banda de disco poderá ser utilizada.

Neste projeto, i-nodes ainda existem e têm até a mesma estrutura que no UNIX, mas estão dispersos agora por todo o diário, em vez de ter uma posição fixa no disco. Mesmo assim, quando um i-node é localizado, a localização dos blocos acontece da maneira usual. É claro, encontrar um i-node é muito mais difícil agora, já que seu endereço não pode ser simplesmente calculado a partir do seu i-número, como no UNIX. Para tornar possível encontrar i-nodes, é mantido um mapa do i-node, indexado pelo i-número. O registro *i* nesse mapa aponta para o i-node *i* no disco. O mapa fica armazenado no disco, e também é mantido em cache, de maneira que as partes mais intensamente usadas estarão na memória a maior parte do tempo.

Para resumir o que dissemos até o momento, todas as operações de escrita são inicialmente armazenadas na memória, e periodicamente todas as operações de escrita armazenadas são escritas para o disco em um único segmento, ao final do diário. Abrir um arquivo agora consiste em usar o mapa para localizar o i-node para o arquivo. Uma vez que o i-node tenha sido localizado, os endereços dos blocos podem ser encontrados a partir dele. Todos os blocos em si estarão em segmentos, em alguma parte no diário.

Se discos fossem infinitamente grandes, a descrição anterior daria conta de toda a história. No entanto, discos reais são finitos, então finalmente o diário ocupará o disco inteiro, momento em que nenhum segmento novo poderá ser escrito para o diário. Felizmente, muitos segmentos existentes podem ter blocos que não são mais necessários. Por exemplo, se um arquivo for sobrescrito, seu i-node apontará então para os blocos novos, mas os antigos ainda estarão ocupando espaço em segmentos escritos anteriormente.

Para lidar com esse problema, o LFS tem um thread **limpador** que passa o seu tempo escaneando o diário circularmente para compactá-lo. Ele começa lendo o resumo do primeiro segmento no diário para ver quais i-nodes e arquivos estão ali. Então confere o mapa do i-node atual para ver se os i-nodes ainda são atuais e se os blocos de arquivos ainda estão sendo usados. Em caso negativo, essa informação é descartada. Os i-nodes e blocos que ainda estão sendo usados vão para a memória para serem escritos no próximo segmento. O segmento original é então marcado como disponível, de maneira que o arquivo pode usá-lo para novos dados. Dessa maneira, o limpador se movimenta ao longo do diário, removendo velhos segmentos do final e colocando quaisquer dados ativos na memória para serem reescritos no segmento seguinte. Em consequência, o disco é um grande buffer circular, com o thread de escrita adicionando novos segmentos ao início e o thread limpador removendo os antigos do final.

Aqui o sistema de registro não é trivial, visto que, quando um bloco de arquivo é escrito de volta para um novo segmento, o i-node do arquivo (em alguma parte no diário) deve ser localizado, atualizado e colocado na memória para ser escrito no segmento seguinte. O mapa do i-node deve então ser atualizado para apontar para a cópia nova. Mesmo assim, é possível fazer a administração, e os resultados do desempenho mostram que toda essa complexidade vale a pena. As medidas apresentadas nos estudos citados mostram que o LFS supera o UNIX em desempenho por uma ordem de magnitude em escritas pequenas, enquanto tem um desempenho que é tão bom quanto, ou melhor que o UNIX para leituras e escritas grandes.

4.3.6 Sistemas de arquivos journaling

Embora os sistemas de arquivos estruturados em diário sejam uma ideia interessante, eles não são tão usados, em parte por serem altamente incompatíveis com os sistemas de arquivos existentes. Mesmo assim, uma das ideias inerentes a eles, a robustez diante de

falhas, pode ser facilmente aplicada a sistemas de arquivos mais convencionais. A ideia básica aqui é manter um diário do que o sistema de arquivos vai fazer antes que ele o faça; então, se o sistema falhar antes que ele possa fazer seu trabalho planejado, ao ser reinicializado, ele pode procurar no diário para ver o que acontecia no momento da falha e concluir o trabalho. Esse tipo de sistema de arquivos, chamado de **sistemas de arquivos journaling**, já está em uso na realidade. O sistema de arquivos NTFS da Microsoft e os sistemas de arquivos Linux ext3 e ReiserFS todos usam journaling. O OS X oferece sistemas de arquivos journaling como uma opção. A seguir faremos uma breve introdução a esse tópico.

Para ver a natureza do problema, considere uma operação corriqueira simples que acontece todo o tempo: remover um arquivo. Essa operação (no UNIX) exige três passos:

1. Remover o arquivo do seu diretório.
2. Liberar o i-node para o conjunto de i-nodes livres.
3. Retornar todos os blocos de disco para o conjunto de blocos de disco livres.

No Windows, são exigidos passos análogos. Na ausência de falhas do sistema, a ordem na qual esses passos são dados não importa; na presença de falhas, ela importa. Suponha que o primeiro passo tenha sido concluído e então haja uma falha no sistema. O i-node e os blocos de arquivos não serão acessíveis a partir de arquivo algum, mas também não serão acessíveis para realocação; eles apenas estarão em algum limbo, diminuindo os recursos disponíveis. Se a falha ocorrer após o segundo passo, apenas os blocos serão perdidos.

Se a ordem das operações for mudada e o i-node for liberado primeiro, então após a reinicialização, o i-node poderá ser realocado, mas a antiga entrada de diretório continuará apontando para ele, portanto para o arquivo errado. Se os blocos forem liberados primeiro, então uma falha antes de o i-node ser removido significará que uma entrada de diretório válida aponta para um i-node listando blocos que pertencem agora ao conjunto de armazenamento livre e que provavelmente serão reutilizados em breve, levando dois ou mais arquivos a compartilhar ao acaso os mesmos blocos. Nenhum desses resultados é bom.

O que o sistema de arquivos journaling faz é primeiro escrever uma entrada no diário listando as três ações a serem concluídas. A entrada no diário é então escrita para o disco (e de maneira providente, quem sabe lendo de novo do disco para verificar que ela foi, de fato, escrita corretamente). Apenas após a entrada no diário ter

sido escrita é que começam as várias operações. Após as operações terem sido concluídas de maneira bem-sucedida, a entrada no diário é apagada. Se o sistema falhar agora, ao se recuperar, o sistema de arquivos poderá conferir o diário para ver se havia alguma operação pendente. Se afirmativo, todas elas podem ser reexecutadas (múltiplas vezes no caso de falhas repetidas) até que o arquivo seja corretamente removido.

Para que o journaling funcione, as operações registradas no diário devem ser **idempotentes**, isto é, elas podem ser repetidas quantas vezes forem necessárias sem prejuízo algum. Operações como “Atualize o mapa de bits para marcar i-node k ou bloco n como livres” podem ser repetidas sem nenhum problema até o objetivo ser consumado. Do mesmo modo, buscar um diretório e remover qualquer entrada chamada *foobar* também é uma operação idempotente. Por outro lado, adicionar os blocos recentemente liberados do i-node K para o final da lista livre não é uma operação idempotente, pois eles talvez já estejam ali. A operação mais cara “Pesquise a lista de blocos livres e inclua o bloco n se ele ainda não estiver lá” é idempotente. Sistemas de arquivos journaling têm de arranjar suas estruturas de dados e operações ligadas ao diário de maneira que todos sejam idempotentes. Nessas condições, a recuperação de falhas pode ser rápida e segura.

Para aumentar a confiabilidade, um sistema de arquivos pode introduzir o conceito do banco de dados de uma **transação atômica**. Quando esse conceito é usado, um grupo de ações pode ser formado pelas operações *begin transaction* e *end transaction*. O sistema de arquivos sabe então que ele precisa completar todas as operações do grupo ou nenhuma delas, mas não qualquer outra combinação.

O NTFS possui um amplo sistema de journaling e sua estrutura raramente é corrompida por falhas no sistema. Ela está em desenvolvimento desde seu primeiro lançamento com o Windows NT em 1993. O primeiro sistema de arquivos Linux a fazer journaling foi o ReiserFS, mas sua popularidade foi impedida porque ele era incompatível com o então sistema de arquivos ext2 padrão. Em comparação, o ext3, que é um projeto menos ambicioso do que o ReiserFS, também faz journaling enquanto mantém a compatibilidade com o sistema ext2 anterior.

4.3.7 Sistemas de arquivos virtuais

Muitos sistemas de arquivos diferentes estão em uso — muitas vezes no mesmo computador — mesmo para o mesmo sistema operacional. Um sistema Windows

pode ter um sistema de arquivos NTFS principal, mas também uma antiga unidade ou partição FAT-16 ou FAT-32, que contenha dados antigos, porém ainda necessários, e de tempos em tempos um flash drive, um antigo CD-ROM ou um DVD (cada um com seu sistema de arquivos único) podem ser necessários também. O Windows lida com esses sistemas de arquivos díspares identificando cada um com uma letra de unidade diferente, como em *C:*, *D:* etc. Quando um processo abre um arquivo, a letra da unidade está implícita ou explicitamente presente, então o Windows sabe para qual sistema de arquivos passar a solicitação. Não há uma tentativa de integrar sistemas de arquivos heterogêneos em um todo unificado.

Em comparação, todos os sistemas UNIX fazem uma tentativa muito séria de integrar múltiplos sistemas de arquivos em uma única estrutura. Um sistema Linux pode ter o ext2 como o diretório-raiz, com a partição ext3 montada em */usr* e um segundo disco rígido com o sistema de arquivos ReiserFS montado em */home*, assim como um CD-ROM ISO 9660 temporariamente montado em */mnt*. Do ponto de vista do usuário, existe uma hierarquia de sistema de arquivos única. O fato de ela lidar com múltiplos sistemas de arquivos (incompatíveis) não é visível para os usuários ou processos.

No entanto, a presença de múltiplos sistemas de arquivos é definitivamente visível à implementação, e desde o trabalho pioneiro da Sun Microsystems (KLEIMAN, 1986), a maioria dos sistemas UNIX usou o conceito de um **VFS (Virtual File System** — sistema de arquivos virtuais) para tentar integrar múltiplos sistemas de arquivos em uma estrutura ordeira. A ideia fundamental é abstrair a parte do sistema de arquivos que é comum a todos os sistemas de arquivos e colocar aquele código em uma camada separada que chama os sistemas de arquivos subjacentes para realmente gerenciar os dados. A estrutura como um todo está ilustrada na Figura

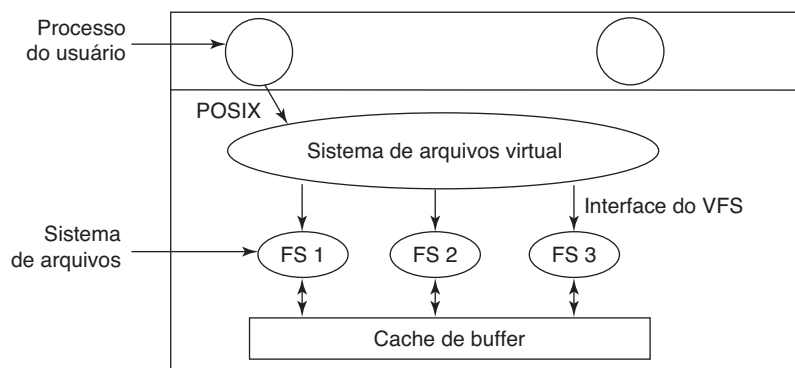
4.18. A discussão a seguir não é específica ao Linux, FreeBSD ou qualquer outra versão do UNIX, mas dá uma ideia geral de como os sistemas de arquivos virtuais funcionam nos sistemas UNIX.

Todas as chamadas de sistemas relativas a arquivos são direcionadas ao sistema de arquivos virtual para processamento inicial. Essas chamadas, vindas de outros processos de usuários, são as chamadas POSIX padrão, como *open*, *read*, *write*, *lseek* e assim por diante. Desse modo, o VFS tem uma interface “superior” para os processos do usuário, e é a já conhecida interface POSIX.

O VFS também tem uma interface “inferior” para os sistemas de arquivos reais, que são rotulados de **interface do VFS** na Figura 4.18. Essa interface consiste em várias dúzias de chamadas de funções que os VFS podem fazer para cada sistema de arquivos para realizar o trabalho. Assim, para criar um novo sistema de arquivos que funcione com o VFS, os projetistas do novo sistema de arquivos devem certificar-se de que ele proporcione as chamadas de funções que o VFS exige. Um exemplo óbvio desse tipo de função é aquela que lê um bloco específico do disco, coloca-o na cache de buffer do sistema de arquivos e retorna um ponteiro para ele. Desse modo, o VFS tem duas interfaces distintas: a superior para os processos do usuário e a inferior para os sistemas de arquivos reais.

Embora a maioria dos sistemas de arquivos sob o VFS represente partições em um disco local, este nem sempre é o caso. Na realidade, a motivação original para a Sun produzir o VFS era dar suporte a sistemas de arquivos remotos usando o protocolo **NFS (Network File System** — sistema de arquivos de rede). O projeto VFS foi feito de tal forma que enquanto o sistema de arquivos real fornecer as funções que o VFS exigir, o VFS não sabe ou se preocupa onde estão armazenados os dados ou como é o sistema de arquivos subjacente.

FIGURA 4.18 Posição do sistema de arquivos virtual.



Internamente, a maioria das implementações de VFS é na essência orientada para objetos, mesmo que todos sejam escritos em C em vez de C++. Há vários tipos de objetos fundamentais que são em geral aceitos. Esses incluem o superbloco (que descreve um sistema de arquivos), o v-node (que descreve um arquivo) e o diretório (que descreve um diretório de sistemas de arquivos). Cada um desses tem operações associadas (métodos) a que os sistemas de arquivos reais têm de dar suporte. Além disso, o VFS tem algumas estruturas internas de dados para seu próprio uso, incluindo a tabela de montagem e um conjunto de descritores de arquivos para monitorar todos os arquivos abertos nos processos do usuário.

Para compreender como o VFS funciona, vamos repassar um exemplo cronologicamente. Quando o sistema é inicializado, o sistema de arquivos raiz é registrado com o VFS. Além disso, quando outros sistemas de arquivos são montados, seja no momento da inicialização ou durante a operação, também devem registrar-se com o VFS. Quando um sistema de arquivos se registra, o que ele basicamente faz é fornecer uma lista de endereços das funções que o VFS exige, seja como um longo vetor de chamada (tabela) ou como vários deles, um por objeto de VFS, como demanda o VFS. Então, assim que um sistema de arquivos tenha se registrado com o VFS, este sabe como, digamos, ler um bloco a partir dele — ele simplesmente chama a quarta (ou qualquer que seja) função no vetor fornecido pelo sistema de arquivos. De modo similar, o VFS então também sabe como realizar todas as funções que o sistema de arquivos real deve fornecer: ele apenas chama a função cujo endereço foi fornecido quando o sistema de arquivos registrou.

Após um sistema de arquivos ter sido montado, ele pode ser usado. Por exemplo, se um sistema de arquivos foi montado em */usr* e um processo fizer a chamada

```
open("/usr/include/unistd.h", O_RDONLY)
```

durante a análise do caminho, o VFS vê que um novo sistema de arquivos foi montado em */usr* e localiza seu superbloco pesquisando a lista de superblocos de sistemas de arquivos montados. Tendo feito isso, ele pode encontrar o diretório-raiz do sistema de arquivos montado e examinar o caminho *include/unistd.h* ali. O VFS então cria um v-node e faz uma chamada para o sistema de arquivos real para retornar todas as informações no i-node do arquivo. Essa informação é copiada para o v-node (em RAM), junto com outras informações, e, o mais importante, cria o ponteiro para a tabela de funções para chamar operações em v-nodes, como *read*, *write*, *close* e assim por diante.

Após o v-node ter sido criado, o VFS registra uma entrada na tabela de descritores de arquivo para o processo que fez a chamada e faz que ele aponte para o novo v-node. (Para os puristas, o descritor de arquivos na realidade aponta para outras estruturas de dados que contêm a posição atual do arquivo e um ponteiro para o v-node, mas esse detalhe não é importante para nossas finalidades aqui.) Por fim, o VFS retorna o descritor de arquivos para o processo que chamou, assim ele pode usá-lo para ler, escrever e fechar o arquivo.

Mais tarde, quando o processo realiza um *read* usando o descritor de arquivos, o VFS localiza o v-node do processo e das tabelas de descritores de arquivos e segue o ponteiro até a tabela de funções, na qual estão os endereços dentro do sistema de arquivos real, no qual reside o arquivo solicitado. A função responsável pelo *read* é chamada agora e o código dentro do sistema de arquivos real vai e busca o bloco solicitado. O VFS não faz ideia se os dados estão vindo do disco local, um sistema de arquivos remoto através da rede, um pen-drive ou algo diferente. As estruturas de dados envolvidas são mostradas na Figura 4.19. Começando com o número do processo chamador e o descritor do arquivo, então o v-node, o ponteiro da função de leitura e a função de acesso dentro do sistema de arquivos real são localizados.

Dessa maneira, adicionar novos sistemas de arquivos torna-se algo relativamente direto. Para realizar a operação, os projetistas primeiro tomam uma lista de chamadas de funções esperadas pelo VFS e então escrevem seu sistema de arquivos para prover todas elas. Como alternativa, se o sistema de arquivos já existe, então eles têm de prover funções adaptadoras que façam o que o VFS precisa, normalmente realizando uma ou mais chamadas nativas ao sistema de arquivos real.

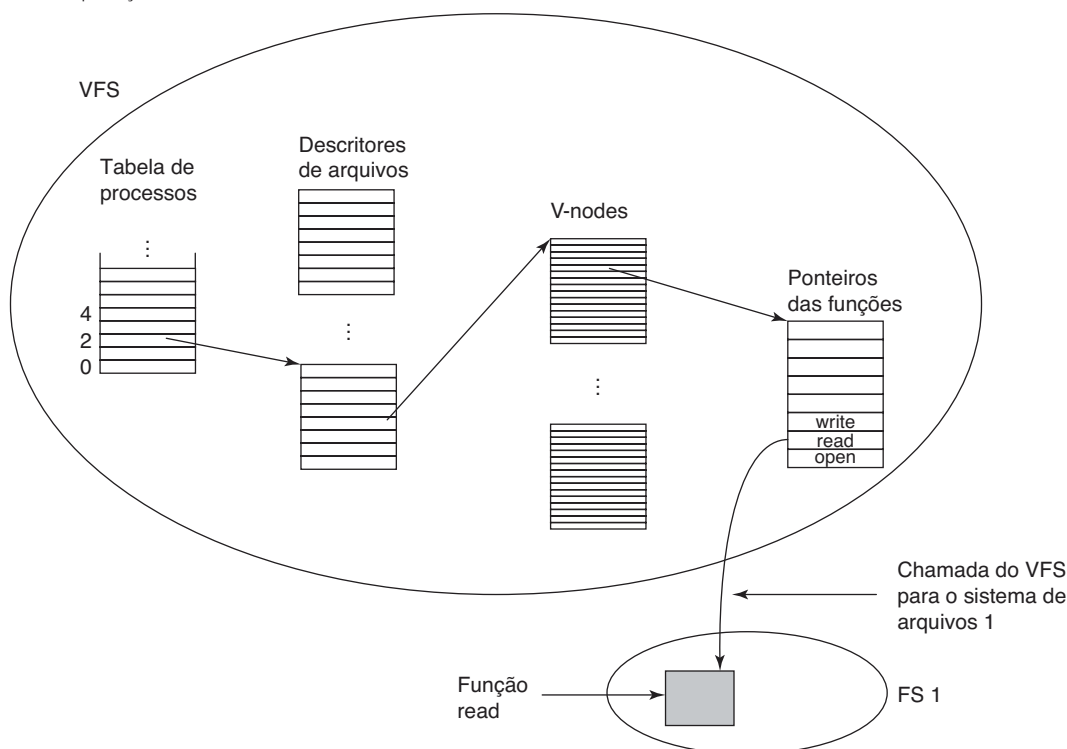
4.4 Gerenciamento e otimização de sistemas de arquivos

Fazer o sistema de arquivos funcionar é uma coisa: fazê-lo funcionar de forma eficiente e robustamente na vida real é algo bastante diferente. Nas seções a seguir examinaremos algumas das questões envolvidas no gerenciamento de discos.

4.4.1 Gerenciamento de espaço em disco

Arquivos costumam ser armazenados em disco, portanto o gerenciamento de espaço de disco é uma preocupação importante para os projetistas de sistemas de

FIGURA 4.19 Uma visão simplificada das estruturas de dados e código usadas pelo VFS e pelo sistema de arquivos real para realizar uma operação read.



arquivos. Duas estratégias gerais são possíveis para armazenar um arquivo de n bytes: ou são alocados n bytes consecutivos de espaço, ou o arquivo é dividido em uma série de blocos (não necessariamente) contíguos. A mesma escolha está presente em sistemas de gerenciamento de memória entre a segmentação pura e a paginação.

Como vimos, armazenar um arquivo como uma sequência contígua de bytes tem o problema óbvio de que se um arquivo crescer, ele talvez tenha de ser movido dentro do disco. O mesmo problema ocorre para segmentos na memória, exceto que mover um segmento na memória é uma operação relativamente rápida em comparação com mover um arquivo de uma posição no disco para outra. Por essa razão, quase todos os sistemas de arquivos os dividem em blocos de tamanho fixo que não precisam ser adjacentes.

Tamanho do bloco

Uma vez que tenha sido feita a opção de armazenar arquivos em blocos de tamanho fixo, a questão que surge é qual tamanho o bloco deve ter. Dado o modo como os discos são organizados, o setor, a trilha e o cilindro são candidatos óbvios para a unidade de

alocação (embora sejam todos dependentes do dispositivo, o que é um ponto negativo). Em um sistema de paginação, o tamanho da página também é um argumento importante.

Ter um tamanho de bloco grande significa que todos os arquivos, mesmo um de 1 byte, ocuparão um cilindro inteiro. Também significa que arquivos pequenos desperdiçam uma grande quantidade de espaço de disco. Por outro lado, um tamanho de bloco pequeno significa que a maioria dos arquivos ocupará múltiplos blocos e, desse modo, precisará de múltiplas buscas e atrasos rotacionais para lê-los, reduzindo o desempenho. Então, se a unidade de alocação for grande demais, desperdiçamos espaço; se ela for pequena demais, desperdiçamos tempo.

Fazer uma boa escolha exige ter algumas informações sobre a distribuição do tamanho do arquivo. Tanenbaum et al. (2006) estudaram a distribuição do tamanho do arquivo no Departamento de Ciências de Computação de uma grande universidade de pesquisa (a Universidade Vrije) em 1984 e então novamente em 2005, assim como em um servidor da web comercial hospedando um site de política (www.electoral-vote.com). Os resultados são mostrados na Figura 4.20, na qual é listada, para cada grupo, a porcentagem de todos os arquivos menores ou iguais ao tamanho (representado por potência de base dois).

FIGURA 4.20 Porcentagem de arquivos menores do que um determinado tamanho (em bytes).

Tamanho	UV 1984	UV 2005	Web
1	1,79	1,38	6,67
2	1,88	1,53	7,67
4	2,01	1,65	8,33
8	2,31	1,80	11,30
16	3,32	2,15	11,46
32	5,13	3,15	12,33
64	8,71	4,98	26,10
128	14,73	8,03	28,49
256	23,09	13,29	32,10
512	34,44	20,62	39,94
1 KB	48,05	30,91	47,82
2 KB	60,87	46,09	59,44
4 KB	75,31	59,13	70,64
8 KB	84,97	69,96	79,69

Tamanho	UV 1984	UV 2005	Web
16 KB	92,53	78,92	86,79
32 KB	97,21	85,87	91,65
64 KB	99,18	90,84	94,80
128 KB	99,84	93,73	96,93
256 KB	99,96	96,12	98,48
512 KB	100,00	97,73	98,99
1 MB	100,00	98,87	99,62
2 MB	100,00	99,44	99,80
4 MB	100,00	99,71	99,87
8 MB	100,00	99,86	99,94
16 MB	100,00	99,94	99,97
32 MB	100,00	99,97	99,99
64 MB	100,00	99,99	99,99
128 MB	100,00	99,99	100,00

Por exemplo, em 2005, 59,13% de todos os arquivos na Universidade de Vrije tinham 4 KB ou menos e 90,84% de todos eles, 64 KB ou menos. O tamanho de arquivo médio era de 2.475 bytes. Algumas pessoas podem achar esse tamanho pequeno surpreendente.

Que conclusões podemos tirar desses dados? Por um lado, com um tamanho de bloco de 1 KB, apenas em torno de 30-50% de todos os arquivos cabem em um único bloco, enquanto com um bloco de 4 KB, a porcentagem de arquivos que cabem em um bloco sobe para a faixa de 60-70%. Outros dados no estudo mostram que com um bloco de 4 KB, 93% dos blocos do disco são usados por 10% dos maiores arquivos. Isso significa que o desperdício de espaço ao fim de cada pequeno arquivo é insignificante, pois o disco está cheio por uma pequena quantidade de arquivos grandes (vídeos) e o montante total de espaço tomado pelos arquivos pequenos pouco importa. Mesmo dobrando o espaço requerido por 90% dos menores arquivos, isso mal seria notado.

Por outro lado, utilizar um pequeno bloco significa que cada arquivo consistirá em muitos blocos. Ler cada bloco exige uma busca e um atraso rotacional (exceto em um disco em estado sólido), então a leitura de um arquivo consistindo em muitos blocos pequenos será lenta.

Como exemplo, considere um disco com 1 MB por trilha, um tempo de rotação de 8,33 ms e um tempo de

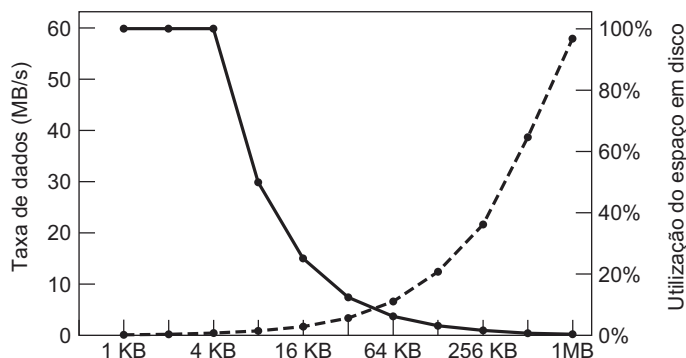
busca de 5 ms. O tempo em milissegundos para ler um bloco de k bytes é então a soma dos tempos de busca, atraso rotacional e transferência:

$$5 + 4,165 + (k/1000000) \times 8,33$$

A curva tracejada da Figura 4.21 mostra a taxa de dados para um disco desses como uma função do tamanho do bloco. Para calcular a eficiência de espaço, precisamos fazer uma suposição a respeito do tamanho médio do arquivo. Para simplificar, vamos presumir que todos os arquivos tenham 4 KB. Embora esse número seja ligeiramente maior do que os dados medidos na Universidade de Vrije, os estudantes provavelmente têm mais arquivos pequenos do que os existentes em um centro de dados corporativo, então, como um todo, talvez seja um palpite melhor. A curva sólida da Figura 4.21 mostra a eficiência de espaço como uma função do tamanho do bloco.

As duas curvas podem ser compreendidas como a seguir. O tempo de acesso para um bloco é completamente dominado pelo tempo de busca e atraso rotacional, então levando-se em consideração que serão necessários 9 ms para acessar um bloco, quanto mais dados forem buscados, melhor. Assim, a taxa de dados cresce quase linearmente com o tamanho do bloco (até as transferências demorarem tanto que o tempo de transferência começa a importar).

FIGURA 4.21 A curva tracejada (escala da esquerda) mostra a taxa de dados de um disco. A curva contínua (escala da direita) mostra a eficiência do espaço em disco. Todos os arquivos têm 4 KB.



Agora considere a eficiência de espaço. Com arquivos de 4 KB e blocos de 1 KB, 2 KB ou 4 KB, os arquivos usam 4, 2 e 1 bloco, respectivamente, sem desperdício. Com um bloco de 8 KB e arquivos de 4 KB, a eficiência de espaço cai para 50% e com um bloco de 16 KB ela chega a 25%. Na realidade, poucos arquivos são um múltiplo exato do tamanho do bloco do disco, então algum espaço sempre é desperdiçado no último bloco de um arquivo.

O que as curvas mostram, no entanto, é que o desempenho e a utilização de espaço estão inerentemente em conflito. Pequenos blocos são ruins para o desempenho, mas bons para a utilização do espaço do disco. Para esses dados, não há equilíbrio que seja razoável. O tamanho mais próximo de onde as duas curvas se cruzam é 64 KB, mas a taxa de dados é de apenas 6,6 MB/s e a eficiência de espaço é de cerca de 7%, nenhum dos dois valores é muito bom. Historicamente, sistemas de arquivos escolheram tamanhos na faixa de 1 KB a 4 KB, mas com discos agora excedendo 1 TB, pode ser melhor aumentar o tamanho do bloco para 64 KB e aceitar o espaço de disco desperdiçado. Hoje é muito pouco provável que falte espaço de disco.

Em um experimento para ver se o uso de arquivos do Windows NT era apreciavelmente diferente do uso de arquivos do UNIX, Vogels tomou medidas nos arquivos na Universidade de Cornell (VOGELS, 1999). Ele observou que o uso de arquivos no NT é mais complicado que no UNIX. Ele escreveu:

Quando digitamos alguns caracteres no editor de texto do Notepad, o salvamento dessa digitação em um arquivo desencadeará 26 chamadas de sistema, incluindo 3 tentativas de abertura que falharam, 1 arquivo sobrescrito e 4 sequências adicionais de abertura e fechamento.

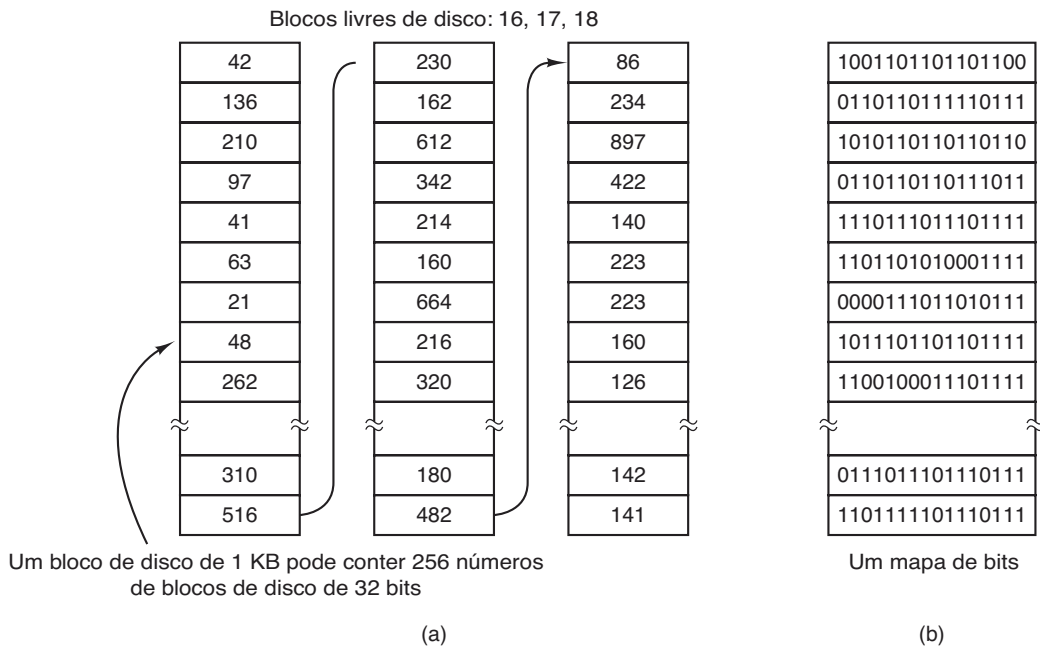
Não obstante isso, Vogels observou um tamanho médio (ponderado pelo uso) de arquivos apenas lidos como

1 KB, arquivos apenas escritos como 2,3 KB e arquivos lidos e escritos como 4,2 KB. Considerando as diferentes técnicas de mensuração de conjuntos de dados, e o ano, esses resultados são certamente compatíveis com os da Universidade de Vrije.

Monitoramento dos blocos livres

Uma vez que um tamanho de bloco tenha sido escolhido, a próxima questão é como monitorar os blocos livres. Dois métodos são amplamente usados, como mostrado na Figura 4.22. O primeiro consiste em usar uma lista encadeada de blocos de disco, com cada bloco contendo tantos números de blocos livres de disco quantos couberem nele. Com um bloco de 1 KB e um número de bloco de disco de 32 bits, cada bloco na lista livre contém os números de 255 blocos livres. (Uma entrada é reservada para o ponteiro para o bloco seguinte.) Considere um disco de 1 TB, que tem em torno de 1 bilhão de blocos de disco. Armazenar todos esses endereços em blocos de 255 exige cerca de 4 milhões de blocos. Em geral, blocos livres são usados para conter a lista livre, de maneira que o armazenamento seja essencialmente gratuito.

A outra técnica de gerenciamento de espaço livre é o mapa de bits. Um disco com n blocos exige um mapa de bits com n bits. Blocos livres são representados por 1s no mapa, blocos alocados por 0s (ou vice-versa). Para nosso disco de 1 TB de exemplo, precisamos de 1 bilhão de bits para o mapa, o que exige em torno de 130.000 blocos de 1 KB para armazenar. Não surpreende que o mapa de bits exija menos espaço, tendo em vista que ele usa 1 bit por bloco, *versus* 32 bits no modelo de lista encadeada. Apenas se o disco estiver praticamente cheio (isto é, tiver poucos blocos livres) o esquema da lista encadeada exigirá menos blocos do que o mapa de bits.

FIGURA 4.22 (a) Armazenamento da lista de blocos livres em uma lista encadeada. (b) Um mapa de bits.

Se os blocos livres tenderem a vir em longos conjuntos de blocos consecutivos, o sistema da lista de blocos livres pode ser modificado para controlar conjuntos de blocos em vez de blocos individuais. Um contador de 8, 16 ou 32 bits poderia ser associado com cada bloco dando o número de blocos livres consecutivos. No melhor caso, um disco basicamente vazio seria representado por dois números: o endereço do primeiro bloco livre seguido pelo contador de blocos livres. Por outro lado, se o disco se tornar severamente fragmentado, o controle de conjuntos de blocos será menos eficiente do que o controle de blocos individuais, pois não apenas o endereço deverá ser armazenado, mas também o contador.

Essa questão ilustra um problema que os projetistas de sistemas operacionais muitas vezes enfrentam. Existem múltiplas estruturas de dados e algoritmos que podem ser usados para solucionar um problema, mas a escolha do melhor exige dados que os projetistas não têm e não terão até que o sistema seja distribuído e amplamente utilizado. E, mesmo assim, os dados podem não estar disponíveis. Por exemplo, nossas próprias medidas de tamanhos de arquivos na Universidade de Vrije em 1984 e 1995, os dados do site e os dados de Cornell são apenas quatro amostras. Embora muito melhor do que nada, temos pouca certeza se eles são também representativos de computadores pessoais, computadores corporativos, computadores do governo e outros. Com algum esforço poderíamos ser capazes de conseguir algumas amostras de outros tipos de computadores, mas

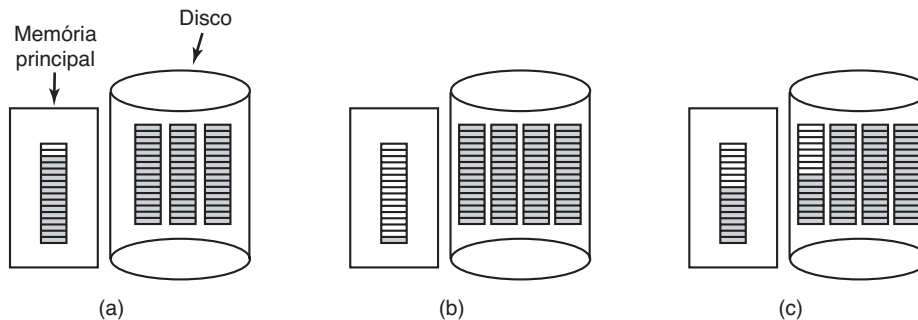
mesmo assim seria uma bobagem extrapolar para todos os computadores dos tipos mensurados.

Voltando para o método da lista de blocos livres por um momento, apenas um bloco de ponteiros precisa ser mantido na memória principal. Quando um arquivo é criado, os blocos necessários são tomados do bloco de ponteiros. Quando ele se esgota, um novo bloco de ponteiros é lido do disco. De modo similar, quando um arquivo é removido, seus blocos são liberados e adicionados ao bloco de ponteiros na memória principal. Quando esse bloco completa, ele é escrito no disco.

Em determinadas circunstâncias, esse método leva a operações desnecessárias de E/S em disco. Considere a situação da Figura 4.23(a), na qual o bloco de ponteiros na memória tem espaço para somente duas entradas. Se um arquivo de três blocos for liberado, o bloco de ponteiros transbordará e ele deverá ser escrito para o disco, levando à situação da Figura 4.23(b). Se um arquivo de três blocos for escrito agora, o bloco de ponteiros cheio deverá ser lido novamente, trazendo-nos de volta para a Figura 4.23(a). Se o arquivo de três blocos recém-escrito constituir um arquivo temporário, quando ele for liberado, será necessária outra operação de escrita para escrever novamente o bloco de ponteiros cheio no disco. Resumindo, quando o bloco de ponteiros estiver quase vazio, uma série de arquivos temporários de vida curta pode causar muitas operações de E/S em disco.

Uma abordagem alternativa que evita a maior parte dessas operações de E/S em disco é dividir o bloco

FIGURA 4.23 (a) Um bloco na memória quase cheio de ponteiros para blocos de disco livres e três blocos de ponteiros em disco. (b) Resultado da liberação de um arquivo de três blocos. (c) Uma estratégia alternativa para lidar com os três blocos livres. As entradas sombreadas representam ponteiros para blocos de discos livres.



cheio de ponteiros. Desse modo, em vez de ir da Figura 4.23(a) para a Figura 4.23(b), vamos da Figura 4.23(a) para a Figura 4.23(c) quando três blocos são liberados. Agora o sistema pode lidar com uma série de arquivos temporários sem realizar qualquer operação de E/S em disco. Se o bloco na memória encher, ele será escrito para o disco e o bloco meio cheio será lido do disco. A ideia aqui é manter a maior parte dos blocos de ponteiros cheios em disco (para minimizar o uso deste), mas manter o bloco na memória cheio pela metade, de maneira que ele possa lidar tanto com a criação quanto com a remoção de arquivos, sem uma operação de E/S em disco para a lista de livres.

Com um mapa de bits, também é possível manter apenas um bloco na memória, usando o disco para outro bloco apenas quando ele ficar completamente cheio ou vazio. Um benefício adicional dessa abordagem é que ao realizar toda a alocação de um único bloco do mapa de bits, os blocos de disco estarão mais próximos, minimizando assim os movimentos do braço do disco. Já que o mapa de bits é uma estrutura de dados de tamanho fixo, se o núcleo estiver (parcialmente) paginado, o mapa de bits pode ser colocado na memória virtual e ter suas páginas paginadas conforme a necessidade.

Cotas de disco

Para evitar que as pessoas exagerem no uso do espaço de disco, sistemas operacionais de múltiplos usuários muitas vezes fornecem um mecanismo para impor cotas de disco. A ideia é que o administrador do sistema designe a cada usuário uma cota máxima de arquivos e blocos, e o sistema operacional se certifique de que os usuários não excedam essa cota. Um mecanismo típico é descrito a seguir.

Quando um usuário abre um arquivo, os atributos e endereços de disco são localizados e colocados em uma

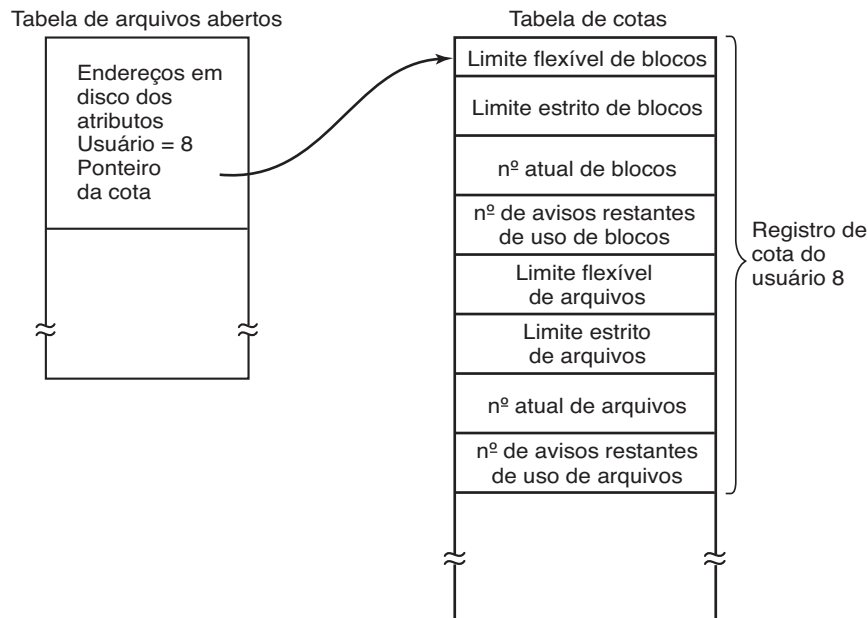
tabela de arquivos aberta na memória principal. Entre os atributos há uma entrada dizendo quem é o proprietário. Quaisquer aumentos no tamanho do arquivo serão cobrados da cota do proprietário.

Uma segunda tabela contém os registros de cotas de todos os usuários com um arquivo aberto, mesmo que esse arquivo tenha sido aberto por outra pessoa. Essa tabela está mostrada na Figura 4.24. Ela foi extraída de um arquivo de cotas no disco para os usuários cujos arquivos estão atualmente abertos. Quando todos os arquivos são fechados, o registro é escrito de volta para o arquivo de cotas.

Quando uma nova entrada é feita na tabela de arquivos abertos, um ponteiro para o registro de cota do proprietário é atribuído a ela, a fim de facilitar encontrar os vários limites. Toda vez que um bloco é adicionado a um arquivo, o número total de blocos cobrados do proprietário é incrementado, e os limites flexíveis e estritos são verificados. O limite flexível pode ser excedido, mas o limite estrito não. Uma tentativa de adicionar blocos a um arquivo quando o limite de blocos estrito tiver sido alcançado resultará em um erro. Verificações análogas também existem para o número de arquivos a fim de evitar que algum usuário sobrecarregue todos os i-nodes.

Quando um usuário tenta entrar no sistema, este examina o arquivo de cotas para ver se ele excedeu o limite flexível para o número de arquivos ou o número de blocos de disco. Se qualquer um dos limites foi violado, um aviso é exibido, e o contador de avisos restantes é reduzido para um. Se o contador chegar a zero, o usuário ignorou o aviso vezes demais, e não tem permissão para entrar. Conseguir a autorização para entrar novamente exigirá alguma conversa com o administrador do sistema.

Esse método tem a propriedade de que os usuários podem ir além de seus limites flexíveis durante uma sessão de uso, desde que removam o excesso antes de se desconnectarem. Os limites estritos jamais podem ser excedidos.

FIGURA 4.24 As cotas são relacionadas aos usuários e monitoradas em uma tabela de cotas.

4.4.2 Backups (cópias de segurança) do sistema de arquivos

A destruição de um sistema de arquivos é quase sempre um desastre muito maior do que a destruição de um computador. Se um computador for destruído pelo fogo, por uma descarga elétrica ou uma xícara de café derrubada no teclado, isso é irritante e custará dinheiro, mas geralmente uma máquina nova pode ser comprada com um mínimo de incômodo. Computadores pessoais baratos podem ser substituídos na mesma hora, bastando uma ida à loja (menos nas universidades, onde emitir uma ordem de compra exige três comitês, cinco assinaturas e 90 dias).

Se o sistema de arquivos de um computador estiver irrevogavelmente perdido, seja pelo hardware ou pelo software, restaurar todas as informações será difícil, exigirá tempo e, em muitos casos, será impossível. Para as pessoas cujos programas, documentos, registros tributários, arquivos de clientes, bancos de dados, planos de marketing, ou outros dados estiverem perdidos para sempre as consequências podem ser catastróficas. Apesar de o sistema de arquivos não conseguir oferecer qualquer proteção contra a destruição física dos equipamentos e da mídia, ele pode ajudar a proteger as informações. A solução é bastante clara: fazer cópias de segurança (backups). Mas isso pode não ser tão simples quanto parece. Vamos examinar a questão.

A maioria das pessoas não acredita que fazer backups dos seus arquivos valha o tempo e o esforço — até que

um belo dia seu disco morre abruptamente, momento que a maioria delas jamais esquecerá. As empresas, no entanto, compreendem (normalmente) bem o valor dos seus dados e costumam realizar um backup ao menos uma vez ao dia, muitas vezes em fita. As fitas modernas armazenam centenas de gigabytes e custam centavos por gigabyte. Não obstante isso, realizar backups não é algo tão trivial quanto parece, então examinaremos algumas das questões relacionadas a seguir.

Backups para fita são geralmente feitos para lidar com um de dois problemas potenciais:

1. Recuperação em caso de um desastre.
2. Recuperação de uma bobagem feita.

O primeiro problema diz respeito a fazer o computador funcionar novamente após uma quebra de disco, fogo, enchente ou outra catástrofe natural. Na prática, essas coisas não acontecem com muita frequência, razão pela qual muitas pessoas não se preocupam em fazer backups. Essas pessoas também tendem a não ter seguro contra incêndio em suas casas pela mesma razão.

A segunda razão é que os usuários muitas vezes removem acidentalmente arquivos de que precisam mais tarde outra vez. Esse problema ocorre com tanta frequência que, quando um arquivo é “removido” no Windows, ele não é apagado de maneira alguma, mas simplesmente movido para um diretório especial, a **cesta de reciclagem**, de maneira que ele possa buscado e restaurado facilmente mais tarde. Backups levam esse princípio mais longe ainda e permitem que arquivos que

foram removidos há dias, mesmo semanas, sejam restaurados de velhas fitas de backup.

Fazer backup leva um longo tempo e ocupa um espaço significativo, portanto é importante fazê-lo de maneira eficiente e conveniente. Essas considerações levantam as questões a seguir. Primeiro, será que todo o sistema de arquivos deve ser copiado ou apenas parte dele? Em muitas instalações, os programas executáveis (binários) são mantidos em uma parte limitada da árvore do sistema de arquivos. Não é necessário realizar backup de todos esses arquivos se todos eles podem ser reinstalados a partir do site do fabricante ou de um DVD de instalação. Também, a maioria dos sistemas tem um diretório para arquivos temporários. Em geral não há uma razão para fazer um backup dele também. No UNIX, todos os arquivos especiais (dispositivos de E/S) são mantidos em um diretório `/dev`. Fazer um backup desse diretório não só é desnecessário, como é realmente perigoso, pois o programa de backup poderia ficar pendurado para sempre se ele tentasse ler cada um desses arquivos até terminar. Resumindo, normalmente é desejável fazer o backup apenas de diretórios específicos e tudo neles em vez de todo o sistema de arquivos.

Segundo, é um desperdício fazer o backup de arquivos que não mudaram desde o último backup, o que leva à ideia de **cópias incrementais**. A forma mais simples de cópia incremental é realizar uma cópia (backup) completa periodicamente, digamos por semana ou por mês, e realizar uma cópia diária somente daqueles arquivos que foram modificados desde a última cópia completa. Melhor ainda é copiar apenas aqueles arquivos que foram modificados desde a última vez em que foram copiados. Embora esse esquema minimize o tempo de cópia, ele torna a recuperação mais complicada, pois primeiro a cópia mais recente deve ser restaurada e depois todas as cópias incrementais têm de ser restauradas na ordem inversa. Para facilitar a recuperação, muitas vezes são usados esquemas de cópias incrementais mais sofisticados.

Terceiro, visto que quantidades imensas de dados geralmente são copiadas, pode ser desejável comprimir os dados antes de escrevê-los na fita. No entanto, com muitos algoritmos de compressão, um único defeito na fita de backup pode estragar o algoritmo e tornar um arquivo inteiro ou mesmo uma fita inteira ilegível. Desse modo, a decisão de comprimir os dados de backup deve ser cuidadosamente considerada.

Quarto, é difícil realizar um backup em um sistema de arquivos ativo. Se os arquivos e diretórios estão sendo adicionados, removidos e modificados durante o processo de cópia, a cópia resultante pode ficar

inconsistente. No entanto, como realizar uma cópia pode levar horas, talvez seja necessário deixar o sistema off-line por grande parte da noite para realizar o backup, algo que nem sempre é aceitável. Por essa razão, algoritmos foram projetados para gerar fotografias (snapshots) rápidas do estado do sistema de arquivos copiando estruturas críticas de dados e então exigindo que nas mudanças futuras em arquivos e diretórios sejam realizadas cópias dos blocos em vez de atualizá-los diretamente (HUTCHINSON et al., 1999). Dessa maneira, o sistema de arquivos é efetivamente congelado no momento do snapshot; portanto, pode ser copiado depois quando o usuário quiser.

Quinto e último, fazer backups introduz muitos problemas não técnicos na organização. O melhor sistema de segurança on-line no mundo pode ser inútil se o administrador do sistema mantiver todos os discos ou fitas de backup em seu gabinete e deixá-lo aberto e desguarnecido sempre que for buscar um café no fim do corredor. Tudo o que um espião precisa fazer é aparecer por um segundo, colocar um disco ou fita minúsculos em seu bolso e cair fora lepidamente. Adeus, segurança. Também, realizar um backup diário tem pouco uso se o fogo que queimar os computadores também queimar todos os discos de backup. Por essa razão, discos de backup devem ser mantidos longe dos computadores, mas isso introduz mais riscos (pois agora dois locais precisam contar com segurança). Para uma discussão aprofundada dessas e de outras questões administrativas práticas, ver Nemeth et al. (2013). A seguir discutiremos apenas as questões técnicas envolvidas em realizar backups de sistemas de arquivos.

Duas estratégias podem ser usadas para copiar um disco para um disco de backup: uma cópia física ou uma cópia lógica. Uma **cópia física** começa no bloco 0 do disco, escreve em ordem todos os blocos de disco no disco de saída, e para quando ele tiver copiado o último. Esse programa é tão simples que provavelmente pode ser feito 100% livre de erros, algo que em geral não pode ser dito a respeito de qualquer outro programa útil.

Mesmo assim, vale a pena fazer vários comentários a respeito da cópia física. Por um lado, não faz sentido fazer backup de blocos de disco que não estejam sendo usados. Se o programa de cópia puder obter acesso à estrutura de dados dos blocos livres, ele pode evitar copiar blocos que não estejam sendo usados. No entanto, pular blocos que não estejam sendo usados exige escrever o número de cada bloco na frente dele (ou o equivalente), já que não é mais verdade que o bloco k no backup era o bloco k no disco.

Uma segunda preocupação é copiar blocos defeituosos. É quase impossível manufaturar discos grandes sem quaisquer defeitos. Alguns blocos defeituosos estão sempre presentes. Às vezes, quando é feita uma formatação de baixo nível, os blocos defeituosos são detectados, marcados como tal e substituídos por blocos de reserva guardados ao final de cada trilha para precisamente esse tipo de emergência. Em muitos casos, o controlador de disco gerencia a substituição de blocos defeituosos de forma transparente sem que o sistema operacional nem fique sabendo a respeito.

No entanto, às vezes os blocos passam a apresentar defeitos após a formatação, caso em que o sistema operacional eventualmente vai detectá-los. Em geral, ele soluciona o problema criando um “arquivo” consistindo em todos os blocos defeituosos — somente para certificar-se de que eles jamais apareçam como livres e sejam ocupados. Desnecessário dizer que esse arquivo é completamente ilegível.

Se todos os blocos defeituosos forem remapeados pelo controlador do disco e escondidos do sistema operacional como descrito há pouco, a cópia física funcionará bem. Por outro lado, se eles forem visíveis para o sistema operacional e mantidos em um ou mais arquivos de blocos defeituosos ou mapas de bits, é absolutamente essencial que o programa de cópia física tenha acesso a essa informação e evite copiá-los para evitar erros de leitura de disco intermináveis enquanto tenta fazer o backup do arquivo de bloco defeituoso.

Sistemas Windows têm arquivos de paginação e hibernação que não são necessários no caso de uma restauração e não devem ser copiados em primeiro lugar. Sistemas específicos talvez também tenham outros arquivos internos que não devem ser copiados, então o programa de backup precisa ter consciência deles.

As principais vantagens da cópia física são a simplicidade e a grande velocidade (basicamente, ela pode ser executada na velocidade do disco). As principais desvantagens são a incapacidade de pular diretórios selecionados, realizar cópias incrementais e restaurar arquivos individuais mediante pedido. Por essas razões, a maioria das instalações faz cópias lógicas.

Uma **cópia lógica** começa em um ou mais diretórios especificados e recursivamente copia todos os arquivos e diretórios encontrados ali que foram modificados desde uma determinada data de base (por exemplo, o último backup para uma cópia incremental ou instalação de sistema para uma cópia completa). Assim, em uma cópia lógica, o disco da cópia recebe uma série de diretórios e arquivos cuidadosamente identificados, o que

torna fácil restaurar um arquivo ou diretório específico mediante pedido.

Tendo em vista que a cópia lógica é a forma mais usual, vamos examinar um algoritmo comum em detalhe usando o exemplo da Figura 4.25 para nos orientar. A maioria dos sistemas UNIX usa esse algoritmo. Na figura vemos uma árvore com diretórios (quadrados) e arquivos (círculos). Os itens sombreados foram modificados desde a data de base e desse modo precisam ser copiados. Os arquivos não sombreados não precisam ser copiados.

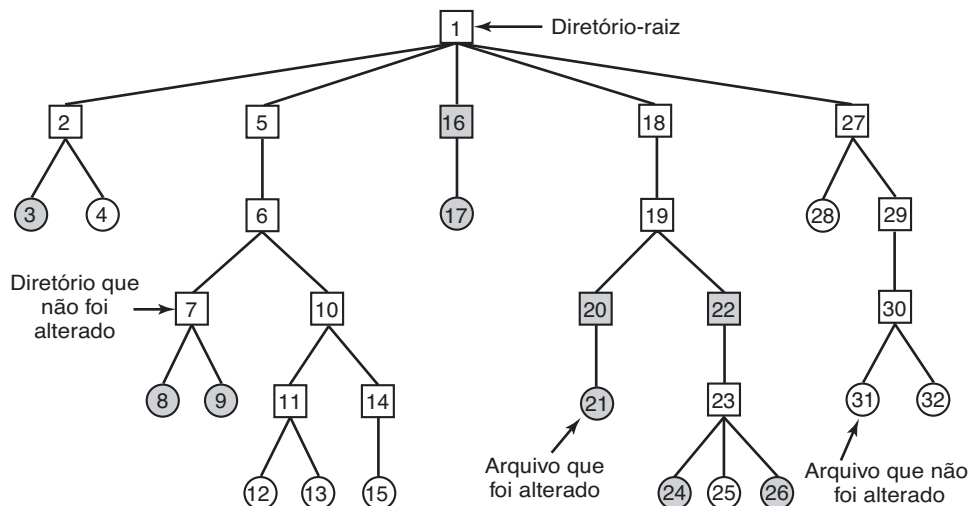
Esse algoritmo também copia todos os diretórios (mesmo os inalterados) que ficam no caminho de um arquivo ou diretório modificado por duas razões. A primeira é tornar possível restaurar os arquivos e diretórios copiados para um sistema de arquivos novos em um computador diferente. Dessa maneira, os programas de cópia e restauração podem ser usados para transportar sistemas de arquivos inteiros entre computadores.

A segunda razão para copiar diretórios inalterados que estejam acima de arquivos modificados é tornar possível restaurar de maneira incremental um único arquivo (possivelmente para recuperar alguma bobagem cometida). Suponha que uma cópia completa do sistema de arquivos seja feita no domingo à noite e uma cópia incremental seja feita segunda-feira à noite. Na terça-feira o diretório `/usr/jhs/proj/nr3` é removido, junto com todos os diretórios e arquivos sob ele. Na manhã ensolarada de quarta-feira suponha que o usuário queira restaurar o arquivo `/usr/jhs/proj/nr3/plans/summary`. No entanto, não é possível apenas restaurar o arquivo `summary` porque não há lugar para colocá-lo. Os diretórios `nr3` e `plans` devem ser restaurados primeiro. Para obter seus proprietários, modos, horários etc. corretos, esses diretórios precisam estar presentes no disco de cópia mesmo que eles mesmos não tenham sido modificados antes da cópia completa anterior.

O algoritmo de cópia mantém um mapa de bits indexado pelo número do i-node com vários bits por i-node. Bits serão definidos como 1 ou 0 nesse mapa conforme o algoritmo é executado. O algoritmo opera em quatro fases. A fase 1 começa do diretório inicial (a raiz neste exemplo) e examina todas as entradas nele. Para cada arquivo modificado, seu i-node é marcado no mapa de bits. Cada diretório também é marcado (modificado ou não) e então inspecionado recursivamente.

Ao fim da fase 1, todos os arquivos modificados e todos os diretórios foram marcados no mapa de bits, como mostrado (pelo sombreamento) na Figura 4.26(a). A fase 2 conceitualmente percorre a árvore de novo de maneira recursiva, desmarcando quaisquer diretórios

FIGURA 4.25 Um sistema de arquivos a ser copiado. Os quadrados são diretórios e os círculos, arquivos. Os itens sombreados foram modificados desde a última cópia. Cada diretório e arquivo estão identificados por seu número de i-node.



que não tenham arquivos ou diretórios modificados neles ou sob eles. Essa fase deixa o mapa de bits como mostrado na Figura 4.26(b). Observe que os diretórios 10, 11, 14, 27, 29 e 30 estão agora desmarcados, pois não contêm nada modificado sob eles. Eles não serão copiados. Por outro lado, os diretórios 5 e 6 serão copiados mesmo que não tenham sido modificados, pois serão necessários para restaurar as mudanças de hoje para uma máquina nova. Para fins de eficiência, as fases 1 e 2 podem ser combinadas para percorrer a árvore uma única vez.

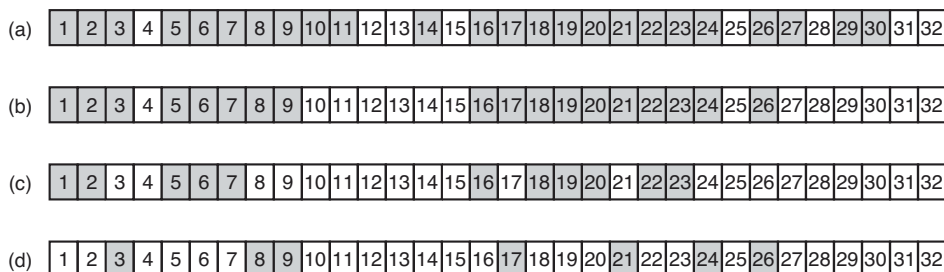
Nesse ponto, sabe-se quais diretórios e arquivos precisam ser copiados. Esses são os arquivos que estão marcados na Figura 4.26(b). A fase 3 consiste em escanear os i-nodes em ordem numérica e copiar todos os diretórios que estão marcados para serem copiados. Esses são mostrados na Figura 4.26(c). Cada diretório é prefixado pelos atributos do diretório (proprietário, horários etc.), de maneira que eles possam ser restaurados. Por fim, na fase 4, os arquivos marcados na Figura 4.26(d) também são copiados, mais uma vez prefixados por seus atributos. Isso completa a cópia.

Restaurar um sistema de arquivos a partir do disco de cópia é algo simples. Para começar, um sistema de arquivos vazio é criado no disco. Então a cópia completa mais recente é restaurada. Já que os diretórios aparecem primeiro no disco de cópia, eles são todos restaurados antes, fornecendo um esqueleto ao sistema de arquivos. Então os arquivos em si são restaurados. Esse processo é repetido com a primeira cópia incremental feita após a cópia completa, depois a seguinte e assim por diante.

Embora a cópia lógica seja simples, há algumas questões complicadas. Por exemplo, já que a lista de blocos livres não é um arquivo, ele não é copiado e assim deve ser reconstruído desde o ponto de partida depois de todas as cópias terem sido restauradas. Realizá-lo sempre é possível já que o conjunto de blocos livres é apenas o complemento do conjunto dos blocos contidos em todos os arquivos combinados.

Outra questão são as ligações. Se um arquivo está ligado a dois ou mais diretórios, é importante que seja restaurado apenas uma vez e que todos os diretórios que supostamente estejam apontando para ele assim o façam.

FIGURA 4.26 Mapas de bits usados pelo algoritmo da cópia lógica.



Ainda outra questão é o fato de que os arquivos UNIX possam conter lacunas. É permitido abrir um arquivo, escrever alguns bytes, então deslocar para uma posição mais distante e escrever mais alguns bytes. Os blocos entre eles não fazem parte do arquivo e não devem ser copiados e restaurados. Arquivos contendo a imagem de processos terminados de modo anormal (core files) apresentam muitas vezes uma lacuna de centenas de megabytes entre os segmentos de dados e a pilha. Se não for tratado adequadamente, cada core file restaurado preencherá essa área com zeros e desse modo terá o mesmo tamanho do espaço de endereço virtual (por exemplo, 2^{32} bytes, ou pior ainda, 2^{64} bytes).

Por fim, arquivos especiais, chamados pipes, e outros similares (qualquer coisa que não seja um arquivo real) jamais devem ser copiados, não importa em qual diretório eles possam ocorrer (eles não precisam estar confinados em */dev*). Para mais informações sobre backups de sistemas de arquivos, ver Chervenak et al. (1998) e Zwicky (1991).

4.4.3 Consistência do sistema de arquivos

Outra área na qual a confiabilidade é um problema é a consistência do sistema de arquivos. Muitos sistemas de arquivos leem blocos, modificam-nos e só depois os escrevem. Se o sistema cair antes de todos os blocos modificados terem sido escritos, o sistema de arquivos pode ser deixado em um estado inconsistente. O problema é especialmente crítico se alguns dos blocos que não foram escritos forem blocos de i-nodes, de diretórios ou blocos contendo a lista de blocos livres.

Para lidar com sistemas de arquivos inconsistentes, a maioria dos programas tem um programa utilitário que confere a consistência do sistema de arquivos. Por exemplo, UNIX tem *fsck*; Windows tem *sfc* (e outros). Esse utilitário pode ser executado sempre que o sistema é iniciado, especialmente após uma queda. A descrição a seguir explica como o *fsck* funciona. *Sfc* é de certa maneira diferente, pois ele funciona em um sistema de arquivos distinto, mas o princípio geral de usar a redundância inerente do sistema de arquivos para repará-lo ainda é válido. Todos os verificadores conferem cada sistema de arquivos (partição do disco) independentemente dos outros.

Dois tipos de verificações de consistência podem ser feitos: blocos e arquivos. Para conferir a consistência do bloco, o programa constrói duas tabelas, cada uma contendo um contador para cada bloco, inicialmente contendo 0. Os contadores na primeira tabela monitoram quantas vezes cada bloco está presente em

um arquivo; os contadores na segunda tabela registram quantas vezes cada bloco está presente na lista de livres (ou o mapa de bits de blocos livres).

O programa então lê todos os i-nodes usando um dispositivo cru, que ignora a estrutura de arquivos e apenas retorna todos os blocos de disco começando em 0. A partir de um i-node, é possível construir uma lista de todos os números de blocos usados no arquivo correspondente. À medida que cada número de bloco é lido, seu contador na primeira tabela é incrementado. O programa então examina a lista de livres ou mapa de bits para encontrar todos os blocos que não estão sendo usados. Cada ocorrência de um bloco na lista de livres resulta em seu contador na segunda tabela sendo incrementado.

Se o sistema de arquivos for consistente, cada bloco terá um 1 na primeira ou na segunda tabela, como ilustrado na Figura 4.27(a). No entanto, como consequência de uma queda no sistema, as tabelas podem ser parecidas com a Figura 4.27(b), na qual o bloco 2 não ocorre em nenhuma tabela. Ele será reportado como um **bloco desaparecido**. Embora blocos desaparecidos não causem nenhum prejuízo real, eles desperdiçam espaço e reduzem assim a capacidade do disco. A solução para os blocos desaparecidos é simples: o verificador do sistema de arquivos apenas os adiciona à lista de blocos livres.

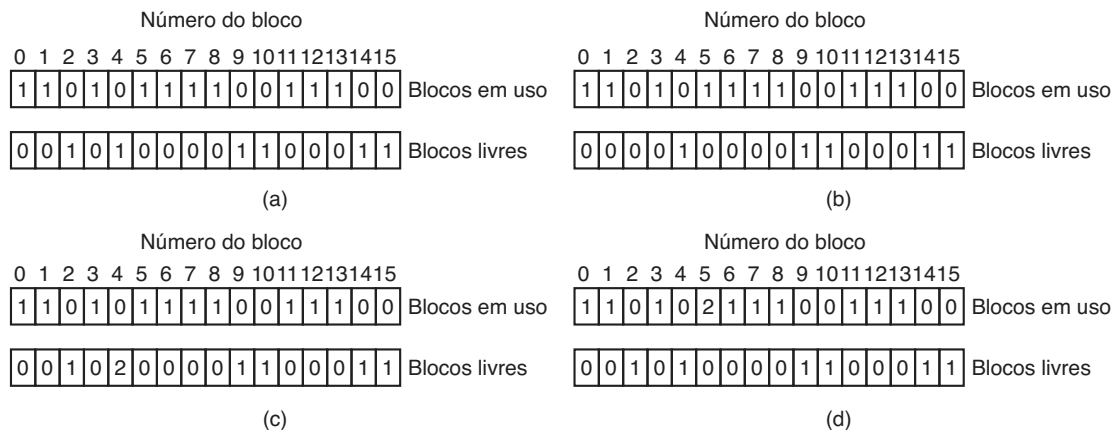
Outra situação que pode ocorrer é aquela da Figura 4.27(c). Aqui vemos um bloco, número 4, que ocorre duas vezes na lista de livres. (Duplicatas podem ocorrer apenas se a lista de livres for realmente uma lista; com um mapa de bits isso é impossível.) A solução aqui também é simples: reconstruir a lista de livres.

A pior coisa que pode ocorrer é o mesmo bloco de dados estar presente em dois ou mais arquivos, como mostrado na Figura 4.27(d) com o bloco 5. Se qualquer um desses arquivos for removido, o bloco 5 será colocado na lista de livres, levando a uma situação na qual o mesmo bloco estará ao mesmo tempo em uso e livre. Se ambos os arquivos forem removidos, o bloco será colocado na lista de livres duas vezes.

A ação apropriada para o verificador de sistema de arquivos é alocar um bloco livre, copiar os conteúdos do bloco 5 nele e inserir a cópia em um dos arquivos. Dessa maneira, o conteúdo de informação dos arquivos ficará inalterado (embora quase certamente adulterado), mas a estrutura do sistema de arquivos ao menos ficará consistente. O erro deve ser reportado, a fim de permitir que o usuário inspecione o dano.

Além de conferir para ver se cada bloco está contabilizado corretamente, o verificador do sistema de arquivos também confere o sistema de diretórios. Ele,

FIGURA 4.27 Estados do sistema de arquivos. (a) Consistente. (b) Bloco desaparecido. (c) Bloco duplicado na lista de livres. (d) Bloco de dados duplicados.



também, usa uma tabela de contadores, mas esses são por arquivo, em vez de por bloco. Ele começa no diretório-raiz e recursivamente percorre a árvore, inspecionando cada diretório no sistema de arquivos. Para cada i-node em cada diretório, ele incrementa um contador para contar o uso do arquivo. Lembre-se de que por causa de ligações estritas, um arquivo pode aparecer em dois ou mais diretórios. Ligações simbólicas não contam e não fazem que o contador incremente para o arquivo-alvo.

Quando o verificador tiver concluído, ele terá uma lista, indexada pelo número do i-node, dizendo quantos diretórios contém cada arquivo. Ele então compara esses números com as contagens de ligações armazenadas nos próprios i-nodes. Essas contagens começam em 1 quando um arquivo é criado e são incrementadas cada vez que uma ligação (estrita) é feita para o arquivo. Em um sistema de arquivos consistente, ambas as contagens concordarão. No entanto, dois tipos de erros podem ocorrer: a contagem de ligações no i-node pode ser alta demais ou baixa demais.

Se a contagem de ligações for mais alta do que o número de entradas de diretório, então mesmo que todos os arquivos sejam removidos dos diretórios, a contagem ainda será diferente de zero e o i-node não será removido. Esse erro não é sério, mas desperdiça espaço no disco com arquivos que não estão em diretório algum. Ele deve ser reparado atribuindo-se o valor correto à contagem de ligações no i-node.

O outro erro é potencialmente catastrófico. Se duas entradas de diretório estão ligadas a um arquivo, mas os i-nodes dizem que há apenas uma, quando qualquer uma das entradas de diretório for removida, a contagem do i-node irá para zero. Quando uma contagem de i-node vai para zero, o sistema de arquivos a marca como

inutilizada e libera todos os seus blocos. Essa ação resultará em um dos diretórios agora apontando para um i-node não usado, cujos blocos logo podem ser atribuídos a outros arquivos. Outra vez, a solução é apenas forçar a contagem de ligações no i-node a assumir o número real de entradas de diretório.

Essas duas operações, conferir os blocos e conferir os diretórios, muitas vezes são integradas por razões de eficiência (por exemplo, apenas uma verificação nos i-nodes é necessária). Outras verificações também são possíveis. Por exemplo, diretórios têm um formato definido, com números de i-nodes e nomes em ASCII. Se um número de i-node é maior do que o número de i-nodes no disco, o diretório foi danificado.

Além disso, cada i-node tem um modo, alguns dos quais são legais, mas estranhos, como o 0007, que possibilita ao proprietário e ao seu grupo não terem acesso a nada, mas permite que pessoas de fora leiam, escrevam e executem o arquivo. Pode ser útil ao menos reportar arquivos que dão aos usuários de fora mais direitos do que ao proprietário. Diretórios com mais de, digamos, 1.000 entradas também são suspeitos. Arquivos localizados nos diretórios de usuários, mas que são de propriedade do superusuário e que tenham o bit SETUID em 1, são problemas de segurança potenciais porque tais arquivos adquirem os poderes do superusuário quando executados por qualquer usuário. Com um pouco de esforço, é possível montar uma lista bastante longa de situações tecnicamente legais, mas peculiares, que vale a pena relatar.

Os parágrafos anteriores discutiram o problema de proteger o usuário contra quedas no sistema. Alguns sistemas de arquivos também se preocupam em proteger o usuário contra si mesmo. Se o usuário quiser digitar

`rm *.o`

para remover todos os arquivos terminando com `.o` (arquivos-objeto gerados pelo compilador), mas acidentalmente digita

```
rm *.o
```

(observe o espaço após o asterisco), `rm` removerá todos os arquivos no diretório atual e então reclamará que não pode encontrar `.o`. No Windows, os arquivos que são removidos são colocados na cesta de reciclagem (um diretório especial), do qual eles podem ser recuperados mais tarde se necessário. É claro, nenhum espaço é liberado até que eles sejam realmente removidos desse diretório.

4.4.4 Desempenho do sistema de arquivos

O acesso ao disco é muito mais lento do que o acesso à memória. Ler uma palavra de 32 bits de memória pode levar 10 ns. A leitura de um disco rígido pode chegar a 100 MB/s, o que é quatro vezes mais lento por palavra de 32 bits, mas a isso têm de ser acrescentados 5-10 ms para buscar a trilha e então esperar pelo setor desejado para chegar sob a cabeça de leitura. Se apenas uma única palavra for necessária, o acesso à memória será da ordem de um milhão de vezes mais rápido que o acesso ao disco. Como consequência dessa diferença em tempo de acesso, muitos sistemas de arquivos foram projetados com várias otimizações para melhorar o desempenho. Nesta seção cobriremos três delas.

Cache de blocos

A técnica mais comum usada para reduzir os acessos ao disco é a **cache de blocos** ou **cache de buffer**. (A palavra “cache” é pronunciada como se escreve e é derivada do verbo francês *cacher*, que significa “esconder”.) Nesse contexto, uma cache é uma coleção de blocos que logicamente pertencem ao disco, mas estão sendo mantidas na memória por razões de segurança.

Vários algoritmos podem ser usados para gerenciar a cache, mas um comum é conferir todas as solicitações para ver se o bloco necessário está na cache. Se estiver, o pedido de leitura pode ser satisfeito sem acesso ao disco. Se o bloco não estiver, primeiro ele é lido na cache e então copiado para onde quer que seja necessário. Solicitações subsequentes para o mesmo bloco podem ser satisfeitas a partir da cache.

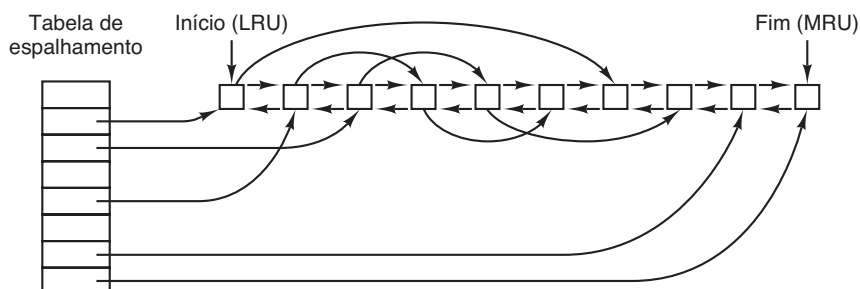
A operação da cache está ilustrada na Figura 4.28. Como há muitos (seguidamente milhares) blocos na cache, alguma maneira é necessária para determinar rapidamente se um dado bloco está presente. A maneira usual é mapear o dispositivo e endereço de disco e olhar o resultado em uma tabela de espalhamento. Todos os blocos com o mesmo valor de espalhamento são encadeados em uma lista de maneira que a cadeia de colisão possa ser seguida.

Quando um bloco tem de ser carregado em uma cache cheia, alguns blocos têm de ser removidos (e reescritos para o disco se eles foram modificados depois de trazidos para o disco). Essa situação é muito parecida com a paginação, e todos os algoritmos de substituição de páginas usuais descritos no Capítulo 3, como FIFO, segunda chance e LRU, são aplicáveis. Uma diferença bem-vinda entre a paginação e a cache de blocos é que as referências de cache são relativamente raras, de maneira que é viável manter todos os blocos na ordem exata do LRU com listas encadeadas.

Na Figura 4.28, vemos que além das colisões encadeadas da tabela de espalhamento, há também uma lista bidirecional ligando todos os blocos na ordem de uso, com o menos recentemente usado na frente dessa lista e o mais recentemente usado no fim. Quando um bloco é referenciado, ele pode ser removido da sua posição na lista bidirecional e colocado no fim. Dessa maneira, a ordem do LRU exata pode ser mantida.

Infelizmente, há um problema. Agora que temos uma situação na qual o LRU exato é possível, ele passa a ser indesejável. O problema tem a ver com as quedas no sistema e consistência do sistema de arquivos

FIGURA 4.28 As estruturas de dados da cache de buffer.



discutidas na seção anterior. Se um bloco crítico, como um bloco do i-node, é lido na cache e modificado, mas não reescrito para o disco, uma queda deixará o sistema de arquivos em estado inconsistente. Se o bloco do i-node for colocado no fim da cadeia do LRU, pode levar algum tempo até que ele chegue à frente e seja reescrito para o disco.

Além disso, alguns blocos, como blocos de i-nodes, raramente são referenciados duas vezes dentro de um intervalo curto de tempo. Essas considerações levam a um esquema de LRU modificado, tomando dois fatores em consideração:

1. É provável que o bloco seja necessário logo novamente?
2. O bloco é essencial para a consistência do sistema de arquivos?

Para ambas as questões, os blocos podem ser divididos em categorias como blocos de i-nodes, indiretos, de diretórios, de dados totalmente preenchidos e de dados parcialmente preenchidos. Blocos que provavelmente não serão necessários logo de novo irão para a frente, em vez de para o fim da lista do LRU, de maneira que seus buffers serão reutilizados rapidamente. Blocos que talvez sejam necessários logo outra vez, como o bloco parcialmente preenchido que está sendo escrito, irão para o fim da lista, de maneira que permanecerão por ali um longo tempo.

A segunda questão é independente da primeira. Se o bloco for essencial para a consistência do sistema de arquivos (basicamente, tudo exceto blocos de dados) e foi modificado, ele deve ser escrito para o disco imediatamente, não importando em qual extremidade da lista LRU será inserido. Ao escrever blocos críticos rapidamente, reduzimos muito a probabilidade de que uma queda arruine o sistema de arquivos. Embora um usuário possa ficar descontente se um de seus arquivos for arruinado em uma queda, é provável que ele fique muito mais descontente se todo o sistema de arquivos for perdido.

Mesmo com essa medida para manter intacta a integridade do sistema de arquivos, é indesejável manter blocos de dados na cache por tempo demais antes de serem escritos. Considere o drama de alguém que está usando um computador pessoal para escrever um livro. Mesmo que o nosso escritor periodicamente diga ao editor para escrever para o disco o arquivo que está sendo editado, há uma boa chance de que tudo ainda esteja na cache e nada no disco. Se o sistema cair, a estrutura do sistema de arquivos não será corrompida, mas um dia inteiro de trabalho será perdido.

Essa situação não precisa acontecer com muita frequência para que tenhamos um usuário descontente. Sistemas adotam duas abordagens para lidar com isso. A maneira UNIX é ter uma chamada de sistema, *sync*, que força todos os blocos modificados para o disco imediatamente. Quando o sistema é inicializado, um programa, normalmente chamado *update*, é inicializado no segundo plano para adentrar um laço infinito que emite chamadas *sync*, dormindo por 30 s entre chamadas. Como consequência, não mais do que 30 s de trabalho são perdidos pela quebra.

Embora o Windows tenha agora uma chamada de sistema equivalente a *sync*, chamada *FlushFileBuffers*, no passado ele não tinha. Em vez disso, ele tinha uma estratégia diferente que era, em alguns aspectos, melhor do que a abordagem do UNIX (e outros, pior). O que ele fazia era escrever cada bloco modificado para o disco tão logo ele fosse escrito para a cache. Caches nas quais todos os blocos modificados são escritos de volta para o disco imediatamente são chamadas de **caches de escrita direta (write-through caches)**. Elas exigem mais E/S de disco do que caches que não são de escrita direta.

A diferença entre essas duas abordagens pode ser vista quando um programa escreve um bloco totalmente preenchido de 1 KB, um caractere por vez. O UNIX coletará todos os caracteres na cache e escreverá o bloco uma vez a cada 30 s, ou sempre que o bloco for removido da cache. Com uma cache de escrita direta, há um acesso de disco para cada caractere escrito. É claro, a maioria dos programas trabalha com buffer interno, então eles normalmente não escrevem um caractere, mas uma linha ou unidade maior em cada chamada de sistema *write*.

Uma consequência dessa diferença na estratégia de cache é que apenas remover um disco de um sistema UNIX sem realizar *sync* quase sempre resultará em dados perdidos, e frequentemente um sistema de arquivos corrompido também. Com as caches de escrita direta não há problema algum. Essas estratégias diferentes foram escolhidas porque o UNIX foi desenvolvido em um ambiente no qual todos os discos eram rígidos e não removíveis, enquanto o primeiro sistema de arquivos Windows foi herdado do MS-DOS, que teve seu início no mundo dos discos flexíveis. Como os discos rígidos tornaram-se a norma, a abordagem UNIX, com sua eficiência melhor (mas pior confiabilidade), tornou-se a norma, e também é usada agora no Windows para discos rígidos. No entanto, o NTFS toma outras medidas (por exemplo, *journaling*) para incrementar a confiabilidade, como discutido anteriormente.

Alguns sistemas operacionais integram a cache de buffer com a cache de páginas. Isso é especialmente atraente quando arquivos mapeados na memória são aceitos. Se um arquivo é mapeado na memória, então algumas das suas páginas podem estar na memória por causa de uma paginação por demanda. Tais páginas dificilmente são diferentes dos blocos de arquivos na cache do buffer. Nesse caso, podem ser tratadas da mesma maneira, com uma cache única para ambos os blocos de arquivos e páginas.

Leitura antecipada de blocos

Uma segunda técnica para melhorar o desempenho percebido do sistema de arquivos é tentar transferir blocos para a cache antes que eles sejam necessários para aumentar a taxa de acertos. Em particular, muitos arquivos são lidos sequencialmente. Quando se pede a um sistema de arquivos para obter o bloco k em um arquivo, ele o faz, mas quando termina, faz uma breve verificação na cache para ver se o bloco $k + 1$ já está ali. Se não estiver, ele programa uma leitura para o bloco $k + 1$ na esperança de que, quando ele for necessário, já terá chegado na cache. No mínimo, ele já estará a caminho.

É claro, essa estratégia de leitura antecipada funciona apenas para arquivos que estão de fato sendo lidos sequencialmente. Se um arquivo estiver sendo acessado aleatoriamente, a leitura antecipada não ajuda. Na realidade, ela piora a situação, pois emperra a largura de banda do disco, fazendo leituras em blocos inúteis e removendo blocos potencialmente úteis da cache (e talvez emperrando mais ainda a largura de banda escrevendo os blocos de volta para o disco se eles estiverem sujos). Para ver se a leitura antecipada vale a pena ser feita, o sistema de arquivos pode monitorar os padrões de acesso para cada arquivo aberto. Por exemplo, um bit associado com cada arquivo pode monitorar se o arquivo está em “modo de acesso sequencial” ou “modo de acesso aleatório”. De início, é dado o benefício da dúvida para o arquivo e ele é colocado no modo de acesso sequencial. No entanto, sempre que uma busca é feita, o bit é removido. Se as leituras sequenciais começarem de novo, o bit é colocado em 1 novamente. Dessa maneira, o sistema de arquivos pode formular um palpite razoável sobre se ele deve ler antecipadamente ou não. Se ele cometer algum erro de vez em quando, não é um desastre, apenas um pequeno desperdício de largura de banda de disco.

Redução do movimento do braço do disco

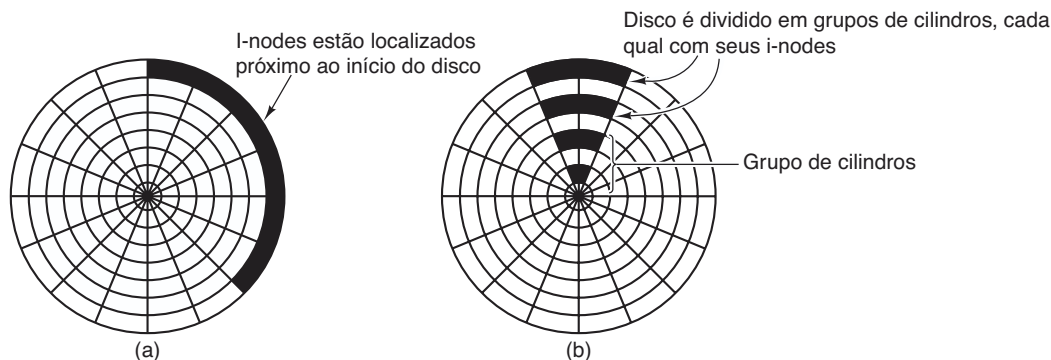
A cache e a leitura antecipada não são as únicas maneiras de incrementar o desempenho do sistema de arquivos. Outra técnica importante é reduzir o montante de movimento do braço do disco colocando blocos que têm mais chance de serem acessados em sequência próximos uns dos outros, de preferência no mesmo cilindro. Quando um arquivo de saída é escrito, o sistema de arquivos tem de alocar os blocos um de cada vez, conforme a demanda. Se os blocos livres forem registrados em um mapa de bits, e todo o mapa de bits estiver na memória principal, será bastante fácil escolher um bloco livre o mais próximo possível do bloco anterior. Com uma lista de blocos livres, na qual uma parte está no disco, é muito mais difícil alocar blocos próximos juntos.

No entanto, mesmo com uma lista de blocos livres, algum agrupamento de blocos pode ser conseguido. O truque é monitorar o armazenamento do disco, não em blocos, mas em grupos de blocos consecutivos. Se todos os setores consistirem em 512 bytes, o sistema poderia usar blocos de 1 KB (2 setores), mas alocar o armazenamento de disco em unidades de 2 blocos (4 setores). Isso não é o mesmo que ter blocos de disco de 2 KB, já que a cache ainda usaria blocos de 1 KB e as transferências de disco ainda seriam de 1 KB, mas a leitura de um arquivo sequencialmente em um sistema de outra maneira ocioso reduziria o número de buscas por um fator de dois, melhorando consideravelmente o desempenho. Uma variação sobre o mesmo tema é levar em consideração o posicionamento rotacional. Quando aloca blocos, o sistema faz uma tentativa de colocar blocos consecutivos em um arquivo no mesmo cilindro.

Outro gargalo de desempenho em sistemas que usam i-nodes (ou qualquer equivalente a eles) é que a leitura mesmo de um arquivo curto exige dois acessos de disco: um para o i-node e outro para o bloco. A localização usual do i-node é mostrada da Figura 4.29(a). Aqui todos os i-nodes estão próximos do início do disco, então a distância média entre um i-node e seus blocos será metade do número de cilindros, exigindo longas buscas.

Uma melhora simples de desempenho é colocar os i-nodes no meio do disco, em vez de no início, reduzindo assim a busca média entre o i-node e o primeiro bloco por um fator de dois. Outra ideia, mostrada na Figura 4.29(b), é dividir o disco em grupos de cilindros, cada um com seus próprios i-nodes, blocos e lista de livres (MCKUSICK et al., 1984). Ao criar um arquivo novo, qualquer i-node pode ser escolhido, mas uma tentativa é feita para encontrar um bloco no

FIGURA 4.29 (a) I-nodes posicionados no início do disco. (b) Disco dividido em grupos de cilindros, cada um com seus próprios blocos e i-nodes.



mesmo grupo de cilindros que o i-node. Se nenhum estiver disponível, então um bloco em um grupo de cilindros próximo é usado.

É claro, o movimento do braço do disco e o tempo de rotação são relevantes somente se o disco os tem. Mais e mais computadores vêm equipados com **discos de estado sólido (SSDs — Solid State Disks)** que não têm parte móvel alguma. Para esses discos, construídos com a mesma tecnologia dos flash cards, acessos aleatórios são tão rápidos quanto os sequenciais e muitos dos problemas dos discos tradicionais deixam de existir. Infelizmente, surgem novos problemas. Por exemplo, SSDs têm propriedades peculiares em suas operações de leitura, escrita e remoção. Em particular, cada bloco pode ser escrito apenas um número limitado de vezes, portanto um grande cuidado é tomado para dispersar uniformemente o desgaste sobre o disco.

4.4.5 Desfragmentação de disco

Quando o sistema operacional é inicialmente instalado, os programas e os arquivos que ele precisa são instalados de modo consecutivo começando no início do disco, cada um seguindo diretamente o anterior. Todo o espaço de disco livre está em uma única unidade contígua seguindo os arquivos instalados. No entanto, à medida que o tempo passa, arquivos são criados e removidos, e o disco prejudica-se com a fragmentação, com arquivos e espaços vazios por toda parte. Em consequência, quando um novo arquivo é criado, os blocos usados para isso podem estar espalhados por todo o disco, resultando em um desempenho ruim.

O desempenho pode ser restaurado movendo os arquivos a fim de deixá-los contíguos e colocando todo (ou pelo menos a maior parte) o espaço livre em uma

ou mais regiões contíguas no disco. O Windows tem um programa, *defrag*, que faz precisamente isso. Os usuários do Windows devem executá-lo com regularidade, exceto em SSDs.

A desfragmentação funciona melhor em sistemas de arquivos que têm bastante espaço livre em uma região contígua ao fim da partição. Esse espaço permite que o programa de desfragmentação selecione arquivos fragmentados próximos do início da partição e copie todos os seus blocos para o espaço livre. Fazê-lo libera um bloco contíguo de espaço próximo do início da partição na qual o original ou outros arquivos podem ser colocados contiguamente. O processo pode então ser repetido com o próximo pedaço de espaço de disco etc.

Alguns arquivos não podem ser movidos, incluindo o arquivo de paginação, o arquivo de hibernação e o log de journaling, pois a administração que seria necessária para fazê-lo daria mais trabalho do que seu valor. Em alguns sistemas, essas áreas são contíguas e de tamanho fixo de qualquer maneira, então elas não precisam ser desfragmentadas. O único momento em que sua falta de mobilidade é um problema é quando elas estão localizadas próximas do fim da partição e o usuário quer reduzir o tamanho da partição. A única maneira de solucionar esse problema é removê-las completamente, redimensionar a partição e então recriá-las depois.

Os sistemas de arquivos Linux (especialmente ext2 e ext3) geralmente sofrem menos com a desfragmentação do que os sistemas Windows pela maneira que os blocos de discos são selecionados, então a desfragmentação manual raramente é exigida. Também, os SSDs não sofrem de maneira alguma com a fragmentação. Na realidade, desfragmentar um SSD é contraproducente. Não apenas não há ganho em desempenho, como os SSDs se desgastam; portanto, desfragmentá-los apenas encurta sua vida.

4.5 Exemplos de sistemas de arquivos

Nas próximas seções discutiremos vários exemplos de sistemas de arquivos, desde os bastante simples aos mais sofisticados. Como os sistemas de arquivos UNIX modernos e o sistema de arquivos nativo do Windows 8 são cobertos no capítulo sobre o UNIX (Capítulo 10) e no capítulo sobre o Windows 8 (Capítulo 11), não os cobriremos aqui. Examinaremos, no entanto, seus predecessores a seguir.

4.5.1 O sistema de arquivos do MS-DOS

O sistema de arquivos do MS-DOS é o sistema com o qual os primeiros PCs da IBM vinham instalados. Foi o principal sistema de arquivos até o Windows 98 e o Windows ME. Ainda é aceito no Windows 2000, Windows XP e Windows Vista, embora não seja mais padrão nos novos PCs exceto para discos flexíveis. No entanto, ele e uma extensão dele (FAT-32) tornaram-se amplamente usados para muitos sistemas embarcados. Muitos players de MP3 o usam exclusivamente, além de câmeras digitais. O popular iPod da Apple o usa como o sistema de arquivos padrão, embora hackers que conhecem do assunto consigam reformatar o iPod e instalar um sistema de arquivos diferente. Desse modo, o número de dispositivos eletrônicos usando o sistema de arquivos MS-DOS é muito maior agora do que em qualquer momento no passado e, decerto, muito maior do que o número usando o sistema de arquivos NTFS mais moderno. Por essa razão somente, vale a pena examiná-lo em detalhe.

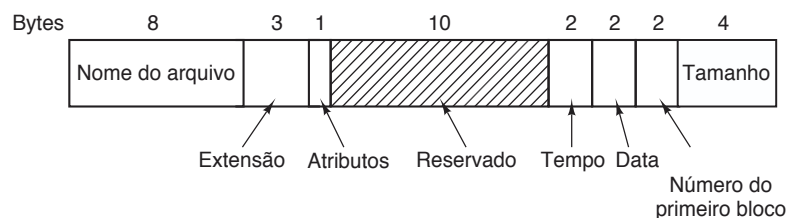
Para ler um arquivo, um programa de MS-DOS deve primeiro fazer uma chamada de sistema `open` para abri-lo. A chamada de sistema `open` especifica um caminho, o qual pode ser absoluto ou relativo ao diretório de trabalho atual. O caminho é analisado componente a componente até que o diretório final seja localizado e lido na memória. Ele então é vasculhado para encontrar o arquivo a ser aberto.

Embora os diretórios de MS-DOS tenham tamanhos variáveis, eles usam uma entrada de diretório de

32 bytes de tamanho fixo. O formato de uma entrada de diretório de MS-DOS é mostrado na Figura 4.30. Ele contém o nome do arquivo, atributos, data e horário de criação, bloco de partida e tamanho exato do arquivo. Nomes de arquivos mais curtos do que 8 + 3 caracteres são ajustados à esquerda e preenchidos com espaços à direita, separadamente em cada campo. O campo *Atributos* é novo e contém bits para indicar que um arquivo é somente de leitura, precisa ser arquivado, está escondido ou é um arquivo de sistema. Arquivos somente de leitura não podem ser escritos. Isso é para protegê-los de danos acidentais. O bit arquivado não tem uma função de sistema operacional real (isto é, o MS-DOS não o examina ou o altera). A intenção é permitir que programas de arquivos em nível de usuário o desliguem quando efetuarem o backup de um arquivo e que os outros programas o liguem quando modificarem um arquivo. Dessa maneira, um programa de backup pode apenas examinar o bit desse atributo em cada arquivo para ver quais arquivos devem ser copiados. O bit oculto pode ser alterado para evitar que um arquivo apareça nas listagens de diretório. Seu uso principal é evitar confundir usuários novatos com arquivos que eles possam não compreender. Por fim, o bit sistema também oculta arquivos. Além disso, arquivos de sistema não podem ser removidos acidentalmente usando o comando `del`. Os principais componentes do MS-DOS têm esse bit ligado.

A entrada de diretório também contém a data e o horário em que o arquivo foi criado ou modificado pela última vez. O tempo é preciso apenas até ± 2 segundos, pois ele está armazenado em um campo de 2 bytes, que pode armazenar somente 65.536 valores únicos (um dia contém 86.400 segundos). O campo do tempo é subdividido em segundos (5 bits), minutos (6 bits) e horas (5 bits). A data conta em dias usando três subcampos: dia (5 bits), mês (4 bits) e ano — 1980 (7 bits). Com um número de 7 bits para o ano e o tempo começando em 1980, o maior valor que pode ser representado é 2107. Então, o MS-DOS tem um problema Y2108 em si. Para evitar a catástrofe, seus usuários devem atentar para esse problema o mais cedo possível. Se o MS-DOS tivesse

FIGURA 4.30 A entrada de diretório do MS-DOS.



usado os campos data e horário combinados como um contador de 32 bits, ele teria representado cada segundo exatamente e atrasado a catástrofe até 2116.

O MS-DOS armazena o tamanho do arquivo como um número de 32 bits, portanto na teoria os arquivos podem ser de até 4 GB. No entanto, outros limites (descritos a seguir) restringem o tamanho máximo do arquivo a 2 GB ou menos. Uma parte surpreendentemente grande da entrada (10 bytes) não é usada.

O MS-DOS monitora os blocos de arquivos mediante uma tabela de alocação de arquivos na memória principal. A entrada do diretório contém o número do primeiro bloco de arquivos. Esse número é usado como um índice em uma FAT de 64 K entradas na memória principal. Seguindo o encadeamento, todos os blocos podem ser encontrados. A operação da FAT está ilustrada na Figura 4.12.

O sistema de arquivos FAT vem em três versões: FAT-12, FAT-16 e FAT-32, dependendo de quantos bits um endereço de disco contém. Na realidade, FAT-32 não é um nome adequado, já que apenas os 28 bits menos significativos dos endereços de disco são usados. Ele deveria chamar-se FAT-28, mas as potências de dois soam bem melhor.

Outra variante do sistema de arquivos FAT é o exFAT, que a Microsoft introduziu para dispositivos removíveis grandes. A Apple licenciou o exFAT, de maneira que há um sistema de arquivos moderno que pode ser usado para transferir arquivos entre computadores Windows e OS X. Como o exFAT é de propriedade da Microsoft e a empresa não liberou a especificação, não o discutiremos mais aqui.

Para todas as FATs, o bloco de disco pode ser alterado para algum múltiplo de 512 bytes (possivelmente diferente para cada partição), com o conjunto de tamanhos de blocos permitidos (chamado **cluster sizes** — tamanhos de aglomerado — pela Microsoft) sendo diferente para cada variante. A primeira versão do MS-DOS usava a FAT-12 com blocos de 512 bytes, dando um tamanho de partição máximo de $2^{12} \times 512$ bytes (na realidade somente 4086×512 bytes, pois 10 dos endereços de disco foram usados como marcadores especiais, como fim de arquivo, bloco defeituoso etc.). Com esses parâmetros, o tamanho de partição de disco máximo era em torno de 2 MB e o tamanho da tabela FAT na memória era de 4096 entradas de 2 bytes cada. Usar uma entrada de tabela de 12 bits teria sido lento demais.

Esse sistema funcionava bem para discos flexíveis, mas quando os discos rígidos foram lançados, ele tornou-se um problema. A Microsoft solucionou o problema permitindo tamanhos de blocos adicionais de 1 KB,

2 KB e 4 KB. Essa mudança preservou a estrutura e o tamanho da tabela FAT-12, mas permitiu partições de disco de até 16 MB.

Como o MS-DOS dava suporte para quatro partições de disco por unidade de disco, o novo sistema de arquivos FAT-12 funcionava para discos de até 64 MB. Além disso, algo tinha de ceder. O que aconteceu foi a introdução do FAT-16, com ponteiros de disco de 16 bits. Adicionalmente, tamanhos de blocos de 8 KB, 16 KB e 32 KB foram permitidos. (32.768 é a maior potência de dois que pode ser representada em 16 bits.) A tabela FAT-16 ocupava 128 KB de memória principal o tempo inteiro, mas com as memórias maiores então disponíveis, ela era amplamente usada e logo substituiu o sistema de arquivos FAT-12. A maior partição de disco a que o FAT-16 pode dar suporte é 2 GB (64 K entradas de 32 KB cada) e o maior disco, 8 GB, a saber quatro partições de 2 GB cada. Por um bom tempo, isso foi o suficiente.

Mas não para sempre. Para cartas comerciais, esse limite não é um problema, mas para armazenar vídeos digitais usando o padrão DV, um arquivo de 2 GB contém apenas um pouco mais de 9 minutos de vídeo. Como um disco de PC suporta apenas quatro partições, o maior vídeo que pode ser armazenado em disco é de mais ou menos 38 minutos, não importa o tamanho do disco. Esse limite também significa que o maior vídeo que pode ser editado on-line é de menos de 19 minutos, pois ambos os arquivos de entrada e saída são necessários.

A partir da segunda versão do Windows 95, foi introduzido o sistema de arquivos FAT-32 com seus endereços de disco de 28 bits e a versão do MS-DOS subjacente ao Windows 95 foi adaptada para dar suporte à FAT-32. Nesse sistema, as partições poderiam ser teoricamente $2^{28} \times 2^{15}$ bytes, mas na realidade elas eram limitadas a 2 TB (2048 GB), pois internamente o sistema monitora os tamanhos de partições em setores de 512 bytes usando um número de 32 bits, e $2^9 \times 2^{32}$ é 2 TB. O tamanho máximo da partição para vários tamanhos de blocos e todos os três tipos FAT é mostrado na Figura 4.31.

Além de dar suporte a discos maiores, o sistema de arquivos FAT-32 tem duas outras vantagens sobre o FAT-16. Primeiro, um disco de 8 GB usando FAT-32 pode ter uma única partição. Usando o FAT-16 ele tem quatro partições, o que aparece para o usuário do Windows como *C:*, *D:*, *E:* e *F:* unidades de disco lógicas. Cabe ao usuário decidir qual arquivo colocar em qual unidade e monitorar o que está onde.

A outra vantagem do FAT-32 sobre o FAT-16 é que para um determinado tamanho de partição de disco, um

FIGURA 4.31 Tamanho máximo da partição para diferentes tamanhos de blocos. As caixas vazias representam combinações proibidas.

Tamanho do bloco	FAT-12	FAT-16	FAT-32
0,5 KB	2 MB		
1 KB	4 MB		
2 KB	8 MB	128 MB	
4 KB	16 MB	256 MB	1 TB
8 KB		512 MB	2 TB
16 KB		1024 MB	2 TB
32 KB		2048 MB	2 TB

tamanho de bloco menor pode ser usado. Por exemplo, para uma partição de disco de 2 GB, o FAT-16 deve usar blocos de 32 KB; de outra maneira, com apenas 64K endereços de disco disponíveis, ele não pode cobrir toda a partição. Em comparação, o FAT-32 pode usar, por exemplo, blocos de 4 KB para uma partição de disco de 2 GB. A vantagem de um tamanho de bloco menor é que a maioria dos arquivos é muito mais curta do que 32 KB. Se o tamanho do bloco for 32 KB, um arquivo de 10 bytes imobiliza 32 KB de espaço de disco. Se o arquivo médio for, digamos, 8 KB, então com um bloco de 32 KB, três quartos do disco serão desperdiçados, o que não é uma maneira muito eficiente de se usar o disco. Com um arquivo de 8 KB e um bloco de 4 KB, não há desperdício de disco, mas o preço pago é mais RAM consumida pelo FAT. Com um bloco de 4 KB e uma partição de disco de 2 GB, há 512 K blocos, de maneira que o FAT precisa ter 512K entradas na memória (ocupando 2 MB de RAM).

O MS-DOS usa a FAT para monitorar blocos de disco livres. Qualquer bloco que no momento não esteja alocado é marcado com um código especial. Quando o MS-DOS precisa de um novo bloco de disco, ele pesquisa a FAT para uma entrada contendo esse código. Desse modo, não são necessários nenhum mapa de bits ou lista de livres.

4.5.2 O sistema de arquivos do UNIX V7

Mesmo as primeiras versões do UNIX tinham um sistema de arquivos multiusuário bastante sofisticado, já que era derivado do MULTICS. A seguir discutiremos o sistema de arquivos V7, aquele do PDP-11 que tornou o UNIX famoso. Examinaremos um sistema de arquivos UNIX moderno no contexto do Linux no Capítulo 10.

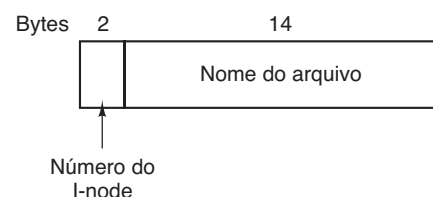
O sistema de arquivos tem forma de uma árvore começando no diretório-raiz, com a adição de ligações, formando um gráfico orientado acíclico. Nomes de arquivos podem ter até 14 caracteres e contêm quaisquer caracteres ASCII exceto / (porque se trata de um separador entre componentes em um caminho) e NUL (pois é usado para preencher os espaços que sobram nos nomes mais curtos que 14 caracteres). NUL tem o valor numérico de 0.

Uma entrada de diretório UNIX contém uma entrada para cada arquivo naquele diretório. Cada entrada é extremamente simples, pois o UNIX usa o esquema i-node ilustrado na Figura 4.13. Uma entrada de diretório contém apenas dois campos: o nome do arquivo (14 bytes) e o número do i-node para aquele arquivo (2 bytes), como mostrado na Figura 4.32. Esses parâmetros limitam o número de arquivos por sistema a 64 K.

Assim como o i-node da Figura 4.13, o i-node do UNIX contém alguns atributos. Os atributos contêm o tamanho do arquivo, três horários (criação, último acesso e última modificação), proprietário, grupo, informação de proteção e uma contagem do número de entradas de diretórios que apontam para o i-node. Este último campo é necessário para as ligações. Sempre que uma ligação nova é feita para um i-node, o contador no i-node é incrementado. Quando uma ligação é removida, o contador é decrementado. Quando ele chega a 0, o i-node é reivindicado e os blocos de disco são colocados de volta na lista de livres.

O monitoramento dos blocos de disco é feito usando uma generalização da Figura 4.13 a fim de lidar com arquivos muito grandes. Os primeiros 10 endereços de disco são armazenados no próprio i-node, então para pequenos arquivos, todas as informações necessárias estão diretamente no i-node, que é buscado do disco para a memória principal quando o arquivo é aberto. Para arquivos um pouco maiores, um dos endereços no i-node é o de um bloco de disco chamado **bloco indireto simples**. Esse bloco contém endereços de disco adicionais. Se isso ainda não for suficiente, outro endereço no i-node, chamado **bloco indireto duplo**, contém o endereço de um bloco com uma lista de blocos indiretos

FIGURA 4.32 Uma entrada do diretório do UNIX V7.



simples. Cada um desses blocos indiretos simples aponta para algumas centenas de blocos de dados. Se mesmo isso não for suficiente, um **bloco indireto triplo** também pode ser usado. O quadro completo é dado na Figura 4.33.

Quando um arquivo é aberto, o sistema deve tomar o nome do arquivo fornecido e localizar seus blocos de disco. Vamos considerar como o nome do caminho `/usr/ast/mbox` é procurado. Usaremos o UNIX como exemplo, mas o algoritmo é basicamente o mesmo para todos os sistemas de diretórios hierárquicos. Primeiro, o sistema de arquivos localiza o diretório-raiz. No UNIX o seu i-node está localizado em um local fixo no disco. A partir desse i-node, ele localiza o diretório-raiz, que pode estar em qualquer lugar, mas digamos bloco 1.

Em seguida, ele lê o diretório-raiz e procura o primeiro componente do caminho, `usr`, no diretório-raiz para encontrar o número do i-node do arquivo `/usr`. Localizar um i-node a partir desse número é algo direto, já que cada i-node tem uma localização fixa no disco. A partir desse i-node, o sistema localiza o diretório para `/usr` e pesquisa o componente seguinte, `ast`, nele. Quando encontrar a entrada para `ast`, ele terá o i-node para o diretório `/usr/ast`. A partir desse i-node ele pode fazer uma busca no próprio diretório e localizar `mbox`. O i-node para esse arquivo é então lido na memória e mantido ali até o arquivo ser fechado. O processo de busca está ilustrado na Figura 4.34.

Nomes de caminhos relativos são procurados da mesma maneira que os absolutos, apenas partindo do diretório de trabalho em vez de do diretório-raiz.

Todo diretório tem entradas para `.` e `..` que são colocadas ali quando o diretório é criado. A entrada `.` tem o número de i-node para o diretório atual, e a entrada para `..` tem o número de i-node para o diretório pai. Desse modo, uma rotina procurando `../dick/prog.c` apenas procura `..` no diretório de trabalho, encontra o número de i-node do diretório pai e busca pelo diretório `dick`. Nenhum mecanismo especial é necessário para lidar com esses nomes. No que diz respeito ao sistema de diretórios, eles são apenas cadeias ASCII comuns, como qualquer outro nome. A única questão a ser observada aqui é que `..` no diretório-raiz aponta para si mesmo.

4.5.3 Sistemas de arquivos para CD-ROM

Como nosso último exemplo de um sistema de arquivos, vamos considerar aqueles usados nos CD-ROMs. Eles são particularmente simples, pois foram projetados para meios de escrita única. Entre outras coisas, por exemplo, eles não têm provisão para monitorar blocos livres, pois em um arquivo de CD-ROM arquivos não podem ser liberados ou adicionados após o disco ter sido fabricado. A seguir examinaremos o principal tipo de sistema de arquivos para CD-ROMs e duas de suas extensões. Embora os CD-ROMs estejam ultrapassados, eles são também simples, e os sistemas de arquivos usados em DVDs e Blu-ray são baseados nos usados para CD-ROMs.

FIGURA 4.33 Um i-node do UNIX.

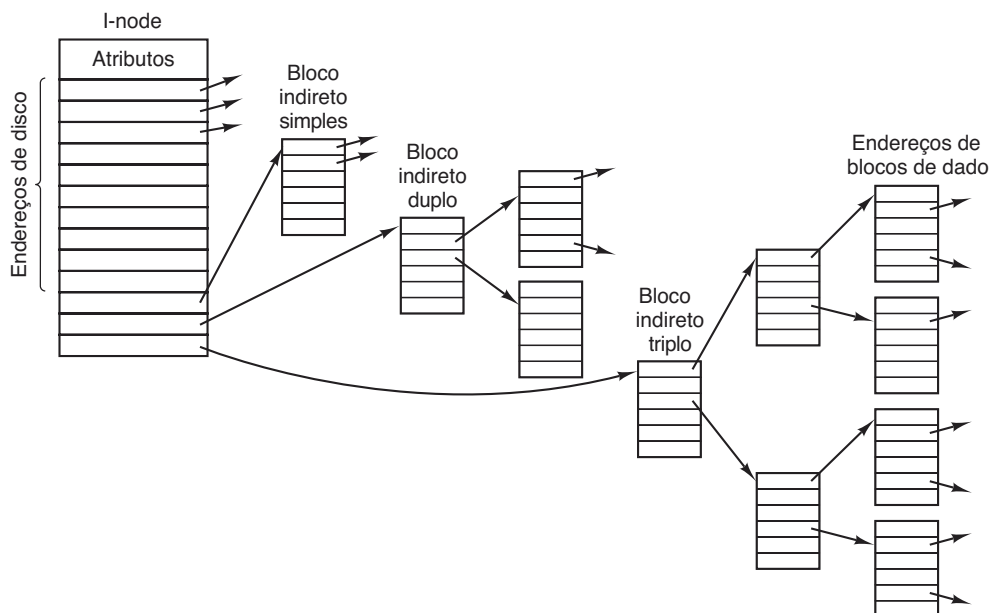
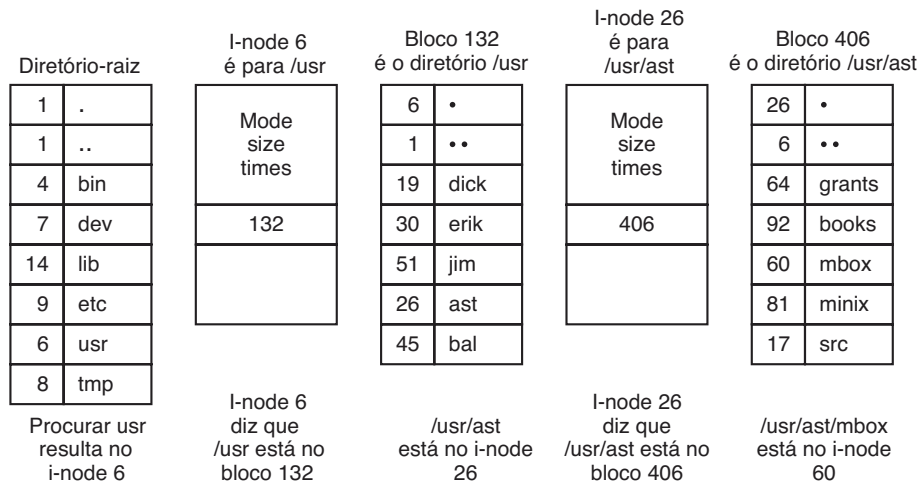


FIGURA 4.34 Os passos para pesquisar em `/usr/ast/mbox`.

Alguns anos após o CD-ROM ter feito sua estreia, foi introduzido o CD-R (**CD Recordable** — CD gravável). Diferentemente do CD-ROM, ele permite adicionar arquivos após a primeira gravação, que são apenas adicionados ao final do CD-R. Arquivos nunca são removidos (embora o diretório possa ser atualizado para esconder arquivos existentes). Como consequência desse sistema de arquivos “somente adicionar”, as propriedades fundamentais não são alteradas. Em particular, todo o espaço livre encontra-se em uma única parte contígua no fim do CD.

O sistema de arquivos ISO 9660

O padrão mais comum para sistemas de arquivos de CD-ROM foi adotado como um Padrão Internacional em 1988 sob o nome **ISO 9660**. Virtualmente, todo CD-ROM no mercado é compatível com esse padrão, às vezes com extensões a serem discutidas a seguir. Uma meta desse padrão era tornar todo CD-ROM legível em todos os computadores, independente do ordenamento de bytes e do sistema operacional usado. Em consequência, algumas limitações foram aplicadas ao sistema de arquivos para possibilitar que os sistemas operacionais mais fracos (como MS-DOS) pudessem lê-los.

Os CD-ROMs não têm cilindros concêntricos como os discos magnéticos. Em vez disso, há uma única espiral contínua contendo os bits em uma sequência linear (embora seja possível buscar transversalmente às espirais). Os bits ao longo da espiral são divididos em blocos lógicos (também chamados setores lógicos) de 2352 bytes. Alguns desses bytes são para preâmbulos, correção de erros e outros destinos. A porção líquida (payload) de cada bloco lógico é 2048 bytes. Quando usados para música,

os CDs têm as posições iniciais, finais e espaços entre as trilhas, mas eles não são usados para CD-ROMs de dados. Muitas vezes a posição de um bloco ao longo da espiral é representada em minutos e segundos. Ela pode ser convertida para um número de bloco linear usando o fator de conversão de 1 s = 75 blocos.

O ISO 9660 dá suporte a conjuntos de CD-ROM com até $2^{16} - 1$ CDs no conjunto. Os CD-ROMs individuais também podem ser divididos em volumes lógicos (partições). No entanto, a seguir, nos concentraremos no ISO 9660 para um único CD-ROM não particionado.

Todo CD-ROM começa com 16 blocos cuja função não é definida pelo padrão ISO 9660. Um fabricante de CD-ROMs poderia usar essa área para oferecer um programa de inicialização que permitisse que o computador fosse inicializado pelo CD-ROM, ou para algum outro propósito nefando. Em seguida vem um bloco contendo o **descriptor de volume primário**, que contém algumas informações gerais sobre o CD-ROM. Entre essas informações estão o identificador do sistema (32 bytes), o identificador do volume (32 bytes), o identificador do editor (128 bytes) e o identificador do preparador dos dados (128 bytes). O fabricante pode preencher esses campos da maneira que quiser, exceto que somente letras maiúsculas, dígitos e um número muito pequeno de caracteres de pontuação podem ser usados para assegurar a compatibilidade entre as plataformas.

O descriptor do volume primário também contém os nomes de três arquivos, que podem conter um resumo, uma notificação de direitos autorais e informações bibliográficas, respectivamente. Além disso, determinados números-chave também estão presentes, incluindo o tamanho do bloco lógico (normalmente 2048, mas

4096, 8192 e valores maiores de potências de 2 são permitidos em alguns casos), o número de blocos no CD-ROM e as datas de criação e expiração do CD-ROM. Por fim, o descritor do volume primário também contém uma entrada de diretório para o diretório-raiz, dizendo onde encontrá-lo no CD-ROM (isto é, em qual bloco ele começa). A partir desse diretório, o resto do sistema de arquivos pode ser localizado.

Além do descritor de volume primário, um CD-ROM pode conter um descritor de volume complementar. Ele contém informações similares ao descritor primário, mas não abordaremos essa questão aqui.

O diretório-raiz, e todos os outros diretórios, quanto a isso, consistem em um número variável de entradas, a última das quais contém um bit marcando-a como a entrada final. As entradas de diretório em si também são de tamanhos variáveis. Cada entrada de diretório consiste em 10 a 12 campos, dos quais alguns são em ASCII e outros são numéricos binários. Os campos binários são codificados duas vezes, uma com os bits menos significativos nos primeiros bytes — little-endian (usados nos Pentiums, por exemplo) e outra com os bits mais significativos nos primeiros bytes — big endian (usados nas SPARCs, por exemplo). Desse modo, um número de 16 bits usa 4 bytes e um número de 32 bits usa 8 bytes.

O uso dessa codificação redundante era necessário para evitar ferir os sentimentos alheios quando o padrão foi desenvolvido. Se o padrão tivesse estabelecido little-endian, então as pessoas de empresas cujos produtos eram big-endian se sentiriam desvalorizadas e não teriam aceitado o padrão. O conteúdo emocional de um CD-ROM pode, portanto, ser quantificado e mensurado exatamente em quilobytes/hora de espaço desperdiçado.

O formato de uma entrada de diretório ISO 9660 está ilustrado na Figura 4.35. Como entradas de diretório têm comprimentos variáveis, o primeiro campo é um byte indicando o tamanho da entrada. Esse byte é definido com o bit de ordem mais alta à esquerda para evitar qualquer ambiguidade.

Entradas de diretório podem opcionalmente ter atributos estendidos. Se essa prioridade for usada, o segundo byte indicará o tamanho dos atributos estendidos.

Em seguida vem o bloco inicial do próprio arquivo. Arquivos são armazenados como sequências contíguas de blocos, assim a localização de um arquivo é completamente especificada pelo bloco inicial e o tamanho, que está contido no próximo campo.

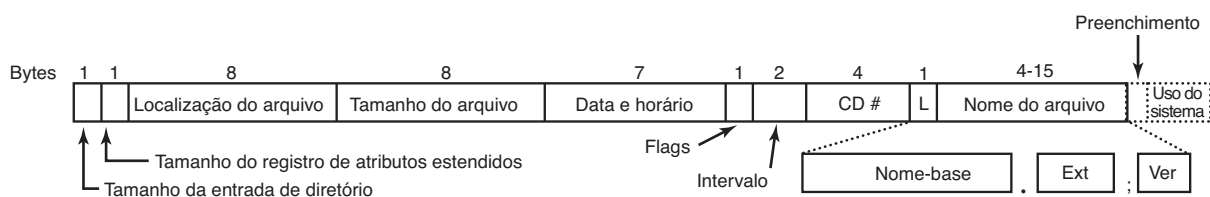
A data e o horário em que o CD-ROM foi gravado estão armazenados no próximo campo, com bytes separados para o ano, mês, dia, hora, minuto, segundo e zona do fuso horário. Os anos começam a contar em 1900, o que significa que os CD-ROMs sofrerão de um problema Y2156, pois o ano seguinte a 2155 será 1900. Esse problema poderia ter sido postergado com a definição da origem do tempo em 1988 (o ano que o padrão foi adotado). Se isso tivesse ocorrido, o problema teria sido postergado até 2244. Cada 88 anos extras ajuda.

O campo *Flags* contém alguns bits diversos, incluindo um para ocultar a entrada nas listagens (um atributo copiado do MS-DOS), um para distinguir uma entrada que é um arquivo de uma entrada que é um diretório, um para capacitar o uso dos atributos estendidos e um para marcar a última entrada em um diretório. Alguns outros bits também estão presentes nesse campo, mas não os abordaremos aqui. O próximo campo lida com a intercalação de partes de arquivos de uma maneira que não é usada na versão mais simples do ISO 9660, portanto não nos aprofundaremos nela.

O campo a seguir diz em qual CD-ROM o arquivo está localizado. É permitido que uma entrada de diretório em um CD-ROM refira-se a um arquivo localizado em outro CD-ROM no conjunto. Dessa maneira, é possível construir um diretório-mestre no primeiro CD-ROM que liste todos os arquivos que estejam em todos os CD-ROMs no conjunto completo.

O campo marcado *L* na Figura 4.35 mostra o tamanho do nome do arquivo em bytes. Ele é seguido pelo nome do próprio arquivo. Um nome de arquivo consiste em um nome base, um ponto, uma extensão, um ponto e vírgula e um número binário de versão (1 ou 2 bytes). O nome base e a extensão podem usar letras maiúsculas, os dígitos 0-9 e o caractere sublinhado. Todos os outros caracteres são proibidos para certificar-se de que todos os computadores possam lidar com todos os nomes de

FIGURA 4.35 A entrada de diretório do ISO 9660.



arquivos. O nome base pode ter até oito caracteres; a extensão até três caracteres. Essas escolhas foram ditadas pela necessidade de tornar o padrão compatível com o MS-DOS. Um nome de arquivo pode estar presente em um diretório múltiplas vezes, desde que cada um tenha um número de versão diferente.

Os últimos dois campos nem sempre estão presentes. O campo *Preenchimento* é usado para forçar que toda entrada de diretório seja um número par de bytes a fim de alinhar os campos numéricos de entradas subsequentes em limites de 2 bytes. Se o preenchimento for necessário, um byte 0 é usado. Por fim, temos o campo *Uso do sistema*. Sua função e tamanho são indefinidos, exceto que ele deve conter um número par de bytes. Sistemas diferentes o utilizam de maneiras diferentes. O Macintosh, por exemplo, mantém os flags do Finder nele.

Entradas dentro de um diretório são listadas em ordem alfabética, exceto para as duas primeiras entradas. A primeira entrada é para o próprio diretório. A segunda é para o pai. Nesse sentido, elas são similares para as entradas de diretório `.` e `..` do UNIX. Os arquivos em si não precisam estar na ordem do diretório.

Não há um limite explícito para o número de entradas em um diretório. No entanto, há um limite para a profundidade de aninhamento. A profundidade máxima de aninhamento de um diretório é oito. Esse limite foi estabelecido arbitrariamente para simplificar algumas implementações.

O ISO 9660 define o que são chamados de três níveis. O nível 1 é o mais restritivo e especifica que os nomes de arquivos sejam limitados a 8 + 3 caracteres como descrevemos, e também exige que todos os arquivos sejam contíguos como descrevemos. Além disso, ele especifica que os nomes dos diretórios sejam limitados a oito caracteres sem extensões. O uso desse nível maximiza as chances de que um CD-ROM seja lido em todos os computadores.

O nível 2 relaxa a restrição de tamanho. Ele permite que arquivos e diretórios tenham nomes de até 31 caracteres, mas ainda do mesmo conjunto de caracteres.

O nível 3 usa os mesmos limites de nomes do nível 2, mas relaxa parcialmente o pressuposto de que os arquivos tenham de ser contíguos. Com esse nível, um arquivo pode consistir em várias seções (extensões), cada uma como uma sequência contígua de blocos. A mesma sequência pode ocorrer múltiplas vezes em um arquivo e pode ocorrer em dois ou mais arquivos. Se grandes porções de dados são repetidas em vários arquivos, o nível 3 oferecerá alguma otimização de espaço ao não exigir que os dados estejam presentes múltiplas vezes.

Extensões Rock Ridge

Como vimos, o ISO 9660 é altamente restritivo em várias maneiras. Logo depois do seu lançamento, as pessoas na comunidade UNIX começaram a trabalhar em uma extensão a fim de tornar possível representar os sistemas de arquivos UNIX em um CD-ROM. Essas extensões foram chamadas de **Rock Ridge**, em homenagem à cidade no filme de Mel Brooks, *Banzé no Oeste (Blazing Saddles)*, provavelmente porque um dos membros do comitê gostou do filme.

As extensões usam o campo *Uso do sistema* para possibilitar a leitura dos CD-ROMs Rock Ridge em qualquer computador. Todos os outros campos mantêm seu significado ISO 9660 normal. Qualquer sistema que não conheça as extensões Rock Ridge apenas as ignora e vê um CD-ROM normal.

As extensões são divididas nos campos a seguir:

1. PX — atributos POSIX.
2. PN — Números de dispositivo principal e secundário.
3. SL — Ligação simbólica.
4. NM — Nome alternativo.
5. CL — Localização do filho.
6. PL — Localização do pai.
7. RE — Realocação.
8. TF — Estampas de tempo (Time stamps).

O campo *PX* contém o padrão UNIX para bits de permissão `rw-rw-rw` para o proprietário, grupo e outros. Ele também contém os outros bits contidos na palavra de modo, como os bits SETUID e SETGID, e assim por diante.

Para permitir que dispositivos sejam representados em um CD-ROM, o campo *PN* está presente. Ele contém os números de dispositivos principais e secundários associados com o arquivo. Dessa maneira, os conteúdos do diretório `/dev` podem ser escritos para um CD-ROM e mais tarde reconstruídos corretamente no sistema de destino.

O campo *SL* é para ligações simbólicas. Ele permite que um arquivo em um sistema refira-se a um arquivo em um sistema diferente.

O campo mais importante é *NM*. Ele permite que um segundo nome seja associado com o arquivo. Esse nome não está sujeito às restrições de tamanho e conjunto de caracteres do ISO 9660, tornando possível expressar nomes de arquivos UNIX arbitrários em um CD-ROM.

Os três campos seguintes são usados juntos para contornar o limite de oito diretórios que podem ser aninhados no ISO 9660. Usando-os é possível especificar que um diretório seja realocado, e dizer onde ele vai

na hierarquia. Trata-se efetivamente de uma maneira de contornar o limite de profundidade artificial.

Por fim, o campo *TF* contém as três estampas de tempo incluídas em cada i-node UNIX, a saber o horário que o arquivo foi criado, modificado e acessado pela última vez. Juntas, essas extensões tornam possível copiar um sistema de arquivos UNIX para um CD-ROM e então restaurá-lo corretamente para um sistema diferente.

Extensões Joliet

A comunidade UNIX não foi o único grupo que não gostou do ISO 9660 e queria uma maneira de estendê-lo. A Microsoft também o achou restritivo demais (embora tenha sido o próprio MS-DOS da Microsoft que causou a maior parte das restrições em primeiro lugar). Portanto, a Microsoft inventou algumas extensões chamadas **Joliet**. Elas foram projetadas para permitir que os sistemas de arquivos Windows fossem copiados para um CD-ROM e então restaurados, precisamente da mesma maneira que o Rock Ridge foi projetado para o UNIX. Virtualmente todos os programas que executam sob o Windows e usam CD-ROMs aceitam Joliet, incluindo programas que gravam em CDs regraváveis. Em geral, esses programas oferecem uma escolha entre os vários níveis de ISO 9660 e Joliet.

As principais extensões oferecidas pelo Joliet são:

1. Nomes de arquivos longos.
2. Conjunto de caracteres Unicode.
3. Aninhamento de diretórios mais profundo que oito níveis.
4. Nomes de diretórios com extensões.

A primeira extensão permite nomes de arquivos de até 64 caracteres. A segunda extensão capacita o uso do conjunto de caracteres Unicode para os nomes de arquivos. Essa extensão é importante para que o software possa ser empregado em países que não usam o alfabeto latino, como Japão, Israel e Grécia. Como os caracteres Unicode ocupam 2 bytes, o nome de arquivo máximo em Joliet ocupa 128 bytes.

Assim como no Rock Ridge, a limitação sobre aninhamentos de diretórios foi removida no Joliet. Os diretórios podem ser aninhados o mais profundamente

quanto necessário. Por fim, nomes de diretórios podem ter extensões. Não fica claro por que essa extensão foi incluída, já que os diretórios do Windows virtualmente nunca usam extensões, mas talvez um dia venham a usar.

4.6 Pesquisas em sistemas de arquivos

Os sistemas de arquivos sempre atraíram mais pesquisas do que outras partes do sistema operacional e até hoje é assim. Conferências inteiras como FAST, MSST e NAS são devotadas em grande parte a sistemas de arquivos e armazenamento. Embora os sistemas de arquivos-padrão sejam bem compreendidos, ainda há bastante pesquisa sendo feita sobre backups (SMALDONE et al., 2013; e WALLACE et al., 2012), cache (KOLLER et al.; Oh, 2012; e ZHANG et al., 2013a), exclusão de dados com segurança (WEI et al., 2011), compressão de arquivos (HARNIK et al., 2013), sistemas de arquivos para dispositivos flash (NO, 2012; PARK e SHEN, 2012; e NARAYANAN, 2009), desempenho (LEVENTHAL, 2013; e SCHINDLER et al., 2011), RAID (MOON e REDDY, 2013), confiabilidade e recuperação de erros (CHIDAMBARAM et al., 2013; MA et al., 2013; MCKUSICK, 2012; e VAN MOOLENBROEK et al., 2012), sistemas de arquivos em nível do usuário (RAJGARHIA e GEHANI, 2010), verificações de consistência (FRYER et al., 2012) e sistemas de arquivos com controle de versões (MASHTIZADEH et al., 2013). Apenas mensurar o que está realmente acontecendo em um sistema de arquivos também é um tópico de pesquisa (HARTER et al., 2012).

A segurança é um tópico sempre presente (BOTE-LHO et al., 2013; LI et al., 2013c; e LORCH et al., 2013). Por outro lado, um novo tópico refere-se aos sistemas de arquivos na nuvem (MAZUREK et al., 2012; e VRABLE et al., 2012). Outra área que tem ganhado atenção recentemente é a procedência — o monitoramento da história dos dados, incluindo de onde vêm, quem é o proprietário e como eles foram transformados (GHOSHAL e PLALE, 2013; e SULTANA e BERTINO, 2013). Manter os dados seguros e úteis por décadas também interessa às empresas que têm um compromisso legal de fazê-lo (BAKER et al., 2006). Por fim, outros pesquisadores estão repensando a pilha do sistema de arquivos (APPUSWAMY et al., 2011).

4.7 Resumo

Quando visto de fora, um sistema operacional é uma coleção de arquivos e diretórios, mais as operações

sobre eles. Arquivos podem ser lidos e escritos, diretórios criados e destruídos e arquivos podem ser movidos

de diretório para diretório. A maioria dos sistemas de arquivos modernos dá suporte a um sistema de diretórios hierárquico no qual os diretórios podem ter subdiretórios, e estes podem ter “subsubdiretórios” *ad infinitum*.

Quando visto de dentro, um sistema de arquivos parece bastante diferente. Os projetistas do sistema de arquivos precisam estar preocupados com como o armazenamento é alocado e como o sistema monitora qual bloco vai com qual arquivo. As possibilidades incluem arquivos contíguos, listas encadeadas, tabelas de alocação de arquivos e i-nodes. Sistemas diferentes têm estruturas de diretórios diferentes. Os atributos podem ficar nos diretórios ou em algum outro lugar (por exemplo, um i-node). O espaço de disco pode ser gerenciado usando listas de espaços livres ou mapas de bits. A

confiabilidade do sistema de arquivos é reforçada a partir da realização de cópias incrementais e um programa que possa reparar sistemas de arquivos danificados. O desempenho do sistema de arquivos é importante e pode ser incrementado de diversas maneiras, incluindo cache de blocos, leitura antecipada e a colocação cuidadosa de um arquivo próximo do outro. Sistemas de arquivos estruturados em diário (log) também melhoram o desempenho fazendo escritas em grandes unidades.

Exemplos de sistemas de arquivos incluem ISO 9660, MS-DOS e UNIX. Eles diferem de muitas maneiras, incluindo pelo modo de monitorar quais blocos vão para quais arquivos, estrutura de diretórios e gerenciamento de espaço livre em disco.

PROBLEMAS

1. Dê cinco nomes de caminhos diferentes para o arquivo `/etc./passwd`. (Dica: lembre-se das entradas de diretório “.” e “..”).
2. No Windows, quando um usuário clica duas vezes sobre um arquivo listado pelo Windows Explorer, um programa é executado e dado aquele arquivo como parâmetro. Liste duas maneiras diferentes através das quais o sistema operacional poderia saber qual programa executar.
3. Nos primeiros sistemas UNIX, os arquivos executáveis (arquivos *a.out*) começavam com um número mágico, bem específico, não um número escolhido ao acaso. Esses arquivos começavam com um cabeçalho, seguido por segmentos de texto e dados. Por que você acha que um número bem específico era escolhido para os arquivos executáveis, enquanto os outros tipos de arquivos tinham um número mágico mais ou menos aleatório como primeiro caractere?
4. A chamada de sistema `open` no UNIX é absolutamente essencial? Quais seriam as consequências de não a ter?
5. Sistemas que dão suporte a arquivos sequenciais sempre têm uma operação para voltar arquivos para trás (rewind). Os sistemas que dão suporte a arquivos de acesso aleatório precisam disso, também?
6. Alguns sistemas operacionais fornecem uma chamada de sistema `rename` para dar um nome novo para um arquivo. Existe alguma diferença entre usar essa chamada para renomear um arquivo e apenas copiar esse arquivo para um novo com o nome novo, seguido pela remoção do antigo?
7. Em alguns sistemas é possível mapear parte de um arquivo na memória. Quais restrições esses sistemas precisam impor? Como é implementado esse mapeamento parcial?
8. Um sistema operacional simples dá suporte a apenas um único diretório, mas permite que ele tenha nomes arbitrariamente longos de arquivos. Seria possível simular algo próximo de um sistema de arquivos hierárquico? Como?
9. No UNIX e no Windows, o acesso aleatório é realizado por uma chamada de sistema especial que move o ponteiro “posição atual” associado com um arquivo para um determinado byte nele. Proponha uma forma alternativa para o acesso aleatório sem ter essa chamada de sistema.
10. Considere a árvore de diretório da Figura 4.8. Se `/usr/jim` é o diretório de trabalho, qual é o nome de caminho absoluto para o arquivo cujo nome de caminho relativo é `../ast/x`?
11. A alocação contígua de arquivos leva à fragmentação de disco, como mencionado no texto, pois algum espaço no último bloco de disco será desperdiçado em arquivos cujo tamanho não é um número inteiro de blocos. Estamos falando de uma fragmentação interna, ou externa? Faça uma analogia com algo discutido no capítulo anterior.
12. Descreva os efeitos de um bloco de dados corrompido para um determinado arquivo: (a) contíguo, (b) encadeado e (c) indexado (ou baseado em tabela).
13. Uma maneira de usar a alocação contígua do disco e não sofrer com espaços livres é compactar o disco toda vez que um arquivo for removido. Já que todos os arquivos são contíguos, copiar um arquivo exige uma busca e atraso rotacional para lê-lo, seguido pela transferência em velocidade máxima. Escrever um arquivo de volta exige o mesmo trabalho. Presumindo um tempo de busca de 5 ms, um atraso rotacional de 4 ms, uma taxa de

- transferência de 80 MB/s e o tamanho de arquivo médio de 8 KB, quanto tempo leva para ler um arquivo para a memória principal e então escrevê-lo de volta para o disco na nova localização? Usando esses números, quanto tempo levaria para compactar metade de um disco de 16 GB?
14. Levando em conta a resposta da pergunta anterior, a compactação do disco faz algum sentido?
 15. Alguns dispositivos de consumo digitais precisam armazenar dados, por exemplo, como arquivos. Cite um dispositivo moderno que exija o armazenamento de arquivos e para o qual a alocação contígua seria uma boa ideia.
 16. Considere o i-node mostrado na Figura 4.13. Se ele contém 10 endereços diretos e esses tinham 8 bytes cada e todos os blocos do disco eram de 1024 KB, qual seria o tamanho do maior arquivo possível?
 17. Para uma determinada turma, os históricos dos estudantes são armazenados em um arquivo. Os registros são acessados aleatoriamente e atualizados. Presuma que o histórico de cada estudante seja de um tamanho fixo. Qual dos três esquemas de alocação (contíguo, encadeado e indexado por tabela) será o mais apropriado?
 18. Considere um arquivo cujo tamanho varia entre 4 KB e 4 MB durante seu tempo de vida. Qual dos três esquemas de alocação (contíguo, encadeado e indexado por tabela) será o mais apropriado?
 19. Foi sugerido que a eficiência poderia ser incrementada e o espaço de disco poupado armazenando os dados de um arquivo curto dentro do i-node. Para o i-node da Figura 4.13, quantos bytes de dados poderiam ser armazenados dentro dele?
 20. Duas estudantes de computação, Carolyn e Elinor, estão tendo uma discussão sobre i-nodes. Carolyn sustenta que as memórias ficaram tão grandes e baratas que, quando um arquivo é aberto, é mais simples e mais rápido buscar uma cópia nova do i-node na tabela de i-nodes, em vez de procurar na tabela inteira para ver se ela já está ali. Elinor discorda. Quem está certa?
 21. Nomeie uma vantagem de ligações estritas sobre ligações simbólicas e uma vantagem de ligações simbólicas sobre ligações estritas.
 22. Explique como as ligações estritas e as ligações flexíveis diferem em relação às alocações de i-nodes.
 23. Considere um disco de 4 TB que usa blocos de 4 KB e o método da lista de livres. Quantos endereços de blocos podem ser armazenados em um bloco?
 24. O espaço de disco livre pode ser monitorado usando-se uma lista de livres e um mapa de bits. Endereços de disco exigem D bits. Para um disco com B blocos, F dos quais estão disponíveis, estabeleça a condição na qual a lista de livres usa menos espaço do que o mapa de bits. Para D tendo um valor de 16 bits, expresse a resposta como uma porcentagem do espaço de disco que precisa estar livre.
 25. O começo de um mapa de bits de espaço livre fica assim após a partição de disco ter sido formatada pela primeira vez: 1000 0000 0000 0000 (o primeiro bloco é usado pelo diretório-raiz). O sistema sempre busca por blocos livres começando no bloco de número mais baixo, então após escrever o arquivo A , que usa seis blocos, o mapa de bits fica assim: 1111 1110 0000 0000. Mostre o mapa de bits após cada uma das ações a seguir:
 - (a) O arquivo B é escrito usando cinco blocos.
 - (b) O arquivo A é removido.
 - (c) O arquivo C é escrito usando oito blocos.
 - (d) O arquivo B é removido.
 26. O que aconteceria se o mapa de bits ou a lista de livres contendo as informações sobre blocos de disco livres fossem perdidos por uma queda no computador? Existe alguma maneira de recuperar-se desse desastre ou é “adeus, disco”? Discuta suas respostas para os sistemas de arquivos UNIX e FAT-16 separadamente.
 27. O trabalho noturno de Oliver Owl no centro de computadores da universidade é mudar as fitas usadas para os backups de dados durante a noite. Enquanto espera que cada fita termine, ele trabalha em sua tese que prova que as peças de Shakespeare foram escritas por visitantes extraterrestres. Seu processador de texto executa no sistema sendo copiado, pois esse é o único que eles têm. Há algum problema com esse arranjo?
 28. Discutimos como realizar cópias incrementais detalhadamente no texto. No Windows é fácil dizer quando copiar um arquivo, pois todo arquivo tem um bit de arquivamento. Esse bit não existe no UNIX. Como os programas de backup do UNIX sabem quais arquivos copiar?
 29. Suponha que o arquivo 21 na Figura 4.25 não foi modificado desde a última cópia. De qual maneira os quatro mapas de bits da Figura 4.26 seriam diferentes?
 30. Foi sugerido que a primeira parte de cada arquivo UNIX fosse mantida no mesmo bloco de disco que o seu i-node. Qual a vantagem que isso traria?
 31. Considere a Figura 4.27. Seria possível que, para algum número de bloco em particular, os contadores em *ambas* as listas tivessem o valor 2? Como esse problema poderia ser corrigido?
 32. O desempenho de um sistema de arquivos depende da taxa de acertos da cache (fração de blocos encontrados na cache). Se for necessário 1 ms para satisfazer uma solicitação da cache, mas 40 ms para satisfazer uma solicitação se uma leitura de disco for necessária, dê uma fórmula para o tempo médio necessário para satisfazer uma solicitação se a taxa de acerto é h . Represente graficamente essa função para os valores de h variando de 0 a 1,0.

33. Para um disco rígido USB externo ligado a um computador, o que é mais adequado: uma cache de escrita direta ou uma cache de bloco?
34. Considere uma aplicação em que os históricos dos estudantes são armazenados em um arquivo. A aplicação pega a identidade de um estudante como entrada e subsequentemente lê, atualiza e escreve o histórico correspondente; isso é repetido até a aplicação desistir. A técnica de “leitura antecipada de bloco” seria útil aqui?
35. Considere um disco que tem 10 blocos de dados começando do bloco 14 até o 23. Deixe 2 arquivos no disco: *f1* e *f2*. A estrutura do diretório lista que os primeiros blocos de dados de *f1* e *f2* são respectivamente 22 e 16. Levando-se em consideração as entradas de tabela FAT a seguir, quais são os blocos de dados designados para *f1* e *f2*?
(14,18); (15,17); (16,23); (17,21); (18,20); (19,15); (20, -1); (21, -1); (22,19); (23,14).
Nessa notação, (x, y) indicam que o valor armazenado na entrada de tabela *x* aponta para o bloco de dados *y*.
36. Considere a ideia por trás da Figura 4.21, mas agora para um disco com um tempo de busca médio de 6 ms, uma taxa rotacional de 15.000 rpm e 1.048.576 bytes por trilha. Quais são as taxas de dados para os tamanhos de blocos de 1 KB, 2 KB e 4 KB, respectivamente?
37. Um determinado sistema de arquivos usa blocos de disco de 4 KB. O tamanho de arquivo médio é 1 KB. Se todos os arquivos fossem exatamente de 1 KB, qual fração do espaço do disco seria desperdiçada? Você acredita que o desperdício para um sistema de arquivos real será mais alto do que esse número ou mais baixo do que ele? Explique sua resposta.
38. Levando-se em conta um tamanho de bloco de 4 KB e um valor de endereço de ponteiro de disco de 4 bytes, qual é o maior tamanho de arquivo (em bytes) que pode ser acessado usando 10 endereços diretos e um bloco indireto?
39. Arquivos no MS-DOS têm de competir por espaço na tabela FAT-16 na memória. Se um arquivo usa *k* entradas, isto é, *k* entradas que não estão disponíveis para qualquer outro arquivo, qual restrição isso ocasiona sobre o tamanho total de todos os arquivos combinados?
40. Um sistema de arquivos UNIX tem blocos de 4 KB e endereços de disco de 4 bytes. Qual é o tamanho de arquivo máximo se os i-nodes contêm 10 entradas diretas, e uma entrada indireta única, dupla e tripla cada?
41. Quantas operações de disco são necessárias para buscar o i-node para um arquivo com o nome de caminho */usr/ast/courses/os/handout.t*? Presuma que o i-node para o diretório-raiz está na memória, mas nada mais ao longo do caminho está na memória. Também presuma que todo diretório caiba em um bloco de disco.
42. Em muitos sistemas UNIX, os i-nodes são mantidos no início do disco. Um projeto alternativo é alocar um i-node quando um arquivo é criado e colocar o i-node no começo do primeiro bloco do arquivo. Discuta os prós e contras dessa alternativa.
43. Escreva um programa que inverta os bytes de um arquivo, para que o último byte seja agora o primeiro e o primeiro, o último. Ele deve funcionar com um arquivo arbitrariamente longo, mas tente torná-lo razoavelmente eficiente.
44. Escreva um programa que comece em um determinado diretório e percorra a árvore de arquivos a partir daquele ponto registrando os tamanhos de todos os arquivos que encontrar. Quando houver concluído, ele deve imprimir um histograma dos tamanhos dos arquivos usando uma largura de célula especificada como parâmetro (por exemplo, com 1024, tamanhos de arquivos de 0 a 1023 são colocados em uma célula, 1024 a 2047 na seguinte etc.).
45. Escreva um programa que escaneie todos os diretórios em um sistema de arquivos UNIX e encontre e localize todos os i-nodes com uma contagem de ligações estritas de duas ou mais. Para cada arquivo desses, ele lista juntos todos os nomes que apontam para o arquivo.
46. Escreva uma nova versão do programa *ls* do UNIX. Essa versão recebe como argumentos um ou mais nomes de diretórios e para cada diretório lista todos os arquivos nele, uma linha por arquivo. Cada campo deve ser formatado de uma maneira razoável considerando o seu tipo. Liste apenas o primeiro endereço de disco, se houver.
47. Implemente um programa para mensurar o impacto de tamanhos de buffer no nível de aplicação nos tempos de leitura. Isso consiste em ler para e escrever a partir de um grande arquivo (digamos, 2 GB). Varie o tamanho do buffer de aplicação (digamos, de 64 bytes para 4 KB). Use rotinas de mensuração de tempo (como *gettimeofday* e *getitimer* no UNIX) para mensurar o tempo levado por diferentes tamanhos de buffers. Analise os resultados e relate seus achados: o tamanho do buffer faz uma diferença para o tempo de escrita total e tempo por escrita?
48. Implemente um sistema de arquivos simulado que será completamente contido em um único arquivo regular armazenado no disco. Esse arquivo de disco conterá diretórios, i-nodes, informações de blocos livres, blocos de dados de arquivos etc. Escolha algoritmos adequados para manter informações sobre blocos livres e para alocar blocos de dados (contíguos, indexados, encadeados). Seu programa aceitará comandos de sistema do usuário para criar/remover diretórios, criar/remover/abrir arquivos, ler/escrever de/para um arquivo selecionado e listar conteúdos de diretórios.



Além de oferecer abstrações como processos, espaços de endereçamentos e arquivos, um sistema operacional também controla todos os dispositivos de E/S (entrada/saída) do computador. Ele deve emitir comandos para os dispositivos, interceptar interrupções e lidar com erros. Também deve fornecer uma interface entre os dispositivos e o resto do sistema que seja simples e fácil de usar. Na medida do possível, a interface deve ser a mesma para todos os dispositivos (independentemente do dispositivo). O código de E/S representa uma fração significativa do sistema operacional total. Como o sistema operacional gerencia a E/S é o assunto deste capítulo.

Este capítulo é organizado da seguinte forma: examinaremos primeiro alguns princípios do hardware de E/S e então o software de E/S em geral. O software de E/S pode ser estruturado em camadas, com cada uma tendo uma tarefa bem definida. Examinaremos cada uma para ver o que fazem e como se relacionam entre si.

Em seguida, exploraremos vários dispositivos de E/S detalhadamente: discos, relógios, teclados e monitores. Para cada dispositivo, examinaremos o seu hardware e software. Por fim, estudaremos o gerenciamento de energia.

5.1 Princípios do hardware de E/S

Diferentes pessoas veem o hardware de E/S de maneiras diferentes. Engenheiros elétricos o veem em termos de chips, cabos, motores, suprimento de energia e todos os outros componentes físicos que compreendem o hardware. Programadores olham para a

interface apresentada ao software — os comandos que o hardware aceita, as funções que ele realiza e os erros que podem ser reportados de volta. Neste livro, estamos interessados na programação de dispositivos de E/S, e não em seu projeto, construção ou manutenção; portanto, nosso interesse é saber como o hardware é programado, não como ele funciona por dentro. Não obstante isso, a programação de muitos dispositivos de E/S está muitas vezes intimamente ligada à sua operação interna. Nas próximas três seções, apresentaremos uma pequena visão geral sobre hardwares de E/S à medida que estes se relacionam com a programação. Podemos considerar isso como uma revisão e expansão do material introdutório da Seção 1.3.

5.1.1 Dispositivos de E/S

Dispositivos de E/S podem ser divididos de modo geral em duas categorias: **dispositivos de blocos** e **dispositivos de caractere**. O primeiro armazena informações em blocos de tamanho fixo, cada um com seu próprio endereço. Tamanhos de blocos comuns variam de 512 a 65.536 bytes. Todas as transferências são em unidades de um ou mais blocos inteiros (consecutivos). A propriedade essencial de um dispositivo de bloco é que cada bloco pode ser lido ou escrito independentemente de todos os outros. Discos rígidos, discos Blu-ray e pendrives são dispositivos de bloco comuns.

Se você observar bem de perto, o limite entre dispositivos que são endereçáveis por blocos e aqueles que não são não é bem definido. Todo mundo concorda que um disco é um dispositivo endereçável por bloco, pois

não importa a posição em que se encontra o braço no momento, é sempre possível buscar em outro cilindro e então esperar que o bloco solicitado gire sob a cabeça. Agora considere um velho dispositivo de fita magnética ainda usado, às vezes, para realizar backups de disco (porque fitas são baratas). Fitas contêm uma sequência de blocos. Se o dispositivo de fita receber um comando para ler o bloco N , ele sempre pode rebobiná-la e ir direto até chegar ao bloco N . Essa operação é análoga a um disco realizando uma busca, exceto por levar muito mais tempo. Também pode ou não ser possível reescrever um bloco no meio de uma fita. Mesmo que fosse possível usar as fitas como dispositivos de bloco com acesso aleatório, isso seria forçar o ponto de algum modo: em geral elas não são usadas dessa maneira.

O outro dispositivo de E/S é o de caractere. Um dispositivo de caractere envia ou aceita um fluxo de caracteres, desconsiderando qualquer estrutura de bloco. Ele não é endereçável e não tem qualquer operação de busca. Impressoras, interfaces de rede, mouses (para apontar), ratos (para experimentos de psicologia em laboratórios) e a maioria dos outros dispositivos que não são parecidos com discos podem ser vistos como dispositivos de caracteres.

Esse esquema de classificação não é perfeito. Alguns dispositivos não se enquadram nele. Relógios, por exemplo, não são endereçáveis por blocos. Tampouco geram ou aceitam fluxos de caracteres. Tudo o que fazem é causar interrupções em intervalos bem definidos. As telas

mapeadas na memória não se enquadram bem no modelo também. Tampouco as telas de toque, quanto a isso. Ainda assim, o modelo de dispositivos de blocos e de caractere é suficientemente geral para ser usado como uma base para fazer alguns dos softwares do sistema operacional que tratam de E/S independentes dos dispositivos. O sistema de arquivos, por exemplo, lida apenas com dispositivos de blocos abstratos e deixa a parte dependente de dispositivos para softwares de nível mais baixo.

Dispositivos de E/S cobrem uma ampla gama de velocidades, o que coloca uma pressão considerável sobre o software para desempenhar bem através de muitas ordens de magnitude em taxas de transferência de dados. A Figura 5.1 mostra as taxas de dados de alguns dispositivos comuns. A maioria desses dispositivos tende a ficar mais rápida com o passar do tempo.

5.1.2 Controladores de dispositivos

Unidades de E/S consistem, em geral, de um componente mecânico e um componente eletrônico. É possível separar as duas porções para permitir um projeto mais modular e geral. O componente eletrônico é chamado de **controlador do dispositivo** ou **adaptador**. Em computadores pessoais, ele muitas vezes assume a forma de um chip na placa-mãe ou um cartão de circuito impresso que pode ser inserido em um slot de expansão (PCIe). O componente mecânico é o dispositivo em si. O arranjo é mostrado na Figura 1.6.

FIGURA 5.1 Algumas taxas de dados típicas de dispositivos, placas de redes e barramentos.

Dispositivo	Taxa de dados
Teclado	10 bytes/s
Mouse	100 bytes/s
Modem 56 K	7 KB/s
Scanner em 300 dpi	1 MB/s
Filmadora <i>camcorder</i> digital	3,5 MB/s
Disco Blu-ray 4x	18 MB/s
Wireless 802.11n	37,5 MB/s
USB 2.0	60 MB/s
FireWire 800	100 MB/s
Gigabit Ethernet	125 MB/s
Drive de disco SATA 3	600 MB/s
USB 3.0	625 MB/s
Barramento SCSI Ultra 5	640 MB/s
Barramento de faixa única PCIe 3.0	985 MB/s
Barramento Thunderbolt2	2,5 GB/s
Rede SONET OC-768	5 GB/s

O cartão controlador costuma ter um conector, no qual um cabo levando ao dispositivo em si pode ser conectado. Muitos controladores podem lidar com dois, quatro ou mesmo oito dispositivos idênticos. Se a interface entre o controlador e o dispositivo for padrão, seja um padrão oficial ANSI, IEEE ou ISO — ou um padrão *de facto* —, então as empresas podem produzir controladores ou dispositivos que se enquadrem àquela interface. Muitas empresas, por exemplo, produzem controladores de disco compatíveis com as interfaces SATA, SCSI, USB, Thunderbolt ou FireWire (IEEE 1394).

A interface entre o controlador e o dispositivo muitas vezes é de nível muito baixo. Um disco, por exemplo, pode ser formatado com 2 milhões de setores de 512 bytes por trilha. No entanto, o que realmente sai da unidade é um fluxo serial de bits, começando com um **preâmbulo**, então os 4096 bits em um setor, e por fim uma soma de verificação (checksum), ou **código de correção de erro (ECC — Error Correcting Code)**. O preâmbulo é escrito quando o disco é formatado e contém o cilindro e número de setor, e dados similares, assim como informações de sincronização.

O trabalho do controlador é converter o fluxo serial de bits em um bloco de bytes, assim como realizar qualquer correção de erros necessária. O bloco de bytes é tipicamente montado primeiro, bit por bit, em um buffer dentro do controlador. Após a sua soma de verificação ter sido verificada e o bloco ter sido declarado livre de erros, ele pode então ser copiado para a memória principal.

O controlador para um monitor LCD também funciona um pouco como um dispositivo serial de bits em um nível igualmente baixo. Ele lê os bytes contendo os caracteres a serem exibidos da memória e gera os sinais para modificar a polarização da retroiluminação para os pixels correspondentes a fim de escrevê-los na tela. Se não fosse pelo controlador de tela, o programador do sistema operacional teria de programar explicitamente os campos elétricos de todos os pixels. Com o controlador, o sistema operacional o inicializa com alguns parâmetros, como o número de caracteres ou pixels por linha e o número de linhas por tela, e deixa o controlador cuidar realmente da orientação dos campos elétricos.

Em um tempo muito curto, telas de LCD substituíram completamente os velhos monitores **CRT (Cathode Ray Tube — Tubo de Raios Catódicos)**. Monitores CRT disparam um feixe de elétrons em uma tela fluorescente. Usando campos magnéticos, o sistema é capaz

de curvar o feixe e atrair pixels sobre a tela. Comparado com as telas de LCD, os monitores CRT eram grandalhões, gastadores de energia e frágeis. Além disso, a resolução das telas de LCD (Retina) de hoje é tão boa que o olho humano é incapaz de distinguir pixels individuais. É difícil de imaginar que os laptops no passado vinham com uma pequena tela CRT que os deixava com mais de 20 cm de fundo e um peso de aproximadamente 12 quilos.

5.1.3 E/S mapeada na memória

Cada controlador tem alguns registradores que são usados para comunicar-se com a CPU. Ao escrever nesses registradores, o sistema operacional pode comandar o dispositivo a fornecer e aceitar dados, ligar-se e desligar-se, ou de outra maneira realizar alguma ação. Ao ler a partir desses registradores, o sistema operacional pode descobrir qual é o estado do dispositivo, se ele está preparado para aceitar um novo comando e assim por diante.

Além dos registradores de controle, muitos dispositivos têm um buffer de dados a partir do qual o sistema operacional pode ler e escrever. Por exemplo, uma maneira comum para os computadores exibirem pixels na tela é ter uma RAM de vídeo, que é basicamente apenas um buffer de dados, disponível para ser escrita pelos programas ou sistema operacional.

A questão que surge então é como a CPU se comunica com os registradores de controle e também com os buffers de dados do dispositivo. Existem duas alternativas. Na primeira abordagem, para cada registrador de controle é designado um número de **porta de E/S**, um inteiro de 8 ou 16 bits. O conjunto de todas as portas de E/S formam o **espaço de E/S**, que é protegido de maneira que programas de usuário comuns não consigam acessá-lo (apenas o sistema operacional). Usando uma instrução de E/S especial como

IN REG,PORT,

a CPU pode ler o registrador de controle PORT e armazenar o resultado no registrador de CPU REG. Similarmente, usando

OUT PORT,REG

a CPU pode escrever o conteúdo de REG para um controlador de registro. A maioria dos primeiros computadores, incluindo quase todos os de grande porte, como o IBM 360 e todos os seus sucessores, funcionava dessa maneira.

Nesse esquema, os espaços de endereçamento para memória e E/S são diferentes, como mostrado na Figura 5.2(a). As instruções

IN R0,4

e

MOV R0,4

são completamente diferentes nesse projeto. A primeira lê o conteúdo da porta de E/S 4 e o coloca em R0, enquanto a segunda lê o conteúdo da palavra de memória 4 e o coloca em R0. Os 4 nesses exemplos referem-se a espaços de endereçamento diferentes e não relacionados.

A segunda abordagem, introduzida com o PDP-11, é mapear todos os registradores de controle no espaço da memória, como mostrado na Figura 5.2(b). Para cada registrador de controle é designado um endereço de memória único para o qual nenhuma memória é designada. Esse sistema é chamado de **E/S mapeada na memória**. Na maioria dos sistemas, os endereços designados estão no — ou próximos do — topo do espaço de endereçamento. Um esquema híbrido, com buffers de dados de E/S mapeados na memória e portas de E/S separadas para os registradores de controle, é mostrado na Figura 5.2(c). O x86 usa essa arquitetura, com endereços de 640K a 1M – 1 sendo reservados para buffers de dados de dispositivos em PCs compatíveis com a IBM, além de portas de E/S de 0 a 64K – 1.

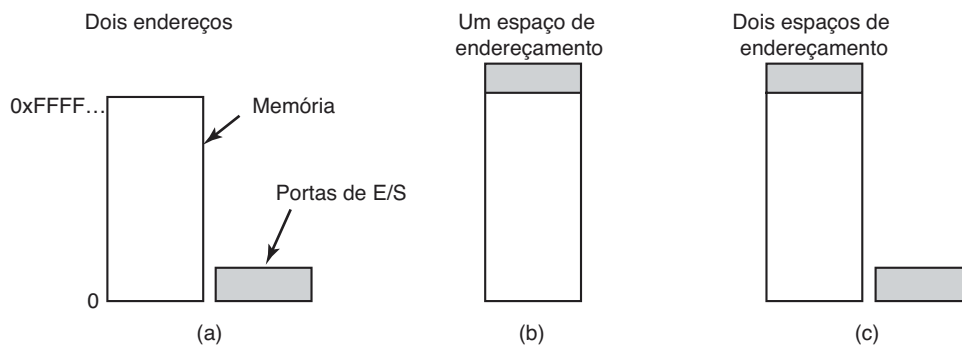
Como esses esquemas realmente funcionam na prática? Em todos os casos, quando a CPU quer ler uma palavra, seja da memória ou de uma porta de E/S, ela coloca o endereço de que precisa nas linhas de endereçamento do barramento e então emite um sinal READ sobre uma linha de controle do barramento. Uma segunda linha de sinal é usada para dizer se o espaço de E/S ou o espaço de memória é necessário. Se for o espaço de memória, a memória responde ao pedido. Se for o espaço de E/S, o dispositivo de E/S responde ao pedido. Se houver apenas

espaço de memória [como na Figura 5.2(b)], cada módulo de memória e cada dispositivo de E/S comparam as linhas de endereços com a faixa de endereços que elas servem. Se o endereço cair na sua faixa, ela responde ao pedido. Tendo em vista que nenhum endereço jamais é designado tanto à memória quanto a um dispositivo de E/S, não há ambiguidade ou conflito.

Esses dois esquemas de endereçamento dos controladores têm diferentes pontos fortes e fracos. Vamos começar com as vantagens da E/S mapeada na memória. Primeiro, se instruções de E/S especiais são necessárias para ler e escrever os registradores de controle do dispositivo, acessá-los exige o uso de código de montagem, já que não há como executar uma instrução IN ou OUT em C ou C++. Uma chamada a esse procedimento acarreta um custo adicional ao controle de E/S. Por outro lado, com a E/S mapeada na memória, os registradores de controle do dispositivo são apenas variáveis na memória e podem ser endereçados em C da mesma maneira que quaisquer outras variáveis. Desse modo, com a E/S mapeada na memória, um driver do dispositivo de E/S pode ser escrito inteiramente em C. Sem a E/S mapeada na memória, é necessário algum código em linguagem de montagem.

Segundo, com a E/S mapeada na memória, nenhum mecanismo de proteção especial é necessário para evitar que processos do usuário realizem E/S. Tudo o que o sistema operacional precisa fazer é deixar de colocar aquela porção do espaço de endereçamento contendo os registros de controle no espaço de endereçamento virtual de qualquer usuário. Melhor ainda, se cada dispositivo tem os seus registradores de controle em uma página diferente do espaço de endereçamento, o sistema operacional pode dar a um usuário controle sobre dispositivos específicos, mas não dar a outros, ao simplesmente incluir as páginas desejadas em sua tabela de páginas. Esse esquema pode permitir que diferentes drivers de dispositivos sejam colocados em diferentes espaços de

FIGURA 5.2 (a) Espaços de memória e E/S separados. (b) E/S mapeada na memória. (c) Híbrido.



endereçamento, não apenas reduzindo o tamanho do núcleo, mas também impedindo que um driver interfira nos outros.

Terceiro, com a E/S mapeada na memória, cada instrução capaz de referenciar a memória também referencia os registradores de controle. Por exemplo, se houver uma instrução, TEST, que testa se uma palavra de memória é 0, ela também poderá ser usada para testar se um registrador de controle é 0, o que pode ser o sinal de que o dispositivo está ocioso e pode aceitar um novo comando. O código em linguagem de montagem pode parecer da seguinte maneira:

```
LOOP: TEST PORT_4    // verifica se a porta 4 é 0
      BEQ READY      // se for 0, salta para READY
      BRANCH LOOP     // caso contrario, continua
                        testando
```

READY:

Se a E/S mapeada na memória não estiver presente, o registrador de controle deve primeiro ser lido na CPU, então testado, exigindo duas instruções em vez de uma. No caso do laço mostrado, uma quarta instrução precisa ser adicionada, atrasando ligeiramente a detecção de ociosidade do dispositivo.

No projeto de computadores, praticamente tudo envolve uma análise de custo-benefício, e este é o caso aqui também. A E/S mapeada na memória também tem suas desvantagens. Primeiro, a maioria dos computadores hoje tem alguma forma de cache para as palavras de memória. O uso de cache para um registrador de controle do dispositivo seria desastroso. Considere o laço em código de montagem dado anteriormente na presença de cache. A primeira referência a PORT_4 o faria ser colocado em cache. Referências subsequentes simplesmente tomariam o valor da cache e nem perguntariam ao dispositivo. Então quando o dispositivo por fim estivesse pronto, o software não teria como descobrir. Em vez disso, o laço entraria em repetição para sempre.

Para evitar essa situação com a E/S mapeada na memória, o hardware tem de ser capaz de desabilitar seletivamente a cache, por exemplo, em um sistema por página. Essa característica acrescenta uma complexidade de extra tanto para o hardware, quanto para o sistema operacional, o qual deve gerenciar a cache seletiva.

Segundo, se houver apenas um espaço de endereçamento, então todos os módulos de memória e todos os dispositivos de E/S terão de examinar todas as referências de memória para ver quais devem ser respondidas por cada um. Se o computador tiver um único

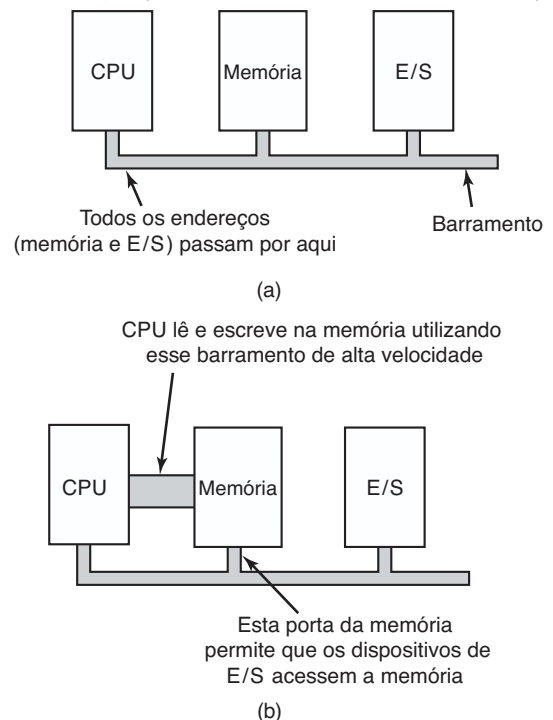
barramento, como na Figura 5.3(a), cada componente poderá olhar para cada endereço diretamente.

No entanto, a tendência nos computadores pessoais modernos é ter um barramento de memória de alta velocidade dedicado, como mostrado na Figura 5.3(b). O barramento é feito sob medida para otimizar o desempenho da memória, sem concessões para o bem de dispositivos de E/S lentos. Os sistemas x86 podem ter múltiplos barramentos (memória, PCIe, SCSI e USB), como mostrado na Figura 1.12.

O problema de ter um barramento de memória separado em máquinas mapeadas na memória é que os dispositivos de E/S não têm como enxergar os endereços de memória quando estes são lançados no barramento da memória, de maneira que eles não têm como responder. Mais uma vez, medidas especiais precisam ser tomadas para fazer que a E/S mapeada na memória funcione em um sistema com múltiplos barramentos. Uma possibilidade pode ser enviar primeiro todas as referências de memória para a memória. Se esta falhar em responder, então a CPU tenta outros barramentos. Esse projeto pode se tornar exequível, mas ele exige uma complexidade adicional do hardware.

Um segundo projeto possível é colocar um dispositivo de escuta no barramento de memória para passar todos os endereços apresentados para os dispositivos de E/S potencialmente interessados. O problema aqui

FIGURA 5.3 (a) Arquitetura com barramento único.
(b) Arquitetura de memória com barramento duplo.



é que os dispositivos de E/S podem não ser capazes de processar pedidos na mesma velocidade da memória.

Um terceiro projeto possível, e que se enquadraria bem naquele desenhado na Figura 1.12, é filtrar endereços no controlador de memória. Nesse caso, o chip controlador de memória contém registradores de faixa que são pré-carregados no momento da inicialização. Por exemplo, de 640K a 1M – 1 poderia ser marcado como uma faixa de endereços reservada não utilizável como memória. Endereços que caem dentro dessas faixas marcadas são transferidos para dispositivos em vez da memória. A desvantagem desse esquema é a necessidade de descobrir no momento da inicialização quais endereços de memória são realmente endereços de memória. Desse modo, cada esquema tem argumentos favoráveis e contrários, de maneira que concessões e avaliações de custo-benefício são inevitáveis.

5.1.4 Acesso direto à memória (DMA)

Não importa se uma CPU tem ou não E/S mapeada na memória, ela precisa endereçar os controladores dos dispositivos para poder trocar dados com eles. A CPU pode requisitar dados de um controlador de E/S um byte de cada vez, mas fazê-lo desperdiça o tempo da CPU, de maneira que um esquema diferente, chamado de **acesso direto à memória (Direct Memory Access — DMA)** é usado muitas vezes. Para simplificar a explicação, presumimos que a CPU acessa todos os dispositivos e memória mediante um único barramento de sistema que conecta a CPU, a memória e os dispositivos de E/S, como mostrado na Figura 5.4. Já sabemos que a organização real em sistemas modernos é mais complicada, mas todos os princípios são os mesmos. O sistema operacional pode usar somente DMA se o hardware tiver

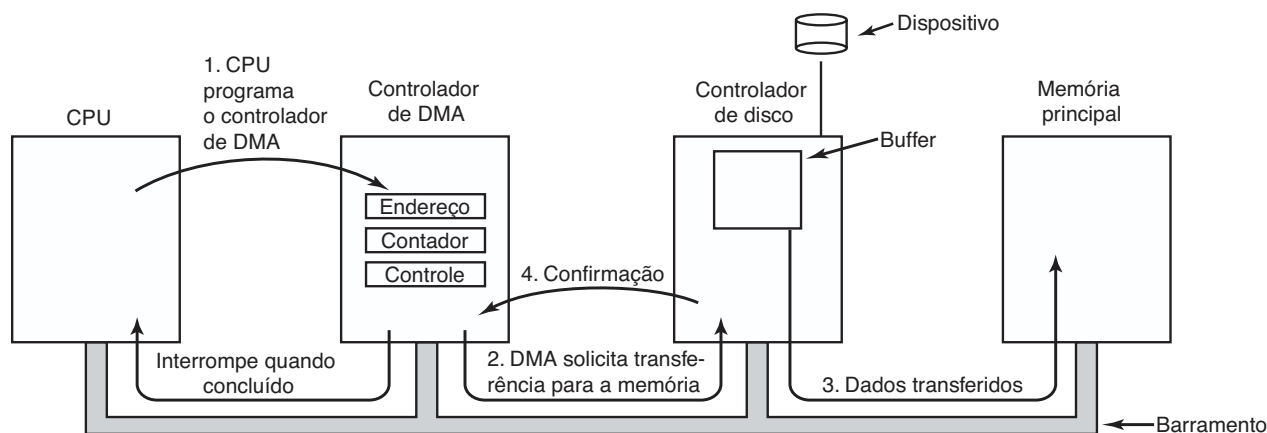
um controlador de DMA, o que a maioria dos sistemas tem. Às vezes esse controlador é integrado em controladores de disco e outros, mas um projeto desses exige um controlador de DMA separado para cada dispositivo. Com mais frequência, um único controlador de DMA está disponível (por exemplo, na placa-mãe) a fim de controlar as transferências para múltiplos dispositivos, muitas vezes simultaneamente.

Não importa onde esteja localizado fisicamente, o controlador de DMA tem acesso ao barramento do sistema independente da CPU, como mostrado na Figura 5.4. Ele contém vários registradores que podem ser escritos e lidos pela CPU. Esses incluem um registrador de endereço de memória, um registrador contador de bytes e um ou mais registradores de controle. Os registradores de controle especificam a porta de E/S a ser usada, a direção da transferência (leitura do dispositivo de E/S ou escrita para o dispositivo de E/S), a unidade de transferência (um byte por vez ou palavra por vez) e o número de bytes a ser transferido em um surto.

Para explicar como o DMA funciona, vamos examinar primeiro como ocorre uma leitura de disco quando o DMA não é usado. Primeiro o controlador de disco lê o bloco (um ou mais setores) do dispositivo serialmente, bit por bit, até que o bloco inteiro esteja no buffer interno do controlador. Em seguida, ele calcula a soma de verificação para verificar que nenhum erro de leitura tenha ocorrido. Então o controlador causa uma interrupção. Quando o sistema operacional começa a ser executado, ele pode ler o bloco de disco do buffer do controlador um byte ou uma palavra de cada vez executando um laço, com cada iteração lendo um byte ou palavra de um registrador do controlador e armazenando-a na memória principal.

Quando o DMA é usado, o procedimento é diferente. Primeiro a CPU programa o controlador de DMA

FIGURA 5.4 Operação de transferência utilizando DMA.



configurando seus registradores para que ele saiba o que transferir para onde (passo 1 na Figura 5.4). Ela também emite um comando para o controlador de disco dizendo para ele ler os dados do disco para o seu buffer interno e verificar a soma de verificação. Quando os dados que estão no buffer do controlador de disco são válidos, o DMA pode começar.

O controlador de DMA inicia a transferência emitindo uma solicitação de leitura via barramento para o controlador de disco (passo 2). Essa solicitação de leitura se parece com qualquer outra, e o controlador de disco não sabe (ou se importa) se ela veio da CPU ou de um controlador de DMA. Tipicamente, o endereço de memória para onde escrever está nas linhas de endereçamento do barramento, então quando o controlador de disco busca a palavra seguinte do seu buffer interno, ele sabe onde escrevê-la. A escrita na memória é outro ciclo de barramento-padrão (passo 3). Quando a escrita está completa, o controlador de disco envia um sinal de confirmação para o controlador de DMA, também via barramento (passo 4). O controlador de DMA então incrementa o endereço de memória e diminui o contador de bytes. Se o contador de bytes ainda for maior do que 0, os passos 2 até 4 são repetidos até que o contador chegue a 0. Nesse momento, o controlador de DMA interrompe a CPU para deixá-la ciente de que a transferência está completa agora. Quando o sistema operacional é inicializado, ele não precisa copiar o bloco de disco para a memória, pois ele já está lá.

Controladores de DMA variam consideravelmente em sofisticação. Os mais simples lidam com uma transferência de cada vez, como acabamos de descrever. Os mais complexos podem ser programados para lidar com múltiplas transferências ao mesmo tempo. Esses controladores têm múltiplos conjuntos de registradores internamente, um para cada canal. A CPU inicializa carregando cada conjunto de registradores com os parâmetros relevantes para sua transferência. Cada transferência deve usar um controlador de dispositivos diferente. Após cada palavra ser transferida (passos 2 a 4) na Figura 5.4, o controlador de DMA decide qual dispositivo servir em seguida. Ele pode ser configurado para usar um algoritmo de alternância circular (*round-robin*), ou ter um esquema de prioridade projetado para favorecer alguns dispositivos em detrimento de outros. Múltiplas solicitações para diferentes controladores de dispositivos podem estar pendentes ao mesmo tempo, desde que exista uma maneira clara de identificar separadamente os sinais de confirmação. Por esse motivo, muitas vezes uma linha diferente de confirmação no barramento é usada para cada canal de DMA.

Muitos barramentos podem operar em dois modos: modo uma palavra de cada vez (*word-at-a-time mode*) e modo bloco. Alguns controladores de DMA também podem operar em ambos os modos. No primeiro, a operação funciona como descrito: o controlador de DMA solicita a transferência de uma palavra e consegue. Se a CPU também quiser o barramento, ela tem de esperar. O mecanismo é chamado de **roubo de ciclo**, pois o controlador do dispositivo entra furtivamente e rouba um ciclo de barramento ocasional da CPU de vez em quando, atrasando-a ligeiramente. No modo bloco, o controlador de DMA diz para o dispositivo para adquirir o barramento, emitir uma série de transferências, então libera o barramento. Essa forma de operação é chamada de **modo de surto** (*burst*). Ela é mais eficiente do que o roubo de ciclo, pois adquirir o barramento leva tempo e múltiplas palavras podem ser transferidas pelo preço de uma aquisição de barramento. A desvantagem do modo de surto é que ele pode bloquear a CPU e outros dispositivos por um período substancial caso um surto longo esteja sendo transferido.

No modelo que estivemos discutindo, também chamado de **modo direto** (*fly-by mode*), o controlador do DMA diz para o controlador do dispositivo para transferir os dados diretamente para a memória principal. Um modo alternativo que alguns controladores de DMA usam estabelece que o controlador do dispositivo deve enviar a palavra para o controlador de DMA, que então emite uma segunda solicitação de barramento para escrever a palavra para qualquer que seja o seu destino. Esse esquema exige um ciclo de barramento extra por palavra transferida, mas é mais flexível no sentido de que ele pode também desempenhar cópias dispositivo-para-dispositivo e mesmo cópias memória-para-memória (ao emitir primeiro uma requisição de leitura à memória e então uma requisição de escrita à memória, em endereços diferentes).

A maioria dos controladores de DMA usa endereços físicos de memória para suas transferências. O uso de endereços físicos exige que o sistema operacional converta o endereço virtual do buffer de memória pretendido em um endereço físico e escreva esse endereço físico no registrador de endereço do controlador de DMA. Um esquema alternativo usado em alguns controladores de DMA é em vez disso escrever o próprio endereço virtual no controlador de DMA. Então o controlador de DMA deve usar a unidade de gerenciamento de memória (*Memory Management Unit* — MMU) para fazer a tradução de endereço virtual para físico. Apenas no caso em que a MMU faz parte da memória (possível, mas raro), em

vez de parte da CPU, os endereços virtuais podem ser colocados no barramento.

Mencionamos anteriormente que o disco primeiro lê dados para seu buffer interno antes que o DMA possa ser inicializado. Você pode estar imaginando por que o controlador não armazena simplesmente os bytes na memória principal tão logo ele os recebe do disco. Em outras palavras, por que ele precisa de um buffer interno? Há duas razões. Primeiro, ao realizar armazenamento interno, o controlador de disco pode conferir a soma de verificação antes de começar uma transferência. Se a soma de verificação estiver incorreta, um erro é sinalizado e nenhuma transferência é feita.

A segunda razão é que uma vez inicializada uma transferência de disco os bits continuam chegando do disco a uma taxa constante, não importa se o controlador estiver pronto para eles ou não. Se o controlador tentasse escrever dados diretamente na memória, ele teria de acessar o barramento do sistema para cada palavra transferida. Se o barramento estivesse ocupado por algum outro dispositivo usando-o (por exemplo, no modo surto), o controlador teria de esperar. Se a próxima palavra de disco chegasse antes que a anterior tivesse sido armazenada, o controlador teria de armazená-la em outro lugar. Se o barramento estivesse muito ocupado, o controlador poderia terminar armazenando um número considerável de palavras e tendo bastante gerenciamento para fazer também. Quando o bloco é armazenado internamente, o barramento não se faz necessário até que o DMA comece; portanto, o projeto do controlador é muito mais simples, pois utilizando DMA o momento de transferência para a memória não é um fator crítico. (Alguns controladores mais antigos iam, na realidade, diretamente para a memória com apenas uma pequena quantidade de armazenamento interno, mas quando o barramento estava muito ocupado, uma transferência talvez tivesse de ser terminada com um erro de transbordo de pilha.)

Nem todos os computadores usam DMA. O argumento contra ele é que a CPU principal muitas vezes é muito mais rápida do que o controlador de DMA e pode fazer o trabalho muito mais rápido (quando o fator limitante não é a velocidade do dispositivo de E/S). Se não há outro trabalho para ela realizar, fazer a CPU (rápida) esperar pelo controlador de DMA (lento) terminar, não faz sentido. Também, livrar-se do controlador de DMA e ter a CPU realizando todo o trabalho via software economiza dinheiro, algo importante em computadores de baixo custo (embarcados).

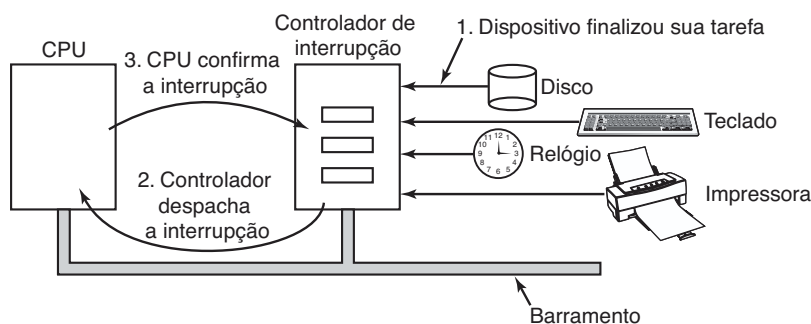
5.1.5 Interrupções revisitadas

Introduzimos brevemente as interrupções na Seção 1.3.4, mas há mais a ser dito. Em um sistema típico de computador pessoal, a estrutura de interrupção é como a mostrada na Figura 5.5. No nível do hardware, as interrupções funcionam como a seguir. Quando um dispositivo de E/S termina o trabalho dado a ele, gera uma interrupção (presumindo que as interrupções tenham sido habilitadas pelo sistema operacional). Ele faz isso enviando um sinal pela linha de barramento à qual está associado. Esse sinal é detectado pelo chip controlador de interrupções na placa-mãe, que então decide o que fazer.

Se nenhuma outra interrupção estiver pendente, o controlador de interrupção processa a interrupção imediatamente. No entanto, se outra interrupção estiver em andamento, ou outro dispositivo tiver feito uma solicitação simultânea em uma linha de requisição de interrupção de maior prioridade no barramento, o dispositivo é simplesmente ignorado naquele momento. Nesse caso, ele continua a gerar um sinal de interrupção no barramento até ser atendido pela CPU.

Para tratar a interrupção, o controlador coloca um número sobre as linhas de endereço especificando qual

FIGURA 5.5 Como ocorre uma interrupção. As conexões entre os dispositivos e o controlador usam na realidade linhas de interrupção no barramento em vez de cabos dedicados.



dispositivo requer atenção e repassa um sinal para interromper a CPU.

O sinal de interrupção faz a CPU parar aquilo que ela está fazendo e começar outra atividade. O número nas linhas de endereço é usado como um índice em uma tabela chamada de **vetor de interrupções** para buscar um novo contador de programa. Esse contador de programa aponta para o início da rotina de tratamento da interrupção correspondente. Em geral, interrupções de software (traps ou armadilhas) e de hardware usam o mesmo mecanismo desse ponto em diante, muitas vezes compartilhando o mesmo vetor de interrupções. A localização do vetor de interrupções pode ser estabelecida fisicamente na máquina ou estar em qualquer lugar na memória, com um registrador da CPU (carregado pelo sistema operacional) apontando para sua origem.

Logo após o início da execução, a rotina de tratamento da execução reconhece a interrupção escrevendo um determinado valor para uma das portas de E/S do controlador de interrupção. Esse reconhecimento diz ao controlador que ele está livre para gerar outra interrupção. Ao fazer a CPU atrasar esse reconhecimento até que ela esteja pronta para lidar com a próxima interrupção, podem ser evitadas condições de corrida envolvendo múltiplas (quase simultâneas) interrupções. Como nota, alguns computadores (mais velhos) não têm um controlador de interrupções centralizado, de maneira que cada controlador de dispositivo solicita as suas próprias interrupções.

O hardware sempre armazena determinadas informações antes de iniciar o procedimento de serviço. Quais informações e onde elas são armazenadas varia muito de CPU para CPU. No mínimo, o contador do programa deve ser salvo, de maneira que o processo interrompido possa ser reiniciado. No outro extremo, todos os registradores visíveis e um grande número de registradores internos podem ser salvos também.

Uma questão é onde salvar essas informações. Uma opção é colocá-las nos registradores internos que o sistema operacional pode ler conforme a necessidade. Um problema com essa abordagem é que então o controlador de interrupções não pode ser reconhecido até que todas as informações potencialmente relevantes tenham sido lidas, a fim de que uma segunda informação não sobreponha os registradores internos durante o salvamento. Essa estratégia leva a longos períodos de tempo desperdiçados quando as interrupções são desabilitadas e possivelmente a interrupções e dados perdidos.

Em consequência, a maioria das CPUs salva as informações em uma pilha. No entanto, essa abordagem também tem problemas. Para começo de conversa, de

quem é a pilha? Se a pilha atual for usada, ela pode muito bem ser uma pilha do processo do usuário. O ponteiro da pilha pode não ter legitimidade, o que causaria um erro fatal quando o hardware tentasse escrever algumas palavras no endereço apontado. Também, ele poderia apontar para o fim de uma página. Após várias escritas na memória, o limite da página pode ser excedido e uma falta de página gerada. A ocorrência de uma falta de página durante o processamento de uma interrupção de hardware cria um problema maior: onde salvar o estado para tratar a falta de página?

Se for usada a pilha de núcleo, há uma chance muito maior de o ponteiro de pilha ser legítimo e estar apontando para uma página na memória. No entanto, o chaveamento para o modo núcleo pode requerer a troca de contextos da MMU e provavelmente invalidará a maior parte da cache — ou toda ela — e a tabela de tradução de endereços (translation look aside table — TLB). A recarga de toda essa informação, estática ou dinamicamente, aumenta o tempo para processar uma interrupção e, desse modo, desperdiça tempo de CPU.

Interrupções precisas e imprecisas

Outro problema é causado pelo fato de a maioria das CPUs modernas ser projetada com pipelines profundos e, muitas vezes, superescalares (paralelismo interno). Em sistemas mais antigos, após cada instrução finalizar a sua execução, o microprograma ou hardware era verificado para ver se havia uma interrupção pendente. Se fosse o caso, o contador de programa e a palavra de estado do programa (Program Status Word — PSW) eram colocados na pilha e a sequência de interrupção começava. Após o fim do tratamento da interrupção, ocorria o processo reverso, com a velha PSW e o contador do programa retirados da pilha e o processo anterior continuado.

Esse modelo faz a suposição implícita de que se ocorrer uma interrupção logo após alguma instrução, todas as instruções até ela (incluindo-a) foram executadas completamente, e nenhuma interrupção posterior foi executada de maneira alguma. Em máquinas mais antigas, tal suposição sempre foi válida. Nas modernas, talvez não seja.

Para começo de conversa, considere o modelo de pipeline da Figura 1.7(a). O que acontece se uma interrupção ocorrer enquanto o pipeline está cheio (o caso usual)? Muitas instruções estão em vários estágios de execução. Quando ocorre a interrupção, o valor do contador do programa pode não refletir o limite correto entre as instruções executadas e as não executadas. Na realidade, muitas instruções talvez tenham sido

parcialmente executadas, com diferentes instruções estando mais ou menos completas. Nessa situação, o contador do programa provavelmente refletirá o endereço da próxima instrução a ser buscada e colocada no pipeline em vez do endereço da instrução que acabou de ser processada pela unidade de execução.

Em uma máquina superescalar, como a da Figura 1.7(b), as coisas são ainda piores. As instruções podem ser decompostas em micro-operações e estas podem ser executadas fora de ordem, dependendo da disponibilidade dos recursos internos, como unidades funcionais e registradores. No momento de uma interrupção, algumas instruções enviadas há muito tempo talvez não tenham sido iniciadas e outras mais recentes talvez estejam quase concluídas. No momento em que uma interrupção é sinalizada, pode haver muitas instruções em vários estados de completude, com uma relação menor entre elas e o contador do programa.

Uma interrupção que deixa a máquina em um estado bem definido é chamada de uma **interrupção precisa** (WALKER e CRAGON, 1995). Uma interrupção assim possui quatro propriedades:

1. O contador do programa (Program Counter — PC) é salvo em um lugar conhecido.
2. Todas as instruções anteriores àquela apontada pelo PC foram completadas.
3. Nenhuma instrução posterior à apontada pelo PC foi concluída.
4. O estado de execução da instrução apontada pelo PC é conhecido.

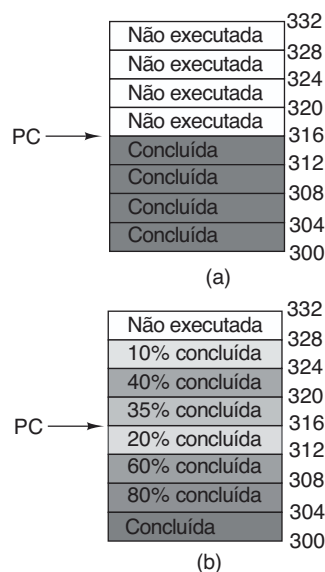
Observe que não existe proibição para que as instruções posteriores à apontada pelo PC sejam iniciadas. A questão é apenas que quaisquer alterações que elas façam aos registradores ou memória devem ser desfeitas antes que a interrupção ocorra. É permitido que a instrução apontada tenha sido executada. Também é permitido que ela não tenha sido executada. No entanto, deve ficar claro qual caso se aplica à situação. Muitas vezes, se a interrupção é de E/S, a instrução não terá começado ainda. No entanto, se ela é uma interrupção de software ou uma falta de página, então o PC geralmente aponta para a instrução que causou a falta para que ele possa ser reinicializado mais tarde. A situação da Figura 5.6(a) ilustra uma interrupção precisa. Todas as instruções até o contador do programa (316) foram concluídas e nenhuma das que estão além dele foi iniciada (ou foi retrocedida para desfazer os seus efeitos).

Uma interrupção que não atende a essas exigências é chamada de **interrupção imprecisa** e dificulta bastante a vida do projetista do sistema operacional, que

agora tem de descobrir o que aconteceu e o que ainda está para acontecer. A Figura 5.6(b) ilustra uma interrupção imprecisa, em que diferentes instruções próximas do contador do programa estão em diferentes estágios de conclusão, com as mais antigas não necessariamente mais completas que as mais novas. Máquinas com interrupções imprecisas despejam em geral uma grande quantidade de estado interno sobre a pilha para proporcionar ao sistema operacional a possibilidade de descobrir o que está acontecendo. O código necessário para reiniciar a máquina costuma ser muito complicado. Também, salvar uma quantidade grande de informações na memória em cada interrupção torna as interrupções lentas e a recuperação ainda pior. Isso gera a situação irônica de termos CPUs superescalares muito rápidas sendo, às vezes, inadequadas para o trabalho em tempo real por causa das interrupções lentas.

Alguns computadores são projetados de maneira que alguns tipos de interrupções de software e de hardware são precisos e outros não. Por exemplo, ter interrupções de E/S precisas, mas interrupções de software imprecisas por erros de programação fatais não é algo tão ruim, pois nenhuma tentativa precisa ser feita para reiniciar um processo em execução após ele ter feito uma divisão por zero. Algumas máquinas têm um bit que pode ser configurado para forçar que todas as interrupções sejam precisas. A desvantagem de se configurar esse bit é que ele força a CPU a registrar cuidadosamente tudo o que ela está fazendo e manter cópias de proteção dos registradores a fim de poder gerar uma interrupção precisa a

FIGURA 5.6 (a) Uma interrupção precisa. (b) Uma interrupção imprecisa.



qualquer instante. Toda essa sobrecarga tem um impacto importante sobre o desempenho.

Algumas máquinas superescalares, como as da família x86, têm interrupções precisas para permitir que softwares antigos funcionem corretamente. O custo por essa compatibilidade com interrupções precisas é uma lógica de interrupção extremamente complexa dentro da CPU para certificar-se de que, quando o controlador da interrupção sinaliza que ele quer gerar uma interrupção, deixem-se terminar todas as instruções até um determinado ponto e nada além daquele ponto tenha qualquer efeito perceptível sobre o estado da máquina. Aqui o custo não é em tempo, mas em área de chip e na complexidade do projeto. Se interrupções precisas não fossem necessárias para fins de compatibilidade com versões antigas, essa área de chip seria disponível para caches maiores dentro do chip, tornando a CPU mais rápida. Por outro lado, interrupções imprecisas tornam o sistema operacional muito mais complicado e lento, então é difícil dizer qual abordagem é realmente melhor.

5.2 Princípios do software de E/S

Vamos agora deixar de lado por enquanto o hardware de E/S e examinar o software de E/S. Primeiro analisaremos suas metas e então as diferentes maneiras que a E/S pode ser feita do ponto de vista do sistema operacional.

5.2.1 Objetivos do software de E/S

Um conceito fundamental no projeto de software de E/S é conhecido como **independência de dispositivo**. O que isso significa é que devemos ser capazes de escrever programas que podem acessar qualquer dispositivo de E/S sem ter de especificá-lo antecipadamente. Por exemplo, um programa que lê um arquivo como entrada deve ser capaz de ler um arquivo em um disco rígido, um DVD ou em um pen-drive sem ter de ser modificado para cada dispositivo diferente. Similarmente, deveria ser possível digitar um comando como

```
sort <input >output
```

que trabalhe com uma entrada vinda de qualquer tipo de disco ou teclado e a saída indo para qualquer tipo de disco ou tela. Fica a cargo do sistema operacional cuidar dos problemas causados pelo fato de que esses dispositivos são realmente diferentes e exigem sequências de comando muito diferentes para ler ou escrever.

Um objetivo muito relacionado com a independência do dispositivo é a **nomeação uniforme**. O nome de um

arquivo ou um dispositivo deve simplesmente ser uma cadeia de caracteres ou um número inteiro e não depender do dispositivo de maneira alguma. No UNIX, todos os discos podem ser integrados na hierarquia do sistema de arquivos de maneiras arbitrárias, então o usuário não precisa estar ciente de qual nome corresponde a qual dispositivo. Por exemplo, um pen-drive pode ser **montado** em cima do diretório `/usr/ast/backup` de maneira que, ao copiar um arquivo para `/usr/ast/backup/Monday`, você copia o arquivo para o pen-drive. Assim, todos os arquivos e dispositivos são endereçados da mesma maneira: por um nome de caminho.

Outra questão importante para o software de E/S é o **tratamento de erros**. Em geral, erros devem ser tratados o mais próximo possível do hardware. Se o controlador descobre um erro de leitura, ele deve tentar corrigi-lo se puder. Se ele não puder, então o driver do dispositivo deverá lidar com ele, talvez simplesmente tentando ler o bloco novamente. Muitos erros são transitórios, como erros de leitura causados por grãos de poeira no cabeçote de leitura, e muitas vezes desaparecerão se a operação for repetida. Apenas se as camadas mais baixas não forem capazes de lidar com o problema as camadas superiores devem ser informadas a respeito. Em muitos casos, a recuperação de erros pode ser feita de modo transparente em um nível baixo sem que os níveis superiores sequer tomem conhecimento do erro.

Ainda outra questão importante é a das transferências **síncronas** (bloqueantes) *versus* **assíncronas** (orientadas à interrupção). A maioria das E/S físicas são assíncronas — a CPU inicializa a transferência e vai fazer outra coisa até a chegada da interrupção. Programas do usuário são muito mais fáceis de escrever se as operações de E/S forem bloqueantes — após uma chamada de sistema `read`, o programa é automaticamente suspenso até que os dados estejam disponíveis no buffer. Fica a cargo do sistema operacional fazer operações que são realmente orientadas à interrupção parecerem bloqueantes para os programas do usuário. No entanto, algumas aplicações de muito alto desempenho precisam controlar todos os detalhes da E/S, então alguns sistemas operacionais disponibilizam a E/S assíncrona para si.

Outra questão para o software de E/S é a **utilização de buffer**. Muitas vezes, dados provenientes de um dispositivo não podem ser armazenados diretamente em seu destino final. Por exemplo, quando um pacote chega da rede, o sistema operacional não sabe onde armazená-lo definitivamente até que o tenha colocado em algum lugar para examiná-lo. Também, alguns dispositivos têm severas restrições de tempo real (por exemplo, dispositivos de áudio digitais), portanto os dados devem

ser colocados antecipadamente em um buffer de saída para separar a taxa na qual o buffer é preenchido da taxa na qual ele é esvaziado, a fim de evitar seu completo esvaziamento. A utilização do buffer envolve consideráveis operações de cópia e muitas vezes tem um impacto importante sobre o desempenho de E/S.

O conceito final que mencionaremos aqui é o de dispositivos compartilhados *versus* dedicados. Alguns dispositivos de E/S, como discos, podem ser usados por muitos usuários ao mesmo tempo. Nenhum problema é causado por múltiplos usuários terem arquivos abertos no mesmo disco ao mesmo tempo. Outros dispositivos, como impressoras, têm de ser dedicados a um único usuário até ele ter concluído sua operação. Então outro usuário pode ter a impressora. Ter dois ou mais usuários escrevendo caracteres de maneira aleatória e intercalada na mesma página definitivamente não funcionará. Introduzir dispositivos dedicados (não compartilhados) também introduz uma série de problemas, como os impasses. Novamente, o sistema operacional deve ser capaz de lidar com ambos os dispositivos — compartilhados e dedicados — de uma maneira que evite problemas.

5.2.2 E/S programada

Há três maneiras fundamentalmente diferentes de realizar E/S. Nesta seção examinaremos a primeira (E/S programada). Nas duas seções seguintes examinaremos as outras (E/S orientada à interrupções e E/S usando DMA). A forma mais simples de E/S é ter a CPU realizando todo o trabalho. Esse método é chamado de **E/S programada**.

É mais simples ilustrar como a E/S programada funciona mediante um exemplo. Considere um processo de usuário que quer imprimir a cadeia de oito caracteres “ABCDEFGH” na impressora por meio de uma interface serial. Telas em pequenos sistemas embutidos

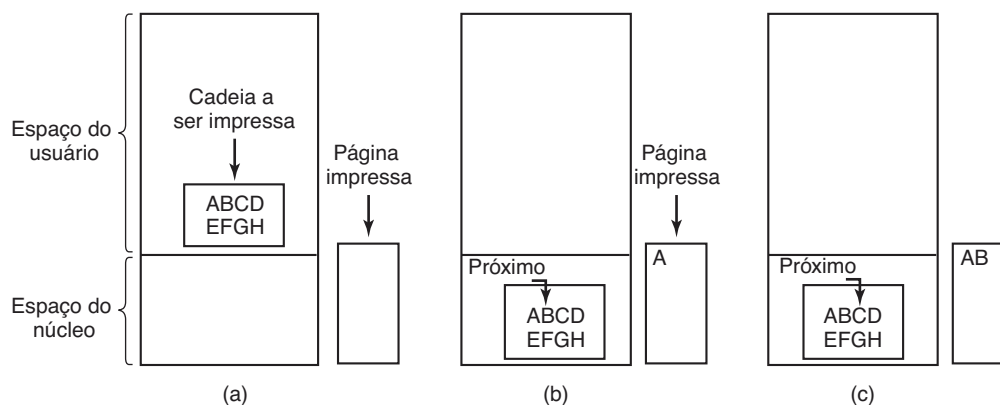
funcionam assim às vezes. O software primeiro monta a cadeia de caracteres em um buffer no espaço do usuário, como mostrado na Figura 5.7(a).

O processo do usuário requisita então a impressora para escrita fazendo uma chamada de sistema para abri-la. Se a impressora estiver atualmente em uso por outro processo, a chamada fracassará e retornará um código de erro ou bloqueará até que a impressora esteja disponível, dependendo do sistema operacional e dos parâmetros da chamada. Uma vez que ele tenha a impressora, o processo do usuário faz uma chamada de sistema dizendo ao sistema operacional para imprimir a cadeia de caracteres na impressora.

O sistema operacional então (normalmente) copia o buffer com a cadeia de caracteres para um vetor — digamos, p — no espaço do núcleo, onde ele é mais facilmente acessado (pois o núcleo talvez tenha de mudar o mapa da memória para acessar o espaço do usuário). Ele então confere para ver se a impressora está disponível no momento. Se não estiver, ele espera até que ela esteja. Tão logo a impressora esteja disponível, o sistema operacional copia o primeiro caractere para o registrador de dados da impressora, nesse exemplo usando a E/S mapeada na memória. Essa ação ativa a impressora. O caractere pode não aparecer ainda porque algumas impressoras armazenam uma linha ou uma página antes de imprimir qualquer coisa. Na Figura 5.7(b), no entanto, vemos que o primeiro caractere foi impresso e que o sistema marcou o “B” como o próximo caractere a ser impresso.

Tão logo copiado o primeiro caractere para a impressora, o sistema operacional verifica se ela está pronta para aceitar outro. Geralmente, a impressora tem um segundo registrador, que contém seu estado. O ato de escrever para o registrador de dados faz que o estado torne-se “indisponível”. Quando o controlador da impressora tiver processado o caractere atual, ele indica a

FIGURA 5.7 Estágios na impressão de uma cadeia de caracteres.



sua disponibilidade marcando algum bit em seu registrador de *status* ou colocando algum valor nele.

Nesse ponto, o sistema operacional espera que a impressora fique pronta de novo. Quando isso acontece, ele imprime o caractere seguinte, como mostrado na Figura 5.7(c). Esse laço continua até que a cadeia inteira tenha sido impressa. Então o controle retorna para o processo do usuário.

As ações seguidas pelo sistema operacional estão brevemente resumidas na Figura 5.8. Primeiro os dados são copiados para o núcleo. Então o sistema operacional entra em um laço fechado, enviando um caractere de cada vez para a saída. O aspecto essencial da E/S programada, claramente ilustrado nessa figura, é que, após a saída de um caractere, a CPU continuamente verifica o dispositivo para ver se ele está pronto para aceitar outro. Esse comportamento é muitas vezes chamado de **espera ocupada** (*busy waiting*) ou **polling**.

A E/S programada é simples, mas tem a desvantagem de segurar a CPU o tempo todo até que toda a E/S tenha sido feita. Se o tempo para “imprimir” um caractere for muito curto (pois tudo o que a impressora está fazendo é copiar o novo caractere para um buffer interno), então a espera ocupada estará bem. No entanto, em sistemas mais complexos, em que a CPU tem outros trabalhos a fazer, a espera ocupada será ineficiente, e será necessário um método de E/S melhor.

5.2.3 E/S orientada a interrupções

Agora vamos considerar o caso da impressão em uma impressora que não armazena caracteres, mas imprime cada

um à medida que ele chega. Se a impressora puder imprimir, digamos, 100 caracteres/segundo, cada caractere levará 10 ms para imprimir. Isso significa que após cada caractere ter sido escrito no registrador de dados da impressora, a CPU vai permanecer em um laço ocioso por 10 ms esperando a permissão para a saída do próximo caractere. Isso é mais tempo do que o necessário para realizar um chaveamento de contexto e executar algum outro processo durante os 10 ms que de outra maneira seriam desperdiçados.

A maneira de permitir que a CPU faça outra coisa enquanto espera que a impressora fique pronta é usar interrupções. Quando a chamada de sistema para imprimir a cadeia é feita, o buffer é copiado para o espaço do núcleo, como já mostramos, e o primeiro caractere é copiado para a impressora tão logo ela esteja disposta a aceitar um caractere. Nesse ponto, a CPU chama o escalonador e algum outro processo é executado. O processo que solicitou que a cadeia seja impressa é bloqueado até que a cadeia inteira seja impressa. O trabalho feito durante a chamada de sistema é mostrado na Figura 5.9(a).

Quando a impressora imprimiu o caractere e está preparada para aceitar o próximo, ela gera uma interrupção. Essa interrupção para o processo atual e salva seu estado. Então a rotina de tratamento de interrupção da impressora é executada. Uma versão simples desse código é mostrada na Figura 5.9(b). Se não houver mais caracteres a serem impressos, o tratador de interrupção executa alguma ação para desbloquear o usuário. Caso contrário, ele sai com o caractere seguinte, reconhece a interrupção e retorna ao processo que estava sendo executado um momento antes da interrupção, o qual continua a partir do ponto em que ele parou.

FIGURA 5.8 Escrevendo uma cadeia de caracteres para a impressora usando E/S programada.

```
copy_from_user(buffer, p, count);          /* p e o buffer do núcleo */
for (i=0; i < count; i++) {                /* executa o laço para cada caractere */
    while (*printer_status_reg != READY) ;  /* executa o laço até a impressora estar pronta */
    *printer_data_register = p[i];          /* envia um caractere para a saída */
}
return_to_user();
```

FIGURA 5.9 Escrevendo uma cadeia de caracteres na impressora usando E/S orientada à interrupção. (a) Código executado no momento em que a chamada de sistema para execução é feita. (b) Rotina de tratamento da execução para a impressora.

```
copy_from_user(buffer, p, count);          if (count == 0) {
enable_interrupts();                       unblock_user();
while (*printer_status_reg != READY) ;    } else {
*printer_data_register = p[0];             *printer_data_register = p[i];
scheduler();                              count = count - 1;
                                           i = i + 1;
                                           }
                                           acknowledge_interrupt();
                                           return_from_interrupt();
```

(a)

(b)

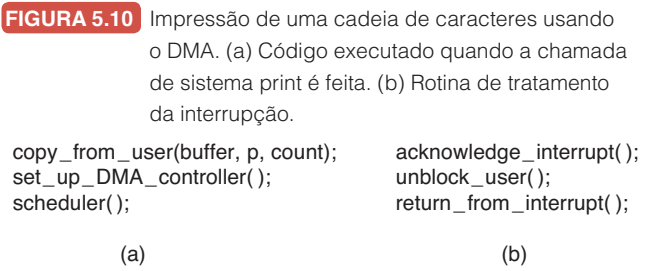
5.2.4 E/S usando DMA

Uma desvantagem óbvia do mecanismo de E/S orientado a interrupções é que uma interrupção ocorre em cada caractere. Interrupções levam tempo; portanto, esse esquema desperdiça certa quantidade de tempo da CPU. Uma solução é usar o acesso direto à memória (DMA). Aqui a ideia é deixar que o controlador de DMA alimente os caracteres para a impressora um de cada vez, sem que a CPU seja incomodada. Na essência, o DMA executa E/S programada, apenas com o controlador do DMA realizando todo o trabalho, em vez da CPU principal. Essa estratégia exige um hardware especial (o controlador de DMA), mas libera a CPU durante a E/S para fazer outros trabalhos. Uma linha geral do código é dada na Figura 5.10.

A grande vantagem do DMA é reduzir o número de interrupções de uma por caractere para uma por buffer impresso. Se houver muitos caracteres e as interrupções forem lentas, esse sistema poderá representar uma melhoria importante. Por outro lado, o controlador de DMA normalmente é muito mais lento do que a CPU principal. Se o controlador de DMA não é capaz de dirigir o dispositivo em velocidade máxima, ou a CPU normalmente não tem nada para fazer de qualquer forma enquanto esperando pela interrupção do DMA, então a E/S orientada à interrupção ou mesmo a E/S programada podem ser melhores. Na maioria das vezes, no entanto, o DMA vale a pena.

5.3 Camadas do software de E/S

O software de E/S costuma ser organizado em quatro camadas, como mostrado na Figura 5.11. Cada camada



tem uma função bem definida a desempenhar e uma interface bem definida para as camadas adjacentes. A funcionalidade e as interfaces diferem de sistema para sistema; portanto, a discussão que se segue, que examina todas as camadas começando de baixo, não é específica para uma máquina.

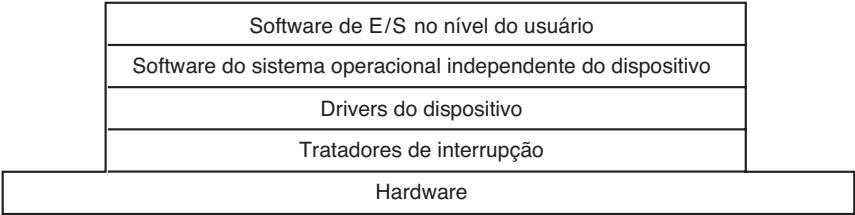
5.3.1 Tratadores de interrupção

Embora a E/S programada seja útil ocasionalmente, para a maioria das E/S, as interrupções são um fato desagradável da vida e não podem ser evitadas. Elas devem ser escondidas longe, nas profundezas do sistema operacional, de maneira que a menor parcela possível do sistema operacional saiba delas. A melhor maneira de escondê-las é bloquear o driver que inicializou uma operação de E/S até que ela se complete e a interrupção ocorra. O driver pode bloquear a si mesmo, por exemplo, realizando um down em um semáforo, um wait em uma variável de condição, um receive em uma mensagem, ou algo similar.

Quando a interrupção acontece, a rotina de interrupção faz o que for necessário a fim de lidar com ela. Então ela pode desbloquear o driver que a chamou. Em alguns casos, apenas completará a operação up em um semáforo. Em outras, ela emitirá um signal sobre uma variável de condição em um monitor. Em outras ainda, enviará uma mensagem para o driver bloqueado. Em todos os casos, o efeito resultante da interrupção será de que um driver que estava anteriormente bloqueado estará agora disponível para executar. Esse modelo funciona melhor se os drivers estiverem estruturados como processos do núcleo, com seus próprios estados, pilhas e contadores de programa.

É claro que a realidade não é tão simples. Processar uma interrupção não é apenas uma questão de interceptar a interrupção, realizar um up em algum semáforo e então executar uma instrução IRET para retornar da interrupção para o processo anterior. Há bem mais trabalho envolvido para o sistema operacional. Faremos agora um resumo desse trabalho como uma série de passos que devem ser realizados no software após a interrupção de hardware ter sido completada. Deve ser

FIGURA 5.11 Camadas do sistema de software de E/S.



observado que os detalhes são altamente dependentes no sistema, de maneira que alguns dos passos listados a seguir podem não ser necessários em uma máquina em particular, e passos não listados podem ser necessários. Além disso, os passos que ocorrem podem acontecer em uma ordem diferente em algumas máquinas.

1. Salvar quaisquer registros (incluindo o PSW) que ainda não foram salvos pelo hardware de interrupção.
2. Estabelecer um contexto para a rotina de tratamento da interrupção. Isso pode envolver a configuração de TLB, MMU e uma tabela de páginas.
3. Estabelecer uma pilha para a rotina de tratamento da interrupção.
4. Sinalizar o controlador de interrupções. Se não houver um controlador delas centralizado, reabilitá-las.
5. Copiar os registradores de onde eles foram salvos (possivelmente alguma pilha) para a tabela de processos.
6. Executar a rotina de tratamento de interrupção. Ela extrairá informações dos registradores do controlador do dispositivo que está interrompendo.
7. Escolher qual processo executar em seguida. Se a interrupção deixou pronto algum processo de alta prioridade que estava bloqueado, ele pode ser escolhido para executar agora.
8. Escolher o contexto de MMU para o próximo processo a executar. Algum ajuste na TBL também pode ser necessário.
9. Carregar os registradores do novo processo, incluindo sua PSW.
10. Começar a execução do novo processo.

Como podemos ver, o processamento da interrupção está longe de ser trivial. Ele também exige um número considerável de instruções da CPU, especialmente em máquinas nas quais a memória virtual está presente e as tabelas de páginas precisam ser atualizadas ou o estado da MMU armazenado (por exemplo, os bits *R* e *M*). Em algumas máquinas, a TLB e a cache da CPU talvez também tenham de ser gerenciadas quando trocam entre modos núcleo e usuário, que usam ciclos de máquina adicionais.

5.3.2 Drivers dos dispositivos

No início deste capítulo, examinamos o que fazem os controladores dos dispositivos. Vimos que cada controlador tem alguns registradores do dispositivo usados para dar a ele comandos ou para ler seu estado ou

ambos. O número de registradores do dispositivo e a natureza dos comandos variam radicalmente de dispositivo para dispositivo. Por exemplo, um driver de mouse tem de aceitar informações do mouse dizendo a ele o quanto ele se moveu e quais botões estão pressionados no momento. Em contrapartida, um driver de disco talvez tenha de saber tudo sobre setores, trilhas, cilindros, cabeçotes, movimento do braço, unidades do motor, tempos de ajuste do cabeçote e todas as outras mecânicas que fazem um disco funcionar adequadamente. Obviamente, esses drivers serão muito diferentes.

Em consequência, cada dispositivo de E/S ligado a um computador precisa de algum código específico do dispositivo para controlá-lo. Esse código, chamado **driver do dispositivo**, geralmente é escrito pelo fabricante do dispositivo e fornecido junto com ele. Tendo em vista que cada sistema operacional precisa dos seus próprios drivers, os fabricantes de dispositivos comumente fornecem drivers para vários sistemas operacionais populares.

Cada driver de dispositivo normalmente lida com um tipo, ou no máximo, uma classe de dispositivos muito relacionados. Por exemplo, um driver de disco SCSI em geral pode lidar com múltiplos discos SCSI de tamanhos e velocidades diferentes, e talvez um disco Blu-ray SCSI também. Por outro lado, um mouse e um joystick diferem tanto que drivers diferentes são normalmente necessários. No entanto, não há nenhuma restrição técnica sobre ter um driver do dispositivo controlando múltiplos dispositivos não relacionados. Apenas não é uma boa ideia *na maioria dos casos*.

Às vezes, no entanto, dispositivos completamente diferentes são baseados na mesma tecnologia subjacente. O exemplo mais conhecido é provavelmente o USB, uma tecnologia de barramento serial que não é chamada de “universal” por acaso. Dispositivos USB incluem discos, pen-drives, câmeras, mouses, teclados, miniventiladores, cartões de rede wireless, robôs, leitores de cartão de crédito, barbeadores recarregáveis, picotadores de papel, scanners de códigos de barras, bolas de espelhos e termômetros portáteis. Todos usam USB e, no entanto, todos realizam coisas muito diferentes. O truque é que drivers de USB são tipicamente empilhados, como uma pilha de TCP/IP em redes. Na parte de baixo, em geral no hardware, encontramos a camada do link do USB (E/S serial) que lida com questões de hardware como sinalização e decodificação de um fluxo de sinais para os pacotes do USB. Ele é usado por camadas mais altas que lidam com pacotes de dados e a funcionalidade comum para USB que é compartilhada pela maioria dos dispositivos.

Em cima disso, por fim, encontramos as APIs de camadas superiores, como as interfaces para armazenamento em massa, câmeras etc. Desse modo, ainda temos drivers de dispositivos em separado, embora eles compartilhem de parte da pilha de protocolo.

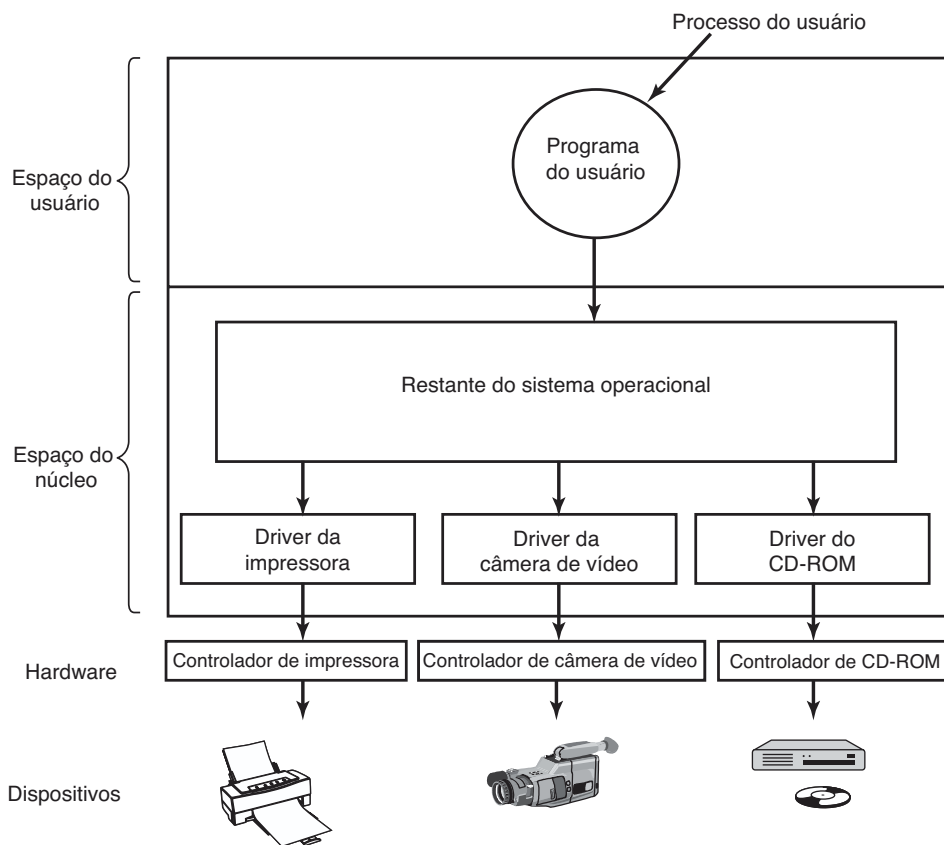
Para acessar o hardware do dispositivo, isto é, os registradores do controlador, o driver do dispositivo deve fazer parte do núcleo do sistema operacional, pelo menos com as arquiteturas atuais. Na realidade, é possível construir drivers que são executados no espaço do usuário, com chamadas de sistema para leitura e escrita nos registradores do dispositivo. Esse projeto isola o núcleo dos drivers e os drivers entre si, eliminando uma fonte importante de quedas no sistema — drivers defeituosos que interferem com o núcleo de uma maneira ou outra. Para construir sistemas altamente confiáveis, este é definitivamente o caminho a seguir. Um exemplo de um sistema no qual os drivers do dispositivo executam como processos do usuário é o MINIX 3 (<www.minix3.org>). No entanto, como a maioria dos sistemas operacionais de computadores de mesa espera que os drivers executem no núcleo, este será um modelo que consideraremos aqui.

Tendo em vista que os projetistas de cada sistema operacional sabem que pedaços de código (drivers) escritos por terceiros serão instalados no sistema operacional, este precisa ter uma arquitetura que permita essa instalação. Isso significa ter um modelo bem definido do que faz um driver e como ele interage com o resto do sistema operacional. Drivers de dispositivos são normalmente posicionados abaixo do resto do sistema operacional, como está ilustrado na Figura 5.12.

Sistemas operacionais normalmente classificam os drivers entre um número pequeno de categorias. As categorias mais comuns são os **dispositivos de blocos** — como discos, que contêm múltiplos blocos de dados que podem ser endereçados independentemente — e **dispositivos de caracteres**, como teclados e impressoras, que geram ou aceitam um fluxo de caracteres.

A maioria dos sistemas operacionais define uma interface padrão a que todos os drivers de blocos devem dar suporte e uma segunda interface padrão a que todos os drivers de caracteres devem dar suporte. Essas interfaces consistem em uma série de rotinas que o resto do sistema operacional pode utilizar para fazer o driver trabalhar para ele. Procedimentos típicos são aqueles que

FIGURA 5.12 Posicionamento lógico dos drivers de dispositivos. Na realidade, toda comunicação entre os drivers e os controladores dos dispositivos passa pelo barramento.



leem um bloco (dispositivo de blocos) ou escrevem uma cadeia de caracteres (dispositivo de caracteres).

Em alguns sistemas, o sistema operacional é um programa binário único que contém compilado em si todos os drivers de que ele precisará. Esse esquema era a norma por anos com sistemas UNIX, pois eles eram executados por centros computacionais e os dispositivos de E/S raramente mudavam. Se um novo dispositivo era acrescentado, o administrador do sistema simplesmente recompilava o núcleo com o driver novo para construir o binário novo.

Com o advento dos computadores pessoais, com sua miríade de dispositivos de E/S, esse modelo não funcionava mais. Poucos usuários são capazes de recompilar ou religar o núcleo, mesmo que eles tenham o código-fonte ou módulos-objeto, o que nem sempre é o caso. Em vez disso, sistemas operacionais, começando com o MS-DOS, se converteram em um modelo no qual os drivers eram dinamicamente carregados no sistema durante a execução. Sistemas diferentes lidam com o carregamento de drivers de maneiras diferentes.

Um driver de dispositivo apresenta diversas funções. A mais óbvia é aceitar solicitações abstratas de leitura e escrita de um software independente de dispositivo localizado na camada acima dele e verificar que elas sejam executadas. Mas há também algumas outras funções que ele deve realizar. Por exemplo, o driver deve inicializar o dispositivo, se necessário. Ele também pode precisar gerenciar suas necessidades de energia e registrar seus eventos.

Muitos drivers de dispositivos têm uma estrutura geral similar. Um driver típico inicia verificando os parâmetros de entrada para ver se eles são válidos. Se não, um erro é retornado. Se eles forem válidos, uma tradução dos termos abstratos para os termos concretos pode ser necessária. Para um driver de disco, isso pode significar converter um número de bloco linear em números de cabeçote, trilha, setor e cilindro para a geometria do disco.

Em seguida, o driver pode conferir se o dispositivo está em uso no momento. Se ele estiver, a solicitação entrará em uma fila para processamento posterior. Se o dispositivo estiver ocioso, o estado do hardware será examinado para ver se a solicitação pode ser cuidada agora. Talvez seja necessário ligar o dispositivo ou um motor antes que as transferências possam começar. Uma vez que o dispositivo esteja ligado e pronto para trabalhar, o controle de verdade pode começar.

Controlar o dispositivo significa emitir uma sequência de comandos para ele. O driver é o local onde a sequência de comandos é determinada, dependendo do

que precisa ser feito. Após o driver saber qual comando ele vai emitir, ele começa escrevendo-os nos registradores do controlador do dispositivo. Após cada comando ter sido escrito para o controlador, talvez seja necessário conferir para ver se este aceitou o comando e está preparado para aceitar o seguinte. Essa sequência continua até que todos os comandos tenham sido emitidos. Alguns controladores podem receber uma lista encadeada de comandos (na memória), tendo de ler e processar todos eles sem mais ajuda alguma do sistema operacional.

Após os comandos terem sido emitidos, ocorrerá uma de duas situações. Na maioria dos casos, o driver do dispositivo deve esperar até que o controlador realize algum trabalho por ele, de maneira que ele bloqueia a si mesmo até que a interrupção chegue para desbloqueá-lo. Em outros casos, no entanto, a operação termina sem atraso, então o driver não precisa bloquear. Como um exemplo da segunda situação, a rolagem da tela exige apenas escrever alguns bytes nos registradores do controlador. Nenhum movimento mecânico é necessário; assim, toda a operação pode ser completada em nanossegundos.

No primeiro caso, o driver bloqueado será despertado pela interrupção. No segundo caso, ele jamais dormirá. De qualquer maneira, após a operação ter sido completada, o driver deverá conferir a ocorrência de erros. Se tudo estiver bem, o driver poderá ter alguns dados para passar para o software independente do dispositivo (por exemplo, um bloco recém-lido). Por fim, ele retorna ao seu chamador alguma informação de estado para o relatório de erros. Se quaisquer outras solicitações estiverem na fila, uma delas poderá então ser selecionada e inicializada. Se nada estiver na fila, o driver bloqueará a espera para a próxima solicitação.

Esse modelo simples é apenas uma aproximação da realidade. Muitos fatores tornam o código muito mais complicado. Por um lado, um dispositivo de E/S pode completar uma tarefa enquanto um driver está sendo executado, interrompendo o driver. A interrupção pode colocar um driver em execução. Na realidade, ela pode fazer que o driver atual execute. Por exemplo, enquanto o driver da rede está processando um pacote que chega, outro pacote pode chegar. Em consequência, os drivers têm de ser **reentrantes**, significando que um driver em execução tem de estar preparado para ser chamado uma segunda vez antes que a primeira chamada tenha sido concluída.

Em um sistema manuseável em operação (hot-pluggable system), dispositivos podem ser adicionados ou removidos enquanto o computador está executando. Como resultado, enquanto um driver está ocupado

lendo de algum dispositivo, o sistema pode informá-lo de que o usuário removeu subitamente aquele dispositivo do sistema. Não apenas a transferência de E/S atual deve ser abortada sem danificar nenhuma estrutura de dados do núcleo, como quaisquer solicitações pendentes para o agora desaparecido dispositivo também devem ser cuidadosamente removidas do sistema e a má notícia ser dada aos processos que as requisitaram. Além disso, a adição inesperada de novos dispositivos pode levar o núcleo a fazer malabarismos com os recursos (por exemplo, linhas de solicitação de interrupção), tirando os mais antigos do driver e colocando novos em seu lugar.

Drivers não têm permissão para fazer chamadas de sistema, mas eles muitas vezes precisam interagir com o resto do núcleo. Em geral, são permitidas chamadas para determinadas rotinas de núcleo. Por exemplo, normalmente há chamadas para alocar e liberar páginas físicas de memória para usar como buffers. Outras chamadas úteis são necessárias para o gerenciamento da MMU, dos relógios, do controlador de DMA, do controlador de interrupção e assim por diante.

5.3.3 Software de E/S independente de dispositivo

Embora parte do software de E/S seja específico do dispositivo, outras partes dele são independentes. O limite exato entre os drivers e o software independente do dispositivo é dependente do sistema (e dispositivo), pois algumas funções que poderiam ser feitas de maneira independente do dispositivo podem na realidade ser feitas nos drivers, em busca de eficiência ou outras razões. As funções mostradas na Figura 5.13 são tipicamente realizadas em softwares independentes do dispositivo.

A função básica do software independente do dispositivo é realizar as funções de E/S que são comuns a todos os dispositivos e fornecer uma interface uniforme

para o software no nível do usuário. Examinaremos agora essas questões mais detalhadamente.

Interface uniforme para os drivers dos dispositivos

Uma questão importante em um sistema operacional é como fazer todos os dispositivos de E/S e drivers parecerem mais ou menos o mesmo. Se discos, impressoras, teclados e assim por diante possuem todos interfaces diferentes, sempre que um dispositivo novo aparece, o sistema operacional tem de ser modificado para o novo dispositivo. Ter de modificar o sistema operacional para cada dispositivo novo não é uma boa ideia.

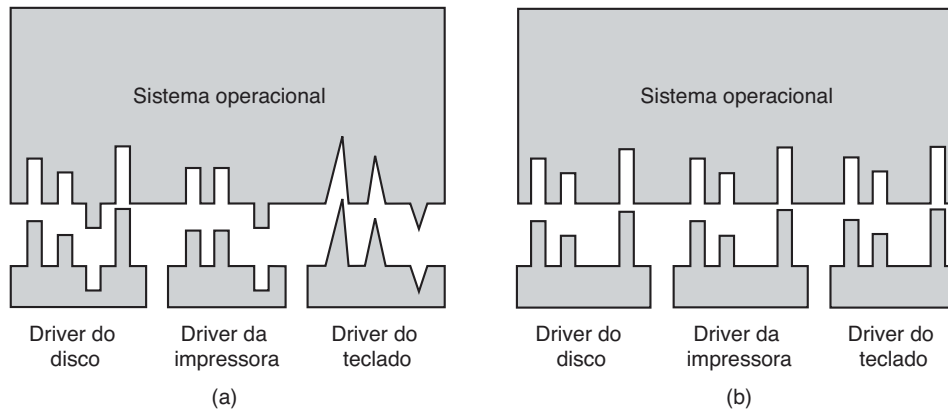
Um aspecto dessa questão é a interface entre os drivers do dispositivo e o resto do sistema operacional. Na Figura 5.14(a), ilustramos uma situação na qual cada driver do dispositivo tem uma interface diferente para o sistema operacional. O que isso significa é que as funções do driver disponíveis para o sistema chamar diferem de driver para driver. Também pode significar que as funções do núcleo de que o driver precisa também diferem de driver para driver. Como um todo, isso significa que fornecer uma interface para cada novo driver exige um novo esforço considerável de programação.

Em comparação, na Figura 5.14(b), mostramos um projeto diferente no qual todos os drivers têm a mesma interface. Nesse caso fica muito mais fácil acoplar um driver novo, desde que ele esteja em conformidade com a interface do driver. Também significa que os escritores de drivers sabem o que é esperado deles. Na prática, nem todos os dispositivos são absolutamente idênticos, mas costuma existir apenas um pequeno número de tipos de dispositivos e mesmo esses são geralmente quase os mesmos.

A maneira como isso funciona é a seguinte: para cada classe de dispositivos, como discos ou impressoras, o sistema operacional define um conjunto de funções que o driver deve fornecer. Para um disco, essas naturalmente incluiriam a leitura e a escrita, além de ligar e desligar a energia, formatação e outras coisas típicas de discos. Muitas vezes o driver contém uma tabela com ponteiros para si mesmo para essas funções. Quando o driver está carregado, o sistema operacional registra o endereço dessa tabela de ponteiros de funções, de maneira que, quando ela precisa chamar uma das funções, pode fazer uma chamada indireta via essa tabela. A tabela de ponteiros de funções define a interface entre o driver e o resto do sistema operacional. Todos os dispositivos de uma determinada classe (discos, impressoras etc.) devem obedecer a ela.

FIGURA 5.13 Funções do software de E/S independente do dispositivo.

Uniformizar interfaces para os drivers de dispositivos
Armazenar no buffer
Reportar erros
Alocar e liberar dispositivos dedicados
Providenciar um tamanho de bloco independente de dispositivo

FIGURA 5.14 (a) Sem uma interface-padrão para o driver. (b) Com uma interface-padrão para o driver.

Outro aspecto que surge quando se tem uma interface uniforme é como os dispositivos de E/S são nomeados. O software independente do dispositivo cuida do mapeamento de nomes de dispositivos simbólicos para o driver apropriado. No UNIX, por exemplo, o nome de um dispositivo como `/dev/disk0` especifica unicamente o i-node para um arquivo especial, e esse i-node contém o **número do dispositivo especial**, que é usado para localizar o driver apropriado. O i-node também contém o **número do dispositivo secundário**, que é passado como um parâmetro para o driver a fim de especificar a unidade a ser lida ou escrita. Todos os dispositivos têm números principal e secundário, e todos os drivers são acessados usando o número de dispositivo especial para selecionar o driver.

Estreitamente relacionada com a nomeação está a proteção. Como o sistema evita que os usuários acessem dispositivos aos quais eles não estão autorizados a acessar? Tanto no UNIX quanto no Windows os dispositivos aparecem no sistema de arquivos como objetos nomeados, o que significa que as regras de proteção usuais para os arquivos também se aplicam a dispositivos de E/S. O administrador do sistema pode então estabelecer as permissões adequadas para cada dispositivo.

Utilização de buffer

A utilização de buffer também é uma questão, tanto para dispositivos de bloco quanto de caracteres, por uma série de razões. Para ver uma delas, considere um processo que quer ler dados de um modem (ADSL — Asymmetric Digital Subscriber Line — linha de assinante digital assimétrica), algo que muitas pessoas usam em casa para conectar-se à internet. Uma estratégia possível para lidar com os caracteres que chegam é fazer o processo do usuário realizar uma chamada de

sistema `read` e bloquear a espera por um caractere. Cada caractere que chega causa uma interrupção. A rotina de tratamento da interrupção passa o caractere para o processo do usuário e o desbloqueia. Após colocar o caractere em algum lugar, o processo lê outro caractere e bloqueia novamente. Esse modelo está indicado na Figura 5.15(a).

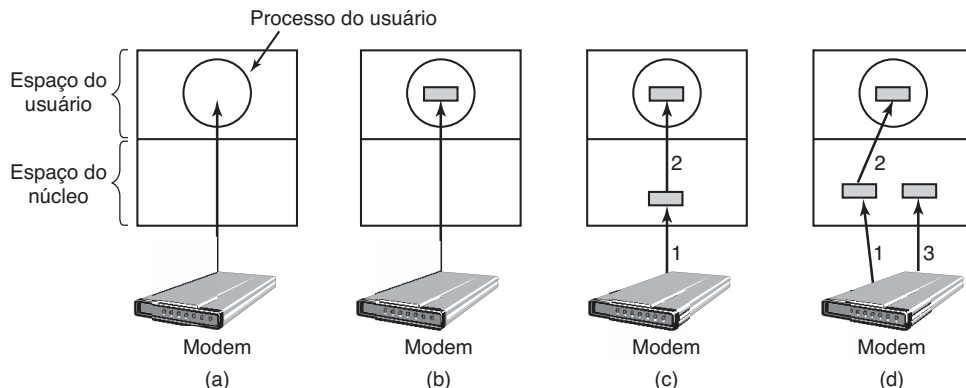
O problema com essa maneira de fazer negócios é que o processo do usuário precisa ser inicializado para cada caractere que chega. Permitir que um processo execute muitas vezes durante curtos intervalos de tempo é algo ineficiente; portanto, esse projeto não é uma boa opção.

Uma melhoria é mostrada na Figura 5.15(b). Aqui o processo do usuário fornece um buffer de n caracteres no espaço do usuário e faz uma leitura de n caracteres. A rotina de tratamento da interrupção coloca os caracteres que chegam nesse buffer até que ele esteja completamente cheio. Apenas então ele desperta o processo do usuário. Esse esquema é muito mais eficiente do que o anterior, mas tem uma desvantagem: o que acontece se o buffer for paginado para o disco quando um caractere chegar? O buffer poderia ser trancado na memória, mas se muitos processos começarem a trancar páginas na memória descuidadamente, o conjunto de páginas disponíveis diminuirá e o desempenho cairá.

Outra abordagem ainda é criar um buffer dentro do núcleo e fazer o tratador de interrupção colocar os caracteres ali, como mostrado na Figura 5.15(c). Quando esse buffer estiver cheio, a página com o buffer do usuário é trazida, se necessário, e o buffer copiado ali em uma operação. Esse esquema é muito mais eficiente.

No entanto, mesmo esse esquema melhorado sofre de um problema: o que acontece com os caracteres que chegam enquanto a página com o buffer do usuário está sendo trazida do disco? Já que o buffer está cheio, não há um lugar para colocá-los. Uma solução é ter um

FIGURA 5.15 (a) Entrada não enviada para buffer. (b) Utilização de buffer no espaço do usuário. (c) Utilização de buffer no núcleo seguido da cópia para o espaço do usuário. (d) Utilização de buffer duplo no núcleo.



segundo buffer de núcleo. Quando o primeiro buffer está cheio, mas antes que ele seja esvaziado, o segundo buffer é usado, como mostrado na Figura 5.15(d). Quando o segundo buffer enche, ele está disponível para ser copiado para o usuário (presumindo que o usuário tenha pedido por isso). Enquanto o segundo buffer está sendo copiado para o espaço do usuário, o primeiro pode ser usado para novos caracteres. Dessa maneira, os dois buffers se revezam: enquanto um está sendo copiado para o espaço do usuário, o outro está acumulando novas entradas. Esse tipo de esquema é chamado de **utilização de buffer duplo (double buffering)**.

Outra forma comum de utilização de buffer é o **buffer circular**. Ele consiste em uma região da memória e dois ponteiros. Um ponteiro aponta para a próxima palavra livre, onde novos dados podem ser colocados. O outro ponteiro aponta para a primeira palavra de dados no buffer que ainda não foi removida. Em muitas situações, o hardware avança o primeiro ponteiro à medida que ele acrescenta novos dados (por exemplo, recém-chegado da rede) e o sistema operacional avança o segundo ponteiro à medida que ele remove e processa dados. Ambos os ponteiros dão a volta, voltando para o início quando chegam ao topo.

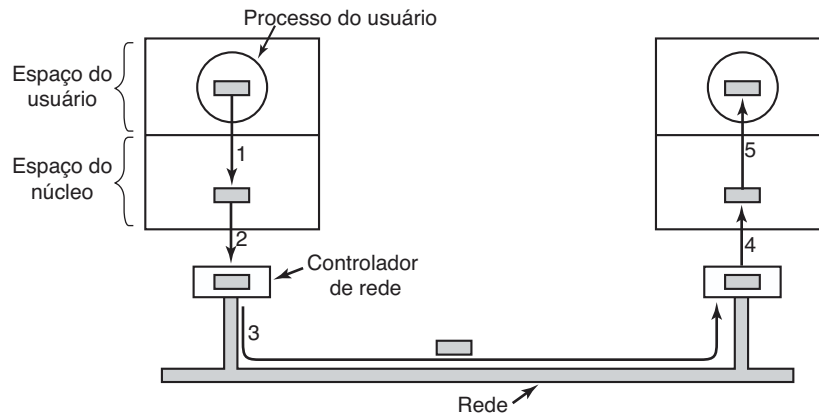
A utilização de buffer também é importante na saída de dados. Considere, por exemplo, como a saída é feita para o modem sem a utilização do buffer e usando o modelo da Figura 5.15(b). O processo do usuário executa uma chamada de sistema `write` para escrever n caracteres. O sistema tem duas escolhas a essa altura. Ele pode bloquear o usuário até que todos os caracteres tenham sido escritos, mas isso poderia levar muito tempo em uma linha de telefone lenta. Ele também poderia liberar o usuário imediatamente e realizar a E/S enquanto o usuário processa algo mais, mas isso leva a um problema ainda pior: como o processo do usuário vai

saber que a saída foi concluída e que ele pode reutilizar o buffer? O sistema poderia gerar um sinal ou interrupção de sinal, mas esse estilo de programação é difícil e propenso a condições de corrida. Uma solução muito melhor é o núcleo copiar os dados para um buffer de núcleo, análogo à Figura 5.15(c) (mas no outro sentido), e desbloquear o processo que chamou imediatamente. Agora não importa quando a E/S efetiva foi concluída. O usuário está livre para reutilizar o buffer no instante em que ele for desbloqueado.

A utilização de buffer é uma técnica amplamente utilizada, mas ele tem uma desvantagem também: se os dados forem armazenados em buffer vezes demais, o desempenho sofre. Considere, por exemplo, a rede da Figura 5.16. Aqui um usuário realiza uma chamada de sistema para escrever para a rede. O núcleo copia um pacote para um buffer do núcleo para permitir que o usuário proceda imediatamente (passo 1). Nesse ponto, o programa do usuário pode reutilizar o buffer.

Quando o driver é requisitado, ele copia o pacote para o controlador para saída (passo 2). A razão pela qual ele não transfere da memória do núcleo diretamente para o barramento é que uma vez inicializada uma transmissão do pacote, ela deve continuar a uma velocidade uniforme. O driver não pode garantir essa velocidade uniforme, pois canais de DMA e outros dispositivos de E/S podem estar roubando muitos ciclos. Uma falha na obtenção de uma palavra a tempo arruinaria o pacote. A utilização de buffer para o pacote dentro do controlador pode contornar esse problema.

Após o pacote ter sido copiado para o buffer interno do controlador, ele é copiado para a rede (passo 3). Os bits chegam ao receptor logo após terem sido enviados, de maneira que logo após o último bit ter sido enviado, aquele bit chega ao receptor, onde o pacote foi armazenado no buffer do controlador. Em seguida o pacote

FIGURA 5.16 O envio de dados pela rede pode envolver muitas cópias de um pacote.

é copiado para o buffer do núcleo do receptor (passo 4). Por fim, ele é copiado para o buffer do processo receptor (passo 5). Em geral, o receptor envia de volta uma confirmação do recebimento. Quando o emissor obtém a confirmação, ele está livre para enviar o próximo pacote. No entanto, deve ficar claro que toda essa operação de cópia vai retardar a taxa de transmissão de modo considerável, pois todos os passos devem acontecer sequencialmente.

Relatório de erros

Erros são muito mais comuns no contexto de E/S do que em outros contextos. Quando ocorrem, o sistema operacional deve lidar com eles da melhor maneira possível. Muitos erros são específicos de dispositivos e devem ser tratados pelo driver apropriado, mas o modelo para o tratamento de erros é independente do dispositivo.

Uma classe de erros de E/S é a dos erros de programação. Eles ocorrem quando um processo pede por algo impossível, como escrever em um dispositivo de entrada (teclado, scanner, mouse etc.) ou ler de um dispositivo de saída (impressora, plotter etc.). Outros erros incluem fornecer um endereço de buffer inválido ou outro parâmetro e especificar um dispositivo inválido (por exemplo, disco 3 quando o sistema tem apenas dois discos), e assim por diante. A ação a ser tomada a respeito desses erros é direta: simplesmente relatar de volta um código de erro para o chamador.

Outra classe de erros é a que engloba erros de E/S reais, por exemplo, tentar escrever em um bloco de disco que foi danificado ou tentar ler de uma câmera de vídeo que foi desligada. Se o driver não sabe o que fazer, ele pode passar o problema de volta para o software independente do dispositivo.

O que esse software faz depende do ambiente e da natureza do erro. Se for um simples erro de leitura e houver um usuário interativo disponível, ele poderá exibir uma caixa de diálogo perguntando ao usuário o que fazer. As opções podem incluir tentar de novo um determinado número de vezes, ignorar o erro, ou acabar com o processo que emitiu a chamada. Se não houver um usuário disponível, provavelmente a única opção real será relatar um código de erro indicando uma falha na chamada de sistema.

No entanto, alguns erros não podem ser gerenciados dessa maneira. Por exemplo, uma estrutura de dados crítica, como o diretório-raiz ou a lista de blocos livres, pode ter sido destruída. Nesse caso, o sistema pode ter de exibir uma mensagem de erro e desligar. Não há muito mais que possa ser feito.

Alocação e liberação de dispositivos dedicados

Alguns dispositivos, como impressoras, podem ser usados somente por um único processo a qualquer dado momento. Cabe ao sistema operacional examinar solicitações para o uso de dispositivos e aceitá-los ou rejeitá-los, dependendo de o dispositivo solicitado estar disponível ou não. Uma maneira simples de lidar com essas solicitações é exigir que os processos executem chamadas de sistema `open` diretamente nos arquivos especiais para os dispositivos. Se o dispositivo estiver indisponível, a chamada `open` falha. O fechamento desse dispositivo dedicado então o libera.

Uma abordagem alternativa é ter mecanismos especiais para solicitação e liberação de serviços dedicados. Uma tentativa de adquirir um dispositivo que não está disponível bloqueia o processo chamador em vez de causar uma falha. Processos bloqueados são colocados em uma fila. Mais cedo ou mais tarde, o dispositivo

solicitado fica disponível e o primeiro processo na fila tem permissão para adquiri-lo e continua a execução.

Tamanho de bloco independente de dispositivo

Discos diferentes podem ter tamanhos de setores diferentes. Cabe ao software independente do dispositivo esconder esse fato e fornecer um tamanho de bloco uniforme para as camadas superiores, por exemplo, tratando vários setores como um único bloco lógico. Dessa maneira, as camadas superiores lidam apenas com dispositivos abstratos, que usam o mesmo tamanho de bloco lógico, independentemente do tamanho do setor físico. De modo similar, alguns dispositivos de caracteres entregam seus dados um byte de cada vez (por exemplo, o modem), enquanto outros entregam seus dados em unidades maiores (por exemplo, interfaces de rede). Essas diferenças também podem ser escondidas.

5.3.4 Software de E/S do espaço do usuário

Embora a maior parte do software de E/S esteja dentro do sistema operacional, uma pequena porção dele consiste em bibliotecas ligadas aos programas do usuário e mesmo programas inteiros sendo executados fora do núcleo. Chamadas de sistema, incluindo chamadas de sistema de E/S, são normalmente feitas por rotinas de biblioteca. Quando um programa C contém a chamada

```
count = write(fd, buffer, nbytes);
```

a rotina de biblioteca *write* pode estar ligada com o programa e contida no programa binário presente na memória no tempo de execução. Em outros sistemas, bibliotecas podem ser carregadas durante a execução do programa. De qualquer maneira, a coleção de todas essas rotinas de biblioteca faz claramente parte do sistema de E/S.

Embora essas rotinas façam pouco mais do que colocar seus parâmetros no lugar apropriado para a chamada de sistema, outras rotinas de E/S na realidade fazem o trabalho de verdade. Em particular, a formatação de entrada e saída é feita pelas rotinas de biblioteca. Um exemplo de C é *printf*, que recebe uma cadeia de caracteres e possivelmente algumas variáveis como entrada, constrói uma cadeia ASCII, e então chama o *write* para colocá-la na saída. Como um exemplo de *printf*, considere o comando

```
printf("O quadrado de %3d e %6d\n", i, i*i);
```

Ele formata uma cadeia de caracteres constituída de 14 caracteres, "O quadrado de " seguido pelo valor *i* como uma cadeia de 3 caracteres, então a cadeia de 3

caracteres " e ", em seguida *i*² como 6 caracteres, e por fim, mais um caractere para mudança de linha.

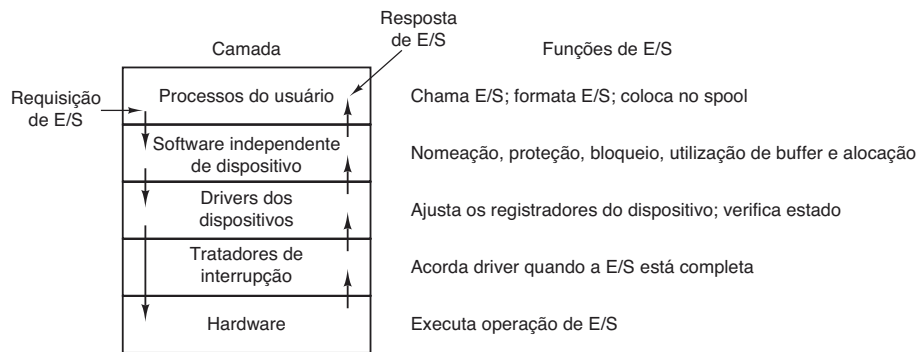
Um exemplo de uma rotina similar para entrada é *scanf*, que lê uma entrada e a armazena nas variáveis descritas em um formato de cadeia de caracteres usando a mesma sintaxe que *printf*. A biblioteca de E/S padrão contém um número de rotinas que envolvem E/S e todas executam como parte dos programas do usuário.

Nem todo software de E/S no nível do usuário consiste em rotinas de biblioteca. Outra categoria importante é o sistema de **spooling**. O uso de spool é uma maneira de lidar com dispositivos de E/S dedicados em um sistema de multiprogramação. Considere um dispositivo "spooled" típico: uma impressora. Embora seja tecnicamente fácil deixar qualquer processo do usuário abrir o arquivo especial de caractere para a impressora, suponha que um processo o abriu e então não fez nada por horas. Nenhum outro processo poderia imprimir nada.

Em vez disso, o que é feito é criar um processo especial, chamado **daemon**, e um diretório especial, chamado de **diretório de spooling**. Para imprimir um arquivo, um processo primeiro gera o arquivo inteiro a ser impresso e o coloca no diretório de spooling. Cabe ao daemon, que é o único processo com permissão de usar o arquivo especial da impressora, imprimir os arquivos no diretório. Ao proteger o arquivo especial contra o uso direto pelos usuários, o problema de ter alguém mantendo-o aberto por um tempo desnecessariamente longo é eliminado.

O spooling é usado não somente para impressoras. Ele também é empregado em outras situações de E/S. Por exemplo, a transferência de arquivos por uma rede muitas vezes usa um daemon de rede. Para enviar um arquivo para algum lugar, um usuário o coloca em um diretório de spooling da rede. Mais tarde, o daemon da rede o retira do diretório e o transmite. Um uso em particular da transmissão "spooled" de arquivos é o sistema de notícias USENET (agora parte do Google Groups). Essa rede consiste em milhões de máquinas mundo afora comunicando-se usando a internet. Milhares de grupos de notícias existem em muitos tópicos. Para postar uma mensagem de notícias, o usuário invoca um programa de notícias, o qual aceita a mensagem a ser postada e então a deposita em um diretório de spooling para transmissão para outras máquinas mais tarde. Todo o sistema de notícias executa fora do sistema operacional.

A Figura 5.17 resume o sistema de E/S, mostrando todas as camadas e as principais funções de cada uma. Começando na parte inferior, as camadas são o hardware, tratadores de interrupção, drivers do dispositivo, software independente de dispositivos e, por fim, os processos do usuário.

FIGURA 5.17 Camadas do sistema de E/S e as principais funções de cada camada.

As setas na Figura 5.17 mostram o fluxo de controle. Quando um programa do usuário tenta ler um bloco de um arquivo, por exemplo, o sistema operacional é invocado para executar a chamada. O software independente de dispositivos procura, digamos, na cache do buffer. Se o bloco necessário não está lá, ele chama o driver do dispositivo para emitir a solicitação para o hardware ir buscá-lo do disco. O processo é então bloqueado até que a operação do disco tenha sido completada e os dados estejam seguramente disponíveis no buffer do chamador.

Quando o disco termina, o hardware gera uma interrupção. O tratador de interrupção é executado para descobrir o que aconteceu, isto é, qual dispositivo quer atenção agora. Ele então extrai o estado do dispositivo e desperta o processo dormindo para finalizar a solicitação de E/S e deixar que o processo do usuário continue.

5.4 Discos

Agora começaremos a estudar alguns dispositivos de E/S reais. Começaremos com os discos, que são conceitualmente simples, mas muito importantes. Em seguida examinaremos relógios, teclados e monitores.

5.4.1 Hardware do disco

Existe uma série de tipos de discos. Os mais comuns são os discos rígidos magnéticos. Eles se caracterizam pelo fato de que leituras e escritas são igualmente rápidas, o que os torna adequados como memória secundária (paginação, sistemas de arquivos etc.). Arranjos desses discos são usados às vezes para fornecer um armazenamento altamente confiável. Para distribuição de programas, dados e filmes, discos ópticos (DVDs e Blu-ray) também são importantes. Por fim, discos de

estado sólido são cada dia mais populares à medida que eles são rápidos e não contêm partes móveis. Nas seções a seguir discutiremos discos magnéticos como um exemplo de hardware e então descreveremos o software para dispositivos de discos em geral.

Discos magnéticos

Discos magnéticos são organizados em cilindros, cada um contendo tantas trilhas quanto for o número de cabeçotes dispostos verticalmente. As trilhas são divididas em setores, com o número de setores em torno da circunferência sendo tipicamente 8 a 32 nos discos flexíveis e até várias centenas nos discos rígidos. O número de cabeçotes varia de 1 a cerca de 16.

Discos mais antigos têm pouca eletrônica e transmitem somente um fluxo de bits serial simples. Nesses discos, o controlador faz a maior parte do trabalho. Nos outros discos, em particular nos discos **IDE (Integrated Drive Electronics** — eletrônica integrada ao disco) e **SATA (Serial ATA** — ATA serial), a própria unidade contém um microcontrolador que realiza um trabalho considerável e permite que o controlador real emita um conjunto de comandos de nível mais elevado. O controlador muitas vezes controla a cache, faz o remapeamento de blocos defeituosos e muito mais.

Uma característica do dispositivo que tem implicações importantes para o driver do disco é a possibilidade de um controlador realizar buscas em duas ou mais unidades ao mesmo tempo. Elas são conhecidas como **buscas sobrepostas (overlapped seeks)**. Enquanto o controlador e o software estão esperando que uma busca seja concluída em uma unidade, o controlador pode iniciar uma busca em outra. Muitos controladores também podem ler ou escrever em uma unidade enquanto realizam uma busca em uma ou mais unidades, mas um controlador de disco flexível não pode ler ou escrever em duas unidades ao mesmo tempo. (A leitura

e a escrita exigem que o controlador mova bits em uma escala de tempo de microssegundos, então uma transferência usa quase todo o seu poder de computação.) A situação é diferente para discos rígidos com controladores integrados e, em um sistema com mais de um desses discos rígidos, eles podem operar simultaneamente, pelo menos até o ponto de transferência entre o disco e o buffer de memória do controlador. No entanto, apenas uma transferência entre o controlador e a memória principal é possível ao mesmo tempo. A capacidade de desempenhar duas ou mais operações ao mesmo tempo pode reduzir o tempo de acesso médio consideravelmente.

A Figura 5.18 compara parâmetros da mídia de armazenamento padrão para o PC IBM original com parâmetros de um disco feito três décadas mais tarde a fim de mostrar como os discos evoluíram de lá para cá. É interessante observar que nem todos os parâmetros tiveram a mesma evolução. O tempo médio de busca é quase 9 vezes melhor do que era, a taxa de transferência é 16 mil vezes melhor, enquanto a capacidade aumentou por um fator de 800 mil vezes. Esse padrão tem a ver com as melhorias relativamente graduais nas partes móveis, mas muito mais significativas nas densidades de bits das superfícies de gravação.

Um detalhe para o qual precisamos atentar ao examinarmos as especificações dos discos rígidos modernos é que a geometria especificada, e usada pelo driver, é quase sempre diferente do formato físico. Em discos antigos, o número de setores por trilha era o mesmo para todos os cilindros. Discos modernos são divididos em zonas com mais setores nas zonas externas do que nas

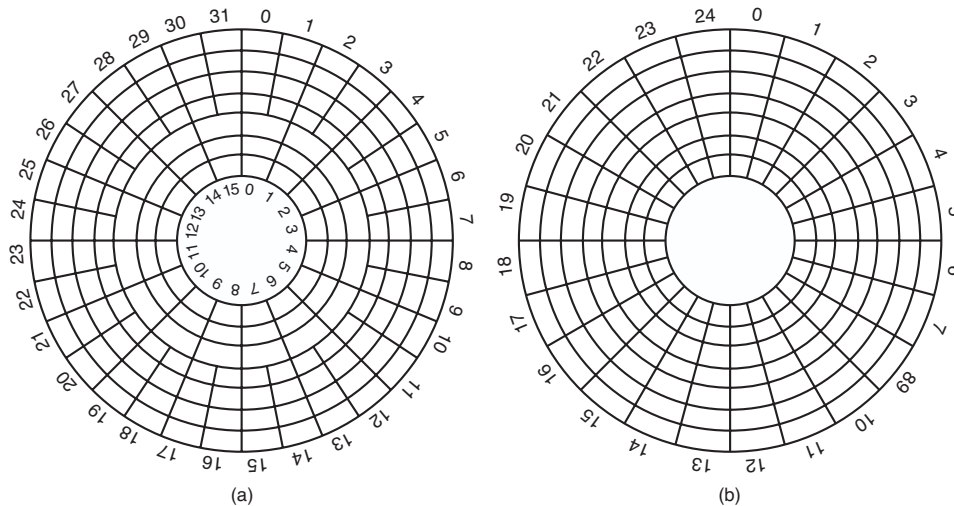
internas. A Figura 5.19(a) ilustra um disco pequeno com duas zonas. A zona externa tem 32 setores por trilha; a interna tem 16 setores por trilha. Um disco real, como o WD 3000 HLFS, costuma ter 16 ou mais zonas, com o número de setores aumentando em aproximadamente 4% por zona à medida que se vai da zona mais interna para a mais externa.

Para esconder os detalhes de quantos setores tem cada trilha, a maioria dos discos modernos tem uma geometria virtual que é apresentada ao sistema operacional. O software é instruído a agir como se houvesse x cilindros, y cabeçotes e z setores por trilha. O controlador então realiza um remapeamento de uma solicitação para (x, y, z) no cilindro, cabeçote e setor real. Uma geometria virtual possível para o disco físico da Figura 5.19(a) é mostrada na Figura 5.19(b). Em ambos os casos o disco tem 192 setores, apenas o arranjo publicado é diferente do real.

Para os PCs, os valores máximos para esses três parâmetros são muitas vezes (65535, 16 e 63), pela necessidade de eles continuarem compatíveis com as limitações do PC IBM original. Nessa máquina, campos de 16, 4 e 6 bits foram usados para especificar tais números, com cilindros e setores numerados começando em 1 e cabeçotes numerados começando em 0. Com esses parâmetros e 512 bytes por setor, o maior disco possível é 31,5 GB. Para contornar esse limite, todos os discos modernos aceitam um sistema chamado **endereço lógico de bloco (logical block addressing)**, no qual os setores do disco são numerados consecutivamente começando em 0, sem levar em consideração a geometria do disco.

FIGURA 5.18 Parâmetros de disco para o disco flexível original do IBM PC 360 KB e um disco rígido Western Digital WD 3000 HLFS (“Velociraptor”).

Parâmetro	Disco flexível IBM 360 KB	Disco rígido WD 3000 HLFS
Número de cilindros	40	36.481
Trilhas por cilindro	2	255
Setores por trilha	9	63 (em média)
Setores por disco	720	586.072.368
Bytes por setor	512	512
Capacidade do disco	360 KB	300 GB
Tempo de busca (cilindros adjacentes)	6 ms	0,7 ms
Tempo de busca (em média)	77 ms	4,2 ms
Tempo de rotação	200 ms	6 ms
Tempo de transferência de um setor	22 ms	1,4 μ s

FIGURA 5.19 (a) Geometria física de um disco com duas zonas. (b) Uma possível geometria virtual para esse disco.

RAID

O desempenho da CPU tem aumentado exponencialmente na última década, dobrando mais ou menos a cada 18 meses, mas nem tanto o desempenho de disco. Na década de 1970, os tempos de busca nos discos de minicomputadores eram de 50 a 100 ms. Hoje os tempos de busca atingem alguns ms. Na maioria das indústrias técnicas (digamos, automobilística e de aviação), um fator de melhoria no desempenho de 5 a 10 em duas décadas seria uma grande notícia (imagine carros andando 125 quilômetros por litro), mas na indústria dos computadores isso é uma vergonha. Desse modo, a diferença entre o desempenho da CPU e o do disco (rígido) tornou-se muito maior com o passar do tempo. Existe algo que possa ser feito para ajudar nessa situação?

Sim! Como vimos, o processamento paralelo está cada vez mais sendo usado para acelerar o desempenho da CPU. Ocorreu a várias pessoas ao longo dos anos que a E/S paralela poderia ser uma boa ideia também. Em seu estudo de 1988, Patterson et al. sugeriram seis organizações de disco específicas que poderiam ser usadas para melhorar o desempenho do disco, sua confiabilidade, ou ambos (PATTERSON et al., 1988). Essas ideias foram rapidamente adotadas pela indústria e levaram a uma nova classe de dispositivo de E/S chamada **RAID**. Patterson et al. definiram RAID como **arranjo redundante de discos baratos (Redundant Array of Inexpensive Disks)**, mas a indústria redefiniu o I como “*Independent*” em vez de “*Inexpensive*” (barato) — quem sabe para cobrarem mais? Já que um vilão

também era necessário (como em RISC *versus* CISC, também por causa de Patterson), o bandido aqui foi o **disco único grande e caro (SLED — Single Large Expensive Disk)**.

A ideia fundamental por trás de um RAID é instalar uma caixa cheia de discos junto ao computador, em geral um grande servidor, substituir a placa controladora de disco com um controlador RAID, copiar os dados para o RAID e então continuar a operação normal. Em outras palavras, um RAID deve parecer com um SLED para o sistema operacional, mas ter um desempenho melhor e mais confiável. No passado, RAIDs consistiam quase exclusivamente em um controlador SCSI RAID mais uma caixa de discos SCSI, pois o desempenho era bom e o SCSI moderno suporta até 15 discos em um único controlador. Hoje, muitos fabricantes também oferecem RAIDs (mais baratos) baseados no SATA. Dessa maneira, nenhuma mudança no software é necessária para usar o RAID, um grande atrativo para muitos administradores de sistemas.

Além de se parecer como um disco único para o software, todos os RAIDs têm a propriedade de que os dados são distribuídos pelos dispositivos, a fim de permitir a operação em paralelo. Vários esquemas diferentes para fazer isso foram definidos por Patterson et al. Hoje, a maioria dos fabricantes refere-se às sete configurações padrão com RAID nível 0 a RAID nível 6. Além deles, existem alguns outros níveis secundários que não discutiremos. O termo “nível” é de certa maneira equivocado, pois nenhuma hierarquia está envolvida; simplesmente há sete organizações diferentes possíveis.

O RAID nível 0 está ilustrado na Figura 5.20(a). Ele consiste em ver o disco único virtual simulado pelo RAID como dividido em faixas de k setores cada, com os setores 0 a $k - 1$ sendo a faixa 0, setores k a $2k - 1$ faixa 1, e assim por diante. Para $k = 1$, cada faixa é um setor, para $k = 2$ uma faixa são dois setores etc. A organização do RAID nível 0 grava faixas consecutivas nos discos em um estilo de alternância circular (round-robin), como descrito na Figura 5.20(a) para um RAID com quatro discos.

Essa distribuição de dados por meio de múltiplos discos é chamada de **striping**. Por exemplo, se o software emitir um comando para ler um bloco de dados consistindo em quatro faixas consecutivas começando no limite de uma faixa, o controlador de RAID dividirá esse comando em quatro comandos separados, um para cada um dos quatro discos, e os fará operarem em paralelo. Desse modo, temos E/S paralela sem que o software saiba a respeito.

O RAID nível 0 funciona melhor com grandes solicitações, quanto maiores melhor. Se uma solicitação for maior que o produto do número de discos pelo tamanho da faixa, alguns discos receberão múltiplas solicitações, assim quando terminam a primeira, eles iniciam a segunda. Cabe ao controlador dividir a solicitação e alimentar os comandos apropriados para os discos apropriados na sequência certa e então montar os resultados na memória corretamente. O desempenho é excelente e a implementação, direta.

O RAID nível 0 funciona pior com sistemas operacionais que habitualmente pedem por dados um setor de cada vez. Os resultados estarão corretos, mas não haverá paralelismo e, por conseguinte, nenhum ganho em desempenho. Outra desvantagem dessa organização é que a sua confiabilidade é potencialmente pior do que ter um SLED. Se um RAID consiste em quatro discos, cada um com um tempo médio de falha de 20 mil horas, cerca de uma vez a cada 5 mil horas um disco falhará e todos os dados serão completamente perdidos. Um SLED com um tempo médio de falha de 20 mil seria quatro vezes mais confiável. Como nenhuma redundância está presente nesse projeto, ele não é de fato um RAID.

A próxima opção, RAID nível 1, mostrada na Figura 5.20(b), é um RAID de fato. Ele duplica todos os discos, portanto há quatro discos primários e quatro de backup. Em uma escrita, cada faixa é escrita duas vezes. Em uma leitura, cada cópia pode ser usada, distribuindo a carga através de mais discos. Em consequência, o desempenho de escrita não é melhor do que para um disco único, mas o desempenho de leitura pode ser duas vezes melhor. A tolerância a falhas é excelente: se um disco quebra, a cópia é simplesmente usada em seu lugar. A recuperação consiste em apenas instalar um disco novo e copiar o backup inteiro para ele.

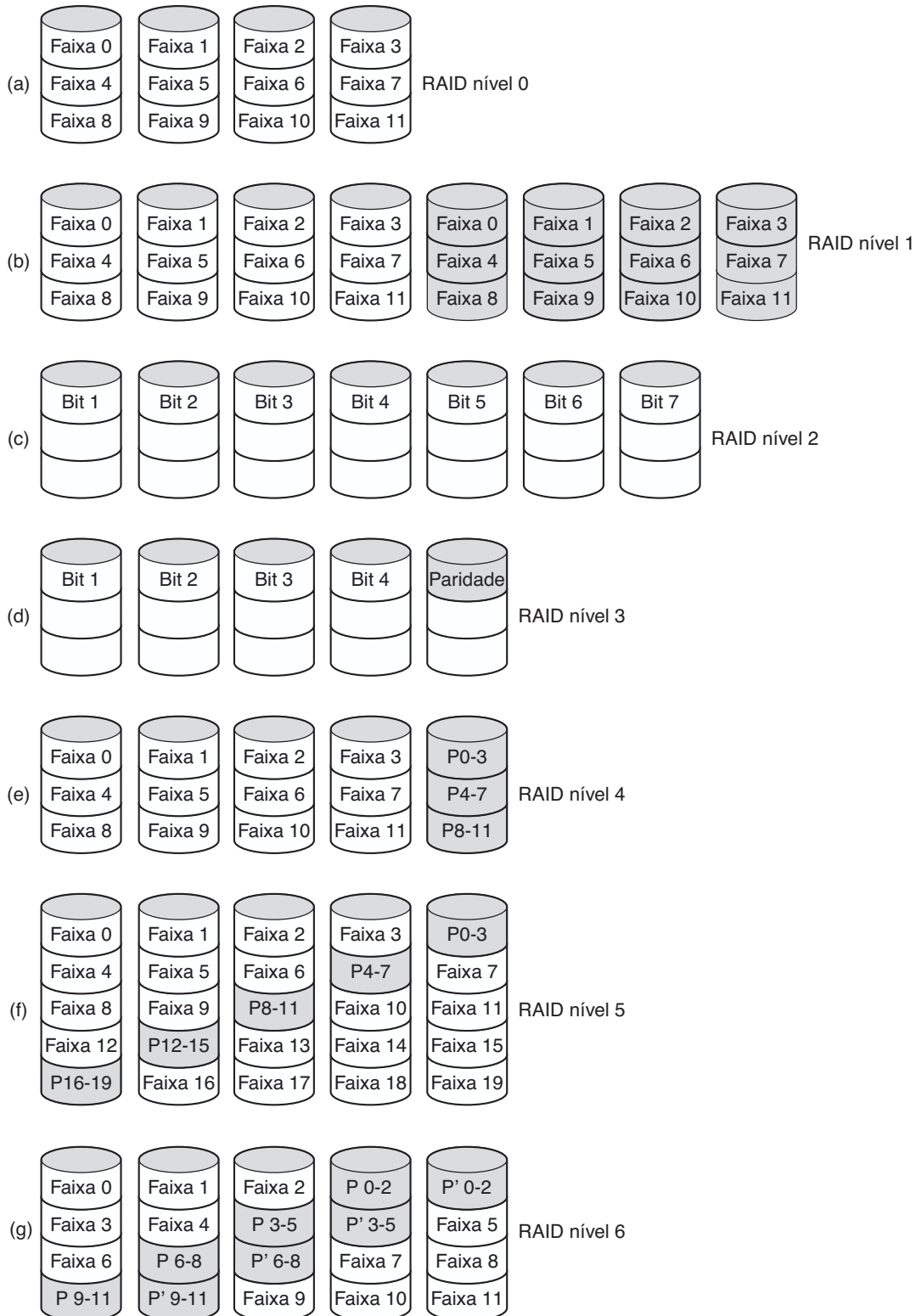
Diferentemente dos níveis 0 e 1, que trabalham com faixas de setores, o RAID nível 2 trabalha com palavras, talvez mesmo com bytes. Imagine dividir cada byte de um único disco virtual em um par de pedaços de 4 bits cada, então acrescentar um código Hamming para cada um para formar uma palavra de 7 bits, das quais os bits 1, 2 e 4 são de paridade. Imagine ainda que os sete discos da Figura 5.20(c) foram sincronizados em termos de posição do braço e posição rotacional. Então seria possível escrever a palavra de 7 bits codificada com Hamming nos sete discos, um bit por disco.

O computador CM-2 da Thinking Machines usava esse esquema, tomando palavras de dados de 32 bits e adicionando 6 bits de paridade para formar uma palavra Hamming de 38 bits, mais um bit extra para paridade de palavras, e espalhando cada palavra sobre 39 discos. O ganho total era imenso, pois em um tempo de setor ele podia escrever o equivalente de 32 setores de dados. Também, a perda de um disco não causava problemas, pois isso equivalia a perder 1 bit em cada palavra de 39 bits, algo que o código Hamming podia manejar sem a necessidade de parar o sistema.

A desvantagem é que esse esquema exige que todos os discos tenham suas rotações sincronizadas, e isso só faz sentido com um número substancial de discos (mesmo com 32 discos de dados e 6 discos de paridade, a sobrecarga é 19%). Ele também exige muito do controlador, visto que é necessário fazer a verificação de erro do código Hamming a cada chegada de bit.

O RAID nível 3 é uma versão simplificada do RAID nível 2. Ele está ilustrado na Figura 5.20(d). Aqui um único bit de paridade é calculado para cada palavra de dados e escrito para o disco de paridade. Como no RAID nível 2, os discos devem estar exatamente sincronizados, tendo em vista que palavras de dados individuais estão espalhadas por múltiplos discos.

Em um primeiro momento, poderia parecer que um único bit de paridade fornece apenas a detecção de erros, não a correção deles. Para o caso de erros não detectados aleatórios, essa observação é verdadeira. No entanto, para o caso de uma quebra de disco, ele fornece a correção completa do erro de 1 bit, pois a posição do bit defeituoso é conhecida. Caso um disco quebre, o controlador apenas finge que todos os bits são 0s. Se uma palavra tem um erro de paridade, o bit do disco quebrado deve ter sido um 1, então ele é corrigido. Embora ambos os níveis RAID 2 e 3 ofereçam taxas de dados muito altas, o número de solicitações de E/S separadas por segundo que eles podem tratar não é melhor do que para um único disco.

FIGURA 5.20 Níveis RAID 0 a 6. Os discos de backup e paridade estão sombreados.

Os níveis RAID 4 e 5 trabalham novamente com faixas, não palavras individuais com paridade, e não exigem discos sincronizados. O RAID nível 4 [ver Figura

5.20(e)] é como o RAID nível 0, com uma paridade faixa-por-faixa escrita em um disco extra. Por exemplo, se cada faixa tiver k bytes de comprimento, todas as faixas

são processadas juntas por meio de um OU EXCLUSIVO, resultando em uma faixa de paridade de k bytes de comprimento. Se um disco quebra, os bytes perdidos podem ser recalculados do disco de paridade por uma leitura do conjunto inteiro de discos.

Esse projeto protege contra a perda de um disco, mas tem um desempenho ruim para atualizações pequenas. Se um setor for modificado, é necessário ler todos os arquivos a fim de recalculer a paridade, que então deve ser reescrita. Como alternativa, ele pode ler os dados antigos do usuário, assim como os dados antigos de paridade, e recalculer a nova paridade a partir deles. Mesmo com essa otimização, uma pequena atualização exige duas leituras e duas escritas.

Em consequência da carga pesada sobre o disco de paridade, ele pode tornar-se um gargalo. Esse gargalo é eliminado no RAID nível 5 distribuindo os bits de paridade uniformemente por todos os discos, de modo circular (round-robin), como mostrado na Figura 5.20(f). No entanto, caso ocorra uma quebra do disco, a reconstrução do disco falhado é um processo complexo.

O RAID nível 6 é similar ao RAID nível 5, exceto que um bloco de paridade adicional é usado. Em outras palavras, os dados são divididos pelos discos com dois blocos de paridade em vez de um. Em consequência, as escritas são um pouco mais caras por causa dos cálculos de paridade, mas as leituras não incorrem em nenhuma penalidade de desempenho. Ele oferece mais confiabilidade (imagine o que acontece se um RAID nível 5 encontra um bloco defeituoso bem no momento em que ele está reconstruindo seu conjunto).

5.4.2 Formatação de disco

Um disco rígido consiste em uma pilha de pratos de alumínio, liga metálica ou vidro, em geral com 8,9 cm de diâmetro (ou 6,35 cm em notebooks). Em cada prato é depositada uma fina camada de um óxido de metal magnetizado. Após a fabricação, não há informação alguma no disco.

Antes que o disco possa ser usado, cada prato deve passar por uma **formatação de baixo nível** feita por software. A formatação consiste em uma série de trilhas concêntricas, cada uma contendo uma série de setores, com pequenos intervalos entre eles. O formato de um setor é mostrado na Figura 5.21.

O preâmbulo começa com um determinado padrão de bits que permite que o hardware reconheça o começo do setor. Ele também contém os números do cilindro e setor, assim como outras informações. O tamanho da porção de dados é determinado pelo programa de formatação de baixo nível. A maioria dos discos usa setores de 512 bytes. O campo ECC contém informações redundantes que podem ser usadas para a recuperação de erros de leitura. O tamanho e o conteúdo desse campo variam de fabricante para fabricante, dependendo de quanto espaço de disco o projetista está disposto a abrir mão em prol de uma maior confiabilidade, assim como o grau de complexidade do código de ECC que o controlador é capaz de manejar. Um campo de ECC de 16 bytes não é incomum. Além disso, todos os discos rígidos têm algum número de setores sobressalentes alocados para serem usados para substituir setores com defeito de fabricação.

A posição do setor 0 em cada trilha é deslocada com relação à trilha anterior quando a formatação de baixo nível é realizada. Esse deslocamento, chamado de **deslocamento de cilindro** (cylinder skew), é feito para melhorar o desempenho. A ideia é permitir que o disco leia múltiplas trilhas em uma operação contínua sem perder dados. A natureza do problema pode ser vista examinando-se a Figura 5.19(a). Suponha que uma solicitação precise de 18 setores começando no setor 0 da trilha mais interna. A leitura dos primeiros 16 setores leva a uma rotação de disco, mas uma busca é necessária para mover o cabeçote de leitura/gravação para a trilha seguinte, mais externa, no setor 17. No momento em que o cabeçote se deslocou uma trilha, o setor 0 já passou por ele, então uma rotação inteira é necessária até que ele volte novamente. Esse problema é eliminado deslocando-se os setores como mostrado na Figura 5.22.

A intensidade de deslocamento de cilindro depende da geometria do disco. Por exemplo, um disco de 10.000 RPM (rotações por minuto) leva 6 ms para realizar uma rotação completa. Se uma trilha contém 300 setores, um novo setor passa sob o cabeçote a cada 20 μ s. Se o tempo de busca de uma trilha para outra for 800 μ s, 40 setores passarão durante a busca, então o deslocamento de cilindro deve ter ao menos 40 setores, em vez dos três mostrados na Figura 5.22. Vale a pena observar que o chaveamento entre cabeçotes também leva um tempo finito; portanto, existe também um **deslocamento de cabeçote** assim como um de cilindro,

FIGURA 5.21 Um setor de disco.

Preâmbulo	Dados	ECC
-----------	-------	-----

mas o deslocamento de cabeçote não é muito grande, normalmente muito menor do que um tempo de setor.

Como resultado da formatação de baixo nível, a capacidade do disco é reduzida, dependendo dos tamanhos do preâmbulo, do intervalo entre setores e do ECC, assim como o número de setores sobressalentes reservado. Muitas vezes a capacidade formatada é 20% mais baixa do que a não formatada. Os setores sobressalentes não contam para a capacidade formatada; então, todos os discos de um determinado tipo têm exatamente a mesma capacidade quando enviados, independentemente de quantos setores defeituosos eles de fato têm (se o número de setores defeituosos exceder o de sobressalentes, o disco será rejeitado e não enviado).

Há uma confusão considerável a respeito da capacidade de disco porque alguns fabricantes anunciavam a capacidade não formatada para fazer seus discos parecerem maiores do que eles eram na realidade. Por exemplo, vamos considerar um disco cuja capacidade não formatada é de 200×10^9 bytes. Ele poderia ser vendido como um disco de 200 GB. No entanto, após a formatação, possivelmente apenas 170×10^9 bytes estavam disponíveis para dados. Para aumentar a confusão, o sistema operacional provavelmente relatará essa capacidade como sendo 158 GB, não 170 GB, pois o software considera uma memória de 1 GB como sendo 2^{30} (1.073.741.824) bytes, não 10^9 (1.000.000.000) bytes. Seria melhor se isso fosse relatado como 158 GiB.

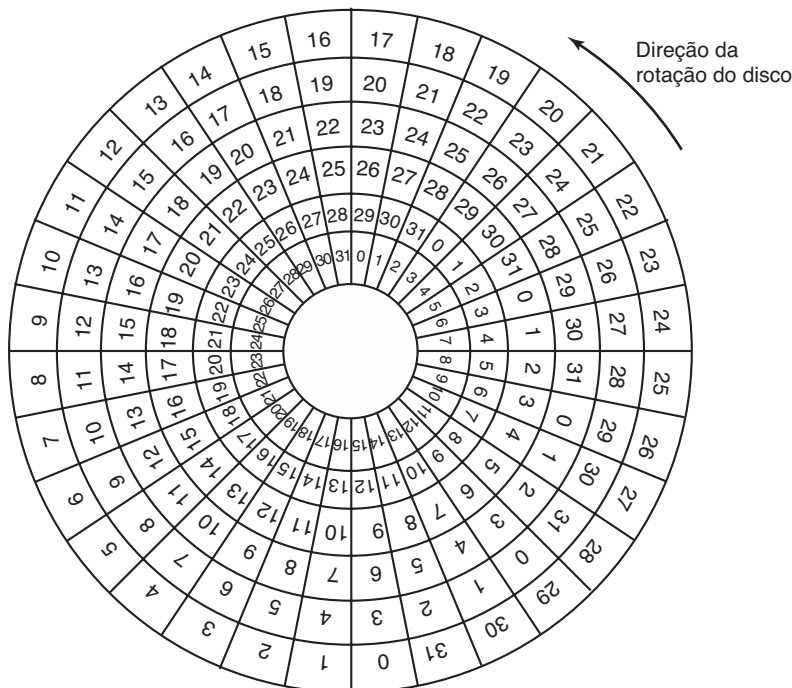
Para piorar ainda mais as coisas, no mundo das comunicações de dados, 1 Gbps significa 1 bilhão de bits/s porque o prefixo giga realmente significa 10^9 (um quilômetro tem 1.000 metros, não 1.024 metros, afinal de contas). Somente para os tamanhos de memória e de disco que as medidas quilo, mega, giga e tera significam 2^{10} , 2^{20} , 2^{30} e 2^{40} , respectivamente.

Para evitar confusão, alguns autores usam os prefixos quilo, mega, giga e tera para significar 10^3 , 10^6 , 10^9 e 10^{12} , respectivamente, enquanto usando kibi, mebi, gibi e tebi para significar 2^{10} , 2^{20} , 2^{30} e 2^{40} , respectivamente. No entanto, o uso dos prefixos “b” é relativamente raro. Apenas caso você goste de números realmente grandes, os prefixos após tebi são pebi, exbi, zebi e yobi, então um yobibyte representa uma quantidade considerável de bytes (2^{80} para ser preciso).

A formatação também afeta o desempenho. Se um disco de 10.000 RPM tem 300 setores por trilha de 512 bytes cada, ele leva 6 ms para ler os 153.600 bytes em uma trilha para uma taxa de dados de 25.600.000 bytes/s ou 24,4 MB/s. Não é possível ir mais rápido do que isso, não importa o tipo de interface que esteja presente, mesmo que seja uma interface SCSI a 80 MB/s ou 160 MB/s.

Na realidade, ler continuamente com essa taxa exige um buffer grande no controlador. Considere, por exemplo, um controlador com um buffer de um setor que tenha recebido um comando para ler dois

FIGURA 5.22 Uma ilustração do deslocamento de cilindro.



setores consecutivos. Após ler o primeiro setor do disco e realizar o cálculo ECC, os dados precisam ser transferidos para a memória principal. Enquanto essa transferência está sendo feita, o setor seguinte passará pelo cabeçote. Quando uma cópia para a memória for concluída, o controlador terá de esperar quase o tempo de uma rotação inteira para que o segundo setor dê a volta novamente.

Esse problema pode ser eliminado numerando os setores de maneira entrelaçada quando se formata o disco. Na Figura 5.23(a), vemos o padrão de numeração usual (ignorando o deslocamento de cilindro aqui). Na Figura 5.23(b), observa-se um **entrelaçamento simples** (single interleaving), que dá ao controlador algum descanso entre os setores consecutivos a fim de copiar o buffer para a memória principal.

Se o processo de cópia for muito lento, o **entrelaçamento duplo** da Figura 5.24(c) poderá ser necessário. Se o controlador tem um buffer de apenas um setor, não importa se a cópia do buffer para a memória principal é feita pelo controlador, a CPU principal ou um chip DMA; ela ainda leva algum tempo. Para evitar a necessidade do entrelaçamento, o controlador deve ser capaz de armazenar uma trilha inteira. A maioria dos controladores modernos consegue armazenar trilhas inteiras.

Após a formatação de baixo nível ter sido concluída, o disco é dividido em partições. Logicamente, cada partição é como um disco separado. Partições são necessárias para permitir que múltiplos sistemas operacionais coexistam. Também, em alguns casos, uma partição pode ser usada como área de troca (swapping). No x86 e na maioria dos outros computadores, o setor 0 contém o **registro mestre de inicialização (Master Boot Record — MBR)**, que contém um código de inicialização mais a tabela de partição no fim. O MBR, e desse modo o suporte para tabelas de partição, apareceu pela primeira vez nos PCs da IBM em 1983 para dar suporte ao então enorme disco rígido de 10 MB no PC XT. Os discos cresceram um pouco desde então. À medida que as entradas de partição MBR na maioria dos

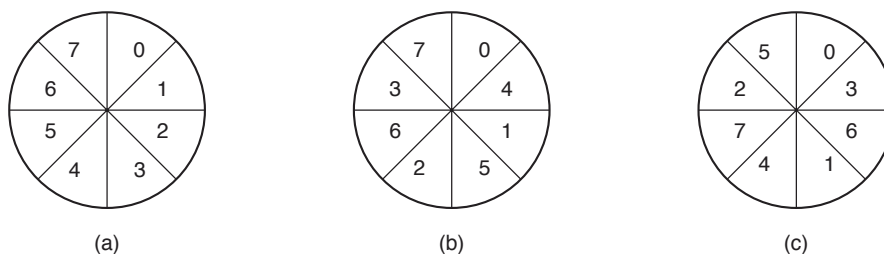
sistemas são limitadas a 32 bits, o tamanho de disco máximo que pode ser suportado pelos setores de 512 B é 2 TB. Por essa razão, a maioria dos sistemas operacionais desde então também suporta o novo **GPT (GUID Partition Table)**, que suporta discos de até 9,4 ZB (9.444.732.965.739.290.426.880 bytes). No momento em que este livro foi para a gráfica, isso era considerado um montão de bytes.

A tabela de partição dá o setor de inicialização e o tamanho de cada partição. No x86, a tabela de partição MBR tem espaço para quatro partições. Se todas forem para o Windows, elas serão chamadas C:, D:, E:, e F: e tratadas como discos separados. Se três delas forem para o Windows e uma para o UNIX, então o Windows chamará suas partições C:, D:, e E:. Se o drive USB for acrescentado, ele será o F:. Para ser capaz de inicializar do disco rígido, uma partição deve ser marcada como ativa na tabela de partição.

O passo final na preparação de um disco para ser usado é realizar uma **formatação de alto nível** de cada partição (separadamente). Essa operação insere um bloco de inicialização, a estrutura de gerenciamento de armazenamento livre (lista de blocos livres ou mapa de bits), diretório-raiz e um sistema de arquivo vazio. Ela também coloca um código na entrada da tabela de partições dizendo qual sistema de arquivos é usado na partição, pois muitos sistemas operacionais suportam múltiplos sistemas de arquivos incompatíveis (por razões históricas). Nesse ponto, o sistema pode ser inicializado.

Quando a energia é ligada, o BIOS entra em execução inicialmente e então carrega o registro mestre de inicialização e salta para ele. Esse programa então confere para ver qual partição está ativa. Então ele carrega o setor de inicialização específico daquela partição e o executa. O setor de inicialização contém um programa pequeno que geralmente carrega um carregador de inicialização maior que busca no sistema de arquivos para encontrar o núcleo do sistema operacional. Esse programa é carregado na memória e executado.

FIGURA 5.23 (a) Sem entrelaçamento. (b) Entrelaçamento simples. (c) Entrelaçamento duplo.



5.4.3 Algoritmos de escalonamento de braço de disco

Nesta seção examinaremos algumas das questões relacionadas com os drivers de disco em geral. Primeiro, considere quanto tempo é necessário para ler ou escrever um bloco de disco. O tempo exigido é determinado por três fatores.

1. Tempo de busca (o tempo para mover o braço para o cilindro correto).
2. Atraso de rotação (o tempo necessário para o setor correto aparecer sob o cabeçote).
3. Tempo de transferência real do dado.

Para a maioria dos discos, o tempo de busca domina os outros dois, então a redução do tempo de busca médio pode melhorar substancialmente o desempenho do sistema.

Se o driver do disco aceita as solicitações uma de cada vez e as atende nessa ordem, isto é, “**primeiro a chegar, primeiro a ser servido**” (FCFS — **First-Come, First-Served**), pouco pode ser feito para otimizar o tempo de busca. No entanto, outra estratégia é possível quando o disco está totalmente carregado. É provável que enquanto o braço está se posicionando para uma solicitação, outras solicitações de disco sejam geradas por outros processos. Muitos drivers de disco mantêm uma tabela, indexada pelo número do cilindro, com todas as solicitações pendentes para cada cilindro encadeadas juntas em uma lista ligada encabeçada pelas entradas da tabela.

Dado esse tipo de estrutura de dados, podemos melhorar o desempenho do algoritmo de escalonamento FCFS. Para ver como, considere um disco imaginário com 40 cilindros. Uma solicitação chega para ler um bloco no cilindro 11. Enquanto a busca para o cilindro 11 está em andamento, chegam novos

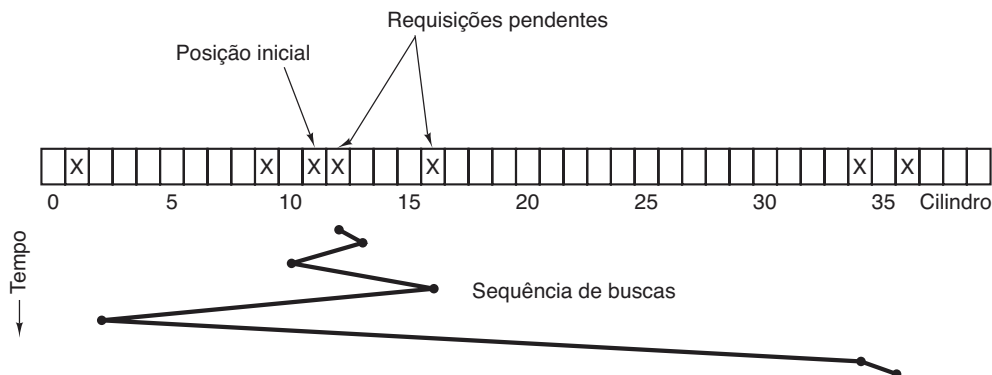
pedidos para os cilindros 1, 36, 16, 34, 9 e 12, nessa ordem. Elas são colocadas na tabela de solicitações pendentes, com uma lista encadeada separada para cada cilindro. As solicitações são mostradas na Figura 5.24.

Quando a solicitação atual (para o cilindro 11) tiver sido concluída, o driver do disco tem uma escolha de qual solicitação atender em seguida. Usando FCFS, ele iria em seguida para o cilindro 1, então para o 36, e assim por diante. Esse algoritmo exigiria movimentos de braço de 10, 35, 20, 18, 25 e 3, respectivamente, para um total de 111 cilindros.

Como alternativa, ele sempre poderia tratar com a solicitação mais próxima em seguida, a fim de minimizar o tempo de busca. Dadas as solicitações da Figura 5.24, a sequência é 12, 9, 16, 1, 34 e 36, como mostra a linha com quebras irregulares na base da Figura 5.24. Com essa sequência, os movimentos de braço são 1, 3, 7, 15, 33 e 2, para um total de 61 cilindros. Esse algoritmo, chamado de **busca mais curta primeiro** (SSF — **Shortest Seek First**), corta o movimento de braço total quase pela metade em comparação com o FCFS.

Infelizmente, o SSF tem um problema. Suponha que mais solicitações continuam chegando enquanto as da Figura 5.24 estão sendo processadas. Por exemplo, se, após ir ao cilindro 16, uma nova solicitação para o cilindro 8 for apresentada, esta terá prioridade sobre o cilindro 1. Se chegar então uma solicitação pelo cilindro 13, o braço irá em seguida para o 13, em vez do 1. Com um disco totalmente carregado, o braço tenderá a ficar no meio do disco na maior parte do tempo, de maneira que as solicitações em qualquer um dos extremos terão de esperar até que uma flutuação estatística na carga faça que não existam solicitações próximas do meio. As solicitações longe do meio talvez sejam mal servidas. As metas de tempo de resposta mínimo e justiça estão em conflito aqui.

FIGURA 5.24 Algoritmo de escalonamento “busca mais curta primeiro” (SSF — Shortest Seek First).



Prédios altos também têm de lidar com essa escolha. O problema do escalonamento de um elevador em um prédio alto é similar ao escalonamento de um braço de disco. Solicitações chegam continuamente chamando o elevador para os andares (cilindros) ao acaso. O computador no comando do elevador poderia facilmente manter a sequência na qual os clientes pressionaram o botão de chamada e servi-los usando FCFS ou SSF.

No entanto, a maioria dos elevadores usa um algoritmo diferente a fim de reconciliar as metas mutuamente conflitantes da eficiência e da justiça. Eles continuam se movimentando na mesma direção até que não existam mais solicitações pendentes naquela direção, então eles trocam de direção. Esse algoritmo, conhecido tanto no mundo dos discos quanto no dos elevadores como o **algoritmo do elevador**, exige que o software mantenha 1 bit: o bit de direção atual, *SOBE* ou *DESCE*. Quando uma solicitação é concluída, o driver do disco ou elevador verifica o bit. Se ele for *SOBE*, o braço ou a cabine se move para a próxima solicitação pendente mais alta. Se nenhuma solicitação estiver pendente em posições mais altas, o bit de direção é invertido. Quando o bit contém *DESCE*, o movimento será para a posição solicitada seguinte mais baixa, se houver alguma. Se nenhuma solicitação estiver pendente, ele simplesmente para e espera.

A Figura 5.25 mostra o algoritmo do elevador usando as mesmas solicitações que a Figura 5.24, presumindo que o bit de direção fosse inicialmente *SOBE*. A ordem na qual os cilindros são servidos é 12, 16, 34, 36, 9 e 1, o que resulta nos movimentos de braço de 1, 4, 18, 2, 27 e 8, para um total de 60 cilindros. Nesse caso, o algoritmo do elevador é ligeiramente melhor do que o SSF, embora ele seja em geral pior. Uma boa propriedade que o algoritmo do elevador tem é que dada qualquer coleção de solicitações, o limite máximo para a distância total é fixo: ele é apenas duas vezes o número de cilindros.

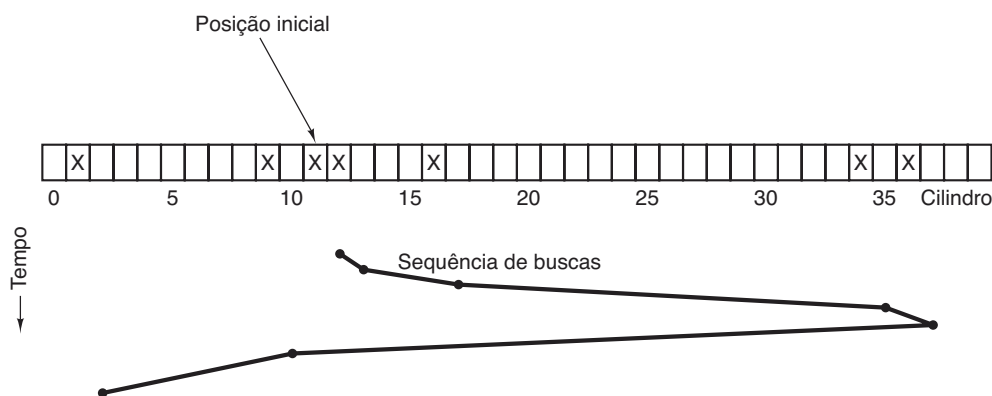
Uma ligeira modificação desse algoritmo que tem uma variação menor nos tempos de resposta (TEORY, 1972) consiste em sempre varrer as solicitações na mesma direção. Quando o cilindro com o número mais alto com uma solicitação pendente tiver sido atendido, o braço vai para o cilindro com o número mais baixo com uma solução pendente e então continua movendo-se para cima. Na realidade, o cilindro com o número mais baixo é visto como logo acima do cilindro com o número mais alto.

Alguns controladores de disco fornecem uma maneira para o software inspecionar o número de setor atual sob o cabeçote. Com esse controlador, outra otimização é possível. Se duas ou mais solicitações para o mesmo cilindro estiverem pendentes, o driver poderá emitir uma solicitação para o setor que passará sob o cabeçote em seguida. Observe que, quando múltiplas trilhas estão presentes em um cilindro, solicitações consecutivas podem ocorrer para diferentes trilhas sem uma penalidade. O controlador pode selecionar qualquer um dos seus cabeçotes quase instantaneamente (a seleção de cabeçotes não envolve nem o movimento de braço, tampouco um atraso rotacional).

Se o disco permitir que o tempo de busca seja muito mais rápido do que o atraso rotacional, então uma otimização diferente deverá ser usada. Solicitações pendentes devem ser ordenadas pelo número do setor, e tão logo o setor seguinte esteja próximo de passar sob o cabeçote, o braço deve ser movido rapidamente para a trilha certa para que a leitura ou a escrita seja realizada.

Com um disco rígido moderno, a busca e os atrasos rotacionais dominam de tal maneira o desempenho que a leitura de um ou dois setores de cada vez é algo ineficiente demais. Por essa razão, muitos controladores de disco sempre leem e armazenam múltiplos setores, mesmo quando apenas um é solicitado. Tipicamente, qualquer solicitação para ler um setor fará que aquele setor e grande parte ou todo o resto da trilha atual seja lida,

FIGURA 5.25 O algoritmo do elevador para o escalonamento de solicitações do disco.



dependendo de quanto espaço há disponível na memória de cache do controlador. O disco rígido descrito na Figura 5.18 tem uma cache de 4 MB, por exemplo. O uso da cache é determinado dinamicamente pelo controlador. No seu modo mais simples, a cache é dividida em duas seções, uma para leituras e outra para escritas. Se uma leitura subsequente puder ser satisfeita da cache do controlador, ele poderá retornar os dados solicitados imediatamente.

Vale a pena observar que a cache do controlador de disco é completamente independente da cache do sistema operacional. A cache do controlador em geral contém blocos que não foram realmente solicitados, mas cuja leitura era conveniente porque eles apenas estavam passando sob o cabeçote como um efeito colateral de alguma outra leitura. Em comparação, qualquer cache mantida pelo sistema operacional consistirá de blocos que foram explicitamente lidos e que o sistema operacional acredita que possam ser necessários novamente em um futuro próximo (por exemplo, um bloco de disco contendo um bloco de diretório).

Quando vários dispositivos estão presentes no mesmo controlador, o sistema operacional deve manter uma tabela de solicitações pendentes para cada dispositivo separadamente. Sempre que um dispositivo estiver ocioso, um comando de busca deve ser emitido para mover o seu braço para o cilindro onde ele será necessário em seguida (presumindo que o controlador permita buscas simultâneas). Quando a transferência atual é concluída, uma verificação deve ser feita para ver se algum dispositivo está posicionado no cilindro correto. Se um ou mais estiverem, a próxima transferência poderá ser inicializada em um drive que já esteja no cilindro correto. Se nenhum dos braços estiver no local correto, o driver deverá emitir um novo comando de busca sobre o dispositivo que acabou de completar uma transferência e esperar até a próxima interrupção para ver qual braço chega ao seu destino primeiro.

É importante perceber que todos os algoritmos de escalonamento de disco acima tacitamente presumem que a geometria de disco real é a mesma que a geometria virtual. Se não for, o escalonamento das solicitações de disco não fará sentido, pois o sistema operacional não poderá realmente dizer se o cilindro 40 ou o cilindro 200 está mais próximo do cilindro 39. Por outro lado, se o controlador do disco for capaz de aceitar múltiplas solicitações pendentes, ele poderá usar esses algoritmos de escalonamento internamente. Nesse caso, os algoritmos ainda serão válidos, mas em um nível abaixo, dentro do controlador.

5.4.4 Tratamento de erros

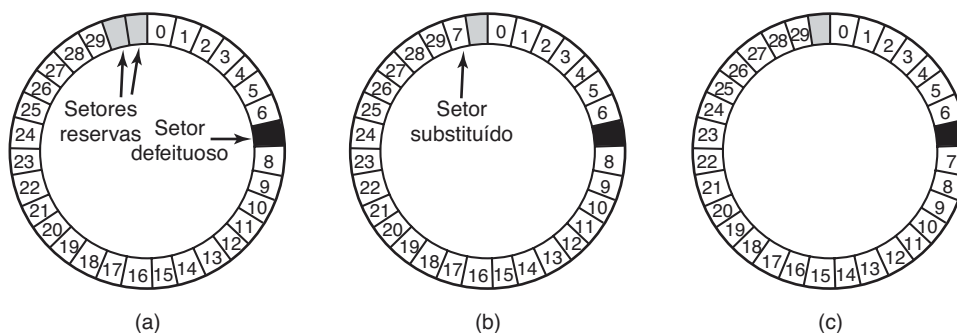
Os fabricantes de discos estão constantemente expandindo os limites da tecnologia por meio do aumento linear das densidades de bits. Uma trilha central de um disco de 13,33 cm tem uma circunferência de aproximadamente 300 mm. Se a trilha contiver 300 setores de 512 bytes, a densidade de gravação linear poderá ser de mais ou menos 5.000 bits/mm, levando em consideração o fato de que algum espaço é perdido para os preâmbulos, ECCs e intervalos entre os setores. A gravação de 5.000 bits/mm exige um substrato extremamente uniforme e uma camada muito fina de óxido. Infelizmente, não é possível fabricar um disco com essas especificações sem defeitos. Tão logo a tecnologia de fabricação melhore a ponto de ser possível operar sem falhas com essas densidades, os projetistas de discos projetarão densidades mais altas para aumentar a capacidade. Ao fazer isso, os defeitos provavelmente serão reintroduzidos.

Os defeitos de fabricação introduzem setores defeituosos, isto é, setores que não leem corretamente de volta o valor recém-escrito neles. Se o defeito for muito pequeno, digamos, apenas alguns bits, será possível usar o setor defeituoso e deixar que o ECC corrija os erros todas as vezes. Se o defeito for maior, o erro não poderá ser mascarado.

Existem duas abordagens gerais para blocos defeituosos: lidar com eles no controlador ou no sistema operacional. Na primeira abordagem, antes que o disco seja enviado da fábrica, ele é testado e uma lista de setores defeituosos é escrita no disco. Para cada setor defeituoso, um dos reservas o substitui.

Há duas maneiras de realizar essa substituição. Na Figura 5.26(a), vemos uma única trilha de disco com 30 setores de dados e dois reservas. O setor 7 é defeituoso. O que o controlador pode fazer é remapear um dos reservas como setor 7, como mostrado na Figura 5.26(b). A outra saída é deslocar todos os setores de uma posição, como mostrado na Figura 5.26(c). Em ambos os casos, o controlador precisa saber qual setor é qual. Ele pode controlar essa informação por meio de tabelas internas (uma por trilha) ou reescrevendo os preâmbulos para dar o novo número dos setores remapeados. Se os preâmbulos forem reescritos, o método da Figura 5.26(c) dará mais trabalho (pois 23 preâmbulos precisam ser reescritos), mas em última análise ele proporcionará um desempenho melhor, pois uma trilha inteira ainda poderá ser lida em uma rotação.

FIGURA 5.26 (a) Uma trilha de disco com um setor defeituoso. (b) Substituindo um setor defeituoso com um reserva. (c) Deslocando todos os setores para pular o setor defeituoso.



Erros também podem ser desenvolvidos durante a operação normal após o dispositivo ter sido instalado. A primeira linha de defesa para tratar um erro com que o ECC não consegue lidar é apenas tentar a leitura novamente. Alguns erros de leitura são passageiros, isto é, são causados por grãos de poeira sob o cabeçote e desaparecerão em uma segunda tentativa. Se o controlador notar que ele está encontrando erros repetidos em um determinado setor, pode trocar para um reserva antes que o setor morra completamente. Dessa maneira, nenhum dado é perdido e o sistema operacional e o usuário nem notam o problema. Em geral, o método da Figura 5.26(b) tem de ser usado, pois os outros setores podem conter dados agora. A utilização do método da Figura 5.26(c) exigiria não apenas reescrever os preâmbulos, como copiar todos os dados também.

Anteriormente dissemos que havia duas maneiras gerais para lidar com erros: lidar com eles no controlador ou no sistema operacional. Se o controlador não tiver a capacidade de remapear setores transparentemente como discutimos, o sistema operacional precisará fazer a mesma coisa no software. Isso significa que ele precisará primeiro adquirir uma lista de setores defeituosos, seja lendo a partir do disco, ou simplesmente testando o próprio disco inteiro. Uma vez que ele saiba quais setores são defeituosos, poderá construir as tabelas de remapeamento. Se o sistema operacional quiser usar a abordagem da Figura 5.26(c), deverá deslocar os dados dos setores 7 a 29 um setor para cima.

Se o sistema operacional estiver lidando com o remapeamento, ele deve certificar-se de que setores defeituosos não ocorram em nenhum arquivo e também não ocorram na lista de blocos livres ou mapa de bits. Uma maneira de fazer isso é criar um arquivo secreto consistindo em todos os setores defeituosos. Se esse arquivo não tiver sido inserido no sistema de arquivos, os usuários não o lerão acidentalmente (ou pior ainda, o liberarão).

No entanto, há ainda outro problema: backups. Se for feito um backup do disco, arquivo por arquivo, é importante que o utilitário usado para realizar o backup não tente copiar o arquivo com o bloco defeituoso. Para evitar isso, o sistema operacional precisa esconder o arquivo com o bloco defeituoso tão bem que mesmo esse utilitário para backup não consiga encontrá-lo. Se o disco for copiado setor por setor em vez de arquivo por arquivo, será difícil, se não impossível, evitar erros de leitura durante o backup. A única esperança é que o programa de backup seja esperto o suficiente para desistir depois de 10 leituras fracassadas e continuar com o próximo setor.

Setores defeituosos não são a única fonte de erros. Erros de busca causados por problemas mecânicos no braço também ocorrem. O controlador controla a posição do braço internamente. Para realizar uma busca, ele emite um comando para o motor do braço para movê-lo para o novo cilindro. Quando o braço chega ao seu destino, o controlador lê o número do cilindro real do preâmbulo do setor seguinte. Se o braço estiver no lugar errado, ocorreu um erro de busca.

A maioria dos controladores de disco rígido corrige erros de busca automaticamente, mas a maioria dos controladores de discos flexíveis usada nos anos de 1980 e 1990 apenas sinalizava um bit de erro e deixava o resto para o driver. O driver lidava com esse erro emitindo um comando *recalibrate*, a fim de mover o braço para o mais longe possível e reconfigurava a ideia interna do cilindro atual do controlador para 0. Normalmente, isso solucionava o problema. Se não solucionasse, o dispositivo tinha de ser reparado.

Como vimos há pouco, o controlador é de fato um pequeno computador especializado, completo com software, variáveis, buffers e, ocasionalmente, defeitos. Às vezes, uma sequência incomum de eventos, como uma interrupção em um dispositivo ocorrendo simultaneamente com um comando *recalibrate* em outro

dispositivo, desencadeia um erro e faz o controlador entrar em um laço infinito ou perder o caminho do que está fazendo. Projetistas de controladores costumam planejar para a pior situação, e assim fornecem um pino no chip que, quando sinalizado, força o controlador a esquecer o que estava fazendo e reiniciar-se. Se todo o resto falhar, o driver do disco poderá invocar esse sinal e reiniciar o controlador. Se isso não funcionar, tudo o que o driver pode fazer é imprimir uma mensagem e desistir.

Recalibrar um disco faz um ruído esquisito, mas de outra forma, não chega a incomodar. No entanto, há uma situação em que a recalibragem é um problema: sistemas com restrições de tempo real. Quando um vídeo está sendo exibido (ou servido) de um disco rígido, ou arquivos de um disco rígido estão sendo gravados em um disco Blu-ray, é fundamental que os bits cheguem do disco rígido a uma taxa uniforme. Nessas circunstâncias, as recalibrações inserem intervalos no fluxo de bits e são inaceitáveis. Dispositivos especiais, chamados **discos AV (audiovisuais)**, que nunca recalibram, estão disponíveis para tais aplicações.

Anedoticamente, uma demonstração muito convincente de quão avançados os controladores de disco se tornaram foi dada pelo hacker holandês Jeroen Domburg, que invadiu um controlador de disco moderno para fazê-lo executar com um código customizado. Na realidade o controlador de disco é equipado com um processador ARM multinúcleo (!) bastante potente e facilmente tem recursos suficientes para executar Linux. Se os hackers entrarem em seu disco rígido dessa maneira, serão capazes de ver e modificar todos os dados que você transfere do e para o disco. Mesmo reinstalar o sistema operacional desde o princípio não removerá a infecção, à medida que o próprio controlador do disco é malicioso e serve como uma porta dos fundos permanente. Em compensação, você pode coletar uma pilha de discos rígidos quebrados do seu centro de reciclagem local e construir seu próprio computador improvisado de graça.

5.4.5 Armazenamento estável

Como vimos, discos às vezes geram erros. Bons setores podem de repente tornar-se defeituosos. Dispositivos inteiros podem pifar inesperadamente. RAID's protegem contra alguns setores de apresentarem defeitos ou mesmo a quebra de um dispositivo. No entanto, não protegem contra erros de gravação que inserem dados corrompidos. Eles também não protegem contra

quedas do sistema durante a gravação corrompendo os dados originais sem substituí-los por dados novos.

Para algumas aplicações, é essencial que os dados nunca sejam perdidos ou corrompidos, mesmo diante de erros de disco e CPU. O ideal é que um disco deve funcionar simplesmente o tempo inteiro sem erros. Infelizmente, isso não é possível. O que é possível é um subsistema de disco que tenha a seguinte propriedade: quando uma escrita é lançada para ele, o disco ou escreve corretamente os dados ou não faz nada, deixando os dados existentes intactos. Esse sistema é chamado de **armazenamento estável** e é implementado no software (LAMPSON e STURGIS, 1979). A meta é manter o disco consistente a todo custo. A seguir descreveremos uma pequena variante da ideia original.

Antes de descrever o algoritmo, é importante termos um modelo claro dos erros possíveis. O modelo presume que, quando um disco escreve um bloco (um ou mais setores), ou a escrita está correta ou ela está incorreta e esse erro pode ser detectado em uma leitura subsequente examinando os valores dos campos ECC. Em princípio, a detecção de erros garantida jamais é possível, pois com um, digamos, campo de ECC de 16 bytes guardando um setor de 512 bytes, há 2^{4096} valores de dados e apenas 2^{144} valores ECC. Desse modo, se um bloco for distorcido durante a escrita — mas o ECC não —, existem bilhões e mais bilhões de combinações incorretas que resultam no mesmo ECC. Se qualquer uma delas ocorrer, o erro não será detectado. Como um todo, a probabilidade de dados aleatórios terem o ECC de 16 bytes apropriado é de mais ou menos 2^{-144} , que é uma probabilidade tão pequena que a chamaremos de zero, embora ela não seja realmente.

O modelo também presume que um setor escrito corretamente pode ficar defeituoso espontaneamente e tornar-se ilegível. No entanto, a suposição é que esses eventos são tão raros que a probabilidade de ter o mesmo setor danificado em um segundo dispositivo (independente) durante um intervalo de tempo razoável (por exemplo, 1 dia) é pequena o suficiente para ser ignorada.

O modelo também presume que a CPU pode falhar, caso em que ela simplesmente para. Qualquer escrita no disco em andamento no momento da falha também para, levando a dados incorretos em um setor e um ECC incorreto que pode ser detectado mais tarde. Com todas essas considerações, o armazenamento estável pode se tornar 100% confiável, no sentido de que as escritas funcionem corretamente ou deixem os dados antigos no lugar. É claro, ele não protege contra desastres físicos, como um terremoto acontecendo e o computador despencando 100 metros em uma fenda e caindo dentro de

um lago de lava fervendo. É difícil recuperar de uma situação dessas por software.

O armazenamento estável usa um par de discos idênticos com blocos correspondentes trabalhando juntos para formar um bloco livre de erros. Na ausência de erros, os blocos correspondentes em ambos os dispositivos são os mesmos. Qualquer um pode ser lido para conseguir o mesmo resultado. Para alcançar essa meta, as três operações a seguir são definidas:

1. **Escritas estáveis.** Uma escrita estável consiste em primeiro escrever um bloco na unidade 1, então lê-lo de volta para verificar que ele foi escrito corretamente. Se ele foi escrito incorretamente, a escrita e a releitura são feitas de novo por n vezes até que estejam corretas. Após n falhas consecutivas, o bloco é remapeado sobre um bloco reserva e a operação repetida até que tenha sucesso, não importa quantos reservas sejam tentados. Após a escrita para a unidade 1 ter sido bem-sucedida, o bloco correspondente na unidade 2 é escrito e relido, repetidamente se necessário, até que ele, também, por fim seja bem-sucedido. Na ausência de falhas da CPU, ao cabo de uma escrita estável, o bloco terá sido escrito e conferido em ambas as unidades.
2. **Leituras estáveis.** Uma leitura estável primeiro lê o bloco da unidade 1. Se isso produzir um ECC incorreto, a leitura é tentada novamente, até n vezes. Se todas elas derem ECCs defeituosos, o bloco correspondente é lido da unidade 2. Levando-se em consideração o fato de que uma escrita estável bem-sucedida deixa duas boas cópias de um bloco para trás, e nossa suposição de que a probabilidade de o mesmo bloco espontaneamente apresentar um defeito em ambas as unidades em um intervalo de tempo razoável é desprezível, uma leitura estável sempre é bem-sucedida.

3. **Recuperação de falhas.** Após uma queda do sistema, um programa de recuperação varre ambos os discos comparando blocos correspondentes. Se um par de blocos está bem e ambos são iguais, nada é feito. Se um deles tiver um erro de ECC, o bloco defeituoso é sobrescrito com o bloco bom correspondente. Se um par de blocos está bem mas eles são diferentes, o bloco da unidade 1 é escrito sobre o da unidade 2.

Na ausência de falhas de CPU, esse esquema sempre funciona, pois escritas estáveis sempre escrevem duas cópias válidas de cada bloco e erros espontâneos são presumidos que jamais ocorram em ambos os blocos correspondentes ao mesmo tempo. E na presença de falhas na CPU durante escritas estáveis? Depende precisamente de quando ocorre a falha. Há cinco possibilidades, como descrito na Figura 5.27.

Na Figura 5.27(a), a falha da CPU ocorre antes que qualquer uma das cópias do bloco seja escrita. Durante a recuperação, nenhuma será modificada e o valor antigo continuará a existir, o que é permitido.

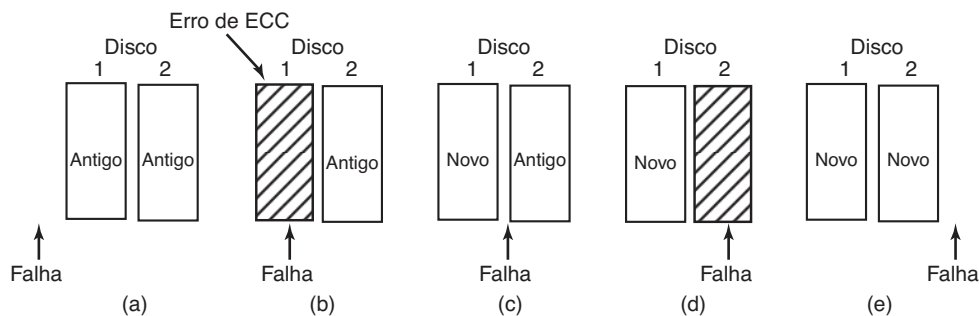
Na Figura 5.27(b), a CPU falha durante a escrita na unidade 1, destruindo o conteúdo do bloco. No entanto, o programa de recuperação detecta o erro e restaura o bloco na unidade 1 da unidade 2. Desse modo, o efeito da falha é apagado e o estado antigo é completamente restaurado.

Na Figura 5.27(c), a falha da CPU acontece após a unidade 1 ter sido escrita, mas antes de a unidade 2 ter sido escrita. O ponto em que não há mais volta foi passado aqui: o programa de recuperação copia o bloco da unidade 1 para a unidade 2. A escrita é bem-sucedida.

A Figura 5.27(d) é como a Figura 5.27(b): durante a recuperação, o bloco bom escreve sobre o defeituoso. Novamente, o valor final de ambos os blocos é o novo.

Por fim, na Figura 5.27(e), o programa de recuperação vê que ambos os blocos são os mesmos, portanto nenhum deles é modificado e a escrita é bem-sucedida aqui também.

FIGURA 5.27 Análise da influência de falhas sobre as escritas estáveis.



Várias otimizações e melhorias são possíveis para esse esquema. Para começo de conversa, comparar todos os blocos em pares após uma falha é possível, mas caro. Um avanço enorme é controlar qual bloco estava sendo escrito durante a escrita estável, de maneira que apenas um bloco precisa ser conferido durante a recuperação. Alguns computadores têm uma pequena quantidade de **RAM não volátil**, que é uma memória CMOS especial mantida por uma bateria de lítio. Essas baterias duram por anos, possivelmente durante a vida inteira do computador. Diferentemente da memória principal, que é perdida após uma queda, a RAM não volátil não é perdida após uma queda. A hora do dia é em geral mantida ali (e incrementada por um circuito especial), razão pela qual os computadores ainda sabem que horas são mesmo depois de terem sido desligados.

Suponha que alguns bytes de RAM não volátil estejam disponíveis para uso do sistema operacional. A escrita estável pode colocar o número do bloco que ela está prestes a atualizar na RAM não volátil antes de começar a escrever. Após completar de maneira bem-sucedida a escrita estável, o número do bloco na RAM não volátil é sobrescrito com um número de bloco inválido, por exemplo, -1. Nessas condições, após uma falha, o programa de recuperação pode conferir a RAM não volátil para ver se havia uma escrita estável em andamento durante a falha e, se afirmativo, qual bloco estava sendo escrito quando a falha aconteceu. As duas cópias podem então ser conferidas quanto à exatidão e consistência.

Se a RAM não volátil não estiver disponível, ela pode ser simulada como a seguir. No começo de uma escrita estável, um bloco de disco fixo na unidade 1 é sobrescrito com o número do bloco a ser escrito estávelmente. Esse bloco é então lido de novo para verificá-lo. Após obtê-lo corretamente, o bloco correspondente na unidade 2 é escrito e verificado. Quando a escrita estável é concluída corretamente, ambos os blocos são sobrescritos com um número de bloco inválido e verificados. Novamente aqui, após uma falha é fácil determinar se uma escrita estável estava ou não em andamento durante a falha. É claro, essa técnica exige oito operações de disco extras para escrever um bloco estável, de maneira que ela deve ser usada o menor número de vezes possível.

Um último ponto que vale a pena destacar: presumimos que apenas uma falha espontânea de um bloco bom para um bloco defeituoso acontece por par de blocos por dia. Se um número suficiente de dias passar, o outro bloco do par também poderá se tornar

defeituoso. Portanto, uma vez por dia uma varredura completa de ambos os discos deve ser feita, reparando qualquer defeito. Dessa maneira, todas as manhãs ambos os discos estarão sempre idênticos. Mesmo que ambos os blocos em um par apresentarem defeitos dentro de um período de alguns dias, todos os erros serão reparados corretamente.

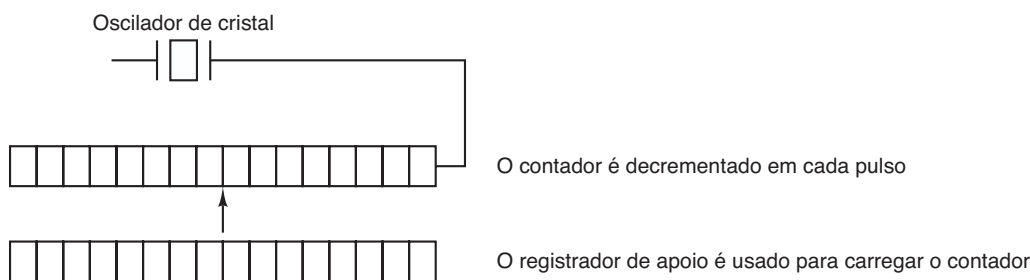
5.5 Relógios

Relógios (também chamados de **temporizadores** — **timers**) são essenciais para a operação de qualquer sistema multiprogramado por uma série de razões. Eles mantêm a hora do dia e evitam que um processo monopolize a CPU, entre outras coisas. O software do relógio pode assumir a forma de um driver de dispositivo, embora um relógio não seja nem um dispositivo de bloco, como um disco, tampouco um dispositivo de caractere, como um mouse. Nosso exame de relógios seguirá o mesmo padrão que na seção anterior: primeiro abordaremos o hardware de relógio e então o software de relógio.

5.5.1 Hardware de relógios

Dois tipos de relógios são usados em computadores, e ambos são bastante diferentes dos relógios de parede e de pulso usados pelas pessoas. Os relógios mais simples são ligados à rede elétrica de 110 ou 220 volts e causam uma interrupção a cada ciclo de voltagem, em 50 ou 60 Hz. Esses relógios costumavam dominar o mercado, mas são raros hoje.

O outro tipo de relógio é construído de três componentes: um oscilador de cristal, um contador e um registrador de apoio, como mostrado na Figura 5.28. Quando um fragmento de cristal é cortado adequadamente e montado sob tensão, ele pode ser usado para gerar um sinal periódico de altíssima precisão, em geral na faixa de várias centenas de mega-hertz até alguns giga-hertz, dependendo do cristal escolhido. Usando a eletrônica, esse sinal básico pode ser multiplicado por um inteiro pequeno para conseguir frequências de até vários giga-hertz ou mesmo mais. Pelo menos um circuito desses normalmente é encontrado em qualquer computador, fornecendo um sinal de sincronização para os vários circuitos do computador. Esse sinal é colocado em um contador para fazê-lo contar regressivamente até zero. Quando o contador chega a zero, ele provoca uma interrupção na CPU.

FIGURA 5.28 Um relógio programável.

Relógios programáveis tipicamente têm vários modos de operação. No **modo disparo único (one-shot mode)**, quando o relógio é inicializado, ele copia o valor do registrador de apoio no contador e então decrementa o contador em cada pulso do cristal. Quando o contador chega a zero, ele provoca uma interrupção e para até que ele é explicitamente inicializado novamente pelo software. No **modo onda quadrada**, após atingir o zero e causar a interrupção, o registrador de apoio é automaticamente copiado para o contador, e todo o processo é repetido de novo indefinidamente. Essas interrupções periódicas são chamadas de **tiques do relógio**.

A vantagem do relógio programável é que a sua frequência de interrupção pode ser controlada pelo software. Se um cristal de 500 MHz for usado, então o contador é pulsado a cada 2 ns. Com registradores de 32 bits (sem sinal), as interrupções podem ser programadas para acontecer em intervalos de 2 ns a 8,6 s. Chips de relógios programáveis costumam conter dois ou três relógios programáveis independentemente e têm muitas outras opções também (por exemplo, contar com incremento em vez de decremento, desabilitar interrupções, e mais).

Para evitar que a hora atual seja perdida quando a energia do computador é desligada, a maioria dos computadores tem um relógio de back-up mantido por uma bateria, implementado com o tipo de circuito de baixo consumo usado em relógios digitais. O relógio de bateria pode ser lido na inicialização. Se o relógio de backup não estiver presente, o software pode pedir ao usuário a data e o horário atuais. Há também uma maneira padrão para um sistema de rede obter o horário atual de um servidor remoto. De qualquer modo, o horário é então traduzido para o número de tiques de relógio desde as 12 horas de 1ª de janeiro de 1970, de acordo com o **Tempo Universal Coordenado (Universal Coordinated Time — UTC)**, antes conhecido como meio-dia de Greenwich, como o UNIX faz, ou de algum outro momento de referência. A origem do tempo para o Windows é o dia 1ª de janeiro de 1980. Em cada tique de

relógio, o tempo real é incrementado por uma contagem. Normalmente programas utilitários são fornecidos para ajustar manualmente o relógio do sistema e o relógio de backup e para sincronizar os dois.

5.5.2 Software de relógio

Tudo o que o hardware de relógios faz é gerar interrupções a intervalos conhecidos. Todo o resto envolvendo tempo deve ser feito pelo software, o driver do relógio. As tarefas exatas do driver do relógio variam entre os sistemas operacionais, mas em geral incluem a maioria das ações seguintes:

1. Manter o horário do dia.
2. Evitar que processos sejam executados por mais tempo do que o permitido.
3. Contabilizar o uso da CPU.
4. Tratar a chamada de sistema alarm feita pelos processos do usuário.
5. Fornecer temporizadores watch dog para partes do próprio sistema.
6. Gerar perfis de execução, realizar monitoramentos e coletar estatísticas.

A primeira função do relógio — a manutenção da hora do dia (também chamada de **tempo real**) não é difícil. Ela apenas exige incrementar um contador a cada tique do relógio, como mencionado anteriormente. A única coisa a ser observada é o número de bits no contador da hora do dia. Com uma frequência de relógio de 60 Hz, um contador de 32 bits ultrapassaria sua capacidade em apenas um pouco mais de dois anos. Claramente, o sistema não consegue armazenar o tempo real como o número de tiques desde 1ª de janeiro de 1970 em 32 bits.

Há três maneiras de resolver esse problema. A primeira maneira é usar um contador de 64 bits, embora fazê-lo torna a manutenção do contador mais cara, pois ela precisa ser feita muitas vezes por segundo. A segunda maneira é manter a hora do dia em segundos,

em vez de em tiques, usando um contador subsidiário para contar tiques até que um segundo inteiro tenha sido acumulado. Como 2^{32} segundos é mais do que 136 anos, esse método funcionará até o século XXII.

A terceira abordagem é contar os tiques, mas fazê-lo em relação ao momento em que o sistema foi inicializado, em vez de em relação a um momento externo fixo. Quando o relógio de backup é lido ou o usuário digita o tempo real, a hora de inicialização do sistema é calculada a partir do valor da hora do dia atual e armazenada na memória de qualquer maneira conveniente. Mais tarde, quando a hora do dia for pedida, a hora do dia armazenada é adicionada ao contador para se chegar à hora do dia atual. Todas as três abordagens são mostradas na Figura 5.29.

A segunda função do relógio é evitar que os processos sejam executados por um tempo longo demais. Sempre que um processo é iniciado, o escalonador inicializa um contador com o valor do quantum do processo em tiques do relógio. A cada interrupção do relógio, o driver decrementa o contador de quantum em 1. Quando chega a zero, o driver do relógio chama o escalonador para selecionar outro processo.

A terceira função do relógio é contabilizar o uso da CPU. A maneira mais precisa de fazer isso é inicializar um segundo temporizador, distinto do temporizador principal do sistema, sempre que um processo é iniciado. Quando um processo é parado, o temporizador pode ser lido para dizer quanto tempo ele esteve em execução. Para fazer as coisas direito, o segundo temporizador deve ser salvo quando ocorre uma interrupção e restaurado mais tarde.

Uma maneira menos precisa, porém mais simples, para contabilizar o uso da CPU é manter um ponteiro para a entrada da tabela de processos relativa ao processo em execução em uma variável global. A cada tique do relógio, um campo na entrada do processo atual é incrementado. Dessa maneira, cada tique do relógio é “cobrado” do processo em execução no momento do tique. Um problema menor com essa estratégia é que se muitas interrupções ocorrerem durante a execução de um

processo, ele ainda será cobrado por um tique completo, mesmo que ele não tenha realizado muito trabalho. A contabilidade apropriada da CPU durante as interrupções é cara demais e feita raramente.

Em muitos sistemas, um processo pode solicitar que o sistema operacional lhe dê um aviso após um determinado intervalo. O aviso normalmente é um sinal, interrupção, mensagem, ou algo similar. Uma aplicação que requer o uso desses avisos é a comunicação em rede, na qual um pacote sem confirmação de recebimento dentro de um determinado intervalo de tempo deve ser retransmitido. Outra aplicação é o ensino auxiliado por computador, onde um estudante que não dê uma resposta dentro de um determinado tempo recebe a resposta do computador.

Se o driver do relógio tivesse relógios o bastante, ele poderia separar um relógio para cada solicitação. Não sendo o caso, ele deve simular múltiplos relógios virtuais com um único relógio físico. Uma maneira é manter uma tabela na qual o tempo do sinal para todos os temporizadores pendentes é mantido, assim como uma variável dando o tempo para o próximo. Sempre que a hora do dia for atualizada, o driver confere para ver se o sinal mais próximo ocorreu. Se ele tiver ocorrido, a tabela é pesquisada para encontrar o próximo sinal a ocorrer.

Se muitos sinais são esperados, é mais eficiente simular múltiplos relógios encadeando juntas todas as solicitações de relógio pendentes, ordenadas no tempo, em uma lista encadeada, como mostra a Figura 5.30. Cada entrada na lista diz quantos tiques do relógio desde o sinal anterior deve-se esperar antes de gerar um novo sinal. Nesse exemplo, os sinais estão pendentes para 4203, 4207, 4213, 4215 e 4216.

Na Figura 5.30, a interrupção seguinte ocorre em 3 tiques. Em cada tique, o “*Próximo sinal*” é decrementado. Quando ele chega a 0, o sinal correspondente para o primeiro item na lista é gerado, e o item é removido da lista. Então “*Próximo sinal*” é ajustado para o valor na entrada agora no início da lista, 4 nesse exemplo.

Observe que durante uma interrupção de relógio, seu driver tem várias coisas para fazer — incrementar

FIGURA 5.29 Três maneiras de manter a hora do dia.

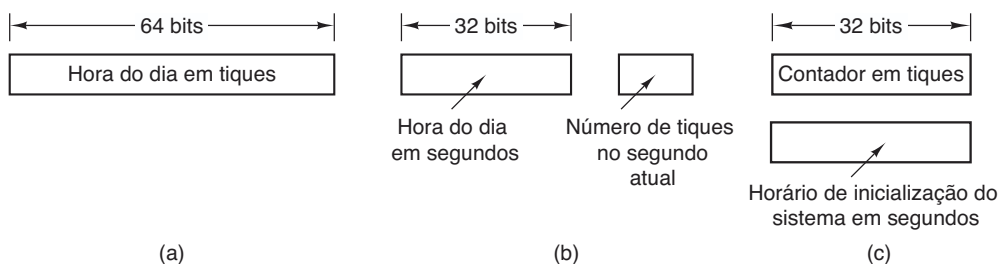
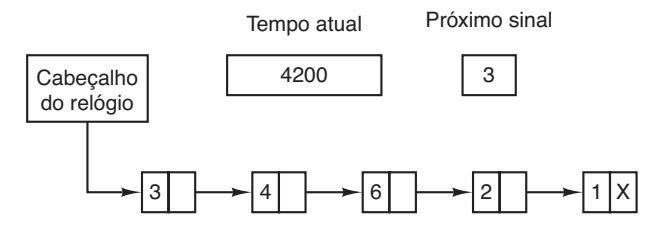


FIGURA 5.30 Simulando múltiplos temporizadores com um único relógio.



o tempo real, decrementar o quantum e verificar o 0, realizar a contabilidade da CPU e decrementar o contador do alarme. No entanto, cada uma dessas operações foi cuidadosamente arranjada para ser muito rápida, pois elas têm de ser feitas muitas vezes por segundo.

Partes do sistema operacional também precisam estabelecer temporizadores. Eles são chamados de **temporizadores watchdog (cão de guarda)** e são frequentemente usados (especialmente em dispositivos embarcados) para detectar problemas como travamentos do sistema. Por exemplo, um temporizador watchdog pode reinicializar um sistema que para de executar. Enquanto o sistema estiver executando, ele regularmente reinicia o temporizador, de maneira que ele nunca expira. Nesse caso, a expiração do temporizador prova que o sistema não executou por um longo tempo e leva a uma ação corretiva — como uma reinicialização completa do sistema.

O mecanismo usado pelo driver do relógio para lidar com temporizadores watchdog é o mesmo que para os sinais do usuário. A única diferença é que, quando um temporizador é desligado, em vez de causar um sinal, o driver do relógio chama uma rotina fornecida pelo chamador. A rotina faz parte do código do chamador. A rotina chamada pode fazer o que for necessário, mesmo causar uma interrupção, embora dentro do núcleo as interrupções sejam muitas vezes inconvenientes e sinais não existem. Essa é a razão pela qual o mecanismo do watchdog é fornecido. É importante observar que o mecanismo de watchdog funciona somente quando o driver do relógio e a rotina a ser chamada estão no mesmo espaço de endereçamento.

A última questão em nossa lista é o perfil de execução. Alguns sistemas operacionais fornecem um mecanismo pelo qual um programa do usuário pode obter do sistema um histograma do seu contador do programa, então ele pode ver onde está gastando seu tempo. Quando esse perfil de execução é uma possibilidade, a cada tique o driver confere para ver se o perfil de execução do processo atual está sendo obtido e, se afirmativo, calcula o intervalo (uma faixa de endereços) correspondente ao contador do programa atual. Ele então incrementa esse intervalo por um.

Esse mecanismo também pode ser usado para extrair o perfil de execução do próprio sistema.

5.5.3 Temporizadores por software

A maioria dos computadores tem um segundo relógio programável que pode ser ajustado para provocar interrupções no temporizador a qualquer frequência de que um programa precisar. Esse temporizador é um acréscimo ao temporizador do sistema principal cujas funções foram descritas antes. Enquanto a frequência de interrupção for baixa, não há problema em se usar esse segundo temporizador para fins específicos da aplicação. O problema ocorre quando a frequência do temporizador específico da aplicação for muito alta. A seguir descreveremos brevemente um esquema de temporizador baseado em software que funciona bem em muitas circunstâncias, mesmo em frequências relativamente altas. A ideia é de autoria de Aron e Druschel (1999). Para mais detalhes, vejam o seu artigo.

Em geral, há duas maneiras de gerenciar E/S: interrupções e polling. Interrupções têm baixa latência, isto é, elas acontecem imediatamente após o evento em si com pouco ou nenhum atraso. Por outro lado, com as CPUs modernas, as interrupções têm uma sobrecarga substancial pela necessidade de chaveamento de contexto e sua influência no pipeline, TLB e cache.

A alternativa às interrupções é permitir que a própria aplicação verifique por meio de polling (verificações ativas) o evento esperado em si. Isso evita interrupções, mas pode haver uma latência substancial, pois um evento pode acontecer imediatamente após uma verificação, caso em que ele esperará quase um intervalo inteiro de verificação. Na média, a latência representa metade do intervalo de verificação.

A latência de interrupção hoje só é um pouco melhor que a dos computadores na década de 1970. Na maioria dos minicomputadores, por exemplo, uma interrupção levava quatro ciclos de barramento: para empilhar o contador do programa e PSW e carregar um novo contador de programa e PSW. Hoje, lidar com a pipeline, MMU, TLB e cache representa um acréscimo à sobrecarga. Esses efeitos provavelmente piorarão antes de melhorar com o tempo, desse modo neutralizando as frequências mais rápidas de relógio. Infelizmente, para determinadas aplicações, não queremos nem a sobrecarga das interrupções, tampouco a latência do polling.

Os **temporizadores por software (soft timers)** evitam interrupções. Em vez disso, sempre que o núcleo está executando por alguma outra razão, imediatamente antes de retornar para o modo do usuário ele verifica o relógio

de tempo real para ver se um temporizador por software expirou. Se ele expirou, o evento escalonado (por exemplo, a transmissão de um pacote ou a verificação da chegada de um pacote) é realizado sem a necessidade de chavear para o modo núcleo, dado que o sistema já está ali. Após o trabalho ter sido realizado, o temporizador por software é reinicializado novamente. Tudo o que precisa ser feito é copiar o valor do relógio atual para o temporizador e acrescentar o intervalo de tempo a ele.

Os temporizadores por software são dependentes da frequência na qual as entradas no núcleo são feitas por outras razões. Essas razões incluem:

1. Chamadas de sistema.
2. Faltas na TLB.
3. Faltas de página.
4. Interrupções de E/S.
5. CPU se tornando ociosa.

Para ver com que frequência esses eventos acontecem, Aron e Druschel tomaram medidas com várias cargas de CPUs, incluindo um servidor da web completamente carregado, um servidor da web com um processo limitado por CPU em segundo plano, executando áudio da internet em tempo real e recompilando o núcleo do UNIX. A frequência média de entrada no núcleo variava de 2 a 18 μ s, com mais ou menos metade dessas entradas sendo chamadas de sistema. Desse modo, para uma aproximação de primeira ordem, a existência de um temporizador por software operando a cada, digamos, 10 μ s, é possível, apesar de esse tempo limite não ser cumprido ocasionalmente. Estar 10 μ s atrasado de tempos em tempos é muitas vezes melhor do que ter interrupções consumindo 35% da CPU.

É claro, existirão períodos em que não haverá chamadas de sistema, faltas na TLB, ou faltas de páginas, caso em que nenhum temporizador por software será disparado. Para colocar um limite superior nesses intervalos, o segundo temporizador de hardware pode ser ajustado para disparar, digamos, a cada 1 ms. Se a aplicação puder viver com apenas 1.000 ativações por segundo em intervalos ocasionais, então a combinação de temporizadores por software e um temporizador de hardware de baixa frequência pode ser melhor do que a E/S orientada somente à interrupção ou controlada apenas por polling.

5.6 Interfaces com o usuário: teclado, mouse, monitor

Todo computador de propósito geral tem um teclado e um monitor (e às vezes um mouse) para permitir

que as pessoas interajam com ele. Embora o teclado e o monitor sejam dispositivos tecnicamente separados, eles funcionam bem juntos. Em computadores de grande porte, frequentemente há muitos usuários remotos, cada um com um dispositivo contendo um teclado e um monitor conectados. Esses dispositivos foram chamados historicamente de **terminais**. As pessoas muitas vezes ainda usam o termo, mesmo quando discutindo teclados e computadores de computadores pessoais (na maior parte das vezes por falta de um termo melhor).

5.6.1 Software de entrada

As informações do usuário vêm fundamentalmente do teclado e do mouse (ou às vezes das telas de toque), então vamos examiná-los. Em um computador pessoal, o teclado contém um microprocessador embutido que em geral comunica-se por uma porta serial com um chip controlador na placa-mãe (embora cada vez mais os teclados estejam conectados a uma porta USB). Uma interrupção é gerada sempre que uma tecla é pressionada e uma segunda é gerada sempre que uma tecla é liberada. Em cada uma dessas interrupções, o driver do teclado extrai as informações sobre o que acontece na porta de E/S associada com o teclado. Todo o restante acontece no software e é bastante independente do hardware.

A maior parte do resto desta seção pode ser compreendida mais claramente se pensarmos na digitação de comandos em uma janela de shell (interface de linha de comando). É assim que os programadores costumam trabalhar. Discutiremos interfaces gráficas a seguir. Alguns dispositivos, em particular telas de toque, são usados para entrada e saída. Fizemos uma escolha (arbitrária) para discuti-los na seção sobre dispositivos de saída. Discutiremos as interfaces gráficas mais adiante neste capítulo.

Software de teclado

O número no registrador de E/S é o número da tecla, chamado de **código de varredura**, não o código de ASCII. Teclados normais têm menos de 128 teclas, então apenas 7 bits são necessários para representar o número da tecla. O oitavo bit é definido como 0 quando a tecla é pressionada e 1 quando ela é liberada. Cabe ao driver controlar o estado de cada tecla (pressionada ou liberada). Assim, tudo o que o hardware faz é gerar interrupções de pressão e liberação. O software faz o resto.

Quando a tecla *A* é pressionada, por exemplo, o código da tecla (30) é colocado em um registrador de E/S. Cabe ao driver determinar se ela é minúscula, maiúscula, CTRL-A, ALT-A, CTRL-ALT-A, ou alguma outra combinação. Tendo em vista que o driver pode dizer quais teclas foram pressionadas, mas ainda não liberadas (por exemplo, SHIFT), ele tem informação suficiente para fazer o trabalho.

Por exemplo, a sequência de teclas

DEPRESS SHIFT, DEPRESS A, RELEASE A, RELEASE SHIFT

indica um caractere A maiúsculo. Contudo, a sequência de teclas

DEPRESS SHIFT, DEPRESS A, RELEASE SHIFT, RELEASE A

também indica um caractere A maiúsculo. Embora essa interface de teclado coloque toda a responsabilidade sobre o software, ela é extremamente flexível. Por exemplo, programas do usuário podem estar interessados se um dígito recém-digitado veio da fileira de teclado do topo do teclado ou do bloco numérico lateral. Em princípio, o driver pode fornecer essa informação.

Duas filosofias possíveis podem ser adotadas para o driver. Na primeira, seu trabalho é apenas aceitar a entrada e passá-la adiante sem modificá-la. Um programa lendo a partir do teclado recebe uma sequência bruta de códigos ASCII. (Dar aos programas do usuário os números das teclas é algo primitivo demais, assim como dependente demais do teclado.)

Essa filosofia é bastante adequada para as necessidades de editores de tela sofisticados, como os *emacs*, os quais permitem que o usuário associe uma ação arbitrária a qualquer caractere ou sequência de caracteres. Ela implica, no entanto, que se o usuário digitar *dste* em vez de *date* e então corrigir o erro digitando três caracteres de retrocesso e *ate*, seguidos de um caractere de retorno de carro (Carriage Return — CR), o programa do usuário receberá todos os 11 caracteres digitados em código ASCII, como a seguir:

```
d s t e ← ← ← a t e CR
```

Nem todos os programas querem tantos detalhes. Muitas vezes eles querem somente a entrada corrigida, não a sequência exata de como ela foi produzida. Essa observação leva à segunda filosofia: o driver lida com toda a edição interna da linha e entrega apenas as linhas corrigidas para os programas do usuário. A primeira filosofia é baseada em caracteres; a segunda em linhas. Na origem elas eram referidas como **modo cru** (*raw mode*) e **modo cozido** (*cooked mode*),

respectivamente. O padrão POSIX usa o termo menos pitoresco **modo canônico** para descrever o modo baseado em linhas. O **modo não canônico** é o equivalente ao modo cru, embora muitos detalhes do comportamento possam ser modificados. Sistemas compatíveis com o POSIX proporcionam várias funções de biblioteca que suportam selecionar qualquer um dos modos e modificar muitos parâmetros.

Se o teclado estiver em modo canônico (cozido), os caracteres devem ser armazenados até que uma linha inteira tenha sido acumulada, pois o usuário pode subsequentemente decidir apagar parte dela. Mesmo que o teclado esteja em modo cru, o programa talvez ainda não tenha solicitado uma entrada, então os caracteres precisam ser armazenados para viabilizar a digitação antecipada. Um buffer dedicado pode ser usado ou buffers podem ser alocados de um reservatório (pool). No primeiro tipo, a digitação antecipada tem um limite; no segundo, não. Essa questão surge mais agudamente quando o usuário está digitando em uma janela de comandos (uma janela de linha de comandos no Windows) e recém-emitiu um comando (como uma compilação) que ainda não foi concluído. Caracteres subsequentes digitados precisam ser armazenados, pois o shell não está pronto para lê-los. Projetistas de sistemas que não permitem que os usuários digitem antecipadamente deveriam ser seriamente punidos, ou pior ainda, ser forçados a usar o próprio sistema.

Embora o teclado e o monitor sejam dispositivos logicamente separados, muitos usuários se acostumaram a ver somente os caracteres que eles recém-digitaram aparecer na tela. Esse processo é chamado de **eco**.

O eco é complicado pelo fato de que um programa pode estar escrevendo para a tela enquanto o usuário está digitando (novamente, pense na digitação em uma janela do shell). No mínimo, o driver do teclado tem de descobrir onde colocar a nova entrada sem que ela seja sobrescrita pela saída do programa.

O eco também fica complicado quando mais de 80 caracteres têm de ser exibidos em uma janela com 80 linhas de caracteres (ou algum outro número). Dependendo da aplicação, pode ser apropriado mostrar na linha seguinte os caracteres excedentes. Alguns drivers simplesmente truncam as linhas em 80 caracteres e descartam todos eles além da coluna 80.

Outro problema é o tratamento da tabulação. Em geral, cabe ao driver calcular onde o cursor está atualmente localizado, levando em consideração tanto a saída dos programas quanto a saída do eco e calcular o número adequado de espaços a ser ecoado.

Agora chegamos ao problema da equivalência. Logicamente, ao final de uma linha de texto, você quer um CR a fim de mover o cursor de volta para a coluna 1, e um caractere de alimentação de linha para avançar para a próxima linha. Exigir que os usuários digitassem ambos ao final de cada linha não venderia bem. Cabe ao driver do dispositivo converter o que entrar para o formato usado pelo sistema operacional. No UNIX, a tecla Enter é convertida para uma alimentação de linha para armazenamento interno; no Windows ela é convertida para um CR seguido de uma alimentação de linha.

Se a forma padrão for apenas armazenar uma alimentação de linha (a convenção UNIX), então CRs (criados pela tecla *Enter*) devem ser transformados em alimentações de linha. Se o formato interno for armazenar ambos (a convenção do Windows), então o driver deve gerar uma alimentação de linha quando recebe um CR e um CR quando recebe uma alimentação de linha. Não importa qual seja a convenção interna, o monitor pode exigir que tanto uma alimentação de linha quanto um CR sejam ecoados a fim de obter uma atualização adequada da tela. Em um sistema com múltiplos usuários como um computador de grande porte, diferentes usuários podem ter diversos tipos de terminais conectados a ele e cabe ao driver do teclado conseguir que todas as combinações de CR/alimentação de linha sejam convertidas ao padrão interno do sistema e arranjar que todos os ecos sejam feitos corretamente.

Quando operando em modo canônico, alguns dos caracteres de entrada têm significados especiais. A Figura 5.31 mostra todos os caracteres especiais exigidos pelo padrão POSIX. Os caracteres-padrão são todos caracteres de controle que não devem entrar em conflito com a entrada de texto ou códigos usados pelos programas;

todos, exceto os últimos dois, podem ser modificados com o controle do programa.

O caractere *ERASE* permite que o usuário apague o caractere recém-digitado. Geralmente é a tecla de retrocesso backspace (CTRL-H). Ele não é acrescentado à fila de caracteres; em vez disso remove o caractere anterior da fila. Ele deve ser ecoado como uma sequência de três caracteres, retrocesso, espaço e retrocesso, a fim de remover o caractere anterior da tela. Se o caractere anterior era uma tabulação, apagá-lo depende de como ele foi processado quando digitado. Se ele for imediatamente expandido em espaços, alguma informação extra é necessária para determinar até onde retroceder. Se a própria tabulação estiver armazenada na fila de entrada, ela pode ser removida e a linha inteira simplesmente enviada outra vez. Na maioria dos sistemas, o uso do retrocesso apenas apagará os caracteres na linha atual. Ele não apagará um CR e retornará para a linha anterior.

Quando o usuário nota um erro no início da linha que está sendo digitada, muitas vezes é conveniente apagar a linha inteira e começar de novo. O caractere *KILL* apaga a linha inteira. A maioria dos sistemas faz a linha apagada desaparecer da tela, mas alguns mais antigos a ecoam mais um CR e uma linha de alimentação, pois alguns usuários gostam de ver a linha antiga. Em consequência, como ecoar *KILL* é uma questão de gosto. Assim como o *ERASE*, normalmente não é possível voltar mais do que a linha atual. Quando um bloco de caracteres é morto, talvez valha a pena para o driver retornar os buffers para o reservatório de buffers, caso um seja usado.

Às vezes, os caracteres *ERASE* ou *KILL* devem ser inseridos como dados comuns. O caractere *LNEXT* serve como um **caractere de escape**. No UNIX, o CTRL-V

FIGURA 5.31 Caracteres tratados especialmente no modo canônico.

Caractere	Nome POSIX	Comentário
CTRL-H	ERASE	Apagar um caractere à esquerda
CTRL-U	KILL	Apagar toda a linha em edição
CTRL-V	LNEXT	Interpretar literalmente o próximo caractere
CTRL-S	STOP	Parar a saída
CTRL-Q	START	Iniciar a saída
DEL	INTR	Interromper processo (SIGINT)
CTRL-\	QUIT	Forçar gravação da imagem da memória (SIGQUIT)
CTRL-D	EOF	Final de arquivo
CTRL-M	CR	Retorno do carro (não modificável)
CTRL-J	NL	Alimentação de linha (não modificável)

é o padrão. Como um exemplo, sistemas UNIX mais antigos muitas vezes usavam o sinal `@` para *KILL*, mas o sistema de correio da internet usa endereços da forma *linda@cs.washington.edu*. Quem se sentir mais confortável com as convenções mais antigas pode redefinir *KILL* como `@`, mas então precisará inserir um sinal `@` literalmente para os endereços eletrônicos. Isso pode ser feito digitando CTRL-V `@`. O CTRL-V em si pode ser inserido literalmente digitando CTRL-V duas vezes em sequência. Após ver um CTRL-V, o driver dá um sinal dizendo que o próximo caractere é isento de processamento especial. O caractere *LNEXT* em si não é inserido na fila de caracteres.

Para permitir que os usuários parem uma imagem na tela que está saindo de seu campo de visão, códigos de controle são fornecidos para congelar a tela e reinicializá-la mais tarde. No UNIX esses são *STOP* (CTRL-S) e *START* (CTRL-Q), respectivamente. Eles não são armazenados, mas são usados para ligar e desligar um sinal na estrutura de dados do teclado. Sempre que ocorre uma tentativa de saída, o sinal é inspecionado. Se ele está ligado, a saída não ocorre. Em geral, o eco também é suprimido junto com a saída do programa.

Muitas vezes é necessário matar um programa descontrolado que está sendo depurado. Os caracteres *INTR* (DEL) e *QUIT* (CTRL-\) podem ser usados para esse fim. No UNIX, DEL envia o sinal SIGINT para todos os processos inicializados a partir daquele teclado. Implementar o DEL pode ser bastante complicado, pois o UNIX foi projetado desde o início para lidar com múltiplos usuários ao mesmo tempo. Desse modo, no caso geral, podem existir muitos processos sendo executados em prol de muitos usuários, e a tecla DEL deve sinalizar apenas os processos do próprio usuário. A parte difícil é fazer chegar a informação do driver para a parte do sistema que lida com sinais, a qual, afinal de contas, não pediu por essa informação.

O CTRL-\ é similar ao DEL, exceto que ele envia o sinal SIGQUIT, que força a gravação da imagem da memória (*core dump*) se não for pego ou ignorado. Quando qualquer uma dessas teclas é acionada, o driver deve ecoar um CR e linha de alimentação e descartar todas as entradas acumuladas para permitir um novo começo. O valor padrão para *INTR* é muitas vezes CTRL-C em vez de DEL, tendo em vista que muitos programas usam DEL e a tecla de retrocesso de modo alternado para a edição.

Outro caractere especial é *EOF* (CTRL-D), que no UNIX faz que quaisquer solicitações de leitura pendentes para o terminal sejam satisfeitas com o que quer que esteja disponível no buffer, mesmo que o buffer esteja

vazio. Digitar CTRL-D no início de uma linha faz que o programa receba uma leitura de 0 byte, o que por convenção é interpretado como fim de arquivo e faz que a maioria dos programas aja do mesmo jeito que eles fariam se vissem o fim de arquivo em um arquivo de entrada.

Software do mouse

A maioria dos PCs tem um mouse, ou às vezes um trackball (que é apenas um mouse deitado de costas). Um tipo comum de mouse tem uma bola de borracha dentro que sai parcialmente através de um buraco na parte de baixo e gira à medida que o mouse é movido sobre uma superfície áspera. À medida que a bola gira, ela desliza contra rolos de borracha colocados em bastões ortogonais. O movimento na direção leste-oeste faz girar o bastão paralelo ao eixo *y*; o movimento na direção norte-sul faz girar o bastão paralelo ao eixo *x*.

Outro tipo popular é o mouse óptico, que é equipado com um ou mais diodos emissores de luz e fotodetectores na parte de baixo. Os primeiros tinham de operar em um mouse pad especial com uma grade retangular traçada nele de maneira que o mouse pudesse contar as linhas cruzadas. Os mouses ópticos modernos contêm um chip de processamento de imagens e tiram fotos de baixa resolução contínuas da superfície debaixo deles, procurando por mudanças de uma imagem para outra.

Sempre que um mouse se move uma determinada distância mínima em qualquer direção ou que um botão é acionado ou liberado, uma mensagem é enviada para o computador. A distância mínima é de mais ou menos 0,1 mm (embora ela possa ser configurada no software). Algumas pessoas chamam essa unidade de **mickey**. Mouses podem ter um, dois, ou três botões, dependendo da estimativa dos projetistas sobre a capacidade intelectual dos usuários de controlar mais do que um botão. Alguns mouses têm rodas que podem enviar dados adicionais de volta para o computador. Os mouses *wireless* são iguais aos conectados, exceto que em vez de enviar seus dados de volta para o computador através de um cabo, eles usam rádios de baixa potência, por exemplo, usando o padrão **Bluetooth**.

A mensagem para o computador contém três itens: Δx , Δy , botões. O primeiro item é a mudança na posição *x* desde a última mensagem. Então vem a mudança na posição *y* desde a última mensagem. Por fim, o estado dos botões é incluído. O formato da mensagem depende do sistema e do número de botões que o mouse tem. Normalmente, são necessários 3 bytes. A maioria dos mouses responde em um máximo de 40 vezes/s,

de maneira que o mouse pode ter percorrido múltiplos mickeys desde a última resposta.

Observe que o mouse indica apenas mudanças na posição, não a posição absoluta em si. Se o mouse for erguido no ar e colocado de volta suavemente sem provocar um giro na bola, nenhuma mensagem será enviada.

Muitas interfaces gráficas fazem distinções entre cliques simples e duplos de um botão de mouse. Se dois cliques estiverem próximos o suficiente no espaço (mickeys) e também próximos o suficiente no tempo (milissegundos), um clique duplo é sinalizado. Cabe ao software definir “próximo o suficiente”, com ambos os parâmetros normalmente sendo estabelecidos pelo usuário.

5.6.2 Software de saída

Agora vamos considerar o software de saída. Primeiro examinaremos uma saída simples para uma janela de texto, que é o que os programadores em geral preferem usar. Então consideraremos interfaces do usuário gráficas, que outros usuários muitas vezes preferem.

Janelas de texto

A saída é mais simples do que a entrada quando a saída está sequencialmente em uma única fonte, tamanho e cor. Na maioria das vezes, o programa envia caracteres para a janela atual e eles são exibidos ali. Normalmente,

um bloco de caracteres, por exemplo, uma linha, é escrito em uma chamada de sistema.

Editores de tela e muitos outros programas sofisticados precisam ser capazes de atualizar a tela de maneiras complexas, como substituindo uma linha no meio da tela. Para acomodar essa necessidade, a maioria dos drivers de saída suporta uma série de comandos para mover o cursor, inserir e deletar caracteres ou linhas no cursor, e assim por diante. Esses comandos são muitas vezes chamados de **sequências de escape**. No auge do terminal burro ASCII 25×80 , havia centenas de tipos de terminais, cada um com suas próprias sequências de escape. Como consequência, era difícil de escrever um software que funcionasse em mais de um tipo de terminal.

Uma solução, introduzida no UNIX de Berkeley, foi um banco de dados terminal chamado de **termcap**. Esse pacote de software definiu uma série de ações básicas, como mover o cursor para uma coordenada (*linha, coluna*). Para mover o cursor para um ponto em particular, o software — digamos, um editor — usava uma sequência de escape genérica que era então convertida para a sequência de escape real para o terminal que estava sendo escrito. Dessa maneira, o editor trabalhava em qualquer terminal que tivesse uma entrada no banco de dados termcap. Grande parte do software UNIX ainda funciona desse jeito, mesmo em computadores pessoais.

Por fim, a indústria viu a necessidade de padronizar a sequência de escape, então foi desenvolvido o padrão ANSI. Alguns dos valores são mostrados na Figura 5.32.

FIGURA 5.32 Sequência de escapes ANSI aceita pelo driver do terminal na saída. ESC representa o caractere de escape ASCII (0x1B), e *n*, *m* e *s* são parâmetros numéricos opcionais.

Sequência de escape	Significado
ESC [<i>n</i> A	Mover <i>n</i> linhas para cima
ESC [<i>n</i> B	Mover <i>n</i> linhas para baixo
ESC [<i>n</i> C	Mover <i>n</i> espaços para a direita
ESC [<i>n</i> D	Mover <i>n</i> espaços para a esquerda
ESC [<i>m</i> ; <i>n</i> H	Mover o cursor para (<i>m</i> , <i>n</i>)
ESC [<i>s</i> J	Limpar a tela a partir do cursor (0 até o final, 1 a partir do início, 2 para ambos)
ESC [<i>s</i> K	Limpar a linha a partir do cursor (0 até o final, 1 a partir do início, 2 para ambos)
ESC [<i>n</i> L	Inserir <i>n</i> linhas a partir do cursor
ESC [<i>n</i> M	Excluir <i>n</i> linhas a partir do cursor
ESC [<i>n</i> P	Excluir <i>n</i> caracteres a partir do cursor
ESC [<i>n</i> @	Inserir <i>n</i> caracteres a partir do cursor
ESC [<i>nm</i>	Habilitar efeito <i>n</i> (0 = normal, 4 = negrito, 5 = piscante, 7 = reverso)
ESC M	Rolar a tela para cima se o cursor estiver na primeira linha

Considere como essas sequências de escape poderiam ser usadas por um editor de texto. Suponha que o usuário digite um comando dizendo ao editor para deletar toda a linha 3 e então reduzir o espaço entre as linhas 2 e 4. O editor pode enviar a sequência de escape a seguir através da linha serial para o terminal:

```
ESC [ 3 ; 1 H ESC [ 0 K ESC [ 1 M
```

(onde os espaços são usados acima somente para separar os símbolos; eles não são transmitidos). Essa sequência move o cursor para o início da linha 3, apaga a linha inteira, e então deleta a linha agora vazia, fazendo que todas as linhas começando em 5 sejam movidas uma linha acima. Então o que era a linha 4 torna-se a linha 3; o que era a linha 5 torna-se a linha 4, e assim por diante. Sequências de escape análogas podem ser usadas para acrescentar texto ao meio da tela. Palavras podem ser acrescentadas ou removidas de uma maneira similar.

O sistema X Window

Quase todos os sistemas UNIX baseiam sua interface de usuário no **Sistema X Window** (muitas vezes chamado simplesmente **X**), desenvolvido no MIT, como parte do projeto Athena na década de 1980. Ele é bastante portátil e executa totalmente no espaço do usuário. Na origem, a ideia era que ele conectasse um grande número de terminais de usuários remotos com um servidor de computadores central, de maneira que ele é logicamente dividido em software cliente e software hospedeiro, que pode executar em diferentes computadores. Nos computadores pessoais modernos, ambas as partes podem executar na mesma máquina. Nos sistemas Linux, os populares ambientes Gnome e KDE executam sobre o X.

Quando o X está executando em uma máquina, o software que coleta a entrada do teclado e mouse e escreve a saída para a tela é chamado de **servidor X**. Ele tem de controlar qual janela está atualmente ativa (onde está o ponteiro do mouse), de maneira que ele sabe para qual cliente enviar qualquer nova entrada do teclado. Ele se comunica com programas em execução (possível sobre uma rede) chamados **clientes X**. Ele lhes envia entradas do teclado e mouse e aceita comandos da tela deles.

Pode parecer estranho que o servidor X esteja sempre dentro do computador do usuário enquanto o cliente X pode estar fora em um servidor de computador remoto, mas pense apenas no trabalho principal do servidor

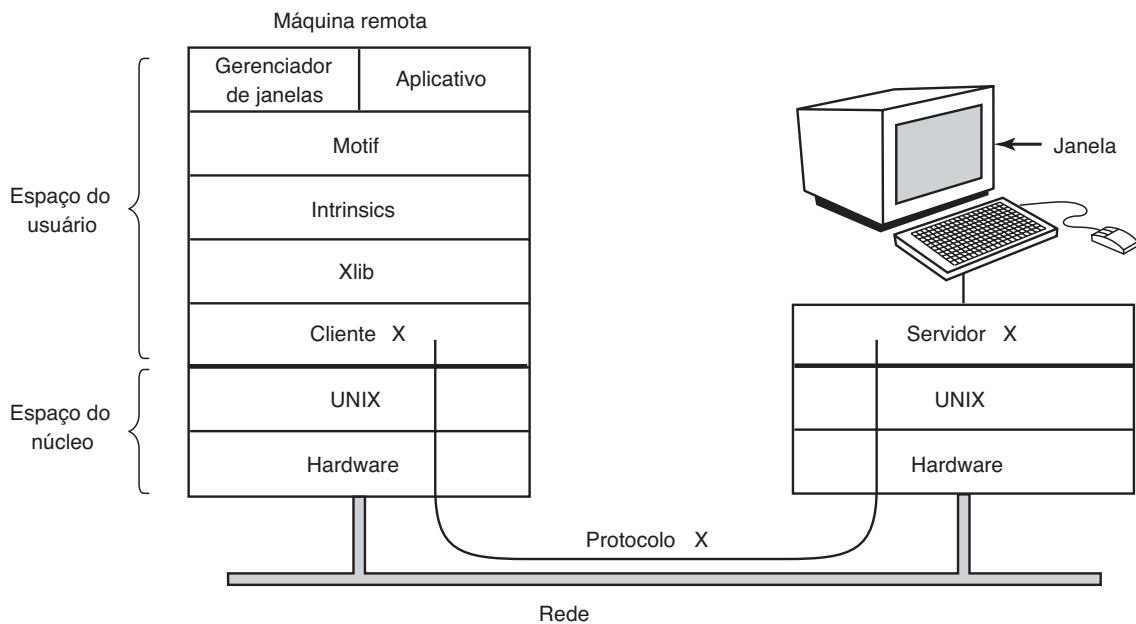
X: exibir bits na tela, então faz sentido estar próximo do usuário. Do ponto de vista do programa, trata-se de um cliente dizendo ao servidor para fazer coisas, como exibir texto e figuras geométricas. O servidor (no PC local) apenas faz o que lhe disseram para fazer, como fazem todos os servidores.

O arranjo do cliente e servidor é mostrado na Figura 5.33 para o caso em que o cliente X e o servidor X estão em máquinas diferentes. Mas quando executa Gnome ou KDE em uma única máquina, o cliente é apenas algum programa de aplicação usando a biblioteca X falando com o servidor X na mesma máquina (mas usando uma conexão TCP através de soquetes, a mesma que ele usaria no caso remoto).

A razão de ser possível executar o sistema X Window no UNIX (ou outro sistema operacional) em uma única máquina ou através de uma rede é o fato de que o que o X realmente define é o protocolo X entre o cliente X e o servidor X, como mostrado na Figura 5.33. Não importa se o cliente e o servidor estão na mesma máquina, separados por 100 metros sobre uma rede de área local, ou distantes milhares de quilômetros um do outro e conectados pela internet. O protocolo e a operação do sistema são idênticos em todos os casos.

O X é apenas um sistema de gerenciamento de janelas. Ele não é uma GUI completa. Para obter uma GUI completa, outras camadas de software são executadas sobre ele. Uma camada é a **Xlib**, um conjunto de rotinas de biblioteca para acessar a funcionalidade do X. Essas rotinas formam a base do Sistema X Window e serão examinadas a seguir, embora sejam primitivas demais para a maioria dos programas de usuário acessar diretamente. Por exemplo, cada clique de mouse é relatado em separado, então determinar que dois cliques realmente formem um clique duplo deve ser algo tratado acima da Xlib.

Para tornar a programação com X mais fácil, um toolkit consistindo em **Intrinsics** é fornecido como parte do X. Essa camada gerencia botões, barras de rolagem e outros elementos da GUI chamados **widgets**. Para produzir uma verdadeira interface GUI, com aparência e sensação uniformes, outra camada é necessária (ou várias delas). Um exemplo é o **Motif**, mostrado na Figura 5.33, que é a base do Common Desktop Environment (Ambiente de Desktop Comum) usado no Solaris e outros sistemas UNIX comerciais. A maioria das aplicações faz uso de chamadas para o Motif em vez da Xlib. Gnome e KDE têm uma estrutura similar à da Figura 5.33, apenas com bibliotecas diferentes. Gnome usa a biblioteca GTK+ e KDE usa a biblioteca Qt. A questão sobre ser melhor ter duas GUIs ou uma é discutível.

FIGURA 5.33 Clientes e servidores no sistema X Window do MIT.

Também vale a pena observar que o gerenciamento de janela não é parte do X em si. A decisão de deixá-lo de fora foi absolutamente intencional. Em vez disso, um processo de cliente X separado, chamado de **gerenciador de janela**, controla a criação, remoção e movimento das janelas na tela. Para gerenciar as janelas, ele envia comandos para o servidor X dizendo-lhe o que fazer. Ele muitas vezes executa na mesma máquina que o cliente X, mas na teoria pode executar em qualquer lugar.

Esse design modular, consistindo em diversas camadas e múltiplos programas, torna o X altamente portátil e flexível. Ele tem sido levado para a maioria das versões do UNIX, incluindo Solaris, todas as variantes do BSD, AIX, Linux e assim por diante, tornando possível aos que desenvolvem aplicações ter uma interface do usuário padrão para múltiplas plataformas. Ele também foi levado para outros sistemas operacionais. Em comparação, no Windows, os sistemas GUI e de gerenciamento de janelas estão misturados na GDI e localizados no núcleo, o que dificulta a sua manutenção e, é claro, torna-os não portáteis.

Agora examinemos brevemente o X como visto do nível Xlib. Quando um programa X inicializa, ele abre uma conexão para um ou mais servidores X — vamos chamá-los de estações de trabalho (workstations), embora possam ser colocados na mesma máquina que o próprio programa X. Este considera a conexão confiável no sentido de que mensagens perdidas e duplicadas são tratadas pelo software de rede e ele não tem de se

preocupar com erros de comunicação. Em geral, TCP/IP é usado entre o cliente e o servidor.

Quatro tipos de mensagens trafegam pela conexão:

1. Comandos gráficos do programa para a estação de trabalho.
2. Respostas da estação de trabalho a perguntas do programa.
3. Teclado, mouse e outros avisos de eventos.
4. Mensagens de erro.

A maioria dos comandos gráficos é enviada do programa para a estação de trabalho como mensagens de uma só via. Não é esperada uma resposta. A razão para esse design é que, quando os processos do cliente e do servidor estão em máquinas diferentes, pode ser necessário um período de tempo substancial para o comando chegar ao servidor e ser executado. Bloquear o programa de aplicação durante esse tempo o atrasaria desnecessariamente. Por outro lado, quando o programa precisa de informações da estação de trabalho, basta-lhe esperar até que a resposta retorne.

Como o Windows, X é altamente impelido por eventos. Eventos fluem da estação de trabalho para o programa, em geral como resposta a alguma ação humana como teclas digitadas, movimentos do mouse, ou uma janela sendo aberta. Cada mensagem de evento tem 32 bytes, com o primeiro byte fornecendo o tipo do evento e os 31 bytes seguintes fornecendo informações adicionais. Várias dúzias de tipos de eventos existem, mas a um programa é enviado somente aqueles eventos que ele disse que está disposto a manejar. Por exemplo, se um programa não quer ouvir falar de liberação de

teclas, ele não recebe quaisquer eventos relacionados ao assunto. Como no Windows, os eventos entram em filas, e os programas leem os eventos da fila de entrada. No entanto, diferentemente do Windows, o sistema operacional jamais chama sozinho rotinas dentro do programa de aplicação por si próprio. Ele não faz nem ideia de qual rotina lida com qual evento.

Um conceito fundamental em X é o **recurso (resource)**. Um recurso é uma estrutura de dados que contém determinadas informações. Programas de aplicação criam recursos em estações de trabalho. Recursos podem ser compartilhados entre múltiplos processos na estação de trabalho. Eles tendem a ter vidas curtas e não sobrevivem às reinicializações das estações de trabalho. Recursos típicos incluem janelas, fontes, mapas de cores (paletas de cores), pixmaps (mapas de bits), cursores e contextos gráficos. Os últimos são usados para associar propriedades às janelas e são similares em conceito com os contextos de dispositivos no Windows.

Um esqueleto incompleto, aproximado, de um programa X é mostrado na Figura 5.34. Ele começa incluindo alguns cabeçalhos obrigatórios e então declarando algumas variáveis. Ele então conecta ao servidor X especificado como o parâmetro para *XOpenDisplay*. Então aloca um recurso de janela e armazena um descritor para ela em *win*.

Na prática, alguma inicialização aconteceria aqui. Após isso, ele diz ao gerenciador da janela que a janela nova existe, assim o gerenciador da janela pode gerenciá-la.

A chamada para *XCreateGC* cria um contexto gráfico no qual as propriedades da janela são armazenadas. Em um programa mais completo, elas podem ser inicializadas aqui. A instrução seguinte, a chamada *XSelectInput*, diz ao servidor X com quais eventos o programa está preparado para lidar. Nesse caso, ele está interessado em cliques do mouse, teclas digitadas e janelas sendo abertas. Na prática, um programa real estaria interessado em outros eventos também. Por fim, a chamada *XMapRaised* mapeia a janela nova na tela como a janela mais acima. Nesse ponto, a janela torna-se visível na tela.

O laço principal consiste em dois comandos e é logicamente muito mais simples do que o laço correspondente no Windows. O primeiro comando obtém um evento e o segundo ativa um processamento de acordo com o tipo do evento. Quando algum evento indica que o programa foi concluído, *running* é colocado em 0 e o laço termina. Antes de sair, o programa libera o contexto gráfico, janela e conexão.

Vale a pena mencionar que nem todas as pessoas gostam da GUI. Muitos programadores preferem uma interface orientada por linha de comando do tipo discutida

FIGURA 5.34 Um esqueleto de um programa de aplicação X Window.

```
#include <X11/Xlib.h>
#include <X11/Xutil.h>

main(int argc, char *argv[])
{
    Display disp;
    Window win;
    GC gc;
    XEvent event;
    int running = 1;

    disp = XOpenDisplay("display_name");
    win = XCreateSimpleWindows(disp, ...);
    XSetStandardProperties(disp, ...);
    gc = XCreateGC(disp, win, 0, 0);
    XSelectInput(disp, win, ButtonPressMask | KeyPressMask | ExposureMask);
    XMapRaised(disp, win);

    while (running) {
        xNextEvent(disp, &event);
        switch (event.type) {
            case Expose:      ...; break;
            case ButtonPress: ...; break;
            case Keypress:    ...; break;
        }
    }

    XFreeGC(disp, gc);
    XDestroyWindow(disp, win);
    XCloseDisplay(disp);
}
```

/* identificador do servidor */
/* identificador da janela */
/* identificador do contexto grafico */
/* armazenamento para um evento */

/* conecta ao servidor X */
/* aloca memoria para a nova janela */
/* anuncia a nova janela para o gerenciador de janelas */
/* cria contexto grafico */
/* mostra a janela; envia evento de exposicao de janela */

/*obtem proximo evento */
/* repinta janela */
/* processa clique do mouse */
/* processa entrada do teclado */

/* libera contexto grafico */
/* desaloca espaco de memoria da janela */
/* termina a conexao de rede */

na Seção 5.6.1. O X trata essa questão mediante um programa cliente chamado *xterm*, que emula um venerável terminal inteligente VT102, completo com todas as sequências de escape. Desse modo, editores como *vi* e *emacs* (e outros softwares que usam *termcap*) trabalham nessas janelas sem modificação.

Interfaces gráficas do usuário

A maioria dos computadores pessoais oferece uma interface gráfica do usuário (**Graphical User Interface** — **GUI**). O acrônimo GUI é pronunciado “gooeey”.

A GUI foi inventada por Douglas Engelbart e seu grupo de pesquisa no Instituto de Pesquisa Stanford. Ela foi então copiada por pesquisadores na Xerox PARC. Um belo dia, Steve Jobs, cofundador da Apple, estava visitando a PARC e viu uma GUI em um computador da Xerox e disse algo como “Meu Deus, esse é o futuro da computação”. A GUI deu a ele a ideia para um novo computador, que se tornou o Apple Lisa. O Lisa era caro demais e foi um fracasso comercial, mas seu sucessor, o Macintosh, foi um sucesso enorme.

Quando a Microsoft conseguiu um protótipo Macintosh para que pudesse desenvolver o Microsoft Office nele, a empresa implorou à Apple para que licenciasse a interface para todos os interessados, de maneira que ela tornar-se-ia o novo padrão da indústria. (A Microsoft ganhou muito mais dinheiro com o Office do que com o MS-DOS, então ela estava disposta a abandonar o MS-DOS para ter uma plataforma melhor para o Office.) O executivo da Apple responsável pelo Macintosh, Jean-Louis Gassée, recusou a oferta e Steve Jobs não estava mais por perto para confrontar a decisão. Por fim, a Microsoft conseguiu uma licença para elementos da interface. Isso formou a base do Windows. Quando o Windows começou a pegar, a Apple processou a Microsoft, alegando que a empresa havia excedido a licença, mas o juiz discordou e o Windows prosseguiu para desbancar o Macintosh. Se Gassée tivesse concordado com as muitas pessoas dentro da Apple que também queriam licenciar o software Macintosh para qualquer um, a Apple teria ficado insanamente rica em taxas de licenciamento e o Windows não existiria hoje em dia.

Deixando de lado as interfaces sensíveis ao toque por ora, a GUI tem quatro elementos essenciais, denotados pelos caracteres WIMP. Essas letras representam Windows, Icons, Menus e Pointing device (Janelas, Ícones, Menus e Dispositivo apontador, respectivamente). As janelas são blocos retangulares de área de tela usados para executar programas. Os ícones são pequenos símbolos que podem ser clicados para fazer

que determinada ação aconteça. Os menus são listas de ações das quais uma pode ser escolhida. Por fim, um dispositivo apontador é um mouse, *trackball*, ou outro dispositivo de hardware usado para mover um cursor pela tela para selecionar itens.

O software GUI pode ser implementado como código no nível do usuário, como nos sistemas UNIX, ou no próprio sistema operacional, como no caso do Windows.

A entrada de dados para sistemas GUI ainda usa o teclado e o mouse, mas a saída quase sempre vai para um dispositivo de hardware especial chamado **adaptador gráfico**. Um adaptador gráfico contém uma memória especial chamada **RAM de vídeo** que armazena as imagens que aparecem na tela. Adaptadores gráficos muitas vezes têm poderosas CPUs de 32 ou 64 bits e até 4 GB de sua própria RAM, separada da memória principal do computador.

Cada adaptador gráfico suporta um determinado número de tamanhos de telas. Tamanhos comuns (horizontal $\times$ vertical em pixels) são 1280×960 , 1600×1200 , 1920×1080 , 2560×1600 , e 3840×2160 . Muitas resoluções na prática encontram-se na proporção 4:3, que se ajusta à proporção de perspectiva dos aparelhos de televisão NTSC e PAL e desse modo fornece pixels quadrados nos mesmos monitores usados para os aparelhos de televisão. Resoluções mais altas são voltadas para os monitores *widescreen* cuja proporção de perspectiva casa com elas. A uma resolução de apenas 1920×1080 (o tamanho dos vídeos HD inteiros), uma tela colorida com 24 bits por pixel exige em torno de 6,2 MB de RAM apenas para conter a imagem, então com 256 MB ou mais, o adaptador gráfico pode conter muitas imagens ao mesmo tempo. Se a tela inteira for renovada 75 vezes/s, a RAM de vídeo tem de ser capaz de fornecer dados continuamente a 445 MB/s.

O software de saída para GUIs é um tópico extenso. Muitos livros de 1500 páginas foram escritos somente sobre o GUI Windows (por exemplo, PETZOLD, 2013; RECTOR e NEWCOMER, 1997; e SIMON, 1997). Claro, nesta seção, podemos apenas arranhar a superfície e apresentar alguns dos conceitos subjacentes. Para tornar a discussão concreta, descreveremos o Win32 API, que é aceito por todas as versões de 32 bits do Windows. O software de saída para outras GUIs é de certa forma comparável em um sentido geral, mas os detalhes são muito diferentes.

O item básico na tela é uma área retangular chamada de **janela**. A posição e o tamanho de uma janela são determinados exclusivamente por meio das coordenadas (em pixels) de dois vértices diagonalmente opostos. Uma janela pode conter uma barra de título, uma barra

de menu, uma barra de ferramentas, uma barra de rolagem vertical e uma barra de rolagem horizontal. Uma janela típica é mostrada na Figura 5.35. Observe que o sistema de coordenadas do Windows coloca a origem no vértice superior esquerdo e estabelece que o y aumenta para baixo, o que é diferente das coordenadas cartesianas usadas na matemática.

Quando uma janela é criada, os parâmetros especificam se ela pode ser movida, redimensionada ou rolada (arrastando a trava deslizante da barra de rolagem) pelo usuário. A principal janela produzida pela maioria dos programas pode ser movida, redimensionada e rolada, o que tem enormes consequências para a maneira como os programas do Windows são escritos. Em particular, os programas precisam ser informados a respeito de mudanças no tamanho de suas janelas e devem estar preparados para redesenhar os conteúdos de suas janelas a qualquer momento, mesmo quando menos esperarem por isso.

Como consequência, os programas do Windows são orientados a mensagens. As ações do usuário envolvendo o teclado ou o mouse são capturadas pelo Windows e convertidas em mensagens para o programa proprietário da janela sendo endereçada. Cada programa tem uma fila de mensagens para a qual as mensagens relacionadas a todas as suas janelas são enviadas. O principal laço do programa consiste em pescar a próxima mensagem e

processá-la chamando uma rotina interna para aquele tipo de mensagem. Em alguns casos, o próprio Windows pode chamar essas rotinas diretamente, ignorando a fila de mensagens. Esse modelo é bastante diferente do código procedimental do modelo UNIX que faz chamadas de sistema para interagir com o sistema operacional. X, no entanto, é orientado a eventos.

A fim de tornar esse modelo de programação mais claro, considere o exemplo da Figura 5.36. Aqui vemos o esqueleto de um programa principal para o Windows. Ele não está completo e não realiza verificação de erros, mas mostra detalhes suficientes para nossos fins. Ele começa incluindo um arquivo de cabeçalho, *windows.h*, que contém muitos macros, tipos de dados, constantes, protótipos de funções e outras informações necessárias pelos programas do Windows.

O programa principal inicializa com uma declaração dando seu nome e parâmetros. A macro *WINAPI* é uma instrução para o compilador usar uma determinada convenção de passagem de parâmetros e não a abordaremos mais neste livro. O primeiro parâmetro, *h*, é o nome da instância e é usado para identificar o programa para o resto do sistema. Até certo ponto, Win32 é orientado a objetos, o que significa que o sistema contém objetos (por exemplo, programas, arquivos e janelas) que têm algum estado e código associado, chamados **métodos**, que operam sobre

FIGURA 5.35 Uma amostra de janela localizada na coordenada (200, 100) em uma tela XGA.

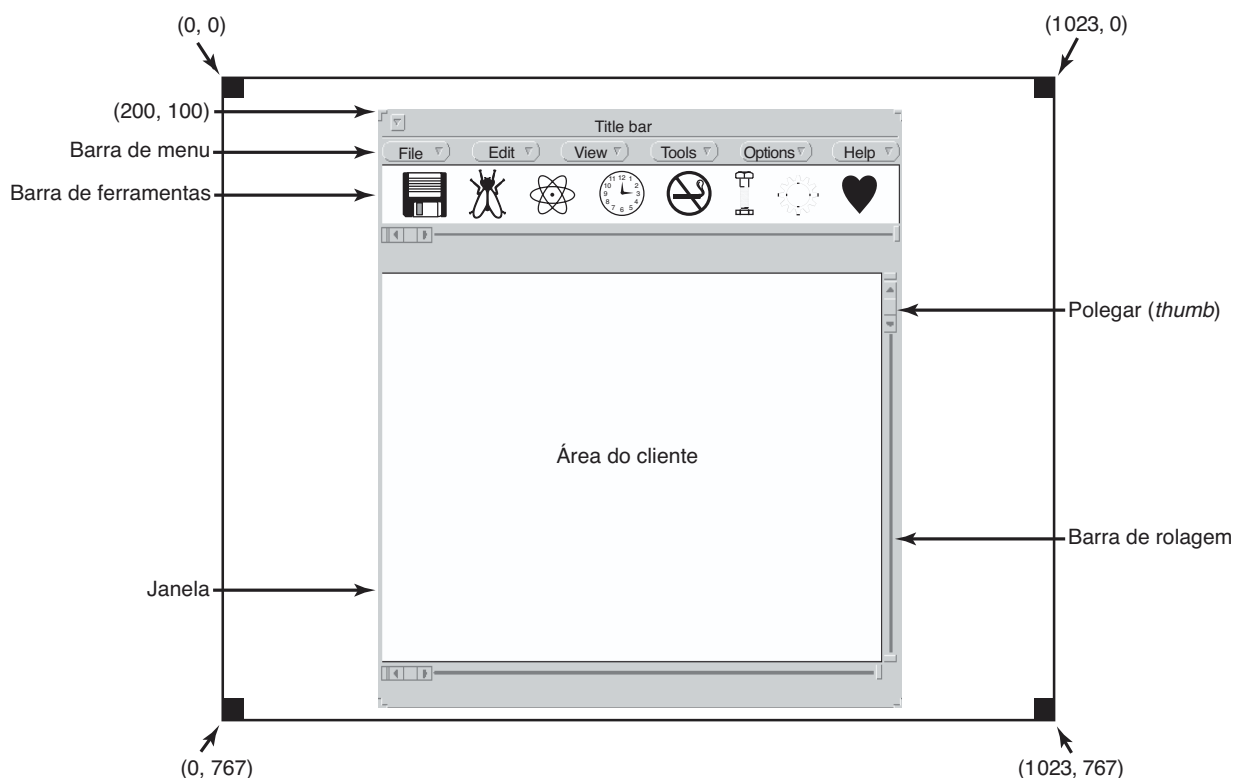


FIGURA 5.36 Um esqueleto de um programa principal do Windows.

```

#include <windows.h>

int WINAPI WinMain(HINSTANCE h, HINSTANCE, hprev, char *szCmd, int iCmdShow)
{
    WNDCLASS wndclass;           /* objeto-classe para esta janela */
    MSG msg;                     /* mensagens que chegam são aqui armazenadas */
    HWND hwnd;                   /* ponteiro para o objeto janela */

    /* Inicializa wndclass */
    wndclass.lpfnWndProc = WndProc; /* indica qual procedimento chamar */
    wndclass.lpszClassName = "Program name"; /* Texto para a Barra de Título */
    wndclass.hIcon = LoadIcon(NULL, IDI_APPLICATION); /* carrega ícone do programa */
    wndclass.hCursor = LoadCursor(NULL, IDC_ARROW); /* carrega cursor do mouse */

    RegisterClass(&wndclass); /* avisa o Windows sobre wndclass */
    hwnd = CreateWindow ( ... ) /* aloca espaço para a janela */
    ShowWindow(hwnd, iCmdShow); /* mostra a janela na tela */
    UpdateWindow(hwnd); /* avisa a janela para pintar-se */

    while (GetMessage(&msg, NULL, 0, 0)) { /* obtém mensagem da fila */
        TranslateMessage(&msg); /* traduz a mensagem */
        DispatchMessage(&msg); /* envia msg para o procedimento apropriado */
    }
    return(msg.wParam);
}

long CALLBACK WndProc(HWND hwnd, UINT message, WPARAM wParam, LPARAM lParam)
{
    /* Declarações são colocadas aqui */

    switch (message) {
        case WM_CREATE: ... ; return ... ; /* cria janela */
        case WM_PAINT: ... ; return ... ; /* repinta conteúdo da janela */
        case WM_DESTROY: ... ; return ... ; /* destrói janela */
    }
    return(DefWindowProc(hwnd, message, wParam, lParam)); /* default */
}

```

esse estado. Os objetos são referenciados usando nomes, e nesse caso *h* identifica o programa. O segundo parâmetro está presente somente por razões de compatibilidade, ele não é mais usado. O terceiro parâmetro, *szCmd*, é uma cadeia terminada em zero contendo a linha de comando que começou o programa, mesmo que ele não tenha sido iniciado por uma linha de comando. O quarto parâmetro, *iCmdShow*, diz se a janela inicial do programa deve ocupar toda a tela, parte ou nada dela (somente a barra de tarefas).

Essa declaração ilustra uma convenção amplamente usada pela Microsoft chamada de **notação húngara**. O nome é uma brincadeira com a notação polonesa, o sistema pós-fixado inventado pelo lógico polonês J. Lukasiewicz para representar fórmulas algébricas sem usar precedência ou parênteses. A notação húngara foi inventada por um programador húngaro na Microsoft, Charles Simonyi, e usa os primeiros caracteres de um identificador para especificar o tipo. Entre as letras e os tipos permitidos estão o *c* (caractere), *w* (*word*, agora significando um inteiro de 16 bits sem sinal), *i* (inteiro de 32 bits com sinal), *l* (longo, também um inteiro de 32 bits com sinal),

s (*string*, cadeia de caracteres), *sz* (cadeia de caracteres terminada por um byte zero), *p* (ponteiro), *fn* (função) e *h* (*handle*, nome). Desse modo, *szCmd* é uma cadeia terminada em zero e *iCmdShow* é um inteiro, por exemplo. Muitos programadores acreditam que codificar o tipo em nomes de variáveis dessa maneira tem pouco valor e dificulta a leitura do código do Windows. Nada análogo a essa convenção existe no UNIX.

Cada janela deve ter um objeto-classe associado que define suas propriedades. Na Figura 5.36, esse objeto-classe é *wndclass*. Um objeto do tipo *WNDCLASS* tem 10 campos, quatro dos quais são inicializados na Figura 5.36. Em um programa real, os outros seis seriam inicializados também. O campo mais importante é *lpfnWndProc*, que é um ponteiro longo (isto é, 32 bits) para a função que lida com as mensagens direcionadas para essa janela. Os outros campos inicializados aqui dizem qual nome e ícone usar na barra do título, e qual símbolo usar para o cursor do mouse.

Após *wndclass* ter sido inicializado, *RegisterClass* é chamado para passá-lo para o Windows. Em particular,

após essa chamada, o Windows sabe qual rotina chamar quando ocorrem vários eventos que não passam pela fila de mensagens. A chamada seguinte, *CreateWindow*, aloca memória para a estrutura de dados da janela e retorna um nome para referenciar futuramente. O programa faz então mais duas chamadas em sequência, para colocar o contorno da janela na tela e por fim preenchê-la por completo.

Nesse ponto chegamos ao laço principal do programa, que consiste em obter uma mensagem, ter determinadas traduções realizadas para ela e então passá-la de volta para o Windows para que ele invoque *WndProc* para processá-la. Respondendo à questão se esse mecanismo todo poderia ter sido mais simples, a resposta é sim, mas ele foi feito dessa maneira por razões históricas e agora estamos presos a ele.

Seguindo o programa principal é a rotina **WndProc**, que lida com várias mensagens que podem ser enviadas para a janela. O uso de *CALLBACK* aqui, como *WINAPI* antes, especifica a sequência de chamadas a ser usada para os parâmetros. O primeiro parâmetro é o nome da janela a ser usada. O segundo é o tipo da mensagem. O terceiro e quarto parâmetros podem ser usados para fornecer informações adicionais quando necessário.

Os tipos de mensagens *WM_CREATE* e *WM_DESTROY* são enviados no início e final do programa, respectivamente. Eles dão ao programa a oportunidade, por exemplo, de alocar memória para estruturas de dados e então devolvê-la.

O terceiro tipo de mensagem, *WM_PAINT*, é uma instrução para o programa para preencher a janela. Ele não é chamado somente quando a janela é desenhada pela primeira vez, mas muitas vezes durante a execução do programa também. Em comparação com os sistemas baseados em texto, no Windows um programa não pode presumir que qualquer coisa que ele desenhe na tela permanecerá lá até sua remoção. Outras janelas podem ser arrastadas para cima dessa janela, menus podem ser trazidos e largados sobre ela, caixas de diálogos e pontas de ferramentas podem cobrir parte dela, e assim por diante. Quando esses itens são removidos, a janela precisa ser redesenhada. A maneira que o Windows diz para um programa redesenhar uma janela é enviar para ela uma mensagem *WM_PAINT*. Como um gesto amigável, ela também fornece informações sobre que parte da janela foi sobrescrita, caso seja mais fácil ou mais rápido regenerar aquela parte da janela em vez de redesenhar tudo desde o início.

Há duas maneiras de o Windows conseguir que um programa faça algo. Uma é postar uma mensagem para sua fila de mensagens. Esse método é usado para a entrada do teclado, entrada do mouse e temporizadores que expiraram. A outra maneira, enviar uma mensagem

para a janela, consiste no Windows chamar diretamente o próprio *WndProc*. Esse método é usado para todos os outros eventos. Tendo em vista que o Windows é notificado quando uma mensagem é totalmente processada, ele pode abster-se de fazer uma nova chamada até que a anterior tenha sido concluída. Desse modo as condições de corrida são evitadas.

Há muitos outros tipos de mensagens. Para evitar um comportamento errático caso uma mensagem inesperada chegue, o programa deve chamar *DefWindowProc* ao fim do *WndProc* para deixar o tratador padrão cuidar dos outros casos.

Resumindo, um programa para o Windows normalmente cria uma ou mais janelas com um objeto-classe para cada uma. Associado com cada programa há uma fila de mensagens e um conjunto de rotinas tratadoras. Em última análise, o comportamento do programa é impelido por eventos que chegam, que são processados pelas rotinas tratadoras. Esse é um modelo do mundo muito diferente da visão mais procedimental adotada pelo UNIX.

A ação de desenhar para a tela é manejada por um pacote que consiste em centenas de rotinas que são reunidas para formar a **interface do dispositivo gráfico (GDI — Graphics Device Interface)**. Ela pode lidar com texto e gráficos e é projetada para ser independente de plataformas e dispositivos. Antes que um programa possa desenhar (isto é, pintar) em uma janela, ele precisa adquirir um **contexto do dispositivo**, que é uma estrutura de dados interna contendo propriedades da janela, como a fonte, cor do texto, cor do segundo plano, e assim por diante. A maioria das chamadas para a GDI usa o contexto de dispositivo, seja para desenhar ou para obter ou ajustar as propriedades.

Existem várias maneiras para adquirir o contexto do dispositivo. Um exemplo simples da sua aquisição e uso é

```
hdc = GetDC(hwnd);  
TextOut(hdc, x, y, psText, iLength);  
ReleaseDC(hwnd, hdc);
```

A primeira instrução obtém um nome para o contexto do dispositivo, *hdc*. A segunda usa o contexto do dispositivo para escrever uma linha de texto na tela, especificando as coordenadas (*x, y*) de onde começa a cadeia, um ponteiro para a cadeia em si e seu comprimento. A terceira chamada libera o contexto do dispositivo para indicar que o programa terminou de desenhar por ora. Observe que *hdc* é usado de maneira análoga a um descritor de arquivos UNIX. Observe também que *ReleaseDC* contém informações redundantes (o uso de *hdc* especifica unicamente uma janela). O uso de informações redundantes que não têm valor real é comum no Windows.

Outra nota interessante é que, quando *hdc* é adquirido dessa maneira, o programa pode escrever apenas na área do cliente da janela, não na barra de títulos e outras partes dela. Internamente, na estrutura de dados do contexto do dispositivo, uma região de pintura é mantida. Qualquer desenho fora dessa região é ignorado. No entanto, existe outra maneira de adquirir um contexto de dispositivo, *GetWindowDC*, que ajusta a região de pintura para toda a janela. Outras chamadas restringem a região de pintura de outras maneiras. A existência de múltiplas chamadas que fazem praticamente a mesma coisa é característica do Windows.

Um tratamento completo sobre GDI está fora de questão aqui. Para o leitor interessado, as referências citadas fornecem informações adicionais. Mesmo assim, dada sua importância, algumas palavras sobre a GDI provavelmente valem a pena. A GDI faz várias chamadas de rotina para obter e liberar contextos de dispositivos, conseguir informações sobre contextos de dispositivos, obter e estabelecer atributos de contextos de dispositivos (por exemplo, a cor do segundo plano), manipular objetos GDI como canetas, pincéis e fontes, cada um dos quais com seus próprios atributos. Por fim, é claro, há um grande número de chamadas GDI para realmente desenhar na tela.

As rotinas de desenho caem em quatro categorias: traçado de linhas e curvas, desenho de áreas preenchidas, gerenciamento de mapas de bits e apresentação de texto. Já vimos um exemplo de tratamento de texto, então vamos dar uma olhada rápida em outro. A chamada

```
Rectangle(hdc, xleft, ytop, xright, ybottom);
```

desenha um retângulo preenchido cujos vértices são (*xleft*, *ytop*) e (*xright*, *ybottom*). Por exemplo,

```
Rectangle(hdc, 2, 1, 6, 4);
```

desenhará o retângulo mostrado na Figura 5.37. A largura e cor da linha, assim como a cor de preenchimento,

são tiradas do contexto do dispositivo. Outras chamadas GDI são similares a essa.

Mapas de bits (*bitmaps*)

As rotinas GDI são exemplos de gráficos vetoriais. Eles são usados para colocar figuras geométricas e texto na tela. Podem ser escalados facilmente para telas maiores ou menores (desde que o número de pixels na tela seja o mesmo). Eles também são relativamente independentes do dispositivo. Uma coleção de chamadas para rotinas GDI pode ser reunida em um arquivo que consiga descrever um desenho complexo. Esse arquivo é chamado de um **meta-arquivo** do Windows, e é amplamente usado para transmitir desenhos de um programa do Windows para outro. Esses arquivos têm uma extensão *.wmf*.

Muitos programas do Windows permitem que o usuário copie (parte de) um desenho e o coloque na área de transferência do Windows. O usuário pode então ir a outro programa e colar os conteúdos da área de transferência em outro documento. Uma maneira de efetuar isso é fazer que o primeiro programa represente o desenho como um meta-arquivo do Windows e o coloque na área de transferência no formato *.wmf*. Existem também outras maneiras.

Nem todas as imagens que os computadores manipulam podem ser geradas usando gráficos vetoriais. Fotografias e vídeos, por exemplo, não usam gráficos vetoriais. Em vez disso, esses itens são escaneados sobrepondo-se uma grade na imagem. Os valores médios do vermelho, verde e azul de cada quadrante da grade são então amostrados e salvos como o valor de um pixel. Esse arquivo é chamado de **mapa de bits (bitmap)**. Existem muitos recursos para manipular os bitmaps no Windows.

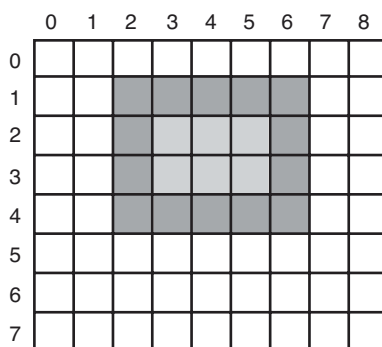
Outro uso para os bitmaps é para o texto. Uma maneira de representar um caractere em particular em alguma fonte é um pequeno bitmap. Acrescentar texto à tela então torna-se uma questão de movimentar bitmaps.

Uma maneira geral de usar os mapas de bits é através de uma rotina chamada *BitBlt*. Ela é chamada como a seguir:

```
BitBlt(dst(hdc), dx, dy, wid, ht, src(hdc), sx, sy, rasterop);
```

Em sua forma mais simples, ela copia um mapa de bits de um retângulo em uma janela para um retângulo em outra janela (ou a mesma). Os primeiros três parâmetros especificam a janela de destino e posição. Então vêm a largura e altura. Em seguida a janela da origem e posição. Observe que cada janela tem seu próprio sistema de coordenadas, com (0, 0) no vértice superior

FIGURA 5.37 Um exemplo de retângulo desenhado usando *Rectangle*. Cada quadrado representa um pixel.



esquerdo da janela. O último parâmetro será descrito a seguir. O efeito de

```
BitBlt(hdc2, 1, 2, 5, 7, hdc1, 2, 2, SRCCOPY);
```

é mostrado na Figura 5.38. Observe cuidadosamente que a área total 5×7 da letra A foi copiada, incluindo a cor do segundo plano.

BitBlt pode fazer mais que somente copiar mapas de bits. O último parâmetro proporciona a possibilidade de realizar operações Booleanas a fim de combinar o mapa de bits fonte com o mapa de bits destino. Por exemplo, a operação lógica OU pode ser aplicada entre a origem e o destino para se fundir com ele. Também pode ser usada a operação OU EXCLUSIVO, que mantém as características tanto da origem quanto do destino.

Um problema com mapas de bits é que eles não são extensíveis. Um caractere que está em uma caixa de 8×12 em uma tela de 640×480 parecerá razoável. No entanto, se esse mapa de bits for copiado para uma página impressa de 1200 pontos/polegada, o que é 10.200 bits $\times$ 13200 bits, a largura do caractere (8 pixels) será de $8/1200$ polegadas ou 0,17 mm. Além disso, copiar entre dispositivos com propriedades de cores diferentes ou entre monocromático e colorido não funciona bem.

Por essa razão, o Windows também suporta uma estrutura de dados chamada **mapa de bits independente de dispositivo (Device Independent Bitmap — DIB)**. Arquivos usando esse formato usam a extensão *.bmp*. Eles têm cabeçalhos de arquivo e informação e uma tabela de cores antes dos pixels. Essa informação facilita a movimentação de mapas de bits entre dispositivos diferentes.

Fontes

Nas versões do Windows antes do 3.1, os caracteres eram representados como mapas de bits e copiados na tela ou impressora usando *BitBlt*. O problema com

isso, como vimos há pouco, é que um mapa de bits que faz sentido na tela é pequeno demais para a impressora. Também, um mapa de bits é necessário para cada caractere em cada tamanho. Em outras palavras, dado o mapa de bits para A no padrão de 10 pontos, não há como calculá-lo para o padrão de 12 pontos. Uma vez que podem ser necessários todos os caracteres de todas as fontes para tamanhos que vão de 4 pontos a 120 pontos, um vasto número de bitmaps era necessário. O sistema inteiro era simplesmente inadequado demais para texto.

A solução foi a introdução das fontes TrueType, que não são bitmaps, mas contornos de caracteres. Cada caractere TrueType é definido por uma sequência de pontos em torno do seu perímetro. Todos os pontos são relativos à origem (0, 0). Usando esse sistema, é fácil colocar os caracteres em uma escala crescente ou decrescente. Tudo o que precisa ser feito é multiplicar cada coordenada pelo mesmo fator da escala. Dessa maneira, um caractere TrueType pode ser colocado em uma escala para cima ou para baixo até qualquer tamanho de pontos, mesmo tamanhos de pontos fracionais. Uma vez no tamanho adequado, os pontos podem ser conectados usando o conhecido algoritmo ligue-os-pontos (*follow-the-dots*) ensinado no jardim de infância (observe que jardins de infância modernos usam superfícies curvas para suavizar os resultados). Após o contorno ter sido concluído, o caractere pode ser preenchido. Um exemplo de alguns caracteres colocados em escalas para três tamanhos de pontos diferentes é dado na Figura 5.39.

Assim que o caractere preenchido estiver disponível em forma matemática, ele pode ser varrido, isto é, convertido em um mapa de bits para qualquer que seja a resolução desejada. Ao colocar primeiro em escala e então varrer, podemos nos certificar de que os caracteres exibidos na tela ou impressos na impressora serão o mais próximos quanto possível, diferenciando-se somente na precisão do erro. Para melhorar ainda mais

FIGURA 5.38 Copiando mapas de bits usando *BitBlt*. (a) Antes. (b) Depois.

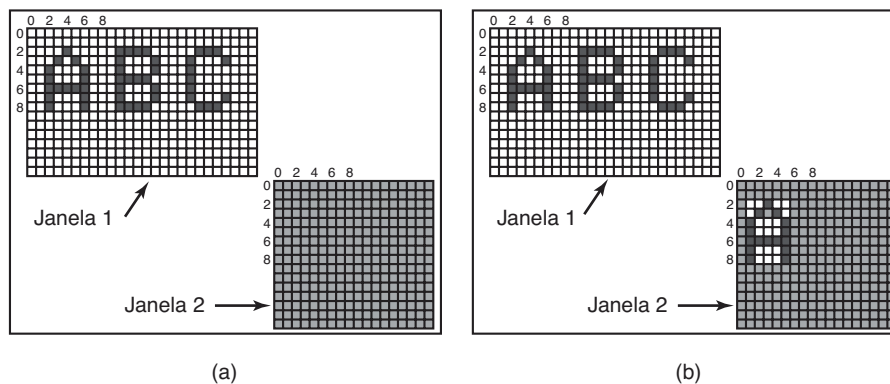


FIGURA 5.39 Alguns exemplos de contornos de caracteres em diferentes tamanhos de pontos.

a qualidade, é possível embutir dicas em cada caractere dizendo como realizar a varredura. Por exemplo, o acabamento das pontas laterais no topo da letra T deve ser idêntico, algo que talvez não fosse o caso devido a um erro de arredondamento. As dicas melhoram a aparência final.

Telas de toque

Mais e mais a tela é usada como um dispositivo de entrada também. Especialmente em smartphones, tablets e outros dispositivos ultraportáteis, é conveniente tocar e “limpar” a tela com seu dedo (ou um stylus). A experiência do usuário é diferente e mais intuitiva do que com um dispositivo como um mouse, tendo em vista que o usuário interage diretamente com objetos na tela. Pesquisas mostraram que mesmo orangotangos, outros primatas e crianças pequenas são capazes de operar dispositivos baseados no toque.

Um dispositivo de toque não é necessariamente uma tela. Dispositivos de toque caem em duas categorias: opacos e transparentes. Um dispositivo de toque opaco típico é o touchpad em um notebook. Um exemplo de um dispositivo transparente é a tela de toque em um smartphone ou tablet. Nessa seção, no entanto, vamos nos limitar às telas de toque.

Assim como muitas coisas que entram na moda na indústria dos computadores, as telas de toque não são exatamente novas. Já em 1965, E.A. Johnson da British Royal Radar Establishment descreveu uma tela de toque (capacitiva) que, embora rudimentar, serviu como

precursora das telas que vemos hoje. A maioria das telas de toque modernas são resistivas ou capacitivas.

Telas resistivas têm uma superfície plástica flexível no topo. O plástico em si não tem nada de especial, exceto que ele é mais resistente a arranhões do que o plástico comum. No entanto, um filme fino de **ITO (Indium Tin Oxide — Óxido de Índio-Estanho)**, ou algum material condutivo similar, é impresso em linhas finas no interior da superfície. Abaixo dela, mas sem tocá-la realmente, há uma segunda superfície também revestida com uma camada de ITO. Na superfície de cima, a carga corre na direção vertical e há conexões condutivas na parte de cima e de baixo. Na camada de baixo a carga corre horizontalmente e há conexões à esquerda e à direita. Ao tocar a tela, você provoca uma “depressão” no plástico de maneira que a camada de cima do ITO toca a camada de baixo. Para descobrir a posição exata do dedo ou stylus tocando-a, tudo o que você precisa fazer é medir a resistência em ambas as direções em todas as posições horizontais da parte de baixo e todas as posições verticais da camada de cima.

Telas capacitivas têm duas superfícies duras, tipicamente vidro, cada uma revestida com ITO. Uma configuração típica é ter o ITO acrescentado a cada superfície em linhas paralelas, onde as linhas na camada de cima são perpendiculares às linhas na camada de baixo. Por exemplo, a camada de cima pode ser revestida de linhas finas em uma direção vertical, enquanto a camada de baixo tem um padrão similarmente em faixas na direção horizontal. As duas superfícies carregadas, separadas pelo ar, formam uma grade de capacitores realmente

pequenos. As voltagens são aplicadas alternativamente às linhas horizontais e verticais, enquanto os valores das voltagens, que são afetados pela capacitância de cada interseção, são lidos nos outros. Quando coloca o seu dedo na tela, você muda a capacitância local. Ao medir muito precisamente as minúsculas mudanças na voltagem em toda parte, é possível descobrir a localização do dedo na tela. Essa operação é repetida muitas vezes por segundo com as coordenadas alimentadas por toque para o driver do dispositivo como um fluxo de pares (x, y) . O processamento além desse ponto, como determinar se o usuário está apontando, apertando, expandindo ou varrendo, é feito pelo sistema operacional.

O que é bacana a respeito das telas resistivas é que a pressão determina o resultado das medidas. Em outras palavras, elas funcionarão mesmo que você esteja usando luvas no frio. Isso não é verdade a respeito de telas capacitivas, a não ser que você use luvas especiais. Por exemplo, você pode costurar um fio condutivo (como nylon banhado em prata) pelas pontas dos dedos das luvas, ou se você não for muito chegado a costura, compre-as prontas. Ou então, você pode cortar as pontas das suas luvas e acabar com o problema em 10 s.

O que não é tão bacana a respeito das telas resistivas é que elas geralmente não suportam **toques múltiplos**, uma técnica que detecta vários toques ao mesmo tempo. Elas permitem que você manipule objetos na tela com dois ou mais dedos. As pessoas gostam dos toques múltiplos porque isso lhes possibilita realizarem gestos de toque-expansão com dois dedos para aumentar ou diminuir uma imagem ou documento. Imagine que os dois dedos estão em $(3, 3)$ e $(8, 8)$. Em consequência, a tela resistiva pode observar uma mudança na resistência nas linhas verticais $x = 3$ e $x = 8$, e nas linhas horizontais $y = 3$ e $y = 8$. Agora considere um cenário diferente com os dedos em $(3, 8)$ e $(8, 3)$, que são os vértices opostos do retângulo cujos vértices são $(3, 3)$, $(8, 3)$, $(8, 8)$ e $(3, 8)$. A resistência em precisamente as mesmas linhas mudou, portanto o software não tem como dizer qual dos dois cenários se mantém. Esse problema é chamado de **ghosting** (efeito fantasma). Como as telas capacitivas enviam um fluxo de coordenadas (x, y) , elas são mais propensas a suportar os toques múltiplos.

Manipular uma tela de toque com apenas um único dedo pode ser considerado WIMP — você simplesmente substitui o ponteiro do mouse com seu *stylus* ou dedo indicador. O uso de múltiplos toques é um pouco mais complicado. Tocar a tela com cinco dedos é como

empurrar cinco ponteiros de mouses sobre a tela ao mesmo tempo e isso claramente muda as coisas de figura para o gerenciador da janela. As telas para múltiplos toques tornaram-se onipresentes e cada vez mais sensíveis e precisas. Mesmo assim, é incerto se a Técnica dos Cinco Pontos que Explodem o Coração¹ tem algum efeito sobre a CPU.

5.7 Clientes magros (thin clients)

Ao longo dos anos, o paradigma do computador principal oscilou entre a computação centralizada e a descentralizada. Os primeiros computadores, como o ENIAC, eram na realidade computadores pessoais (apesar de seu tamanho grande), pois apenas uma pessoa podia usar um de cada vez. Então vieram os sistemas de tempo compartilhado, nos quais muitos usuários remotos em terminais simples compartilhavam de um grande computador central. Em seguida, veio a era do PC, na qual os usuários tinham seus próprios computadores pessoais novamente.

Embora o modelo do PC descentralizado tenha vantagens, ele também tem algumas desvantagens severas que estão apenas começando a ser levadas a sério. Provavelmente o maior problema é que cada PC tem um disco rígido grande e um software complexo que devem ser mantidos. Por exemplo, quando é lançado um novo sistema operacional, é necessário um esforço significativo para atualizar cada máquina em separado. Na maioria das corporações, os custos da mão de obra para realizar esse tipo de manutenção de software são muito superiores aos custos com hardware e software efetivos. Para os usuários caseiros, a mão de obra é tecnicamente gratuita, mas poucas pessoas são capazes de fazê-lo corretamente e menos gente ainda gosta de fazê-lo. Com um sistema centralizado, apenas uma ou algumas poucas máquinas precisam ser atualizadas e elas têm uma equipe de especialistas para realizar o trabalho.

Uma questão relacionada é que os usuários deveriam fazer backups regulares de seus sistemas de arquivos de gigabytes, mas poucos o fazem. Quando acontece um desastre, o que se vê é muita lamentação! Com um sistema centralizado, backups podem ser feitos todas as noites por robôs de fita automatizados.

Outra vantagem é que o compartilhamento de recursos é mais fácil com sistemas centralizados. Um sistema com 256 usuários remotos, cada um com 256 MB de RAM, terá a maior parte dessa RAM ociosa a maior parte do tempo. Com um sistema centralizado com

¹ Brincadeira do autor com um golpe mítico do kung-fu. (N. do T.)

64 GB de RAM, nunca acontece de um algum usuário precisar temporariamente de muita RAM, mas não conseguir porque ela está no PC de outra pessoa. O mesmo argumento se mantém para o espaço de disco e outros recursos.

Por fim, estamos começando a ver uma mudança de uma computação centrada nos PCs para uma computação centrada na web. Uma área já bem adiantada é o e-mail. As pessoas costumavam ter seu e-mail enviado para sua máquina em casa e lê-lo lá. Hoje, muita gente se conecta ao Gmail, Hotmail ou Yahoo e lê seu e-mail ali. O próximo passo será as pessoas se conectando em outros sites na web para editar textos, produzir planilhas e outras coisas que costumavam exigir o software de PCs. É possível até que por fim o único software que as pessoas executarão em seus PCs será um navegador da web, e talvez nem isso.

Dizer que a maioria dos usuários quer uma computação interativa de alto desempenho, mas não quer realmente administrar um computador, provavelmente seja uma conclusão justa. Isso levou os pesquisadores a reexaminar os sistemas de tempo compartilhado usando terminais burros (agora educadamente chamados de **clientes magros**) que atendem às expectativas de terminais modernos. X foi um passo nessa direção e terminais X dedicados foram populares por um tempo, mas caíram em desuso pois eles custavam tanto quanto os PCs, podiam realizar menos e ainda precisavam de alguma manutenção de software. O Santo Graal seria um sistema de computação interativo de alto desempenho no qual as máquinas dos usuários não tivessem software algum. De maneira bastante interessante, essa meta é atingível.

Um dos clientes magros mais conhecidos é o **Chromebook**. Ele é ativamente promovido pelo Google, mas com uma ampla variedade de fabricantes fornecendo uma ampla variedade de modelos. O notebook executa **ChromeOS** que é baseado no Linux e no navegador Chrome Web. Presume-se que esteja on-line o tempo inteiro. A maior parte dos outros softwares está hospedada na web na forma de **Web Apps**, fazendo a pilha de software no próprio Chromebook ser consideravelmente menor do que na maioria dos notebooks tradicionais. Por outro lado, um sistema que executa um sistema Linux inteiro e um navegador Chrome não é exatamente um anoréxico também.

5.8 Gerenciamento de energia

O primeiro computador eletrônico de propósito geral, o ENIAC, tinha 18.000 válvulas e consumia 140.000

watts de energia. Em consequência, ele fez subir muito as contas de energia elétrica. Após a invenção do transistor, o uso de energia caiu dramaticamente e a indústria de computadores perdeu o interesse a respeito das exigências de energia. No entanto, hoje em dia o gerenciamento de energia está de volta ao centro das atenções por várias razões, e o sistema operacional exerce um papel na questão.

Vamos começar com os PCs de mesa. Um PC de mesa muitas vezes tem um suprimento de energia de 200 watts (que é tipicamente 85% eficiente, isto é, perde 15% da energia que recebe para o calor). Se 100 milhões dessas máquinas forem ligadas ao mesmo tempo mundo afora, juntas elas usarão 20.000 megawatts de eletricidade. Isso é uma produção total de 20 usinas nucleares de médio porte. Se as exigências de energia pudessem ser cortadas pela metade, poderíamos nos livrar de 10 usinas nucleares. Do ponto de vista ambiental, livrar-se de 10 usinas nucleares (ou um número equivalente de usinas movidas a combustíveis fósseis) é uma grande vitória que vale a pena ser buscada.

A outra situação em que a energia é uma questão importante é nos computadores movidos a bateria, incluindo notebooks, laptops, palmtops e Webpads, entre outros. O cerne do problema é que as baterias não conseguem conter carga suficiente para durar muito tempo, algumas horas no máximo. Além disso, apesar de enormes esforços por parte das empresas de baterias, fabricantes de computadores e de produtos eletrônicos, o progresso é mínimo. Para uma indústria acostumada a dobrar o seu desempenho a cada 18 meses (lei de Moore), não apresentar progresso algum parece uma violação das leis da física, mas essa é a situação atual. Em consequência, fazer que os computadores usem menos energia de maneira que as baterias existentes durem mais é uma prioridade na agenda de todos. O sistema operacional tem um papel fundamental aqui, como veremos a seguir.

No nível mais baixo os vendedores de hardware estão tentando tornar os seus componentes eletrônicos mais eficientes em termos de energia. As técnicas usadas incluem a redução do tamanho de transistores, o escalonamento dinâmico de tensão, o uso de barramentos adiabáticos e low-swing, e técnicas similares. Essas questões estão fora do escopo deste livro, mas os leitores interessados podem encontrar uma boa pesquisa em um estudo de Venkatachalam e Franz (2005).

Existem duas abordagens gerais para reduzir o consumo de energia. A primeira é o sistema operacional desligar partes do computador (na maior parte dispositivos de E/S) quando elas não estão sendo usadas, pois

um dispositivo que está desligado usa pouca ou nenhuma energia. A segunda abordagem é o programa aplicativo usar menos energia, possivelmente degradando a qualidade da experiência do usuário, a fim de estender o tempo da bateria. Examinaremos cada uma dessas abordagens em sequência, mas primeiro abordaremos brevemente o projeto de hardware em relação ao uso da energia.

5.8.1 Questões de hardware

As baterias vêm em dois tipos gerais: descartáveis e recarregáveis. As baterias descartáveis (mais comumente as do tipo AAA, AA e D) podem ser usadas para executar dispositivos de mão, mas não têm energia suficiente para suprir notebooks com telas grandes e reluzentes. Uma bateria recarregável, por sua vez, pode armazenar energia suficiente para suprir um notebook por algumas horas. Baterias de níquel cádmio costumavam dominar esse nicho, mas deram lugar às baterias híbridas de metal níquel, que duram mais e não poluem tanto o meio ambiente quando são eventualmente descartadas. Baterias de íon lítio são ainda melhores, e podem ser recarregadas sem primeiro serem utilizadas por completo, mas sua capacidade também é severamente limitada.

A abordagem geral que a maioria dos vendedores de computadores escolhe para a conservação da bateria é projetar a CPU, memória e dispositivos de E/S para terem múltiplos estados: ligados, dormindo, hibernando e desligados. Para usar o dispositivo, ele precisa estar ligado. Quando o dispositivo não for necessário por um curto período de tempo, ele pode ser colocado para dormir, o que reduz o consumo de energia. Quando se espera que ele não seja necessário por um intervalo de tempo mais longo, ele pode ser colocado para hibernar, o que reduz o consumo da energia ainda mais. A escolha que precisa ser feita aqui é que tirar um dispositivo da hibernação muitas vezes leva mais tempo e dispende mais energia do que tirá-lo do estado de adormecimento. Por fim, quando um dispositivo está desligado, ele não faz nada e não consome energia. Nem todos os dispositivos têm esses estados, mas quando eles têm, cabe ao sistema operacional gerenciar as transições de estado nos momentos certos.

Alguns computadores têm dois ou até três botões de energia. Um deles pode colocar todo o computador no estado de dormência, do qual ele pode ser acordado rapidamente ao se digitar um caractere ou mover o mouse. Outro pode colocar o computador em hibernação, da qual seu despertar leva bem mais tempo. Em ambos os casos, esses botões não fazem nada além de enviar

um sinal para o sistema operacional, que realiza o resto no software. Em alguns países, dispositivos elétricos devem, por lei, ter uma chave mecânica de energia que interrompa um circuito e remova a energia do dispositivo por questões de segurança. Para obedecer a essa lei, outra chave talvez seja necessária.

O gerenciamento de energia provoca uma série de questões com que o sistema operacional precisa lidar. Muitas delas relacionam-se com a hibernação de recursos — seletiva e temporariamente desligar dispositivos, ou pelo menos reduzir seu consumo de energia quando estão ociosos. As perguntas que devem ser respondidas incluem: quais dispositivos podem ser controlados? Eles têm apenas os estados ligado/desligado, ou existem estados intermediários? Quanta energia é poupada nos estados de baixo consumo? Gasta-se energia para reiniciar o dispositivo? Algum contexto precisa ser salvo quando se vai para o estado de baixo consumo? Quanto tempo leva para retornar para o estado de energia plena? É claro, as respostas para essas questões variam de dispositivo para dispositivo, de maneira que o sistema operacional precisa ser capaz de lidar com uma gama de possibilidades.

Vários pesquisadores examinaram notebooks para ver em que ponto a energia se esgota. Li et al. (1994) mediram várias cargas de trabalho e chegaram às conclusões mostradas na Figura 5.40. Lorch e Smith (1998) tomaram medidas em outras máquinas e chegaram às conclusões mostradas na Figura 5.40. Weiser et al. (1994) também fizeram medidas, mas não publicaram os valores numéricos. Eles apenas declararam que os três maiores consumidores de energia eram a tela, o disco rígido e a CPU, nessa ordem. Embora esses números não concordem totalmente, talvez porque as diferentes marcas de computadores mensuradas de fato tenham exigências diversas de energia, parece claro que a tela, o disco rígido e a CPU são alvos óbvios para poupar energia. Em dispositivos

FIGURA 5.40 Consumo de energia de várias partes de um notebook.

Dispositivo	Li et al. (1994)	Lorch e Smith (1998)
Tela	68%	39%
CPU	12%	18%
Disco rígido	20%	12%
Modem		6%
Som		2%
Memória	0,5%	1%
Outros		22%

como smartphones pode haver outros drenos de energia, como o rádio e o GPS. Embora nos concentremos nas telas, discos, CPUs e memória nesta seção, os princípios são os mesmos para outros periféricos.

5.8.2 Questões do sistema operacional

O sistema operacional tem um papel fundamental no gerenciamento da energia. Ele controla todos os dispositivos, por isso tem de decidir o que desligar e quando fazê-lo. Se desligar um dispositivo e esse dispositivo for necessário imediatamente de novo, pode haver um atraso incômodo enquanto ele é reiniciado. Por outro lado, se ele esperar tempo demais para desligar um dispositivo, a energia é desperdiçada por nada.

O truque é encontrar algoritmos e heurísticas que deixem o sistema operacional tomar boas decisões a respeito do que desligar e quando. O problema é que “boas decisões” é algo muito subjetivo. Um usuário pode achar aceitável que após 30 s de não utilização, o computador leve 2 s para responder a uma tecla pressionada. Outro usuário pode perder a paciência com a mesma espera. Na ausência da entrada de áudio, o computador não sabe distinguir entre esses usuários.

Monitor

Vamos agora examinar os grandes gastadores do orçamento de energia para ver o que pode ser feito com cada um deles. Um dos itens mais importantes no orçamento de energia de todo mundo é o monitor. Para obter uma imagem clara e nítida, a tela deve ser iluminada por trás e isso consome uma energia substancial. Muitos sistemas operacionais tentam poupar energia desligando o monitor quando não há atividade alguma por um número determinado de minutos. Muitas vezes o usuário pode decidir qual deve ser esse intervalo, devendo escolher entre o desligamento frequente da tela e o consumo rápido da bateria (o que provavelmente o usuário não deseja). O desligamento do monitor é um estado de dormência, pois ele pode ser regenerado (da RAM de vídeo) quase instantaneamente quando qualquer tecla for acionada ou o dispositivo apontador for movido.

Uma melhoria possível foi proposta por Flinn e Satyanarayanan (2004). Eles sugeriram que o monitor consista em um determinado número de zonas que podem ser ligadas ou desligadas independentemente. Na Figura 5.41, descrevemos 16 zonas, usando linhas tracejadas para separá-las. Quando o cursor está na janela 2, como mostrado na Figura 5.41(a), apenas quatro zonas do canto inferior direito precisam ser iluminadas.

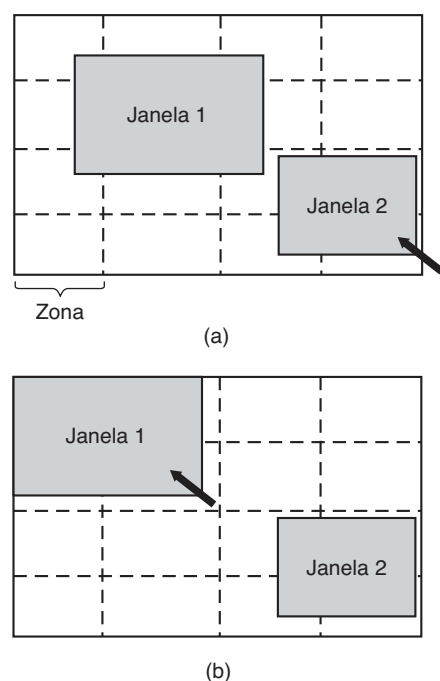
As outras 12 podem ficar escuras, poupando 3/4 da energia da tela.

Quando o usuário move o cursor para a janela 1, as zonas para a janela 2 podem ser escurecidas e as zonas atrás da janela 1 podem ser iluminadas. No entanto, como a janela 1 envolve 9 zonas, mais energia é necessária. Se o gerenciador de janelas consegue perceber o que está acontecendo, ele pode automaticamente mover a janela 1 para que ela se enquadre em quatro zonas, com um tipo de foco instantâneo de zona, como mostrado na Figura 5.41(b). Para realizar essa redução de 9/16 para 4/16 de energia total, o gerenciador de janelas precisa entender de gerenciamento de energia ou ser capaz de aceitar instruções de alguma outra parte do sistema que o compreenda. Ainda mais sofisticada seria a capacidade de iluminar em parte uma janela que não estivesse completamente cheia (por exemplo, uma janela contendo linhas curtas de texto poderia ficar com o lado direito no escuro).

Disco rígido

Outro vilão importante é o disco rígido. Ele consome uma energia substancial para manter-se girando em alta velocidade, mesmo que não haja acessos. Muitos computadores, em especial notebooks, param de girar

FIGURA 5.41 O uso de zonas para a retroiluminação (back lighting) do monitor. (a) Quando a janela 2 é escolhida, ela não é movida. (b) Quando a janela 1 é escolhida, ela é movida para reduzir o número de zonas iluminadas.



o disco após determinado número de minutos com ele ocioso. Quando é requisitado em seguida, o disco é girado outra vez. Infelizmente, um disco parado está hibernando em vez de dormindo, pois ele leva alguns segundos para girar novamente, o que causa atrasos consideráveis para o usuário.

Além disso, reiniciar o disco consome uma energia considerável. Em consequência, todo disco tem um tempo característico, T_d , que é o seu ponto de equilíbrio, muitas vezes na faixa de 5 a 15 s. Suponha que o próximo acesso de disco é esperado que ocorra algum tempo t no futuro. Se $t < T_d$, menos energia é consumida para manter o disco girando do que pará-lo e então reinicializá-lo tão rapidamente. Se $t > T_d$, a energia poupada faz valer a pena parar o disco e então reinicializá-lo de novo bem mais tarde. Se uma boa previsão pudesse ser feita (por exemplo, baseada em padrões de acesso passados), o sistema operacional poderia fazer boas previsões de parada e poupar energia. Na prática, a maioria dos sistemas é conservadora e para o disco somente após alguns minutos de inatividade.

Outra maneira de poupar energia é ter uma cache de disco substancial na RAM. Se um bloco necessário está na cache, um disco ocioso não precisa ser ativado para satisfazer a leitura. Similarmente, se uma escrita para o disco pode ser armazenada na cache, um disco parado não precisa ser ativado apenas para lidar com a escrita. O disco pode permanecer desligado até que a cache esteja cheia ou que ocorra uma lacuna na leitura.

Outra maneira de evitar que um disco seja ativado desnecessariamente é fazer que o sistema operacional mantenha os programas em execução informados a respeito do estado do disco enviando-lhes mensagens ou sinais. Alguns programas têm escritas programadas que podem ser “puladas” ou postergadas. Por exemplo, um editor de texto pode ser ajustado para escrever o arquivo que está sendo editado para o disco de tempos em tempos. Se o editor de texto sabe que o disco está desligado naquele momento em que ele normalmente escreveria o arquivo, pode postergar essa escrita até que o disco esteja ligado.

A CPU

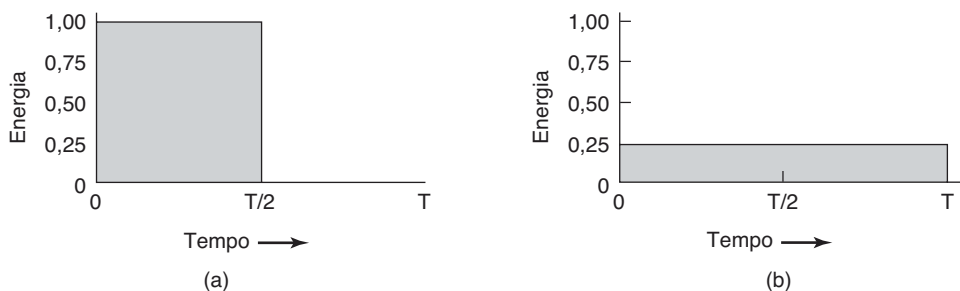
A CPU também pode ser gerenciada para poupar energia. O software pode colocar a CPU de um notebook para dormir, reduzindo o uso de energia a quase zero. A única coisa que ela pode fazer nesse estado é despertar quando ocorre uma interrupção. Portanto, sempre que a CPU fica ociosa, seja esperando por E/S ou porque não há trabalho para fazer, ela vai dormir.

Em muitos computadores, há uma relação entre a voltagem da CPU, ciclo do relógio e consumo de energia. A voltagem da CPU muitas vezes pode ser reduzida no software, o que poupa energia, mas também reduz o ciclo do relógio (mais ou menos linearmente). Tendo em vista que a energia consumida é proporcional ao quadrado da voltagem, cortar a voltagem pela metade torna a CPU cerca de 50% mais lenta, mas a $\frac{1}{4}$ da energia.

Essa propriedade pode ser explorada para programas com prazos bem definidos, como programas de visualização multimídia que devem descomprimir e mostrar no vídeo um quadro a cada 40 ms e ficam ociosos se o fazem mais rapidamente. Suponha que uma CPU usa x joules enquanto executa em velocidade máxima por 40 ms e $x/4$ joules com a metade da velocidade. Se um programa de visualização multimídia pode descomprimir e mostrar no vídeo um quadro em 20 ms, o sistema operacional pode executar em velocidade máxima por 20 ms e então desligar por 20 ms para um consumo de energia total de $x/2$ joules. Alternativamente, ele pode executar com metade da energia e apenas cumprir o prazo, mas usar só $x/4$ joules em vez disso. A Figura 5.42 mostra uma comparação entre a execução em velocidade máxima e em energia máxima por algum intervalo de tempo e na metade da velocidade e em $1/4$ da energia por duas vezes o tempo. Em ambos os casos, o mesmo trabalho é realizado, mas na Figura 5.42(b) somente metade da energia é consumida ao realizá-lo.

De maneira similar, se um usuário está digitando 1 caractere/s, mas o trabalho necessário para processar o caractere leva 100 ms, é melhor para o sistema

FIGURA 5.42 (a) Funcionamento com velocidade total. (b) Redução da voltagem à metade: metade da velocidade e um quarto da energia.



operacional detectar os longos períodos ociosos e desacelerar a CPU por um fator de 10. Resumindo, executar lentamente é mais eficiente em termos de consumo de energia do que executar rapidamente.

De maneira interessante, reduzir a velocidade de núcleos da CPU nem sempre implica uma redução no desempenho. Hruby et al. (2013) mostram que às vezes o desempenho da pilha de software de rede *melhora* com núcleos mais lentos. A explicação é que o núcleo pode ser rápido demais para o seu próprio bem. Por exemplo, imagine uma CPU com diversos núcleos rápidos, onde um núcleo é responsável pela transmissão dos pacotes de rede em prol de um produtor executando em outro núcleo. O produtor e a pilha de rede comunicam-se diretamente via uma memória compartilhada e ambos executam em núcleos dedicados. O produtor realiza uma quantidade considerável de computação e não consegue acompanhar o núcleo da pilha de rede. Em uma execução típica, a rede transmitirá tudo o que ela tem para transmitir e fará uma verificação da memória compartilhada por algum montante de tempo para ver se realmente não há mais dados para transmitir. Por fim, ela vai desistir e ir dormir, pois a verificação contínua é muito ruim para o consumo de energia. Logo em seguida, o produtor fornecerá mais dados, mas agora a pilha de rede está adormecida. Despertar a pilha leva tempo e atrasa a produção. Uma solução possível é jamais dormir, mas isso não é interessante, porque fazê-lo aumentaria o consumo de energia — exatamente o oposto do que estamos tentando conseguir. Uma solução mais atraente é executar a pilha de rede em um núcleo mais lento, de maneira que ele esteja constantemente ocupado (e desse modo, nunca durma), enquanto ainda reduz o consumo de energia. Se o núcleo da rede for desacelerado com cuidado, seu desempenho será melhor do que em uma configuração em que todos os núcleos são incrivelmente rápidos.

A memória

Existem duas opções possíveis para poupar energia com a memória. Primeiro, a cache pode ser esvaziada e então desligada. Ela sempre pode ser recarregada da memória principal sem perda de informações. A recarga pode ser feita dinâmica e rapidamente; portanto, desligar a cache é como entrar em um estado de dormência.

Uma opção mais drástica é escrever os conteúdos da memória principal para o disco, então desligar a própria memória. Essa abordagem é a hibernação, já que virtualmente toda a energia pode ser cortada para a memória à custa de um tempo de recarga substancial, em especial se o disco estiver desligado, também. Quando a memória é desligada, a CPU tem de ser desligada também ou tem

de executar a partir da ROM. Se a CPU estiver desligada, a interrupção que deve acordá-la precisa fazê-la saltar para o código na ROM de modo que a memória possa ser recarregada antes de ser usada. Apesar de toda a sobrecarga, desligar a memória por longos períodos de tempo (por exemplo, horas) pode valer a pena se reinicializá-la em alguns segundos for considerado muito mais desejável do que reinicializar o sistema operacional a partir do disco, o que muitas vezes leva um minuto ou mais.

Comunicação sem fio

Cada vez mais muitos computadores portáteis têm uma conexão sem fio para o mundo exterior (por exemplo, internet). Os transmissores e receptores de rádio necessários são muitas vezes grandes consumidores de energia. Em particular, se o receptor de rádio estiver sempre ligado a fim de receber as mensagens que chegam do e-mail, a bateria pode ser consumida bem rápido. Por outro lado, se o rádio for desligado após, digamos, 1 minuto de ociosidade, as mensagens que chegam podem ser perdidas, o que claramente não é desejável.

Uma solução eficiente para esse problema foi proposta por Kravets e Krishnan (1998). O cerne da sua solução explora o fato de que os computadores móveis comunicam-se com estações-base fixas que têm grandes memórias e discos e nenhuma restrição de energia. O que eles propõem é que o computador móvel envie uma mensagem para a estação-base quando estiver prestes a desligar o rádio. Daquele momento em diante, a estação-base armazena em seu disco as mensagens que chegarem. O computador móvel pode indicar explicitamente quanto tempo ele está planejando dormir, ou simplesmente informar a estação-base quando ele ligar o rádio novamente. A essa altura, quaisquer mensagens acumuladas podem ser enviadas para ele.

Mensagens de saída que são geradas enquanto o rádio está desligado são armazenadas no computador móvel. Se o buffer ameaça saturar, o rádio é ligado e a fila, transmitida para a estação-base.

Quando o rádio deve ser desligado? Uma possibilidade é deixar o usuário ou o programa de aplicação decidir. Outra é desligá-lo após alguns segundos de tempo ocioso. Quando ele deve ser ligado novamente? Mais uma vez, o usuário ou o programa poderiam decidir, ou ele poderia ser ligado periodicamente para conferir o tráfego que chega e transmitir quaisquer mensagens na fila. É claro, ele também deve ser ligado quando o buffer de saída está quase cheio. Várias outras heurísticas são possíveis.

Um exemplo de uma tecnologia sem fio trabalhando com esse tipo de esquema de gerenciamento de energia

pode ser encontrado nas redes (“WiFi”) 802.11. Na 802.11, um computador móvel pode notificar o ponto de acesso que ele vai dormir, mas despertará antes que a estação-base envie o próximo quadro de orientação (beacon frame). O ponto de acesso envia esses sinais periodicamente. Nesse momento, o ponto de acesso pode dizer ao computador que ele tem dados pendentes. Se não houver esses dados, o computador móvel pode dormir novamente até o próximo sinal.

Gerenciamento térmico

Uma questão de certa maneira diferente, mas ainda assim relacionada à energia, é o gerenciamento térmico. As CPUs modernas ficam extremamente quentes por causa de sua alta velocidade. Computadores de mesa em geral têm um ventilador elétrico interno para soprar o ar quente para fora do gabinete. Dado que a redução do consumo de energia normalmente não é uma questão fundamental com os computadores de mesa, o ventilador em geral está ligado o tempo inteiro.

Com os notebooks a situação é diferente. O sistema operacional tem de monitorar a temperatura continuamente. Quando chega próximo da temperatura máxima permitida, o sistema operacional tem uma escolha. Ele pode ligar o ventilador, o que faz barulho e consome energia. Alternativamente, ele pode reduzir o consumo de energia reduzindo a iluminação da tela, desacelerando a CPU, sendo mais agressivo no desligamento do disco, e assim por diante.

Alguma entrada do usuário pode ser valiosa como um guia. Por exemplo, um usuário poderia especificar antecipadamente que o ruído do ventilador é desagradável, assim o sistema operacional reduziria o consumo de energia em vez de ligá-lo.

Gerenciamento de bateria

Antigamente, uma bateria apenas fornecia corrente até se esgotar, momento em que ela parava. Não mais. Os dispositivos móveis usam baterias inteligentes agora, que podem comunicar-se com o sistema operacional. Mediante uma solicitação do sistema operacional, elas podem dar retorno sobre coisas como a sua voltagem máxima, voltagem atual, carga máxima, carga atual, taxa de descarga máxima e mais. A maioria dos dispositivos móveis tem programas que podem ser executados para obter e exibir todos esses parâmetros. Baterias inteligentes também podem ser instruídas a mudar vários parâmetros operacionais sob controle do sistema operacional.

Alguns notebooks têm múltiplas baterias. Quando o sistema operacional detecta que uma bateria está prestes a se esgotar, ele deve desativá-la e ativar outra graciosamente, sem causar nenhuma falha durante a transição. Quando a bateria final está nas suas últimas forças, cabe ao sistema operacional avisar o usuário e então provocar um desligamento ordeiro, por exemplo, certificando-se de que o sistema de arquivos não seja corrompido.

Interface do driver

Vários sistemas operacionais têm um mecanismo elaborado para realizar o gerenciamento de energia chamado de **interface avançada de configuração e energia (Advanced Configuration and Power Interface — ACPI)**. O sistema operacional pode enviar quaisquer comandos para o driver requisitando informações sobre as capacidades dos seus dispositivos e seus estados atuais. Essa característica é especialmente importante quando combinada com a característica plug and play, pois logo após a inicialização, o sistema operacional não faz nem ideia de quais dispositivos estão presentes, muito menos suas propriedades em relação ao consumo ou modo de gerenciamento de energia.

Ele também pode enviar comandos para os drivers instruindo-os para cortar seus níveis de energia (com base nas capacidades a respeito das quais ele ficou sabendo antes, é claro). Há também algum tráfego no outro sentido. Em particular, quando um dispositivo como um teclado ou um mouse detecta atividade após um período de ociosidade, esse é um sinal para o sistema voltar à operação (quase) normal.

5.8.3 Questões dos programas aplicativos

Até o momento examinamos as maneiras que o sistema operacional pode reduzir o uso de energia por meio de vários tipos de dispositivos. Mas existe outra abordagem também: dizer aos programas para usarem menos energia, mesmo que isso signifique fornecer uma experiência do usuário de pior qualidade (melhor uma experiência de pior qualidade do que nenhuma experiência quando a bateria morre e as luzes apagam). Tipicamente, essa informação é passada adiante quando a carga da bateria está abaixo de algum limiar. Cabe aos programas então decidirem entre degradar o desempenho para estender a vida da bateria, ou manter o desempenho e arriscar ficar sem energia.

Uma questão que surge é como um programa pode degradar o seu desempenho para poupar energia. Essa

questão foi estudada por Flinn e Satyanarayanan (2004). Eles forneceram quatro exemplos de como o desempenho degradado pode poupar energia. Vamos examiná-los a seguir.

Nesse estudo, as informações são apresentadas ao usuário de várias formas. Quando nenhuma degradação está presente, é apresentada a melhor informação possível. Quando a degradação está presente, a fidelidade (precisão) da informação apresentada ao usuário é pior do que ela poderia ser. Veremos em breve exemplos disso.

A fim de mensurar o uso de energia, Flinn e Satyanarayanan desenvolveram uma ferramenta de software chamada PowerScope. O que ela faz é fornecer um perfil de uso de energia de um programa. Para usá-la, um computador precisa estar conectado a um suprimento externo de energia através de um multímetro digital controlado por software. Usando o multímetro, o software é capaz de ler o número de miliampères que estão chegando do suprimento de energia e assim determinar a energia instantânea consumida pelo computador. O que a PowerScope faz é amostrar periodicamente o contador do programa e o uso de energia, e escrever esses dados para um arquivo. Após o programa ter sido concluído, o arquivo é analisado para dar o uso de energia de cada rotina. Essas medidas formaram a base das suas observações. Medidas de economia de energia também foram usadas e formaram as linhas de base pelas quais o desempenho degradado foi mensurado.

O primeiro programa mensurado foi um reproduzidor de vídeo. No modo sem degradação, ele reproduz 30 quadros/s em uma resolução total e em cores. Uma forma de degradação é abandonar a informação das cores e exibir o vídeo em preto e branco. Outra forma de degradação é reduzir a frequência de reprodução, o que faz a imagem piscar (flicker) e deixa o filme com uma qualidade irregular. Ainda outra forma de degradação é reduzir o número de pixels em ambas as direções, seja reduzindo a resolução espacial ou tornando a imagem exibida menor. Medidas desse tipo pouparam em torno de 30% da energia.

O segundo programa foi um reconhecedor de voz. Ele coletava amostras do microfone e construía um modelo de onda. Esse modelo de onda podia ser analisado no notebook ou ser enviado por um canal de rádio para análise em um computador fixo. Fazer isso poupa energia da CPU, mas usa energia para o rádio. A degradação foi conseguida usando um vocabulário menor e um modelo acústico mais simples. O ganho aqui foi de aproximadamente 35%.

O exemplo seguinte foi um programa visualizador de mapas que buscava o mapa por um canal de rádio. A degradação consistia em cortar o mapa para dimensões menores ou dizer ao servidor remoto para omitir

estradas menores, desse modo exigindo menos bits para serem transmitidos. Mais uma vez aqui foi conseguido um ganho de aproximadamente 35%.

O quarto experimento foi com a transmissão de imagens JPEG para um navegador na internet. O padrão JPEG permite vários algoritmos, negociando entre a qualidade da imagem e o tamanho do arquivo. Aqui o ganho foi em média 9%. Ainda assim, como um todo, os experimentos mostraram que ao aceitar alguma degradação de qualidade, o usuário pode dispor de uma determinada bateria por mais tempo.

5.9 Pesquisas em entrada/saída

Há uma produção considerável de pesquisas sobre entrada/saída. Parte delas concentra-se em dispositivos específicos, em vez da E/S em geral. Outros trabalhos concentram-se na infraestrutura de E/S inteira. Por exemplo, a arquitetura Streamline busca fornecer E/S sob medida para cada aplicação, que minimize a sobrecarga devido a cópias, chaveamento de contextos, sinalização e uso equivocado da cache e TLB (DEBRUIJN et al., 2011). Ela é baseada na noção de Beltway Buffers, buffers circulares avançados que são mais eficientes do que os sistemas de buffers existentes (DEBRUIJN e BOS, 2008). O streamline é especialmente útil para aplicações com exigentes demandas de rede. Megapipe (HAN et al., 2012) é outra arquitetura de E/S em rede para cargas de trabalho orientadas a mensagens. Ela cria canais bidirecionais por núcleo de CPU entre o núcleo do SO e o espaço do usuário, sobre os quais os sistemas depositam camadas de abstrações como soquetes leves. Esses soquetes não são totalmente compatíveis com o POSIX, de maneira que aplicações precisam ser adaptadas para beneficiar-se da E/S mais eficiente.

Muitas vezes, a meta da pesquisa é melhorar o desempenho de um dispositivo de uma maneira ou de outra. Os sistemas de discos são um desses casos. Os algoritmos de escalonamento de braço de disco são uma área de pesquisa sempre popular. Às vezes o foco é a melhoria do desempenho (GONZALEZ-FEREZ et al., 2012; PRABHAKAR et al., 2013; ZHANG et al., 2012b), mas às vezes é o uso mais baixo de energia (KRISH et al., 2013; NIJIM et al., 2013; ZHANG et al., 2012a). Com a popularidade da consolidação de servidores usando máquinas virtuais, o escalonamento de disco para sistemas virtualizados tornou-se um tópico em alta (JIN et al., 2013; LING et al., 2012).

Nem todos os tópicos são novos, no entanto. O velho RAID ainda é bastante pesquisado (CHEN et al., 2013;

MOON e REDDY, 2013; TIMCENKO e DJORDJEVIC, 2013), assim como os SSDs (DAYAN et al., 2013; KIM et al., 2013; LUO et al., 2013). Na frente teórica, alguns pesquisadores estão examinando a modelagem de sistemas de discos a fim de compreender melhor seu desempenho sob diferentes cargas de trabalho (LI et al., 2013b; SHEN e QI, 2013).

Os discos não são o único dispositivo de E/S na linha de frente das pesquisas. Outra área de pesquisa fundamental relacionada à E/S são as redes. Os tópicos incluem o uso de energia (HEWAGE e VOIGT, 2013; HOQUE et al., 2013), redes para centros de processamento de dados (HAITJEMA, 2013; LIU et al., 2013; SUN et al., 2013), qualidade do serviço (GUPTA, 2013; HEMKUMAR e VINAYKUMAR, 2012; LAI e TANG, 2013) e desempenho (HAN et al., 2012; SOORTY, 2012).

Dado o grande número de cientistas da computação com notebooks e dado o tempo de vida microscópico de bateria na maioria deles, não deve causar surpresa que haja um tremendo interesse na utilização de técnicas de software para a redução do consumo de energia. Entre os tópicos especializados sendo examinados estão o equilíbrio da velocidade do relógio em diferentes núcleos para alcançar um desempenho suficiente sem desperdiçar energia (HRUBY, 2013), uso de energia e qualidade do serviço (HOLMBACKA et al., 2013), estimar o uso de energia em tempo real (DUTTA et al., 2013), fornecer serviços de SO para gerenciar o uso de energia (WEISSEL, 2012), examinar o custo de energia da segurança (KABRI e SERET, 2009) e escalonamento para multimídia (WEI et al., 2010).

Nem todos estão interessados em notebooks, no entanto. Alguns cientistas de computação pensam grande e querem poupar megawatts em centros de processamento de dados (FETZER e KNAUTH, 2012; SCHWARTZ et al., 2012; WANG et al., 2013b; YUAN et al., 2012).

Na outra extremidade do espectro, um tópico muito em alta é o uso de energia em redes de sensores (ALBATH et al., 2013; MIKHAYLOV e TERVONEN, 2013; RASANEH e BANIROSTAM, 2013; SEVERINI et al., 2012).

De certa maneira surpreendente, mesmo o simples relógio ainda é pesquisado. Para fornecer uma boa resolução, alguns sistemas operacionais executam o relógio a 1000 Hz, o que leva a uma sobrecarga substancial. A pesquisa entra com o intuito de livrar-se dessa sobrecarga (TSAFIR et al., 2005).

Similarmente, a latência de interrupções ainda é uma preocupação para os grupos de pesquisa, em especial na área dos sistemas operacionais em tempo real. Tendo em vista que elas são muitas vezes encontradas embarcadas em sistemas críticos (como controles de freio e sistemas de direção), permitir interrupções somente em pontos de preempção bastante específicos capacita o sistema a controlar possíveis entrelaçamentos e permite o uso da verificação formal para melhorar a confiabilidade (BLACKHAM et al., 2012).

Drivers de dispositivos também são uma área de pesquisa muito ativa. Muitas falhas de sistemas operacionais são causadas por drivers de dispositivos defeituosos. Em Symdrive, os autores apresentam uma estrutura para testar drivers de dispositivos sem realmente comunicar-se com os dispositivos (RENZELMANN et al., 2012). Como uma abordagem alternativa, RHYZIK et al. (2009) mostram como drivers de dispositivos podem ser construídos automaticamente a partir de especificações, com uma probabilidade menor de ocorrerem defeitos.

Clientes magros também são um tópico que gera interesse, especialmente dispositivos móveis conectados à nuvem (HOCKING, 2011; TUAN-ANH et al., 2013). Por fim, existem alguns estudos sobre tópicos incomuns como prédios como grandes dispositivos de E/S (DAWSON-HAGGERTY et al., 2013).

5.10 Resumo

A entrada/saída é um tópico importante, mas muitas vezes negligenciado. Uma fração substancial de qualquer sistema operacional diz respeito à E/S. A E/S pode ser conseguida de três maneiras. Primeiro, há a E/S programada, na qual a CPU principal envia ou recebe cada byte ou palavra e aguarda em um laço estreito esperando até que possa receber ou enviar o próximo byte ou palavra. Segundo, há a E/S orientada à interrupção, na qual a CPU inicia uma transferência de E/S para um caractere ou palavra e vai fazer outra coisa até a interrupção chegar sinalizando a conclusão da E/S. Terceiro, há o

DMA, no qual um chip separado gerencia a transferência completa de um bloco de dados, gerando uma interrupção somente quando o bloco inteiro foi transferido.

A E/S pode ser estruturada em quatro níveis: as rotinas de tratamento de interrupção, os drivers de dispositivos, o software de E/S independente do dispositivo, e as bibliotecas de E/S e spoolers que executam no espaço do usuário. Os drivers do dispositivo lidam com os detalhes da execução dos dispositivos e com o fornecimento de interfaces uniformes para o resto do sistema operacional. O software de E/S independente do dispositivo

realiza atividades como o armazenamento em buffers e relatórios de erros.

Os discos vêm em uma série de tipos, incluindo discos magnéticos, RAIDS, pen-drives e discos ópticos. Nos discos rotacionais, os algoritmos de escalonamento do braço do disco podem ser usados muitas vezes para melhorar o desempenho do disco, mas a presença de geometrias virtuais complica as coisas. Pareando dois discos, pode ser construído um meio de armazenamento estável com determinadas propriedades úteis.

Relógios são usados para manter um controle do tempo real — limitando o tempo que os processos podem ser executados —, lidar com temporizadores watchdog e contabilizar o uso da CPU.

Terminais orientados por caracteres têm uma série de questões relativas a caracteres especiais que podem ser entrada e sequências de escape especiais que podem ser saída. A entrada pode ser em modo cru ou modo cozido, dependendo de quanto controle o programa quer sobre ela. Sequências de escape na saída controlam o movimento do cursor e permitem a inserção e remoção de texto na tela.

A maioria dos sistemas UNIX usa o Sistema X Window como base de sua interface do usuário. Ele

consiste em programas que são ligados a bibliotecas especiais que emitem comandos de desenho e um servidor X que escreve na tela.

Muitos computadores usam GUIs para sua saída. Esses são baseados no paradigma WIMP: janelas, ícones, menus e um dispositivo apontador (Windows, Icons, Menus, Pointing device). Programas baseados em GUIs são geralmente orientados a eventos, com eventos do teclado, mouse e outros sendo enviados para o programa para serem processados tão logo eles acontecem. Em sistemas UNIX, os GUIs quase sempre executam sobre o X.

Clientes magros têm algumas vantagens sobre os PCs padrão, notavelmente por sua simplicidade e menos manutenção para os usuários.

Por fim, o gerenciamento de energia é uma questão fundamental para telefones, tablets e notebooks, pois os tempos de vida das baterias são limitados, e para os computadores de mesa e de servidores devido às contas de luz da organização. Várias técnicas podem ser empregadas pelo sistema operacional para reduzir o consumo de energia. Programas também podem ajudar ao sacrificar alguma qualidade por mais tempo de vida das baterias.

PROBLEMAS

1. Avanços na tecnologia de chips tornaram possível colocar o controlador inteiro, incluindo toda a lógica de acesso do barramento, em um chip barato. Como isso afeta o modelo da Figura 1.6?
2. Dadas as velocidades listadas na Figura 5.1, é possível escanear documentos de um scanner e transmiti-los através de uma rede de 802,11 g na velocidade máxima? Defenda sua resposta.
3. A Figura 5.3(b) mostra uma maneira de conseguir a E/S mapeada na memória mesmo na presença de barramentos separados para dispositivos de memória e, E/S, a saber, primeiro tentar o barramento de memória e, se isso falhar, tentar o barramento de E/S. Um estudante de computação inteligente pensou em uma melhoria para essa ideia: tentar ambos em paralelo, a fim de acelerar o processo de acessar dispositivos de E/S. O que você acha dessa ideia?
4. Explique os ganhos e perdas entre interrupções precisas e imprecisas em uma máquina superescalar.
5. Um controlador de DMA tem cinco canais. O controlador é capaz de solicitar uma palavra de 32 bits a cada 40 ns. Uma resposta leva um tempo igualmente longo. Quão rápido o barramento precisar ser para evitar ser um gargalo?
6. Suponha que um sistema usa DMA para a transferência de dados do controlador do disco para a memória principal. Além disso, presuma que são necessários t_1 ns em média para adquirir o barramento e t_2 ns para transferir uma palavra através do barramento ($t_1 \gg t_2$). Após a CPU ter programado o controlador de DMA, quanto tempo será necessário para transferir 1.000 palavras do controlador do disco para a memória principal, se (a) o modo uma-palavra-de-cada-vez for usado, (b) o modo de surto for usado? Presuma que o comando do controlador de disco exige adquirir o barramento para enviar uma palavra e o reconhecimento de uma transferência também exige adquirir o barramento para enviar uma palavra.
7. Um modo que alguns controladores de DMA usam é o controlador do dispositivo enviar a palavra para o controlador de DMA, que então emite uma segunda solicitação de barramento para escrever para a memória. Como esse modo pode ser usado para realizar uma cópia memória para memória? Discuta qualquer vantagem ou desvantagem de usar esse método em vez de usar a CPU para realizar uma cópia memória para memória.
8. Suponha que um computador consiga ler ou escrever uma palavra de memória em 5 ns. Também suponha que, quando uma interrupção ocorrer, todos os 32 registradores da

- CPU, mais o contador do programa e PSW são empurrados para a pilha. Qual é o número máximo de interrupções por segundo que essa máquina pode processar?
9. Projetistas de CPUs sabem que projetistas de sistemas operacionais detestam interrupções imprecisas. Uma maneira de agradar ao pessoal do SO é fazer que a CPU pare de emitir novas instruções quando uma interrupção é sinalizada, mas permitir que todas as instruções que atualmente estão sendo executadas sejam concluídas, então forçar a interrupção. Essa abordagem tem alguma desvantagem? Explique a sua resposta.
 10. Na Figura 5.9(b), a interrupção não é reconhecida até depois de o caractere seguinte ter sido enviado para a impressora. Ele poderia ter sido reconhecido logo no início da rotina do serviço de interrupção? Se a resposta for sim, dê uma razão para fazê-lo ao fim, como no texto. Se não, por quê?
 11. Um computador tem um pipeline de três estágios como mostrado na Figura 1.7(a). Em cada ciclo do relógio, uma instrução nova é buscada da memória no endereço apontado pelo PC, colocado no pipeline e o PC incrementado. Cada instrução ocupa exatamente uma palavra de memória. As instruções que já estão no pipeline são avançadas em um estágio. Quando ocorre uma interrupção, o PC atual é colocado na pilha, e o PC é configurado para o endereço do tratador da interrupção. Então o pipeline é deslocado um estágio para a direita e a primeira instrução do tratador da interrupção é buscada no pipeline. Essa máquina tem interrupções precisas? Defenda sua resposta.
 12. Uma página de texto impressa típica contém 50 linhas de 80 caracteres cada. Imagine que uma determinada impressora possa imprimir 6 páginas por minuto e que o tempo para escrever um caractere para o registrador de saída da impressora é tão curto que ele pode ser ignorado. Faz sentido executar essa impressora usando a E/S orientada pela interrupção se cada caractere impresso exige uma interrupção que leva ao todo 50 μ s para servir?
 13. Explique como um SO pode facilitar a instalação de um dispositivo novo sem qualquer necessidade de recompilar o SO.
 14. Em qual das quatro camadas de software de E/S cada uma das tarefas a seguir é realizada:
 - (a) Calcular a trilha, setor e cabeçote para uma leitura de disco.
 - (b) Escrever comandos para os registradores do dispositivo.
 - (c) Conferir se o usuário tem permissão de usar o dispositivo.
 - (d) Converter inteiros binários em ASCII para impressão.
 15. Uma rede de área local é usada como a seguir. O usuário emite uma chamada de sistema para escrever pacotes de dados para a rede. O sistema operacional então copia os dados para um buffer do núcleo e, a seguir, copia os dados para a placa do controlador da rede. Quando todos os bytes estão seguramente dentro do controlador, eles são enviados através da rede a uma taxa de 10 megabits/s. O controlador da rede receptora armazena cada bit um microssegundo após ele ter sido enviado. Quando o último bit chega, a CPU de destino é interrompida, e o núcleo copia o pacote recém-chegado para um buffer do núcleo inspecioná-lo. Uma vez que ele tenha descoberto para qual usuário é o pacote, o núcleo copia os dados para o espaço do usuário. Se presumirmos que cada interrupção e seu processamento associado leva 1 ms, que pacotes têm 1024 bytes (ignore os cabeçalhos) e que copiar um byte leva 1 μ s, qual é a frequência máxima que um processo pode enviar dados para outro? Presuma que o emissor esteja bloqueado até que o trabalho tenha sido concluído no lado receptor e que uma mensagem de reconhecimento retorne. Para simplificar a questão, presuma que o tempo para receber o reconhecimento de volta é pequeno demais para ser ignorado.
 16. Por que arquivos de saída para a impressora normalmente passam por um spool no disco antes de serem impressos?
 17. Quanto deslocamento de cilindro é necessário para um disco de 7200 RPM com um tempo de busca de trilha para trilha de 1 ms? O disco tem 200 setores de 512 bytes cada em cada trilha.
 18. Um disco gira a 7200 RPM. Ele tem 500 setores de 512 bytes em torno do cilindro exterior. Quanto tempo ele leva para ler um setor?
 19. Calcule a taxa de dados máxima em bytes/s para o disco descrito no problema anterior.
 20. RAID nível 3 é capaz de corrigir erros de bit único usando apenas uma unidade de paridade. Qual é o sentido do RAID nível 2? Afinal de contas, ele só pode corrigir um erro e precisa de mais unidades para fazê-lo.
 21. Um RAID pode falhar se duas ou mais das suas unidades falharem dentro de um intervalo de tempo curto. Suponha que a probabilidade de uma unidade falhar em uma determinada hora é p . Qual é a probabilidade de um RAID com k unidades falhar em uma determinada hora?
 22. Compare o RAID nível 0 até 5 em relação ao desempenho de leitura, desempenho de escrita, sobrecarga de espaço e confiabilidade.
 23. Quantos pebibytes há em um zebibyte?
 24. Por que os dispositivos de armazenamento óptico são inerentemente capazes de uma densidade de dados mais alta que os dispositivos de armazenamento magnético? *Nota:* esse problema exige algum conhecimento de física do Ensino Médio e como os campos magnéticos são gerados.
 25. Quais são as vantagens e as desvantagens dos discos ópticos *versus* os discos magnéticos?

26. Se um controlador de disco escreve os bytes que ele recebe do disco para a memória tão rápido quanto ele os recebe, sem armazenamento interno, o entrelaçamento é concebivelmente útil? Explique.
27. Se um disco tem entrelaçamento duplo, ele também precisa de deslocamento do cilindro a fim de evitar perder dados quando realizando uma busca de trilha para trilha? Discuta sua resposta.
28. Considere um disco magnético consistindo de 16 cabeças e 400 cilindros. Esse disco tem quatro zonas de 100 cilindros com os cilindros nas zonas diferentes contendo 160, 200, 240 e 280 setores, respectivamente. Presuma que cada setor contenha 512 bytes, que o tempo de busca médio entre cilindros adjacentes é 1 ms, e o disco gira a 7200 RPM. Calcule a (a) capacidade do disco, (b) deslocamento de trilha ótimo e (c) taxa de transferência de dados máxima.
29. Um fabricante de discos tem dois discos de 5,25 polegadas, com 10.000 cilindros cada um. O mais novo tem duas vezes a densidade de gravação linear que o mais velho. Quais propriedades de disco são melhores na unidade mais nova e quais são as mesmas? Há alguma pior na unidade mais nova?
30. Um fabricante de computadores decide redesenhar a tabela de partição de um disco rígido Pentium para gerar mais do que quatro partições. Cite algumas das consequências dessa mudança.
31. Solicitações de disco chegam ao driver do disco para os cilindros 10, 22, 20, 2, 40, 6 e 38, nessa ordem. Uma busca leva 6 ms por cilindro. Quanto tempo de busca é necessário para:
- (a) Primeiro a chegar, primeiro a ser servido.
 - (b) Cilindro mais próximo em seguida.
 - (c) Algoritmo do elevador (inicialmente deslocando-se para cima).
- Em todos os casos, o braço está inicialmente no cilindro 20.
32. Uma ligeira modificação do algoritmo do elevador para o escalonamento de solicitações de disco é sempre varrer na mesma direção. Em qual sentido esse algoritmo modificado é melhor do que o algoritmo do elevador?
33. Um vendedor de computadores pessoais visitando uma universidade na região sudoeste de Amsterdã observou durante seu discurso de venda que a sua empresa havia devotado um esforço substancial para tornar a sua versão do UNIX muito rápida. Como exemplo, ele observou que o seu driver do disco usava o algoritmo do elevador e também deixava em fila múltiplas solicitações dentro de um cilindro por ordem do setor. Um estudante, Harry Hacker, ficou impressionado e comprou um. Ele o levou para casa e escreveu um programa para ler aleatoriamente 10.000 blocos espalhados pelo disco. Para seu espanto, o desempenho que ele mensurou foi idêntico ao que era esperado do primeiro a chegar, primeiro a ser servido. O vendedor estava mentindo?
34. Na discussão a respeito de armazenamento estável usando RAM não volátil, o ponto a seguir foi minimizado. O que acontece se a escrita estável termina, mas uma queda do sistema ocorre antes que o sistema operacional possa escrever um número de bloco inválido na RAM não volátil? Essa condição de corrida arruína a abstração do armazenamento estável? Explique a sua resposta.
35. Na discussão sobre armazenamento estável, foi demonstrado que o disco pode ser recuperado para um estado consistente (uma escrita é concluída ou nem chega a ocorrer) se a falha da CPU ocorrer durante uma escrita. Essa propriedade se manterá se a CPU falhar novamente durante a rotina de recuperação? Explique sua resposta.
36. Na discussão sobre armazenamento estável, uma suposição fundamental é de que uma falha na CPU que corrompe um setor leva a um ECC incorreto. Quais problemas podem surgir nos cenários de cinco recuperações de falhas mostrados na Figura 5.27 se essa suposição não se mantiver?
37. O tratador de interrupção do relógio em um determinado computador exige 2 ms (incluindo a sobrecarga de troca de processo) por tique do relógio. O relógio executa a 60 Hz. Qual fração da CPU é devotada ao relógio?
38. Um computador usa um relógio programável em modo de onda quadrada. Se um cristal de 500 MHz for usado, qual deveria ser o valor do registrador (holding register) para atingir uma resolução de relógio de
- (a) um milissegundo (um tique de relógio a cada milissegundo)?
 - (b) 100 microssegundos?
39. Um sistema simula múltiplos relógios encadeando juntas todas as solicitações de relógio pendentes como mostrado na Figura 5.30. Suponha que o tempo atual seja 5000 e existem solicitações de relógio pendentes para o tempo 5008, 5012, 5015, 5029 e 5037. Mostre os valores do cabeçalho do relógio, Tempo atual e próximo sinal nos tempos 5000, 5005 e 5013. Suponha que um novo sinal (pendente) chegue no tempo 5017 para 5033. Mostre os valores do cabeçalho do relógio, Tempo atual e Próximo sinal no tempo 5023.
40. Muitas versões do UNIX usam um inteiro de 32 bits sem sinal para controlar o tempo como o número de segundos desde a origem do tempo. Quando esses sistemas entrarão em colapso (ano e mês)? Você acha que isso vai realmente acontecer?
41. Um terminal de mapas de bits contém 1600 por 1200 pixels. Para deslizar o conteúdo de uma janela, a CPU (ou controlador) tem de mover todas as linhas do texto para

cima copiando seus bits de uma parte da RAM do vídeo para outra. Se uma janela em particular tiver 80 linhas de altura e 80 caracteres de largura (6400 caracteres, total), e uma caixa de caracteres tem 8 pixels de largura por 16 pixels de altura, quanto tempo leva para passar a janela inteira a uma taxa de cópia de 50 ns por byte? Se todas as linhas têm 80 caracteres de comprimento, qual é a taxa de símbolos por segundo (baud rate) do terminal? Colocar um caractere na tela leva 5 μ s. Quantas linhas por segundo podem ser exibidas?

42. Após receber um caractere DEL (SIGINT), o driver do monitor descarta toda a saída atualmente em fila para aquele monitor. Por quê?
43. Um usuário em um terminal emite um comando para um editor remover a palavra na linha 5 ocupando as posições de caractere 7 até e incluindo 12. Presumindo que o cursor não está na linha 5 quando o comando é dado, qual sequência de escape ANSI o editor deve emitir para remover a palavra?
44. Os projetistas de um sistema de computador esperavam que o mouse pudesse ser movido a uma taxa máxima de 20 cm/s. Se um mickey é 0,1 mm e cada mensagem do mouse tem 3 bytes, qual é a taxa de dados máxima do mouse presumindo que cada mickey é relatado separadamente?
45. As cores primárias aditivas são vermelho, verde e azul, o que significa que qualquer cor pode ser construída a partir da superposição linear dessas cores. É possível que uma pessoa tenha uma fotografia colorida que não possa ser representada usando uma cor de 24 bits inteira?
46. Uma maneira de colocar um caractere em uma tela com mapa de bits é usar BitBlt com uma tabela de fontes. Presuma que uma fonte em particular usa caracteres que são 16×24 pixels em cor RGB (red, green, blue) real.
 - (a) Quanto espaço da tabela de fonte cada caractere ocupa?
 - (b) Se copiar um byte leva 100 ns, incluindo sobrecarga, qual é a taxa de saída para a tela em caracteres/s?
47. Presumindo que são necessários 2 ns para copiar um byte, quanto tempo leva para reescrever completamente uma tela de modo texto com 80 caracteres $\times$ 25 linhas, no modo de tela mapeada na memória? E uma tela em modo gráfico com 1024×768 pixels com 24 bits de cores?
48. Na Figura 5.36 há uma classe para *RegisterClass*. No código X Window correspondente, na Figura 5.34, não há uma chamada assim, nem algo parecido. Por que não?
49. No texto demos um exemplo de como desenhar um retângulo na tela usando o Windows GDI:

`Rectangle(hdc, xleft, ytop, xright, ybottom);`

Existe mesmo a necessidade para o primeiro parâmetro (*hdc*), e se afirmativo, qual? Afinal de contas, as

coordenadas do retângulo estão explicitamente especificadas como parâmetros.

50. Um terminal de cliente magro é usado para exibir uma página na web contendo um desenho animado de tamanho $400 \text{ pixels} \times 160 \text{ pixels}$ executando a 10 quadros/s. Qual fração de um Fast Ethernet de 100 Mbps é consumida exibindo o desenho?
51. Foi observado que um sistema de cliente magro funciona bem com uma rede de 1 Mbps em um teste. É provável que ocorram problemas em uma situação de múltiplos usuários? (*Dica*: considere um grande número de usuários assistindo a um programa de TV programado e o mesmo número de usuários navegando na internet.)
52. Descreva duas vantagens e duas desvantagens da computação com clientes magros.
53. Se a voltagem máxima da CPU, V , for cortada para V/n , seu consumo de energia cairá para $1/n^2$ do seu valor original e sua velocidade de relógio cairá para $1/n$ do seu valor original. Suponha que um usuário está digitando a 1 caractere/s, mas o tempo da CPU necessário para processar cada caractere é 100 ms. Qual é o valor ótimo de n e qual é a economia de energia correspondente percentualmente comparada com não cortar a voltagem? Presuma que uma CPU ociosa não consuma energia alguma.
54. Um notebook é configurado para tirar o máximo de vantagem das características de economia de energia, incluindo desligar o monitor e o disco rígido após períodos de inatividade. Uma usuária às vezes executa programas UNIX em modo de texto e outras vezes usa o Sistema X Window. Ela ficou surpresa ao descobrir que a vida da bateria é significativamente mais longa quando usa programas somente de texto. Por quê?
55. Escreva um programa que simule o armazenamento estável. Use dois arquivos grandes de comprimento fixo no seu disco para simular os dois discos.
56. Escreva um programa para implementar os três algoritmos de escalonamento de braço de disco. Escreva um programa de driver que gere uma sequência de números de cilindro (0-999) ao acaso, execute os três algoritmos para essa sequência e imprima a distância total (número de cilindros) que o braço precisa para percorrer nos três algoritmos.
57. Escreva um programa para implementar múltiplos temporizadores usando um único relógio. A entrada para esse programa consiste em uma sequência de quatro tipos de comandos (S <int>, T, E <int>, P): S <int> estabelece o tempo atual para <int>; T é um tique de relógio; e E <int> escala um sinal para ocorrer no tempo <int>; P imprime os valores do Tempo atual, Próximo sinal e Cabeçalho do relógio. O seu programa também deve imprimir um comando sempre que for chegado o momento de gerar um sinal.